ANIL TRANSPORT SERVICE · FLEET MANAGEMENT

Architecture Blueprint

How the ATS Fleet App is built, from the cloud server at the bottom to the button on a driver's phone at the top.

Version of the code described: branch "maintenance", 14,834 lines in app.py, 199 URLs, 36 database tables
Written for two readers at once: someone who has never seen code, and the engineer who has to change it.
Generated 12 October 2026 directly from the project files, so the numbers are real.

From a trip booked on a phone → to a balance, an invoice and a decision.

How to read this document

This is a complete map of the system. It is long on purpose: every layer is explained twice — first in plain words with a picture, then in exact technical detail with file names, function names and line numbers.

If you are...	Start with	You can skip
The owner / manager who wants to understand what you have	Part A (the big picture), the diagrams in Parts B–E, and the findings in Chapter 52 (Part H)	The tables of functions, the data dictionary and the execution cards
A new developer	Part A, then Part C (back-end), Part D (database), Part G (front-end); Part E for how each feature works	Nothing — but read Chapter 30 (money) before changing money code
Someone deploying or fixing a problem at 2 a.m.	Part B and Chapter 53 (runbook, Part I)	Everything else until you need it
An auditor or security reviewer	Chapter 21 (security), Chapter 50 (testing) and Chapter 52 (known gaps)	
Someone who wants to know exactly what one URL or one table does	Part F: Chapter 41 (a card for every URL), Chapter 42 (the life of every table), Chapter 43 (screen to data)	

Conventions used here

Names in `this typeface` are real file, function, table or setting names you can search for. "Line 10071" means line 10071 of app.py at the time of writing. Diagrams are numbered Figure 1, 2, 3… and are drawn from the real design, not from a template.

One idea carries the whole design

Every screen — website or phone — is produced by the same Python program reading the same database file. There is only one copy of every business rule (how a bill is calculated, how a payment is applied), so two screens can never disagree. Most of the testing in this project exists to prove exactly that.

Contents

How to read this document . . . 2

Contents . . . 3

1. What this system is . . . 10

1.1 Who uses it, and how . . . 10

1.2 The journey of one trip through the system . . . 10

2. Plain-language glossary . . . 12

3. The whole system on one page . . . 14

4. The layer cake: base to top . . . 15

4.1 Technology stack, with exact versions . . . 15

5. Map of the project folders . . . 17

6. Hosting: one web service on Render . . . 19

6.1 How the web service starts . . . 19

6.2 Why one machine is enough, and when it is not . . . 19

7. Settings the app reads from its environment . . . 20

8. From the developer's laptop to the live site . . . 21

8.1 The deployment discipline . . . 21

8.2 What the app does by itself every time it starts . . . 21

9. Backups and restore . . . 23

9.1 Net one: Render's snapshots . . . 23

9.2 Net two: your own verified daily copy . . . 23

10. Outside services and what happens when they fail . . . 25

11. Anatomy of app.py . . . 27

11.1 What lives where (line ranges) . . . 27

11.2 Naming habits that make the file navigable . . . 28

12. The life of a request . . . 29

12.1 The hooks, in running order . . . 29

13. The database access layer . . . 31

14. Sign-in and sessions . . . 32

14.1 The numbers that define the behaviour . . . 32

15. Who may do what: roles and access . . . 34

15.1 The four kinds of user . . . 34

15.2 Limited Access in detail . . . 35

15.3 The one-Administrator rule . . . 35

16. Validation and error handling . . . 36

17. The business engines, one by one . . . 37

17.1 Trips . . . 37

17.2 Parties, vendors and the linked-account merge . . . 38

17.3 The ledger engine . . . 38

17.4 Payments and allocation . . . 38

17.5 Invoices . . . 40

17.6 The maintenance family . . . 40

17.7 Expenses (overheads) . . . 42

17.8 Employees, salary, advances and attendance . . . 42

17.9 Vehicles, documents and compliance . . . 42

17.10 Company settings, banking, tax . . . 42

17.11 Exports . . . 42

18. Reports and analytics . . . 43

18.1 The profit rule, stated once . . . 43

18.2 The Route Finder . . . 43

19. The phone app (/app/) on the server side . . . 45

20. Background jobs . . . 45

21. Security architecture . . . 47

22. The database at a glance . . . 50

22.1 How the file is configured . . . 50

23. How the tables relate . . . 51

23.1 Every foreign-key link (generated from the schema) . . . 53

24. Data dictionary . . . 55

25. Conventions, rules and multi-company readiness . . . 72

25.1 Rules enforced by the database itself . . . 72

25.2 Sign and meaning conventions . . . 72

25.3 The PostgreSQL twin . . . 72

26. Migrations: how the database layout has changed . . . 74

27. How to read a walkthrough . . . 76

27.1 Things every write has in common . . . 76

28. Sign-in and sign-out . . . 77

28.1 Password sign-in . . . 77

28.2 Sign-in by SMS code . . . 77

28.3 Sign-out and "sign-up" . . . 78

29. Trips . . . 80

29.1 Add a trip . . . 80

29.2 Edit a trip . . . 81

29.3 End a trip . . . 82

29.4 Delete a trip . . . 82

29.5 The Trips list (how trips are retrieved and shown) . . . 83

29.6 View one trip . . . 83

30. Money: payments, ledgers, parties and vendors . . . 85

30.1 Record money received from a party . . . 85

30.2 Record money paid to a vendor . . . 86

30.3 View a payment . . . 86

30.4 The Ledger list (all parties and vendors) . . . 87

30.5 One party's ledger . . . 87

30.6 Add a party or vendor; edit contact; opening balance; activate/deactivate . . . 88

31. Expenses . . . 90

31.1 Add, edit and delete an expense . . . 90

31.2 The Expenses list . . . 90

32. Maintenance: service, tyres, batteries, urea, tolls, compliance fees and insurance . . . 92

32.1 Add or edit a service bill . . . 92

32.2 Tyres: buy and fit, buy for stock, fit from stock, scrap, delete . . . 93

32.3 Batteries . . . 94
32.4 Urea (AdBlue) . . . 94
32.5 Tolls: manual entry, Excel import, approve / reject, FASTag sync . . . 95
32.6 Compliance fees, insurance policies, generic bills . . . 96
32.7 The Maintenance hub (how everything above is shown) . . . 97

33. Vehicles and their documents . . . 98
33.1 Add, edit, activate and delete a vehicle . . . 98
33.2 Record a renewal; view RC details; refresh from the outside . . . 98
33.3 The Vehicles list and a vehicle's detail page . . . 99

34. People: employees, salaries, advances and attendance . . . 100
34.1 Add or edit an employee . . . 100
34.2 Pay a salary, give an advance, record a repayment . . . 100
34.3 Attendance and payslips . . . 101
34.4 The Employees hub . . . 101

35. Invoices . . . 103
35.1 Choose trips in the Invoice Center . . . 103
35.2 Review, then generate, draft or preview . . . 103
35.3 Generated invoices: list, edit, add/remove lines, PDF, delete; per-trip invoices . . . 104

36. Reports and exports: how the numbers are retrieved . . . 106
36.1 Dashboard . . . 106
36.2 Business Performance . . . 106
36.3 Performance (drivers and vehicles) . . . 106
36.4 Route Analytics . . . 107
36.5 Fleet Utilization . . . 107
36.6 Exports . . . 108

37. Administration: settings, users and your own profile . . . 109
37.1 Save company settings (company, banking, invoice, tax, SMS, keys, sync) . . . 109
37.2 Upload the company logo . . . 110
37.3 Create, edit, hand over and remove users . . . 110
37.4 Your own profile and password . . . 111

38. The Route Finder . . . 112
38.1 Type-ahead, search and stops . . . 112

39. The phone app: how it relates to the website . . . 113

40. Background jobs and operational endpoints . . . 115

40.1 Scheduled vehicle-data refresh . . . 115

40.2 Database backup download . . . 115

40.3 What happens at start-up . . . 116

41. Execution cards: every URL . . . 118

Open pages and sign-in (9) . . . 118

Internal endpoints (token protected) (2) . . . 120

Profile, settings and users (19) . . . 120

Screen: Dashboard (3) . . . 124

Screen: Trips (13) . . . 125

Screen: Vehicles (18) . . . 129

Screen: Maintenance (50) . . . 133

Screen: Ledger (including payments) (22) . . . 144

Screen: Invoices (16) . . . 149

Screen: Expenses (8) . . . 153

Screen: Salaries (19) . . . 155

Screen: Reports (19) . . . 158

Whole-database export (1) . . . 162

42. The life of each table . . . 163

43. Screen-to-data map . . . 171

44. The two layouts . . . 177

44.1 Website layout: templates/base.html . . . 177

44.2 Phone layout: templates/app/base_app.html . . . 178

45. Template inventory . . . 179

Website pages (39) . . . 179

Partials (tabs, modals, shared blocks) (23) . . . 180

Phone-app templates (45) . . . 181

46. Static files: styles, scripts, images . . . 184

47. Behaviours that run in the browser . . . 185

48. The installable app (PWA) and in-app navigation . . . 186

49. Design system . . . 187

50. How the app is tested . . . 189

50.1 The audit harness (test_balance_audit.py, 3,000+ lines) . . . 189

50.2 The other test files . . . 190

51. Performance . . . 192

51.1 Many people at once . . . 192

51.2 What made it fast . . . 192

52. Findings recorded while preparing this document . . . 193

52.1 Data-integrity gaps (left behind rows or silent no-ops) . . . 193

52.2 Design and operations notes . . . 193

52.3 Business decisions still open (screens may differ by definition) . . . 194

53. Runbook . . . 196

53.1 Run it on your computer . . . 196

53.2 Everyday tasks . . . 196

53.3 When something is wrong . . . 196

54. File-by-file reference . . . 197

55. Adding something new without breaking the rules . . . 199

55.1 Roadmap . . . 199

A

The big picture

What the system is, who uses it, the words you need, and the technology in one table.

IN THIS PART

- What the app does and for whom
- Plain-language glossary
- Everything on one page (system map)
- The layer cake: base to top
- Technology stack with versions
- Map of the project folders

1. What this system is

The ATS Fleet App is the day-to-day operating system of a road-transport business, Anil Transport Service. It replaces an Excel workbook (Fleet_Master_Tracker.xlsx) with a web application that records every truck trip, every rupee that is owed or paid, every repair and document, and turns those records into balances, invoices and performance reports.

14,834	199	36	107	17
lines in app.py	URLs (pages and actions)	database tables	page templates	automated test stages

1.1 Who uses it, and how

- **The Administrator** (one person) controls company settings, bank and invoice details, users and what each user may see.
- **Managers and staff** book trips, record payments, enter maintenance and expenses, and generate invoices, with as much access as the Administrator grants (Full, Read Only, or Limited to chosen screens).
- **People on the road or at the yard** use the same system as an installable phone app (a "PWA") that looks like a native app but is really the same web server.
- Today about five people use it; the design has room for a few dozen at once.

1.2 The journey of one trip through the system

The best way to understand the architecture is to follow one piece of business through it. Every step below is a real function in the code; later parts of this document explain each in detail.

Step	What happens in business terms	What happens in the system
1	A truck is booked to carry goods for a customer (a "party").	The Add Trip form posts to `add_trip` (website) or `app_add_trip` (phone). Both call the same `_insert_trip_from_form`, so a trip is calculated identically from either screen.
2	The freight is worked out: quantity × rate (or a fixed amount), plus extras such as detention, minus deductions such as shortage or TDS.	`_trip_freight` and the billed-amount formula; every number first passes `_parse_money` (no letters, no NaN, no negatives).
3	Fuel and a driver advance are given; for a hired truck the owner's terms are recorded.	Stored on the same `trips` row (80 columns); fuel pumps and owners become `vendors` automatically.
4	The customer pays some money, maybe before the trip is finished.	Typed on the trip, or recorded from the ledger with the payment popup: `_insert_party_payment_from_form` splits it over the oldest pending trips and writes `payment_allocations`.
5	The trip is finished; shortage or extra expenses are settled.	`end_trip` / `_end_trip_from_form` records the end date and corrections.
6	An invoice is made for a batch of trips.	Invoice Center → review gate → `invoice_center_generate` → ReportLab PDF; rows go to `invoice_batches` and friends.
7	The owner wants to know who owes what, and whether the month made money.	The ledger engine computes balances from the same allocations; Dashboard and Business Performance read the same numbers.
8	Someone asks "was that trip profitable on that route?"	Route Analytics and Performance group the same trips by route, driver and vehicle.

Own versus hired trucks

A trip on one of the company's own trucks costs fuel, advances, tolls and so on. A trip on a hired truck costs what the vehicle owner is paid; fuel and advances on a hired trip sit inside that owner payment and are not counted twice (only what the *company* itself paid is deducted from the owner's dues). This one rule shows up in the vendor ledger, the payment popup, owner invoices, the Dashboard and Business Performance, and tests keep them identical.

2. Plain-language glossary

If a word in this document is unfamiliar, it is probably here.

Word	Meaning
Server	A computer that is always on and answers requests from other computers. Here: a Render cloud machine.
Browser	The program (Chrome, Safari) that shows web pages.
Request / response	The browser asks for a page or sends a form (request); the server answers with a page, a file or data (response).
HTTP / HTTPS	The language of the web. HTTPS is the same, scrambled so nobody on the way can read it (the padlock).
Python	The programming language the whole back-end is written in.
Flask	A Python toolkit for building websites: it matches a web address to a Python function (a "route") and sends back the result.
Route	One web address handled by one function, e.g. `/trips/add` → `add_trip()`. This app has 199.
Template (Jinja2)	An HTML page with blanks. The server fills the blanks with real data before sending it. 107 templates make all the screens.
Front-end / back-end	Front-end: what runs in the browser (HTML, CSS, JavaScript). Back-end: what runs on the server (Python and the database).
CSS / JavaScript	CSS = colours, sizes, layout. JavaScript = small programs that make a page react without reloading.
Database	Organised storage for records. Here: one SQLite file, `fleet.db`, holding 36 tables.
SQLite	A database that lives in a single file instead of a separate server. Simple, fast and perfect for a small team.
Table / row / column	A table is like one Excel sheet; a row is one record (one trip); a column is one field (the date).
SQL	The language used to ask the database questions and make changes.
Index	A ready-made lookup list (like a book index) that makes searching a big table fast.
Foreign key	A column that points at a row in another table, e.g. a trip's `party_id` points at a party.
Transaction / commit	A group of changes that must all happen or none; "commit" makes them permanent.
WAL mode	A SQLite setting that lets people read while someone else is saving, so screens do not freeze.
Session / cookie	After login the server hands the browser a small signed note (a cookie) that says who you are; the browser sends it back with every click.
Hash	A one-way scramble. Passwords are stored as hashes, so even the database owner cannot read them.
OTP	One-time password: a 6-digit code sent by SMS, valid for 5 minutes, usable once.
MSG91 / DLT	MSG91 is the Indian company that delivers the SMS. DLT is India's mandatory registration of SMS templates and senders.
gunicorn	The production web-server program that runs the Flask app with several workers at once.
Worker / thread	A worker is a copy of the app running as its own process; a thread is one of its lanes. Here 2 workers × 4 threads = 8 requests at the same time.
Render	The cloud company that hosts the app (a "web service" with a "persistent disk").
Persistent disk	Storage that survives restarts and redeployments. Without it, a hosted file system is wiped every time.
Snapshot / backup	A saved copy of data at one moment, used to go back after a mistake or disaster.
Migration	A scripted change to the database layout (add a column, add an index) done safely and repeatably.
PWA	Progressive Web App: a website that can be installed on a phone home screen and opens full-screen like an app.
Service worker	A tiny background script of a PWA. Here it only keeps two icons ready; it deliberately never stores data.
API	A way for programs to talk to each other. The app calls APIs of MSG91, eChallan and OpenStreetMap.
Ledger	The running account of one party or vendor: debits, credits and a balance after each line.
Party / vendor	Party = a customer you carry goods for (they owe you). Vendor = someone you pay: truck owners, fuel pumps, workshops.

Word	Meaning
LR	Lorry Receipt: the consignment note number that identifies a trip for the customer.
Allocation	The record that says "this payment of Rs 50,000 settled these trips".
Multi-tenancy	One system serving several separate companies. Prepared for (every table has company_id), not switched on yet.
Test harness	A program that drives the app automatically with thousands of invented scenarios to catch wrong numbers.

3. The whole system on one page

Read Figure 1 from left to right: people use the website or the phone app; their requests travel over the internet to one web service on Render; the service holds both the program and the database file; and three outside services help with SMS, maps and vehicle data. The bottom row shows how work gets in (the developer's laptop and GitHub) and how safety copies get out (your Mac).

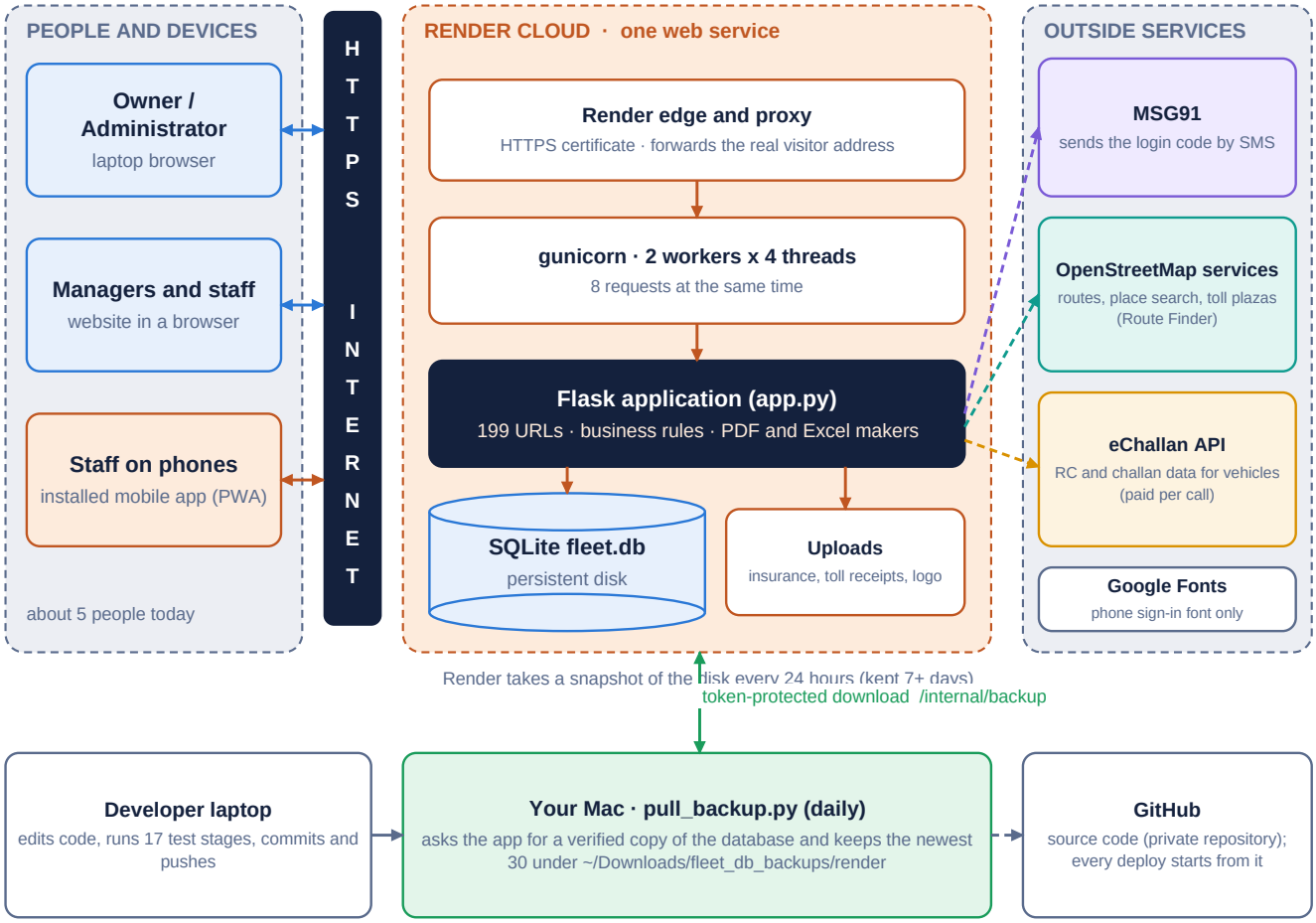


Figure 1 — The whole system on one page: who uses it, where it runs, and what it talks to.

The most important fact about this picture

There is **one** server program and **one** database file. There is no separate "mobile back-end", no cache server, no message queue. That keeps it cheap and simple, and it is why backups and the persistent disk matter so much: the single file *is* the business.

4. The layer cake: base to top

Every web application is a stack of layers, each one standing on the one below. Figure 2 names them from the infrastructure at the base (1) to the people at the top (9). When something goes wrong, the first question is always "which layer?" — a slow page is usually layer 3 or 4 (database), a wrong colour is layer 8, a refused login is layer 7.

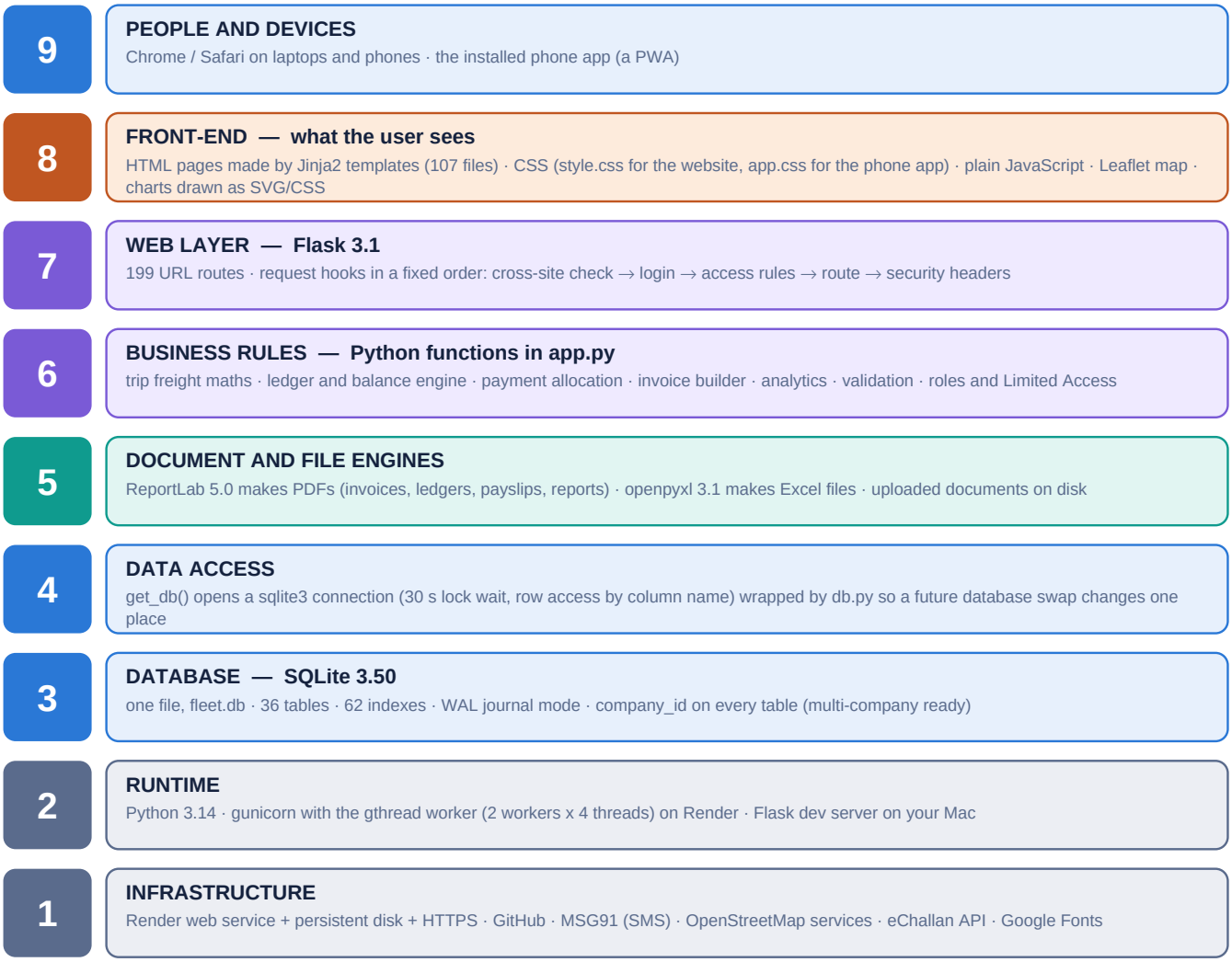


Figure 2 — The layer cake, base (1) to top (9). A request travels from layer 9 down to 3 and the answer climbs back up.

4.1 Technology stack, with exact versions

Layer	Technology	Version	Plain explanation	Where
Language	Python	3.14.6	The language of the back-end	all `.py` files
Web framework	Flask	3.1.3	Matches URLs to functions, handles forms, sessions, cookies	`app.py`
Web toolkit	Werkzeug	3.1.8	Low-level web parts under Flask: password hashing, proxy fix, routing	imported by Flask
Templates	Jinja2 / MarkupSafe	3.1.6 / 3.0.3	Fills HTML pages with data; escapes text so it cannot inject code	`templates/` (107 files)
Cookie signing	itsdangerous	2.2.0	Signs the session cookie so it cannot be forged	inside Flask
Production server	gunicorn (gthread)	26.0.0	Runs the app with 2 workers × 4 threads on Render	`Procfile`
Database	SQLite	3.50.4	A whole database in one file, WAL mode	`fleet.db`

Layer	Technology	Version	Plain explanation	Where
PDF	ReportLab	5.0.0	Draws invoices, ledgers, payslips, reports as PDFs	inside `app.py`
Excel	openpyxl	3.1.5	Reads the toll Excel import, writes every Excel export	inside `app.py`
HTTP client	requests	2.34.2	Calls MSG91, eChallan, OSRM, Nominatim, Photon, Overpass	app.py, `providers/`
Scheduler	APScheduler	3.11.3	Hourly tick that runs background syncs (locally; on Render an outside cron)	`scheduler.py`
Map library	Leaflet (bundled)	1.x	Draws the Route Finder map; shipped inside the project, no CDN	`static/vendor/leaflet`
Font	Poppins (Google Fonts)	—	The one external resource a page loads: only the phone app's sign-in, SMS-code and sign-up screens use it. Every other page uses system fonts (Segoe UI, San Francisco).	templates/app/login*.html, signup, verify_otp
Charts	hand-drawn SVG / CSS	—	No chart library: bars, lines, donuts are SVG and CSS built by the templates	dashboard, performance, route analytics
Hosting	Render	—	Runs the web service, gives HTTPS, a persistent disk and daily snapshots	cloud
Source control	git + GitHub	—	History of every change; Render deploys from it	repository
SMS	MSG91 Send-OTP API	v5	Delivers login codes by SMS in India	`_send_otp_msg91`
Vehicle data	eChallan API	—	RC details and traffic challans (paid per call, switched off by default)	`providers/echallan_client.py`
Maps and routes	OSRM, Nominatim, Photon, Overpass	—	Free OpenStreetMap-based services behind the Route Finder	`providers/osm_routes.py`
Test tools	Flask test client, beautifulsoup4, lxml (dev only)	—	Drive the app and read the rendered pages in tests	`test_*.py`

What is deliberately NOT in the stack

No JavaScript framework (React, Vue), no build step, no Node.js, no ORM, no Redis, no Celery, no separate API server. Fewer moving parts means fewer things to break, patch or pay for. The price is that app.py is large (Chapter 55.1 discusses splitting it).

5. Map of the project folders

Everything lives in one folder. This table lists every top-level item and what it is for; Chapter 54 (Part I) has the file-by-file reference.

Item	Kind	What it is
app.py	code	The application: 14,834 lines, 434 top-level functions (199 URLs + 235 helpers/hooks). Back-end, business rules, PDF/Excel makers, mobile (/app/) routes.
db.py	code	Thin database wrapper — the one place a future switch to PostgreSQL would be wired in.
scheduler.py, compliance_service.py	code	Background syncs of vehicle documents, RC and challans.
providers/	code	Clients for outside services: eChallan (RC/challan), OpenStreetMap services (routes), and mock providers for fitness/PUC/permit/insurance.
config/apis.py	code	Defaults for compliance providers (keys come from environment variables).
templates/	front-end	107 HTML templates: 39 website pages, 23 partials (tabs/modals), 45 phone-app files.
static/	front-end	CSS, JavaScript, icons, images, the Leaflet library and uploaded files (logo, documents).
schema.sql, schema_postgres.sql	database	The authoritative SQLite layout and the prepared PostgreSQL twin (with real multi-company isolation).
migrate_*.sql / .py	database	15 historical, documented database changes. The important ones are also applied automatically at start-up.
fleet.db	data	THE data (never committed to git; backed up separately).
Procfile	deploy	The one line Render runs to start the app.
requirements.txt	deploy	Python packages to install.
test_*.py (16 files)	quality	Automated tests; test_balance_audit.py (3,000+ lines) is the big harness.
load_test.py	quality	Seeds 12,000 trips and times every screen.
db_health_check.py, backup_db.py, pull_backup.py, restore_check.py	operations	Read-only data audit, local backup, off-site pull from Render, and proof-of-restore.
CLAUDE.md	rules	The project rules: keep multi-company readiness, test on disposable copies, deploy database and code separately.
***.md reports**	docs	DEPLOY_TO_CLOUD, BACKUP_AND_RESTORE, production_readiness_report, code_health_report, balance_audit_findings, load_test_report.
docs/blueprint/	docs	The program that generates this PDF from the real files.
archive/, audit_reports/	misc	Retired screens and earlier test reports.

B

Infrastructure and deployment

Where the app runs, how code gets there, what it does when it wakes up, and how the data is protected.

IN THIS PART

- Hosting on Render
- Settings the app reads (environment variables)
- From laptop to live site
- What the app does by itself on every start
- Backups and restore
- Outside services and what happens when they fail

6. Hosting: one web service on Render

The production system is a single Render **web service** that serves the website and the phone app from the same address (the live site is https://ats-fleet-app.onrender.com). Render supplies the machine, the HTTPS certificate, the proxy in front, the persistent disk and daily disk snapshots. Nothing else is needed to run it.

6.1 How the web service starts

```
web: gunicorn app:app --worker-class=gthread --workers=2 --threads=4 --timeout=60
```

- **gunicorn** is the production web server. `app:app` means "inside app.py, use the object named app" (the Flask application).
- **--workers=2** starts two separate copies of the app. They do not share memory, which is why anything that must be shared (login attempt counters, SMS codes, the secret key) lives in the database, not in Python variables.
- **--worker-class=gthread --threads=4** gives each worker four lanes, so 2 × 4 = **8 requests at the same time**. While one thread waits for the database or a PDF, the others keep serving.
- **--timeout=60** kills a request that runs longer than 60 seconds (protects against a stuck report).
- The Flask development server (`python app.py`) is only for your own computer: it listens on 127.0.0.1:5050, has debug mode on by default and handles requests in threads without the gunicorn safety net.

6.2 Why one machine is enough, and when it is not

SQLite allows many readers and one writer at a time; in WAL mode readers never wait for the writer. With 30 seconds of lock waiting, two people saving at the same instant simply queue for a moment. The load test (12,000 trips, ten simultaneous users, real gunicorn) kept the slowest 5% of requests under 1.7 seconds. The honest ceiling is roughly 25–30 people editing simultaneously or very heavy report use; the planned answer is PostgreSQL (Chapter 25), which the code and schema are already prepared for.

> **The persistent disk is not optional**
>
> A Render web service starts from a clean file system every time it is deployed or restarted. The database file (and the uploaded documents) survive only if they live on a **Persistent Disk** mounted on the service, with `DATABASE_PATH` pointing into it (for example /var/data/fleet.db). Render keeps an automatic snapshot of the disk every 24 hours for at least 7 days. A disk can only be reached by its one service, cannot be shared, and can be enlarged but never shrunk. **Status: to be confirmed on the live service.**

7. Settings the app reads from its environment

"Environment variables" are named settings given to the program from outside (in Render's dashboard or the shell), so secrets never have to be written in the code. The table is generated from the real `os.environ.get(...)` calls in app.py.

Variable	What it does	Typical state
`ALLOW_SELF_SIGNUP`	1 reopens the public /app/signup. Default closed: only an Administrator creates users.	default off
`BACKUP_TOKEN`	Secret that unlocks POST /internal/backup (database download). Without it the endpoint refuses everyone.	Render: set
`DATABASE_PATH`	Where fleet.db lives. Default: next to app.py. On Render it must point onto the persistent disk.	Render: set
`DISABLE_ORIGIN_CHECK`	1 disables the cross-site POST check. For diagnosing a proxy problem only.	default off
`ENABLE_CHALLAN_SYNC`	1 allows the paid eChallan challan sync job.	default off
`ENABLE_COMPLIANCE_SCHEDULER`	1 starts the in-process APScheduler (local only).	default off
`ENABLE_RC_SYNC`	1 allows the paid eChallan RC sync job.	default off
`FLASK_DEBUG`	Only used by python app.py. Default 1 (debug on) for your laptop; use 0 whenever the machine is reachable by others (the LAN command in the runbook does).	local
`FLASK_HOST`	Which network address the development server binds to. 127.0.0.1 = this computer only; 0.0.0.0 = also other devices on the Wi-Fi.	local
`FLASK_USE_RELOADER`	1 turns on auto-restart on every file save (off by default so a phone being tested is not disconnected).	local
`INTERNAL_SYNC_TOKEN`	Secret for POST /internal/sync-tick, the outside cron trigger for background syncs. Without it the endpoint refuses everyone.	Render: set if syncing
`RENDER`	Set automatically by Render. Switches on Secure cookies and ProxyFix (so the real visitor address is known for the login limit).	automatic
`SECRET_KEY`	Signs the login cookie. If not set, the app creates a private random key once and keeps it in the database (settings row _auto_secret_key). Set your own long random value on Render.	Render: set
`WERKZEUG_RUN_MAIN`	Internal flag of the auto-reloader; lets the scheduler start only once.	internal

Provider keys for the placeholder compliance providers (`VAHAN_API_KEY`, `INSURANCE_API_KEY`, `PUC_API_KEY`) are read in `config/apis.py`; until real keys exist those providers return clearly labelled mock data and the screens say so. The two real paid keys (eChallan RC lookup, MSG91) are not environment variables: the Administrator saves them in Settings, they are stored in the `settings` table, shown masked, and never printed back into a page.

8. From the developer's laptop to the live site

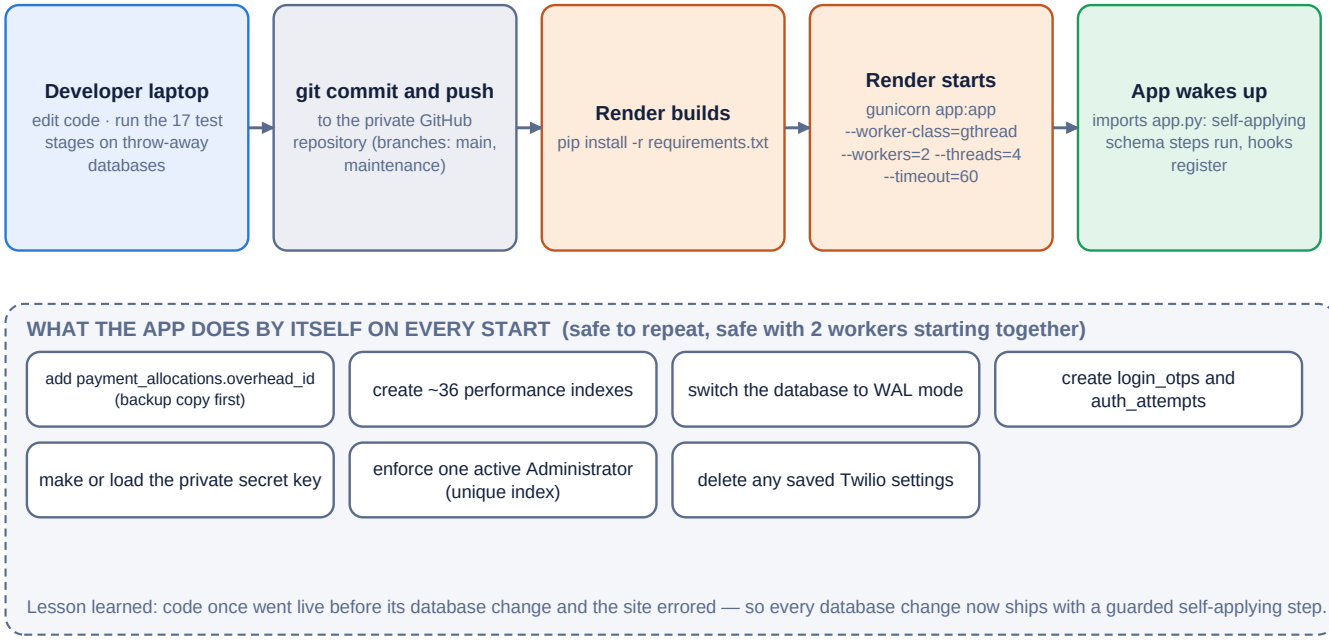


Figure 11 — From a change on the laptop to a running site, and the housekeeping the app does each time it starts.

8.1 The deployment discipline

- **Test first, on copies.** Every change that touches data is tried on a disposable copy of the database (never the real fleet.db), with network calls blocked so a bug can never spend paid API credits or send a real SMS.
- **Database and code are separate steps.** The rule in CLAUDE.md is to migrate and verify the database first, then deploy the code that needs it. Because the user deploys code without running SQL by hand, every new database change also ships a *guarded self-applying step* that runs at start-up (below), so shipping the code first can never take the site down — which is exactly what happened once.
- **Back up before touching the live file.** Before the overhead-allocation change the app saves its own copy (`fleet.db.bak.before_overhead_alloc_*`); manual changes are preceded by a SQLite online backup into ~/Downloads/fleet_db_backups.
- **Branches.** The repository has main, maintenance (the current working branch), feature/ledger-redesign and chore/pycache-gitignore. The database file, uploads of documents and environment files are in .gitignore; eleven old database backups were committed in the past and are flagged in the readiness report for removal from history.

8.2 What the app does by itself every time it starts

These steps run while app.py is being imported — once per gunicorn worker — before the first request. Each is written so that running it again, or running it in two workers at the same moment, does no harm. Progress is printed as `[schema] ...` or `[security] ...` lines in the server log.

#	Function (line)	What it does	Why it is safe to repeat
1	`_ensure_overhead_allocation_column` (129)	Adds payment_allocations.overhead_id (so payments can settle expense bills) and widens the "exactly one target" CHECK.	Saves a backup copy first; one BEGIN IMMEDIATE transaction; re-checks for the column inside it; verifies row counts.
2	`_ensure_performance_indexes` (245)	Creates the lookup indexes (the database holds 62 indexes in total) including partial indexes on mostly-empty allocation columns.	CREATE INDEX IF NOT EXISTS; a missing column is skipped, never fatal.
3	`_ensure_wal_mode` (276)	Switches the file to WAL journaling (readers never block the writer).	Reads the mode first; the setting is persistent.
4	`_ensure_auth_tables` (303)	Creates login_otps and auth_attempts.	CREATE TABLE IF NOT EXISTS.

#	Function (line)	What it does	Why it is safe to repeat
5	`_ensure_secret_key` (344)	Uses SECRET_KEY if set; otherwise creates one random key and stores it in settings (_auto_secret_key).	INSERT OR IGNORE, so both workers end up with the same key.
6	`_ensure_single_admin_index` (372)	Creates the unique index that allows only one active Administrator.	If two active Administrators already exist it skips and warns instead of crashing.
7	`_remove_twilio_settings` (396)	Deletes any saved Twilio credentials left from the old SMS provider.	A plain DELETE of known keys; harmless when none exist.

What the app does NOT do on start

It never deletes business data, never rewrites trips or payments, and never calls a paid outside service. Background syncs are opt-in (Chapter 20).

9. Backups and restore

The business is one file, so protecting that file is the single most important operational duty. There are two independent safety nets (Figure 12) — and one honest gap.

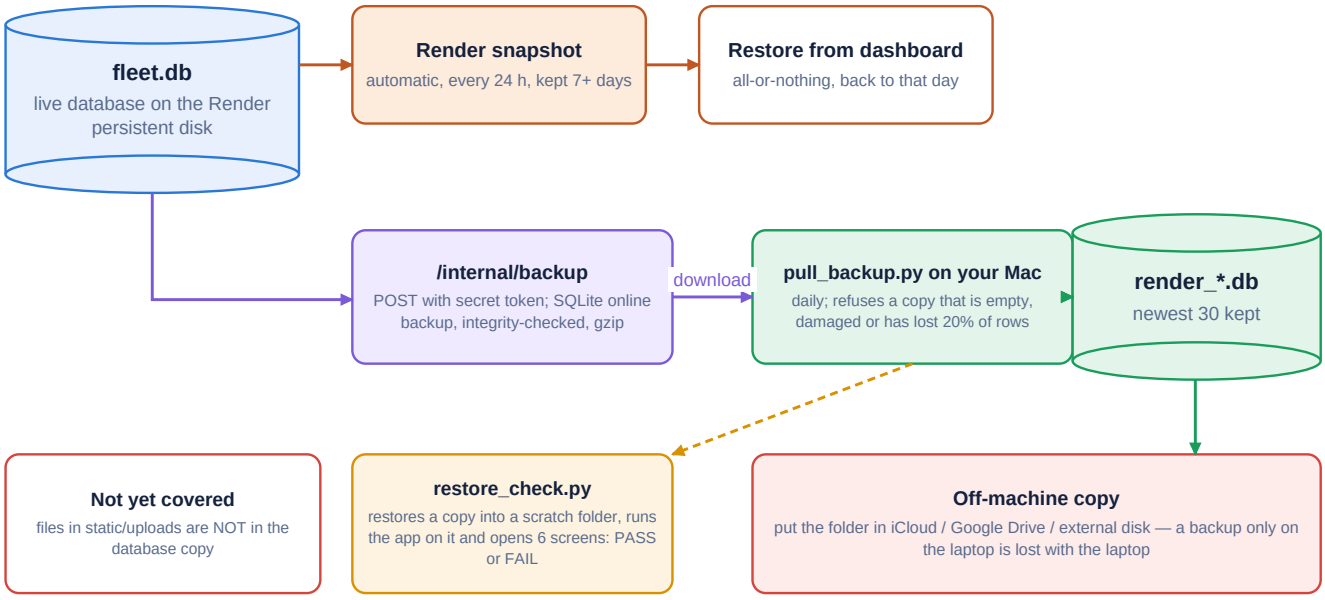


Figure 12 — Two independent safety nets: Render's own snapshots, and your own daily verified copy that you can prove restores.

9.1 Net one: Render's snapshots

Automatic, every 24 hours, kept for at least 7 days, no configuration beyond having the persistent disk. Restore is all-or-nothing from the service's Disks page: the disk returns to that moment and everything after is lost. Use it for "everything went wrong; go back to yesterday".

9.2 Net two: your own verified daily copy

Piece	Where	What it does
POST /internal/backup	app.py line ~12427	Needs the secret in the X-Backup-Token header (BACKUP_TOKEN). Uses SQLite's online backup (safe while people work), integrity-checks the copy, gzips it and streams it. Wrong tries are counted per address; 5 wrong in 15 minutes locks that address out.
pull_backup.py	your Mac, run daily	Calls the endpoint, un-gzips, runs PRAGMA integrity_check, refuses a copy where trips or users are empty or any key table lost more than 20% of its rows since the last copy, saves render_YYYYmmdd_HHMMSS.db and keeps the newest 30.
restore_check.py	your Mac, monthly	Copies a backup to a scratch folder, health-checks it, starts the real app on it and opens login, dashboard, trips, ledger, invoice center, expenses and settings: prints PASS or FAIL. Never touches the live file.
backup_db.py	your Mac	Same safe backup for the local fleet.db (keeps 30).
db_health_check.py	anywhere	Read-only audit: integrity, schema drift versus schema.sql, tables missing company_id, unindexed lookup columns, business-data anomalies (over-received trips, duplicate LRs, desynced payments).

The gap: uploaded files

Insurance papers, toll receipts and the logo are saved under static/uploads/ in the app folder. The database backup does not include them, and unless that folder is on the persistent disk a redeploy deletes them. The planned fix is to make the upload folder follow the disk, accept only PDF/JPG/PNG/WEBP up to about 8 MB, shrink photos on upload, and show disk usage to the Administrator.

10. Outside services and what happens when they fail

The app calls five outside services. None is needed to open a screen or record a trip; each only powers one feature, and each has a defined failure behaviour.

Service	Used for	How it is called	Cost	If it fails
MSG91	SMS login codes	POST https://control.msg91.com/api/v5/otp with the key in an `authkey` header; params template_id, mobile (91 + 10 digits), otp, otp_length, otp_expiry. Success means JSON type = success even on HTTP 200.	a few paise per SMS (about Rs 0.15–0.25); needs DLT registration	User sees "Could not send the code"; the code is voided; password login still works.
eChallan API	Vehicle RC details, traffic challans	GET https://api.echallan.app/vahanfin/vehicle and /echallan, key in an `X-API-Key` header (`providers/echallan_client.py`)	paid per call; background jobs are off unless enabled	Existing cached data is kept; failures are logged in sync_log; nothing is overwritten.
OSRM (router.project-osrm.org)	Driving routes, distance, time	GET route with up to 3 alternatives; timeout 15 s	free, shared	Route Finder shows a plain message; rest of the app unaffected.
Nominatim (OpenStreetMap)	Turn typed text into a map point	GET search; timeout 8 s; limited to 1 request per second by the app	free; autocomplete is forbidden by policy, so Photon is used for typing	Asks the user to add a town/state or pick from suggestions.
Photon (photon.komoot.io)	Suggestions while typing a place	GET, India bounding box, country code IN, 6 s timeout, cached 1 hour	free	Suggestion list stays empty; the user can still type and Search.
Overpass (overpass-api.de, mirror overpass.kumi.systems)	Toll plazas, towns, petrol pumps, airports, junctions along a route	POST one query per route in ~50 km boxes; 22 s per call, 30 s total budget; tries main → mirror → main	free, shared and often busy (7–30 s typical)	Route shown without stops, "Try again" offered; failures are not cached.
Google Fonts	Poppins typeface on the phone app's sign-in, SMS-code and sign-up screens only	CSS link in those four templates	free	Browser falls back to a system font.

Protecting the free services: the Route Finder allows at most 3 searches at once (a bounded semaphore) and caches results for an hour per worker; Nominatim is throttled to 1 request per second; every call has a hard timeout and a shared 30-second budget so one lookup can never hang a thread for long.

C

The back-end

The Python program: how a request travels, who may do what, and how every business rule is calculated.

IN THIS PART

- Anatomy of app.py
- Life of a request
- Database access
- Sign-in, sessions and access control
- Every business engine, step by step
- Analytics, mobile twins, background jobs, security

11. Anatomy of app.py

All back-end code is one Python file of 14,834 lines holding 434 top-level functions: 199 of them are **routes** (a URL that the outside world can ask for) and 235 are **helpers** (calculations, validators, hooks) that routes call. Keeping everything in one file was a choice for speed of change; Chapter 55.1 (Part I) lists splitting it as the main maintainability task.

11.1 What lives where (line ranges)

Lines	Section	What is in it
1–100	Imports, settings, constants	Flask app creation, cookie and upload settings, ProxyFix on Render, the database path, regular expressions for mobile numbers and emails, app name RATH.
103–300	Database connection and self-applying schema	get_db(), then the first three start-up steps (overhead allocation column, performance indexes, WAL).
300–500	Sign-in security core	Auth tables, secret key, single-Administrator index, Twilio cleanup, throttle counters, SMS-code issue and check.
497–590	Input guards and template helpers	Friendly 400 handler, NaN/infinity guard, per-request connection cleanup, asset versioning, site logo.
589–960	Shared helpers	Period and growth maths, pagination, place-name cleaning, vehicle body/wheel helpers, get_or_create for parties, vendors and vehicles.
968–1280	Ledger balance engine and Ledger list	Per-account entries, the fast bulk balance computation, accounts list, create/toggle account.
1281–1670	Website Dashboard	Revenue, expense, profit core shared with the phone Dashboard; charts and tables.
1673–2215	Vehicles	RC lookup/PDF, vehicle detail and list, compliance renew and sync.
2215–2880	Employees and Expenses lists	Employee overview, salary and attendance tabs; expense list data and export.
2879–3370	Trips	List, duplicate-LR check, freight formula, validation, insert, update, add; autocomplete data.
3371–5930	Maintenance family	Hub and Overview, Service, Tyres, Battery, Urea, Toll (incl. Excel import and FASTag sync), Compliance fees, Insurance.
5929–6540	Ledger pages and exports	Party/vendor ledger entries and pages, Excel and PDF export, money parser.
6502–6950	Payment engine and expense writes	Party and vendor payment allocation, duplicate guard, expense insert/update.
6948–7710	Trip delete/end, employees, attendance, payslip, vehicle add/edit, exports, advances	Operational write routes and exports.
7707–8250	Employee ledger, performance, employee add	True running balance for advances; driver and vehicle performance.
8248–9700	Invoices	Invoice Center, review, PDF builder, generate, batches, per-trip invoices.
9701–9830	Opening balances, company name, PWA manifest	Small shared pieces.
9825–10250	Alerts, Limited Access, security hooks	Challan bell, screen map and enforcement, cross-site check, headers, require_login, access context, read-only layer, no-cache.
10249–10500	Login, logout, SMS codes, own profile	Both login flows and the "My profile" page.
10496–10910	Users and Settings	One-Administrator helpers, settings save and validation, add/edit/delete/hand-over user, user export.
10911–11520	Route Analytics and Route Finder	Route rollups, the Route Finder endpoints and cache.
11514–11908	Fleet utilization, edit trip, trip detail	

Lines	Section	What is in it
11908–12430	Business Performance and full Excel export	Monthly/quarterly/yearly comparison.
12427–12533	Internal endpoints	Backup download and sync tick (token protected).
12533–14834	The phone app (/app/)	About 100 routes: login, dashboard, trips, maintenance, expenses, salary, vehicles, analytics, ledger, profile and the Administrator settings screens.

11.2 Naming habits that make the file navigable

- `_insert_X_from_form(conn, f)` / `_update_X_from_form`: the *one* place a record of kind X is written. Both the website route and the phone route call it, so they can never disagree (for trips, payments, expenses, tyres, batteries, urea, tolls, vehicles, employees…).
- `_X_data(conn, ...)`: the one place the numbers of a screen are computed (`_performance_data`, `_fleet_utilization_data`, `_dashboard_financials_data`, `_maintenance_overview_data`); the website page and its phone twin both render from it.
- `_X_tab(conn)`: the tab inside a hub page (Maintenance and Employees are hubs with several tabs, switched by `?tab=`).
- `app_*`: the phone-app twin of a website route; same helper, different template and, for actions, JSON answers.
- A leading underscore means "helper, not a URL". Hooks (`before_request`, `context_processor`, `after_request`) are named for what they do.

Why the twins matter

Early on the website and phone code were separate copies and drifted (a payment recorded on the phone updated a different set of columns than on the website). Extracting the shared helpers was the cure, and the test harness posts to *both* sets of routes and compares every screen so a new drift is caught the day it appears.

12. The life of a request

Every click becomes an HTTP request and passes the same checkpoints in the same order. This is the most useful picture in the whole document for debugging: if a page does not behave, find the first checkpoint that disagrees.

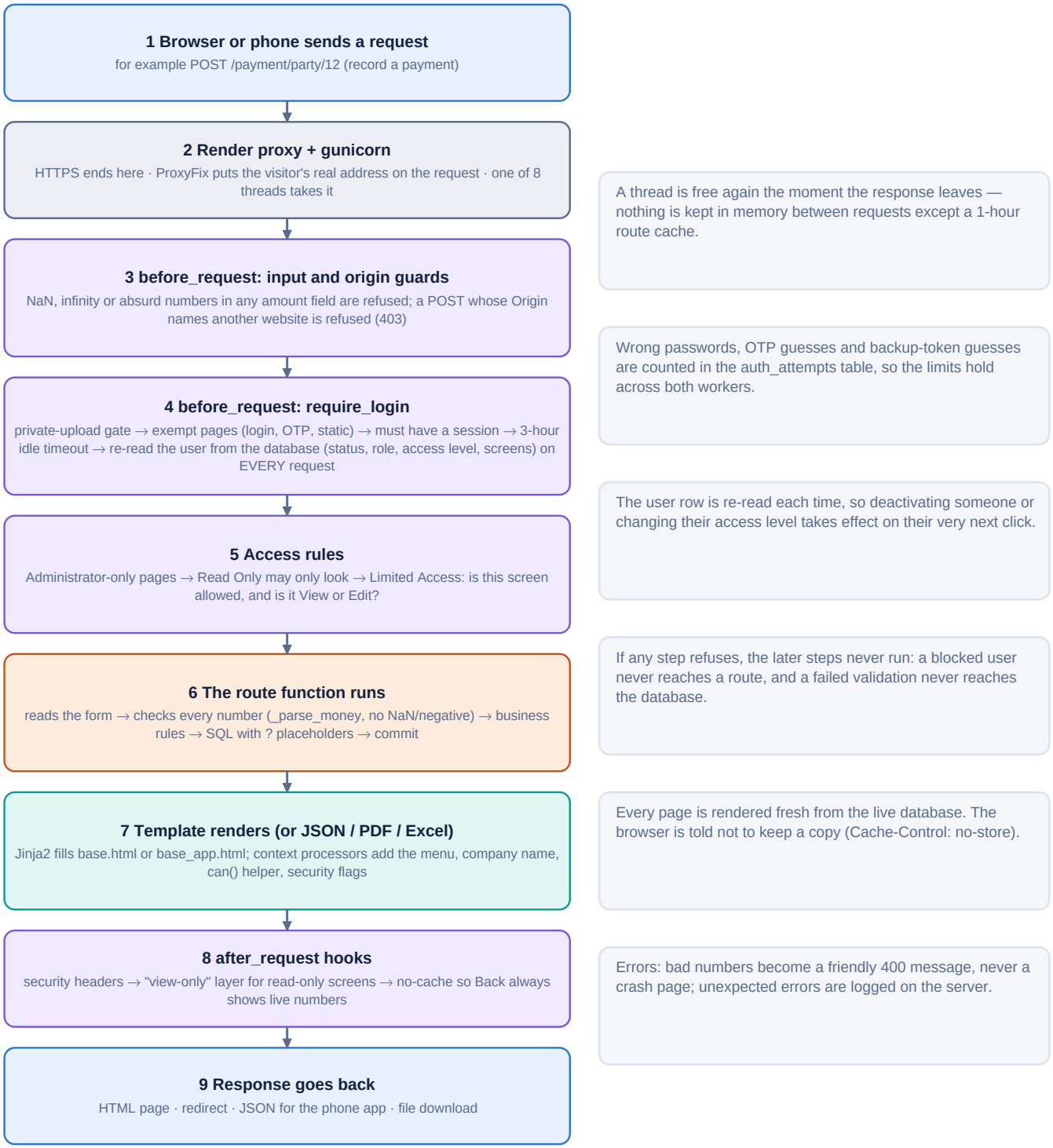


Figure 3 — Life of one request, in the exact order the code runs it.

12.1 The hooks, in running order

Order	Hook	Kind	What it does
1	`_reject_non_finite_numbers` (518)	before_request	For POSTs, looks only at fields whose NAME sounds numeric (amount, rate, qty, toll, …) and refuses "nan", "inf", "1e999" or anything above 1e12, so one typo cannot poison a total. A vendor called "Inf" is never affected because only numeric-looking names are checked.
2	`reject_cross_site_posts` (10048)	before_request	For POST/PUT/PATCH/DELETE: if the Origin (or Referer) names a different site than the one being used, answer 403. Browsers always send Origin on form posts; scripts and tests send none and are unaffected. Escape hatch: DISABLE_ORIGIN_CHECK=1.
3	`require_login` (10071)	before_request	The gatekeeper. (a) Blocks anonymous access to private upload folders. (b) Lets the exempt pages through (login, OTP, signup, static, manifest, internal tokens). (c) Requires a session. (d) 3-hour idle timeout. (e) Re-reads the user's access_level, status, role and screens from the database. (f) Inactive or deleted → signed out. (g) Computes is_administrator. (h) Administrator-only endpoints → 403 for others. (i) Read Only may not change anything except a short allow-list. (j) Limited Access: screen map check.
4	the route function	route	Validates, applies business rules, runs SQL, commits, answers.
5	`_bad_form_input` (497) and the global handlers	error handlers	ValueError, OverflowError and IntegrityError become a friendly 400 ("could not be saved — nothing was changed"), JSON for the phone app.
6	context processors	while rendering	`inject_access_level` (10171) (can(), read_only, is_administrator, limited_nav), `inject_security_flags` (10040), `inject_company_name` (9774), `inject_app_name` (9781), `inject_asset_version` (559) (asset_v), `inject_site_logo` (576), `inject_challan_alerts` (9825) (the bell), `inject_vehicle_type_labels` (550), `inject_current_year` (9820), and the header contexts for website and app.
7	`add_security_headers` (10062)	after_request	X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Referrer-Policy: strict-origin-when-cross-origin, and Strict-Transport-Security on HTTPS.
8	`_read_only_page_enhancements` (10217)	after_request	For Read Only users (and any screen a Limited user holds as View only) injects a small script: a "View-only access" badge, hides obvious Add/Edit/Delete buttons, and blocks any write that is attempted anyway.
9	`_no_bfcache_for_dynamic_pages` (10232)	after_request	Cache-Control: no-store so the browser's Back button re-fetches live numbers instead of showing an old snapshot.
10	`_close_leaked_db_conns` (542)	teardown	Closes every database connection the request opened, even if the route crashed, so a 500 can never leave the file locked for the next person.

13. The database access layer

Every query goes through `get_db` (103). It is deliberately tiny and is the only place connection behaviour is decided.

```
conn = sqlite3.connect(DB, timeout=30)        # wait up to 30 s for a lock instead of failing at 5 s
conn.row_factory = sqlite3.Row                 # read columns by name:  row['billed_amount']
conn.execute("PRAGMA synchronous=NORMAL")      # safe with WAL; far fewer disk syncs per save
wrapper = db.Connection(conn, backend='sqlite')# pure pass-through today; the only place a Postgres swap would happen
g['_open_db_conns'].append(wrapper)            # remembered so teardown can close it even after a crash
```

- **Placeholders, never string-building.** Queries are written with ? and a parameter list, e.g. `conn.execute("SELECT * FROM trips WHERE id=?", (trip_id,))`. That makes SQL injection impossible for values. The few f-strings in the code only splice in fixed table names chosen from a two-value test (`parties` or `vendors`) or a list of "?" marks for an IN (...) list.
- **Commit discipline.** A route opens a connection, does its reads and writes, then commits once and closes. If anything raises before the commit, nothing is saved (SQLite rolls back when the connection closes).
- **WAL mode** (set at start-up) lets readers and the single writer work at once; the 30-second wait turns "two people saved at the same second" into a short queue instead of an error.
- **db.py** (113 lines) exists so that, when PostgreSQL arrives, the ?→%s translation is added in one function. Known manual work for that day: the 34 places using `cursor.lastrowid` (Postgres needs RETURNING id), 2 uses of INSERT OR REPLACE, and about 36 SQLite date functions.
- **Connections inside loops.** Screens that need many accounts pass one open connection down (`conn=conn`) instead of opening one per account; the old Ledger list opened about 9,000 queries for 250 accounts, the current `_bulk_balances` uses a handful.

14. Sign-in and sessions

There are two ways in: a password, or an SMS code. Figure 4 shows both. Both end the same way — the server writes the user's id into a signed cookie and logs the event — and both are guarded by counters stored in the database so the limits hold across both gunicorn workers.



Figure 4 — The two ways to sign in. In both, the secret decisions happen on the server; the browser only holds a signed cookie.

14.1 The numbers that define the behaviour

Rule	Value	Where
Wrong passwords before lock-out	5 per account and 30 per visitor address within 15 minutes; answer HTTP 429 with the minutes left	_AUTH_LIMITS, `_login_throttle_message` (445)
SMS codes that may be requested	3 per mobile number per 15 minutes, 10 per address per hour	`_start_otp_login` (10345)

Rule	Value	Where
SMS code life and tries	6 digits from secrets; valid 5 minutes; 5 wrong tries kill it; one use only	`_otp_issue` (461), `_otp_check` (475)
How a code is protected	Only an HMAC-SHA256 of id+code (keyed with the secret key) is stored in login_otps; nothing readable is kept in the cookie	`_otp_hash` (455)
Unknown number	Same "code sent" screen, nothing sent, so the form cannot reveal who has an account	`_start_otp_login` (10345)
Password rules	minimum 8 characters when changing or setting; stored with werkzeug's salted hash	`_change_own_password` (10458)
Session cookie	signed with the secret key, HttpOnly, SameSite=Lax, Secure on Render; "remember me" keeps it 30 days, otherwise it dies with the browser	app config near line 20
Idle timeout	3 hours without a request signs you out (any request resets the clock)	`require_login` (10071)
Secret key	SECRET_KEY if set, else one random key generated once and kept in settings (_auto_secret_key)	`_ensure_secret_key` (344)
Self sign-up	Closed (403) unless ALLOW_SELF_SIGNUP=1; new accounts would be Staff / Read Only	`app_signup` (12573)

Why the SMS code is kept on the server

An earlier design stored a fingerprint of the code inside the browser cookie. A cookie is signed but readable, so anyone could recover the six digits in a fraction of a second and sign in as the Administrator without receiving the SMS. The code (as a keyed hash) now lives only in the database and the cookie holds just an opaque row number. A regression test repeats that attack and expects it to fail.

15. Who may do what: roles and access

Two separate ideas are combined. A user has a **role** (a label such as Administrator, Manager, Accountant) and an **access level** (Full Access, Read Only or Limited Access). Only the role value "Administrator" opens company-level powers; the access level decides how much of the day-to-day screens a person may use.

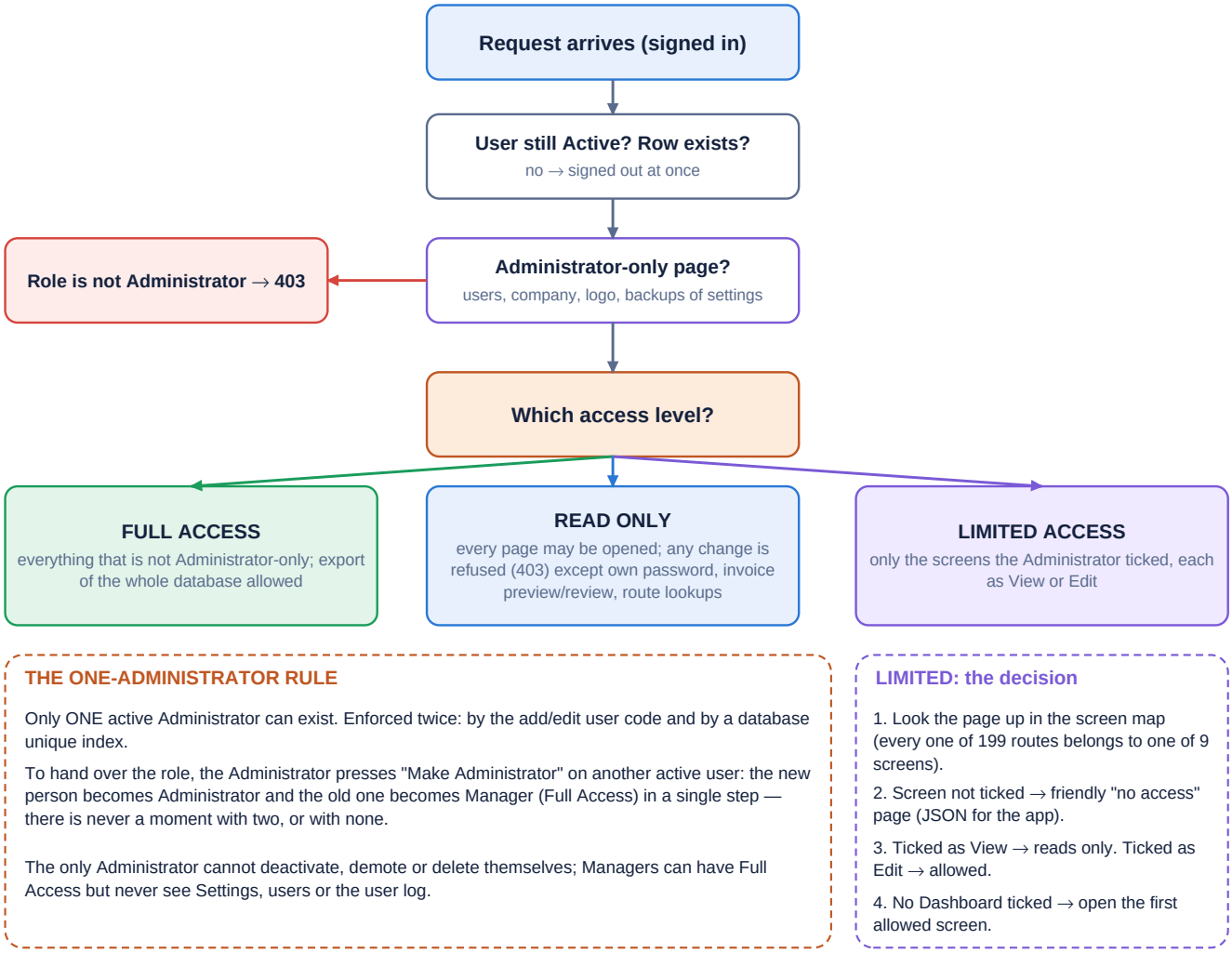


Figure 5 — Who may do what. These checks run on the server for every request, so hiding a button is never the only protection.

15.1 The four kinds of user

Kind	Can see	Can change	Cannot
Administrator (exactly one active)	everything, plus Settings, users, bank/invoice/tax, integrations, access logs	everything, including users and company settings	be removed, demoted or deactivated without handing the seat to another active user
Full Access (Manager etc.)	all operating screens	all operating records	see Settings, other users or the user log; add users
Read Only	all operating screens	nothing (own password only; may preview/review invoices and use route lookups)	save, edit, delete anything; the server refuses even if a button is forced
Limited Access	only the screens ticked for them	only on screens ticked as Edit; View screens are read-only	open any unticked screen, export the whole database, reach Settings or users

15.2 Limited Access in detail

The Administrator ticks which of nine **screens** a user gets, each as View or Edit. The choice is stored in `users.module_access` as text such as `Dashboard:v,Ledger:e,Invoices:e` (e = edit, v = view; old values without a letter count as edit, and the former "Payments" screen is part of Ledger). Every one of the 199 routes is assigned to exactly one screen, or marked common (own profile, logout, static files), Administrator-only, or Full-Access-only (the whole-database export). A test fails if a new route is added without being classified, so a forgotten route can never silently become open or silently become unreachable.

Screen	URLs it protects	Opens on	Examples of what it covers
Dashboard	3	`/dashboard`	website and phone dashboards
Trips	13	`/trips`	trip list, add/edit/end/delete, trip exports
Vehicles	18	`/vehicles`	vehicle list/detail, RC lookup, compliance, challans
Maintenance	50	`/maintenance`	service, tyres, battery, urea, tolls (and Excel import), compliance fees, insurance
Ledger	22	`/accounts`	party and vendor ledgers, payments, opening balances, exports
Invoices	16	`/invoice-center`	Invoice Center, review, generated invoices, PDFs
Expenses	8	`/overheads`	expense list, add/edit/delete, export
Salaries	19	`/salaries`	employees, salary, advances, attendance, payslips, employee ledger
Reports	19	`/route-analytics`	Business Performance, Performance, Route Analytics and Finder, Fleet Utilization

- **Enforcement** is `_limited_block` (9975), called from `require_login` (10071) on every request: it looks the route up in the screen map (`_ENDPOINT_MODULE`) and answers with a friendly "no access" page (JSON for the app), a redirect to the first allowed screen when the Dashboard is not granted, or lets the request through (marking a View screen so the read-only layer is added).
- **Navigation** is built from the same table: the website sidebar and Dashboard cards use `can('Ledger')`; the phone bottom bar, + quick-add fan and More menu are generated by `_limited_nav` (10014) (first three allowed screens in the bar, the rest under More, + only if they can add something). Users who are not Limited keep the original fixed layout.
- **The editor** the Administrator uses is one shared script, `static/limited_access_picker.js`, with presets (Accountant, Operations, Fleet and service), a switch and a View/Edit toggle per screen, and a live phone preview; it writes hidden fields that `_modules_from_form` (9932) validates (at least one screen required).

15.3 The one-Administrator rule

- `_other_active_administrator` (10509) and the checks in `add_user` (10740) / `edit_user` (10791) refuse to create or activate a second Administrator.
- A database unique index (`uq_one_active_administrator`) enforces the same thing even against a bug or two simultaneous requests.
- `make_administrator` (10840) hands over the seat: it demotes the caller to Manager (Full Access) first and promotes the target second inside one transaction, so there is never a moment with two or with none.
- The only active Administrator cannot deactivate, demote or delete themselves (so nobody is ever locked out of Settings).

16. Validation and error handling

A bad number must never be stored, and a bad request must never show a crash page. Four layers cooperate.

Layer	What it catches	Result
Browser (HTML attributes, small scripts)	Required fields, number pads, 10-digit mobile patterns, date pickers	Convenience only; never trusted
`_reject_non_finite_numbers` (518)	nan / inf / 1e999 / over 1e12 in amount-like fields of any form	400 with a friendly message
`_parse_money` (6502)	text, commas and a leading rupee sign tolerated; blanks; negatives (unless allowed); zero (unless allowed); absurdly large; rounds to paise	(value, error) so each route shows a precise message
`_trip_form_error` (3041)	every one of 40 money fields; computed bill cannot be negative; on a NEW trip the party advance cannot exceed the bill (edits may legitimately end up with more received than billed)	form re-shown with the message
`_validated_payment_basics` (6654) + `_payment_is_duplicate` (6645)	amount, date, mode; the exact same payment twice within 10 seconds	"not adding it twice"
`_settings_form_error` (10579)	company name required, GST/TDS 0–100, round-off, due days, next invoice number ≥ 1, sync days, MSG91 key and template present	readable JSON error in the app
`_bad_form_input` (497) (also for IntegrityError)	anything above that still raises ValueError, OverflowError or a constraint violation	friendly 400; nothing saved; original error logged
Database constraints	CHECK that a payment allocation points at exactly one of trip / maintenance / expense; unique usernames, phone and email rules; the single-Administrator index	IntegrityError → friendly 400

The fuzz stage

The test harness has a stage that feeds garbage (huge numbers, NaN, SQL fragments, empty values, wrong types) to every POST route in the app and expects no 500 error and no data change. It is the safety net behind the validation layers above.

17. The business engines, one by one

This chapter explains how each business rule is actually calculated and where it lives. Read it with Figure 6 (money flow) in mind: trips create what is owed, payments settle it, and the ledger engine sums the difference.

17.1 Trips

A trip is one journey carrying one customer's goods. It is the largest table (80 columns) because it holds the whole commercial story of the journey.

Column group	Columns (examples)	Meaning
Identity	id, date, lr_number, vehicle_id, type, party_id, from_loc, to_loc, material, driver_name	which truck, for which party, what route; type is own or hired and follows the vehicle
Rate	quantity, guarantee_qty, rate, rate_type, fixed_rate_amount	PER_MT (tonnes × rate) or FIXED amount; a guarantee quantity bills the minimum weight when one applies
Bill	billed_amount	freight + extra charges – deductions, calculated once on save
Extra charges (added)	detention_charges, gps_cost, loading_charge, unloading_charge, police_charges, sim_tracking, union_charges, weight_charges, other_charges	extras the party is billed
Deductions (subtracted)	brokerage, builty_commission, late_fees, material_damage, shortage_amount, shortage_qty, tds, other_deductions	what reduces the bill
Fuel and driver advance	fuel_amount, fuel_liters, fuel_price, fuel_vendor_id, driver_adv_amount, driver_adv_vendor_id	company costs; vendors are created on the fly
Party money	party_advance, payment_received	money already received against this bill
Hired-vehicle owner terms	owner_name, owner_vendor_id, owner_rate, owner_rate_type, owner_fixed_amount, owner_amount, paid_to_owner, pending_to_owner	what the vehicle owner is paid
Own-vehicle costs	toll, urea, parking, puncture, fine, labour_charges, agent_commission, builty_expense, loading/unloading_expense, weighbridge_charges, permit_charges, other_expense, misc_vendor_id	direct costs of running the trip
Status and timing	lr_received, end_date, end_time, actual_km, shortage_date/unit/remarks, is_empty, remarks	whether it is finished, kilometres, empty runs
Audit	company_id, created_by, created_at, updated_by, updated_at	who and when; company for future multi-tenancy

How a trip is saved

- `add_trip` (3227) (website) and `app_add_trip` (12954) (phone) validate with `_trip_form_error` (3041), refuse a duplicate LR number (`_duplicate_lr_trip` (2990)), then call `_insert_trip_from_form` (3070).
- That function resolves names to ids (`get_or_create_vehicle` (919), `get_or_create_party` (876), `get_or_create_vendor` (897)) so typing a new fuel pump or owner just creates the vendor; the trip type follows the vehicle (`_resolve_trip_type` (955)).
- The freight comes from `_trip_freight` (3010): FIXED → the fixed amount; PER_MT → (guarantee quantity if filled, else actual quantity) × rate. The same function is used by insert, update, invoice preview, invoice PDF and batch lines, so a trip is billed identically everywhere. The guarantee never applies to what the owner is paid.
- `_update_trip_from_form` (3142) recomputes the same way on edit; `end_trip` (7030) / `_end_trip_from_form` (6983) close a running trip (the End Trip box pre-fills the gross freight so shortage is deducted once, not twice); `delete_trip` (6948) removes the trip and its allocations.
- **Own versus hired:** type "own" costs fuel, advance and the direct-cost columns; type "hired" costs the owner payment. Profit = billed – cost under that rule, everywhere.

17.2 Parties, vendors and the linked-account merge

A **party** is a customer; a **vendor** is anyone the company pays. The same organisation can be both (a transporter who hires you trucks and also ships with you); then the vendor row carries `linked_party_id` and the two are shown as **one** merged ledger: what they owe you minus what you owe them. `get_or_create_party` (876) and `get_or_create_vendor` (897) create records from typed names so data entry never stops to ask. Opening balances (money owed before the app existed) are a signed number with a date on each party, vendor and employee.

17.3 The ledger engine

A ledger is a list of dated lines with a debit, a credit and a running balance. `_get_party_ledger_entries` (6093) builds a party's lines and `_get_vendor_ledger_entries` (6176) a vendor's; the ledger page, exports and popups all show them.

Line kind	Debit (they owe us more)	Credit (they owe us less)	Source
Opening Balance	if positive	if negative	parties.opening_balance
Trip Bill	billed_amount	the part of payment_received NOT coming from a ledger payment + party_advance	trips
Payment In	—	one line per allocation ("Applied to LR …"); any unallocated leftover as its own line	payments + payment_allocations
Linked vendor lines	vendor-side bills and owner trips (merged in)	vendor-side payments	_get_vendor_ledger_entries

- Balance = sum of debits – sum of credits; **positive = receivable** (they owe us), **negative = payable** (we owe them). The vendor ledger follows the same sign rule.
- A trip's own "payment received" is floored at zero after subtracting what ledger allocations already account for, so a payment is never subtracted twice.
- **Speed:** `_accounts_rows_by_entries` (968) (the original) builds every account's full ledger — about 9,000 queries for 250 accounts. `_bulk_balances` (1010) restates the same arithmetic over whole tables in a few queries and feeds `_accounts_rows` (1109). A test proves both give identical balances on the real data copy and on the 12,000-trip load database (2 ms vs 10 ms, and 72 ms vs 3,652 ms).

17.4 Payments and allocation

Recording a payment is more than saving a row: the money is also **applied** to the things it settles, so the Trips screen, the Maintenance screen, the Expenses screen and the Ledger can never drift apart.

	Money received from a party	Money paid to a vendor
Function	`_insert_party_payment_from_form` (6682)	`_insert_vendor_payment_from_form` (6745)
Order of application	1) the trips the user ticked, in ticked order (each capped at what is still pending); 2) the party's other pending trips, oldest first	1) ticked owner-hire trips (net of fuel/advance the company paid); 2) other pending owner-hire trips; 3) unpaid maintenance, tyre, battery, service and insurance bills; 4) Pending expense bills naming this vendor, oldest first
Surplus	A true surplus (more than everything owes, e.g. an advance or opening-balance settlement) stays unallocated and shows as its own ledger line	Same
Guards	`_validated_payment_basics` (6654); `_payment_is_duplicate` (6645) (same payment within 10 seconds is refused)	Same, plus the vendor must exist
What gets written	payments row + one payment_allocations row per trip; trips.payment_received increased	payments row + allocation rows pointing at a trip OR a maintenance bill OR an expense; paid_amount / status updated

	Money received from a party	Money paid to a vendor
Consistency helpers	`_party_pending_trips` (6543)	`_vendor_pending_trips` (6554), `_vendor_pending_maintenance` (6567), `_vendor_pending_overheads` (6577), `_settle_overhead_status` (6597), `_enforce_maintenance_paid_floor` (6637)

- **Owner dues rule (hired trips):** only fuel or advance the *company* paid (vendor blank or a third party, not the owner) is deducted from the owner's dues — one function, `_company_paid_on_hired_trip` (6613), used by the vendor ledger, the payment popup and the owner invoice.
- **Deleting** a maintenance bill or expense first removes its allocations (`_delete_maintenance_row` (6628), `_delete_overhead_row` (6604)); a bill's paid amount can never fall below what ledger payments already allocated.

17.5 Invoices



Figure 7 — From a pile of trips to a numbered PDF invoice.

Topic	How it works
Types	Party invoice, Hired-vehicle ("Freight Bill") invoice for owners, Tax invoice (with GST) and Bill invoice. The default type, payment terms, place of supply, due days and four display switches (logo, bank details, signatory, amount in words) are Settings.
Invoice Center (8248)	Filters by type, party/vendor, vehicle, LR, dates and invoiced status. Each trip row shows whether it is already on an invoice and which optional charges can be included.
Review gate	Before Generate or Draft is possible, every selected trip's LR link must have been opened and reviewed; the screen shows a single balloon at the LR column explaining what is missing and the server re-checks `reviewed_trip_ids`. Preview is always allowed.
Charges and deductions	Fields that never fed an invoice before (detention, police, SIM tracking, union, brokerage, builty commission, late fees, material damage, shortage, trip TDS, other deductions) are opt-in per trip; opted-out lines are recorded in invoice_batch_charge_exclusions so regenerating gives the same answer.
Numbering	invoice_prefix + year + zero-padded next_invoice_number from Settings (e.g. ATS/INV/2026/0004); numbers and dates are validated by `_invoice_numbers_error` (6525).
PDF	`_build_invoice_pdf` (8550) (ReportLab) draws letterhead from company settings, party details, trip table, charges, GST (CGST/SGST/IGST), TDS, round-off, amount in words (`number_to_words_inr` (8465)), bank details and RCM clause. With more than 14 extra lines it switches to a stacked full-width layout (200-trip invoices once crashed the compact one).
After saving	Generated invoices can be edited (`invoice_batch_edit` (9329)), have lines added or removed, be deleted, previewed inline or downloaded.

Known design difference

An invoice grand total can differ from the Ledger's billed amount when optional lines (Others, discounts, detention, RCM) are included, because the ledger bills the trip while the invoice may add or remove lines. This is listed as a business decision, not a bug (Chapter 52, Part H).

17.6 The maintenance family

Maintenance is one hub (`maintenance_list` (3409)) with tabs chosen by `?tab=`: Overview, Service, Tyres, Battery, Urea, Toll and Compliance. Every kind of purchase ends as a row in `maintenance` (the bill: amount, paid, vendor, vehicle, km) so the money side is in one place and vendor payments can settle it.

Tab	Data it keeps	Rules worth knowing
Overview	`_maintenance_overview_data` (3433)	Cards and charts for own vehicles only (hired-vehicle bills are excluded by design); category buckets come from `_maintenance_classify` (3371) so every tab agrees.
Service	maintenance + maintenance_items (checklist)	Created by `_insert_service_from_form` (4820); next-due km/date tracked.
Tyres	maintenance (tyre rows) + tyre_stock	Buy into stock, fit on a wheel position, scrap; wheel layout by vehicle wheeler count.
Battery	batteries + battery_checks	Health bands from the latest logged health%: Good ≥70, Weak ≥40, below that Replace Soon, Dead; Mark as Dead is an asserted fact that wins. Warranty end computed from install date + months.
Urea (AdBlue)	urea_transactions	A stock ledger: purchases add to stock (and create the bill), usage subtracts; using stock already paid for is not a second cost. Running balance is computed live, not stored.
Toll	toll_entries (+ receipt files)	Manual entry, Excel import (auto-detects a BlackBuck BOSS statement: only the debit "FasTag Recharge" lines count as toll; the temporary credit and its recovery debit and top-ups are ignored; or a plain template), approve/reject. `sync_fastag_toll` (5465) is currently a **mock** generator (no live FASTag feed exists yet; it seeds at most once per hour).
Compliance	compliance_expenses + vehicle_compliance + insurance_policies	Fees for fitness/permit/PUC become expense bills; insurance policies (with documents) are Active / Expiring (within 30 days) / Expired from dates, Cancelled by assertion.

17.7 Expenses (overheads)

`_overhead_list_data` (2590) powers the list for both website and phone: category, vendor, status and date filters plus search. An expense can be Paid or Pending; a Pending bill that *names a vendor* appears in that vendor's ledger and is settled by vendor payments, oldest first (`_vendor_pending_overheads` (6577)), flipping to Paid when allocations cover it (`_settle_overhead_status` (6597)). Recurring expenses carry a frequency and due date; the app shows overdue items.

17.8 Employees, salary, advances and attendance

- Employees (drivers and office staff) are listed in a hub with Overview, Salary and Attendance tabs (`_employees_overview_tab` (2292), `_employees_salary_tab` (2440), `_employees_attendance_tab` (2465)).
- **Advances and salary are separate ledgers.** `_employee_ledger_entries` (7707) builds Opening balance, Salary paid, Advance given and Advance repaid lines for one employee (matched by name) with a true running *advance* balance over the whole history; a date filter only limits which rows are shown, never the balance. Salary payments do not change the advance balance.
- Attendance is one row per employee per day (`mark_attendance` (7157), `bulk_mark_attendance` (7176)); `download_payslip` (7195) makes a PDF from a salary row (gross, deductions, advance recovery, net).

17.9 Vehicles, documents and compliance

- `vehicles_list` (2049) shows each vehicle's Insurance / Fitness / PUC / Permit badges using `_expiry_bucket` (4593) (expired, expiring within 30 days, ok) and the pending challan count.
- Values come from three places, in this order of trust: what an office user typed (`vehicle_compliance_renew` (7650)), what the eChallan RC lookup returned (cached in vehicles.rc_synced_data), and mock provider data (labelled as mock) when no key exists.
- `vehicle_rc_lookup` (1683) shows the cached RC details; `vehicle_sync_now` (1961) fetches fresh RC and challan data for one vehicle (a paid call, only on demand); `_build_rc_pdf` (1719) prints the RC.

17.10 Company settings, banking, tax

All company-level settings live in the `settings` key/value table (one row per key per company): company details, one bank account, invoice preferences and numbering, GST rates, TDS, e-way bill and round-off switches, the RCM clause, the MSG91 and eChallan keys and the sync intervals. `_save_settings_group` (10552) is the single writer used by **both** the website Settings page and the phone Administrator screens; `_get_all_settings` (10666) and `get_company_name` (9752) read them, always with a company id. Secret values are never printed into a page and a blank box means "keep the saved one".

17.11 Exports

Excel (openpyxl) exports exist for trips, maintenance, expenses, ledger list, party ledger, vendor ledger, users, performance, route analytics, fleet utilization and business performance, plus a full-database Excel (`/export`, Full Access only). PDF (ReportLab) exports exist for trips, maintenance, party and vendor ledgers, invoices, payslips and RC details. Every export reads the same data functions as the screen, so a file always matches what was on screen.

18. Reports and analytics

Every analytical screen is a different grouping of the same trips, bills and expenses. The one shared rule is how a trip's profit is calculated; it is defined once and reused so Dashboard, Business Performance, Performance, Route Analytics and the phone screens agree.

Screen	Function	What it answers
Dashboard (website / phone)	`dashboard` (1371) / `app_dashboard` (12740) using `_dashboard_financials_data` (1281)	Revenue, profit, own-vehicle revenue, hired-vehicle profit, fuel, maintenance, receivables and payables, expense mix, trend and top vehicles for a chosen financial year / period and vehicle.
Business Performance	`business_performance` (11908) with `_period_financials` (641)	This period versus the previous period, quarter and year (growth percentages) — the owner's view.
Performance	`performance` (8057) with `_performance_data` (7883)	Drivers and vehicles ranked by profit on the full real cost basis (own: fuel, advances, tolls and direct costs; hired: owner cost).
Route Analytics	`route_analytics` (11291) with `_route_analytics_data` (10911), `_route_rates_data` (10996)	Best routes by margin or revenue; lowest / average / highest rate per route and vehicle type; plus the Route Finder tab.
Fleet Utilization	`fleet_utilization` (11700) with `_fleet_utilization_data` (11523)	Active versus idle days per own vehicle (a trip is "active" from start to end date), empty runs, kilometres, fuel and efficiency. Hired vehicles are not this company's asset and are excluded.
Trips, Ledger, Expenses lists	the list routes	Operational lists with the same filters on website and phone.

18.1 The profit rule, stated once

- **Own vehicle:** profit = billed – (fuel + driver advance + direct trip costs + toll). Toll uses the linked Toll-Management amount if one exists, otherwise the trip's own estimate — never both.
- **Hired vehicle:** profit = billed – owner cost. Fuel and advance on a hired trip sit inside the owner freight and are not subtracted again.
- **Periods:** the financial year runs April to March; "this month / last month / FY" presets are resolved on the server (`_resolve_dashboard_period` (12712), `_current_fy_bounds` (12654)); a From date after the To date is swapped rather than producing negative days.

18.2 The Route Finder

The Route Finder (Figure 13) is a live look-up, not stored data. It exists on the website (Route Analytics → Route Finder) and in the phone app. Its endpoints are `route_finder_suggest` (11174), `route_finder_search` (11202), `route_finder_extras` (11254); the work is in `providers/osm_routes.py` (726 lines).



Nothing is stored: only a per-worker memory cache (1 hour) and a limit of 3 searches running at once protect the free services.

Used on the website (Route Analytics → Route Finder tab) and in the app (/app/analytics?tab=finder).

Figure 13 — The live Route Finder: five free OpenStreetMap-based services, nothing saved.

19. The phone app (/app/) on the server side

The phone app is not a separate program. It is a second set of routes under `/app/` (about 100 of the 199) that use the same helpers as the website but answer with mobile templates, and for actions with small JSON replies the page's JavaScript understands.

- **Same rules, different face.** A trip entered on the phone calls the same `_insert_trip_from_form`; a payment calls the same allocation function; the same access checks run in the same hook.
- **Own login screens.** /app/login, /app/login/otp and /app/signup render their own templates; an expired session on a /app/ URL redirects to the app login, never to the desktop page.
- **JSON for actions.** Calls made with fetch() (add vehicle, edit profile, change password, Administrator settings saves, user add/edit/delete via the `X-App-Request` header) get {ok, error} answers; the friendly-error handlers return JSON for /app/ requests that do not ask for HTML.
- **The Administrator hub.** `app_profile` (14613) shows the full "Profile and Settings" hub only to the Administrator (Company Profile, Banking, Invoice Settings, Tax and Compliance, Users and Roles, with Integrations inside Users); everyone else keeps the simple profile page. The five settings screens and `app_settings_save` (14693) are Administrator-only endpoints.
- **Phone navigation.** The bottom bar, + fan and More menu are normally fixed; for Limited users they are built from their screens by `_limited_nav` (10014).

Mobile screens that exist

Dashboard · Trips (list, detail, add, edit, end) · Maintenance hub (service, tyres, battery, urea, toll, compliance) · Expenses · Salary and attendance, employee ledger · Vehicles (list, detail, add) · Business Overview hub (analytics and Route Finder, fleet utilization, performance, business performance) · Ledger (list, party, vendor, payments) · Profile. There is no phone invoice screen: invoices are made on the website.

20. Background jobs

Only one thing in the app calls an outside service automatically: the vehicle-document refresh in `scheduler.py` (341 lines) and `compliance_service.py` (279 lines). Everything the Vehicles screens show is read from the database; these jobs keep the database current.

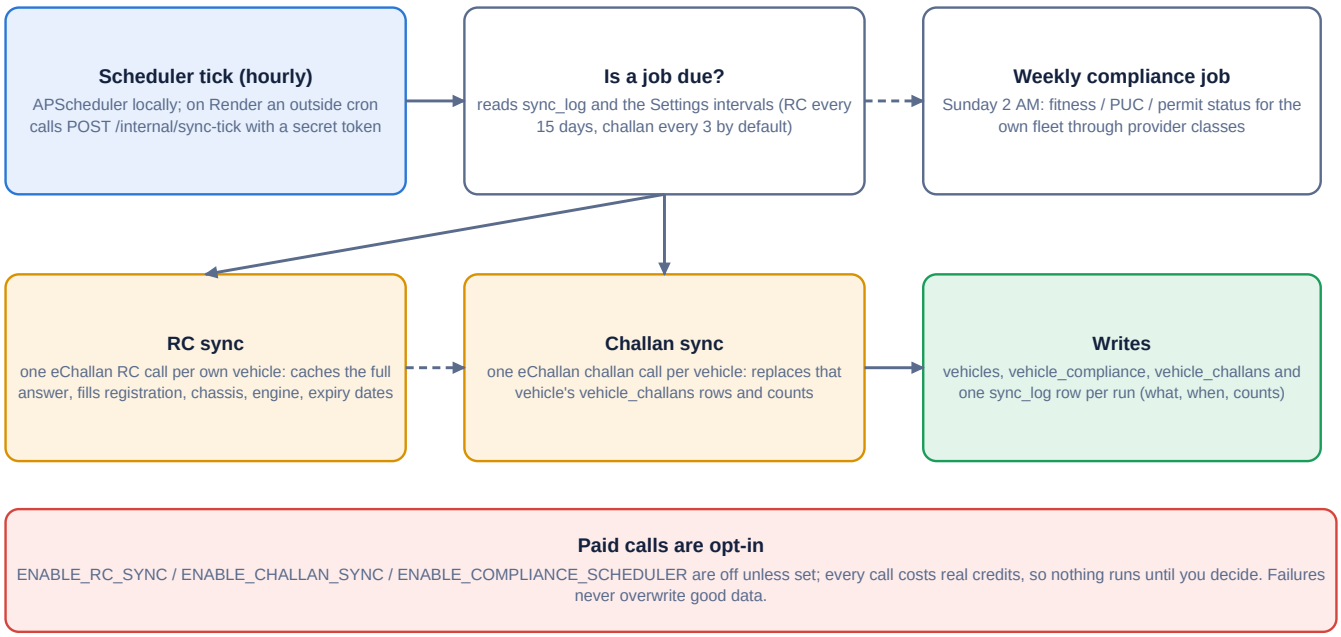


Figure 19 — Background data refresh for vehicle documents and challans.

- **Locally** the scheduler starts from `if __name__ == "__main__"` when ENABLE_COMPLIANCE_SCHEDULER=1. Under gunicorn that block never runs (and would start once per worker), so on Render the tick is triggered by an outside cron calling `POST /internal/sync-tick` with the X-Sync-Token header (INTERNAL_SYNC_TOKEN); the endpoint refuses everyone if the token is not configured.
- The intervals are Settings values (RC every 15 days, challan every 3 by default), so a change applies on the next tick with no restart.
- Providers (`providers/*_provider.py`) share one interface: fetch(vehicle_no) returns status, expiry, document number and "source: mock or live". The VAHAN, PUC and permit providers read slices of the single eChallan RC answer, so one paid call serves all three.
- Each real run writes a sync_log row (job, start, finish, synced, failed, skipped) which the Administrator can read under Users and Roles → Integrations.

21. Security architecture

Security is layered so that a single mistake is caught by the next layer (Figure 17). The table lists each realistic threat, the control, where it lives and the test that proves it.



Figure 17 — Seven layers of protection. An attacker has to get through all of them; one slipped layer is caught by the next.

Threat	Control	Where	Proof
Anyone can create an account	Self sign-up closed (403), link hidden	`app_signup` (12573)	security test: SIGNUP_CLOSED
Guessing passwords	5 failures per account / 30 per address → 15-minute lock; 8-character minimum	`_login_throttle_message` (445)	LOGIN_LOCKOUT, APP_LOGIN_LOCKOUT
Faking or recovering the SMS code	Server-side keyed hash, 5 tries, limits, unknown numbers look identical	`_otp_check` (475)	OTP_NOT_IN_COOKIE, OTP_LOCKED_AFTER_5_WRONG
Forging a login cookie	Private random secret key; the old fixed key is rejected	`_ensure_secret_key` (344)	OLD_SECRET_KEY_REJECTED
A malicious page posting forms as a logged-in user	SameSite=Lax cookie + Origin/Referer check	`reject_cross_site_posts` (10048)	CROSS_SITE_POST_BLOCKED
Reading private documents by link	Insurance and toll-receipt folders need a login	`require_login` (10071)	DOCS_NEED_LOGIN
A user exceeding their rights	Roles, Read Only, Limited screens, one Administrator — all on the server	`require_login` (10071)	roles stage (219 checks)

Threat	Control	Where	Proof
Bad or hostile input	Number guard, _parse_money, friendly 400, parameterised SQL, upload cap 16 MB, local-only return links	`_reject_non_finite_numbers` (518)	fuzz stage (1,200 checks)
Clickjacking / MIME tricks	X-Frame-Options, nosniff, Referrer-Policy, HSTS	`add_security_headers` (10062)	SECURITY_HEADERS
Stolen database copy	Passwords are salted hashes; secrets are not printed in pages; backups are verified	`_save_settings_group` (10552)	KEYS_NEVER_SHOWN_IN_PAGES
Backup endpoint abuse	Token required (refuses everyone when unset), attempt limit, constant-time compare	`internal_backup` (12427)	backup endpoint tests

Still open (see the readiness report)

Full CSRF tokens (today: SameSite + Origin check), a Content-Security-Policy, encrypting the stored SMS/RC keys at rest, confirming the persistent disk, off-site upload backups, error alerts, pinning package versions, closing the old Twilio account, and removing old database backups from the GitHub history.

D

The database

What is stored, how the tables relate, and the rules that keep the numbers honest.

IN THIS PART

- SQLite and the single file
- How the tables relate (three diagrams)
- Data dictionary: every table, every column
- Conventions: signs, statuses, indexes, audit columns
- Multi-company readiness and the PostgreSQL plan
- Migrations

22. The database at a glance

All business data lives in `fleet.db`, a single SQLite file of about 908 KB today, holding 36 tables and 62 indexes. SQLite is not a separate server: the Python process reads and writes the file directly, which removes a whole class of setup, network and password problems for a business of this size.

36	62	472	452	224
tables	indexes	trips	maintenance bills	payments

22.1 How the file is configured

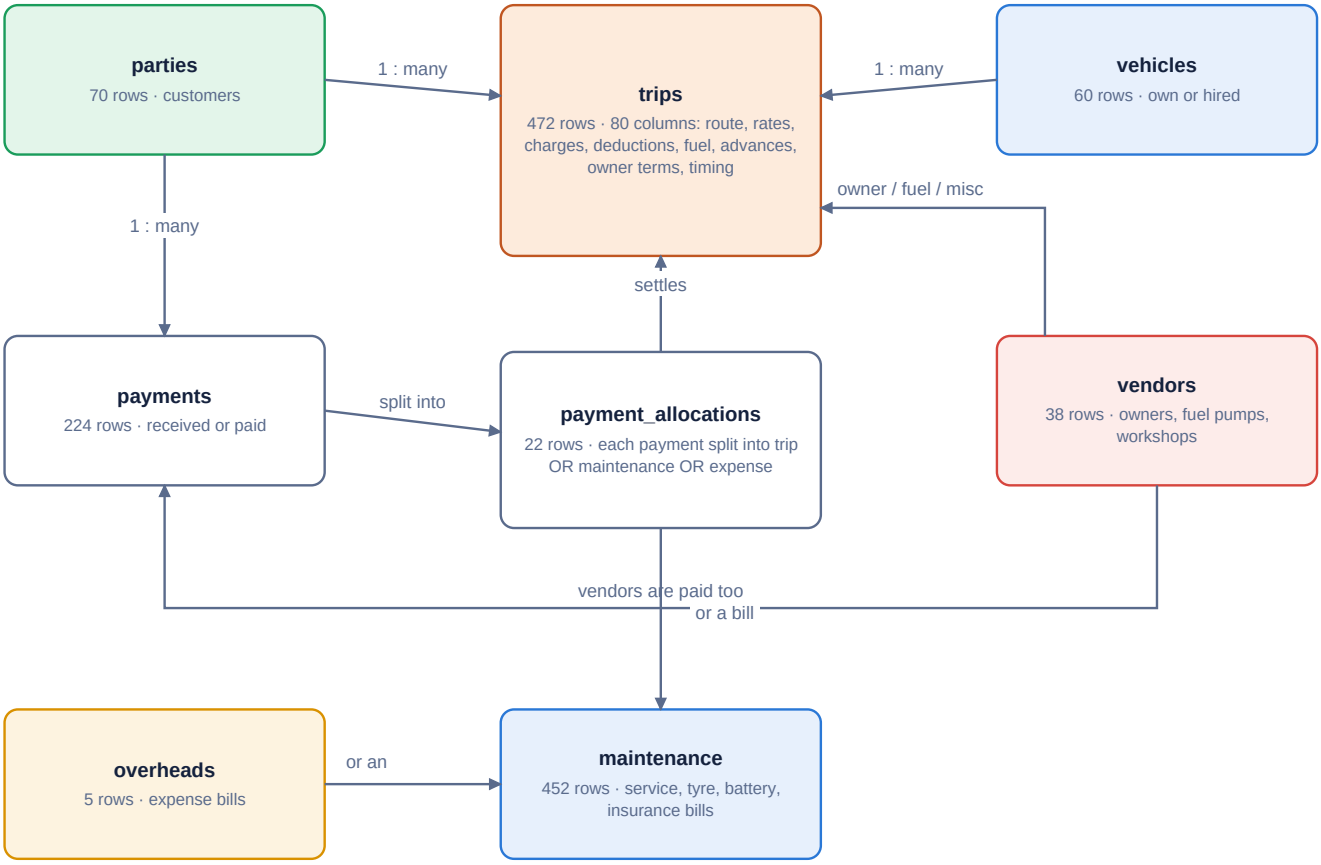
Setting	Value	Why
Journal mode	WAL (write-ahead log)	Readers never wait for the writer; a 0.5-second dashboard no longer blocks saves. Note: the live database is the .db file PLUS its -wal file, so backups use SQLite's backup command, never a plain copy.
Lock wait	30 seconds	Two people saving together queue briefly instead of seeing "database is locked".
synchronous	NORMAL	Safe with WAL; far fewer disk syncs per save.
Foreign keys	declared but not enforced at run time (SQLite default)	Orphans are watched by db_health_check.py instead; deletes remove dependent allocation rows explicitly.
Row access	sqlite3.Row	Columns read by name (row["amount"]).
Dates	text YYYY-MM-DD (times as text)	Sorts correctly as text; the health check flags any other format.
Money	REAL (floating point), rounded to paise on payments	Validated and rounded when written; tests compare screens to independent arithmetic to catch drift.
Audit columns	company_id, created_by, created_at, updated_by, updated_at	On nearly every table: who wrote it and when.

Why SQLite is a good fit now and what changes later

One writer at a time is no problem for five to thirty people entering trips and payments. The honest limit is sustained simultaneous editing by dozens of users or very heavy reporting; the answer is PostgreSQL, for which schema_postgres.sql already exists (Chapter 25).

23. How the tables relate

Three pictures cover the whole model. Arrows read "has many". Numbers in the boxes are the row counts in the database used to prepare this document.



Not drawn here: invoices (see Figure 7) and toll_entries (Figure 9) — both also point at trips.

Figure 8 — Data model, part 1: trading and money. The trips table is the heart; every payment is split into allocations that point at what it settled.

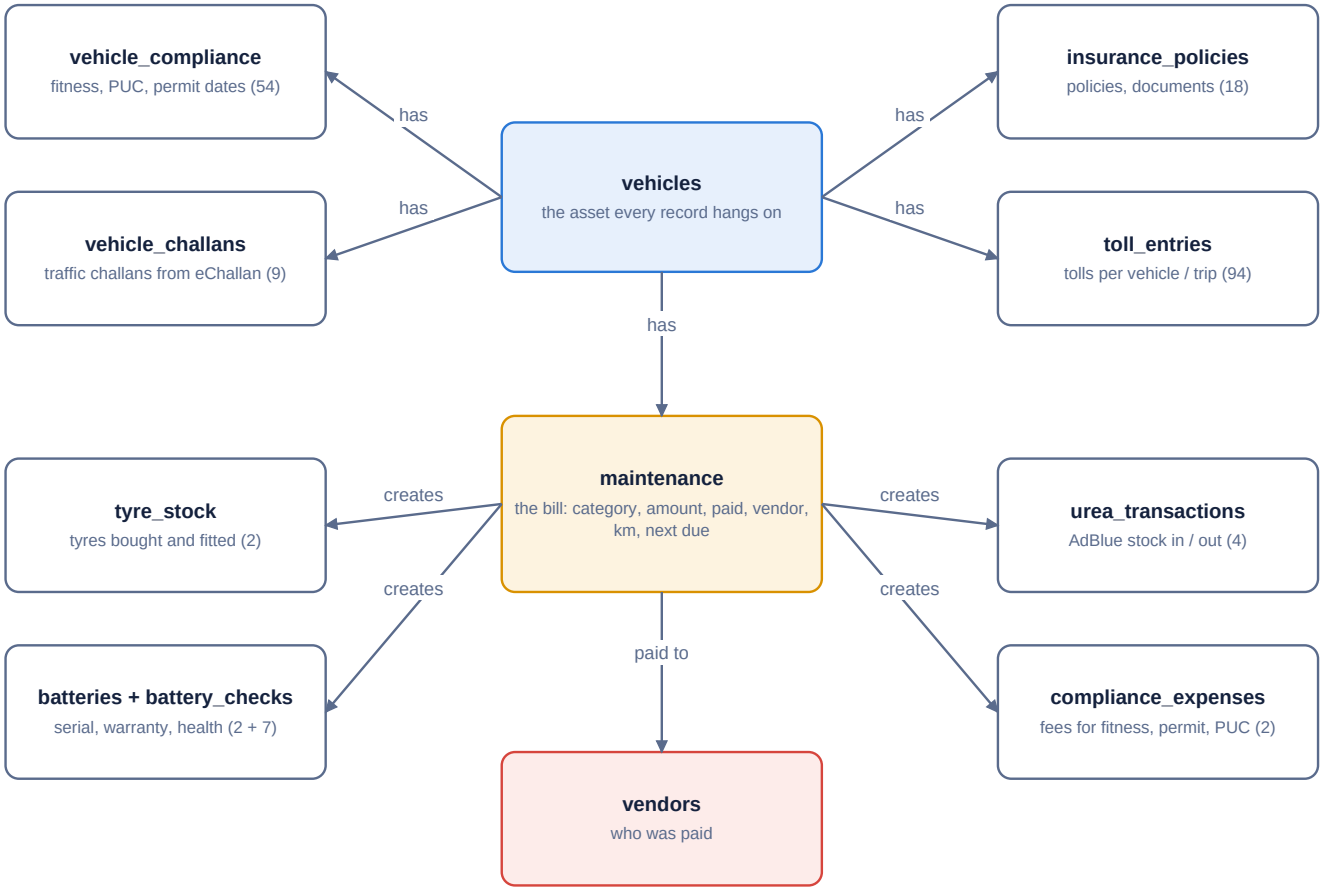


Figure 9 — Data model, part 2: the fleet. Insurance, tyres, batteries, urea and compliance fees all produce a maintenance bill so the money side stays in one place.

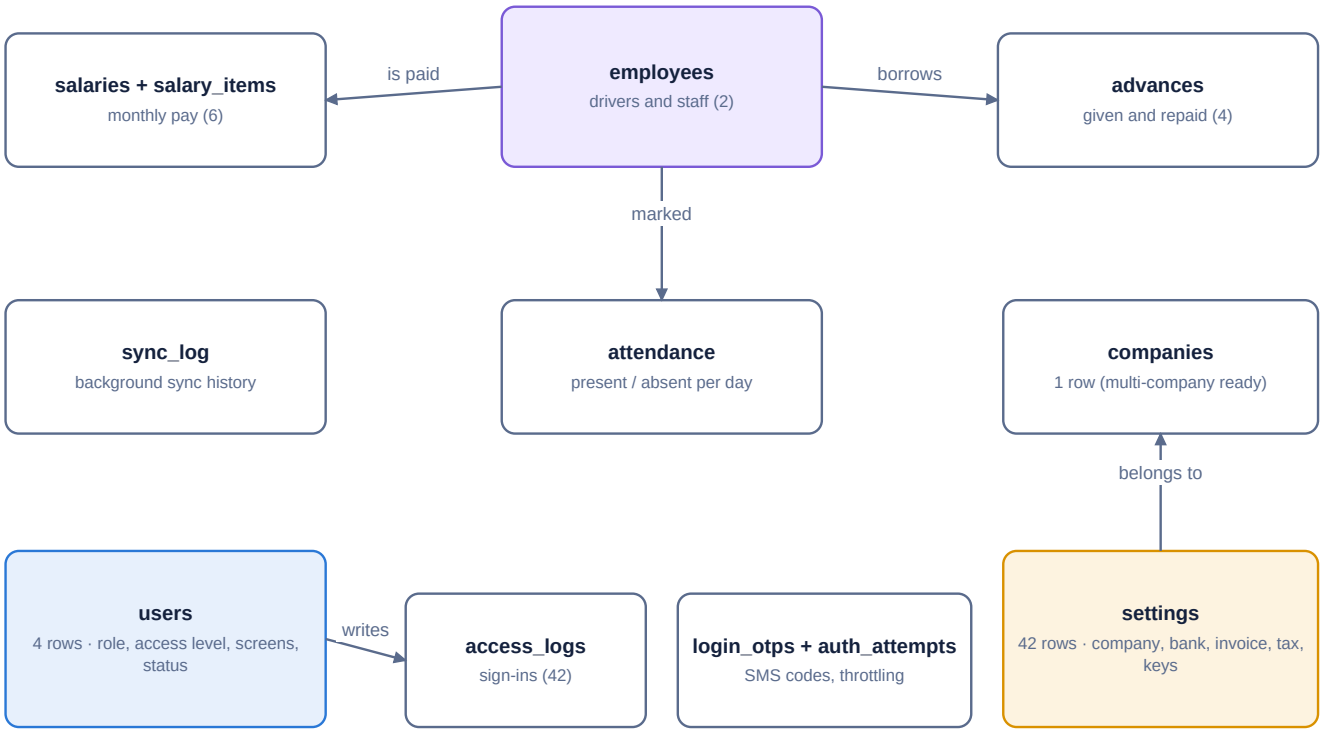


Figure 10 — Data model, part 3: people and the system. Employees are linked to salaries and advances by name; users and settings run the app itself.

23.1 Every foreign-key link (generated from the schema)

From table	Column	Points at
attendance	employee_id	employees
batteries	maintenance_id	maintenance
batteries	vendor_id	vendors
batteries	vehicle_id	vehicles
battery_checks	battery_id	batteries
compliance_expenses	maintenance_id	maintenance
compliance_expenses	vendor_id	vendors
compliance_expenses	vehicle_id	vehicles
insurance_policies	maintenance_id	maintenance
insurance_policies	insurer_id	vendors
insurance_policies	vehicle_id	vehicles
invoice_batch_charge_exclusions	trip_id	trips
invoice_batch_charge_exclusions	invoice_batch_id	invoice_batches
invoice_batch_items	vendor_id	vendors
invoice_items	vendor_id	vendors
maintenance	vendor_id	vendors
maintenance	vehicle_id	vehicles
maintenance_items	maintenance_id	maintenance
payment_allocations	overhead_id	overheads

From table	Column	Points at
payment_allocations	maintenance_id	maintenance
payment_allocations	trip_id	trips
payment_allocations	payment_id	payments
payments	vendor_id	vendors
payments	party_id	parties
salaries	employee_id	employees
salary_items	salary_id	salaries
toll_entries	maintenance_id	maintenance
toll_entries	trip_id	trips
toll_entries	vehicle_id	vehicles
trips	misc_vendor_id	vendors
trips	driver_adv_vendor_id	vendors
trips	fuel_vendor_id	vendors
trips	party_id	parties
trips	vehicle_id	vehicles
tyre_stock	installed_vehicle_id	vehicles
tyre_stock	vendor_id	vendors
tyre_stock	maintenance_id	maintenance
urea_transactions	maintenance_id	maintenance
urea_transactions	vehicle_id	vehicles
urea_transactions	supplier_id	vendors
vehicle_challans	vehicle_id	vehicles
vehicle_compliance	vehicle_id	vehicles

Not listed: the audit columns created_by and updated_by, which point at users on almost every table (a user who is deleted leaves these as empty values; the health check reports them as information, not errors), and links by *name* rather than id: salaries and advances refer to employees by employee name (and employee_id on newer rows), and trips keep owner_name text next to owner_vendor_id.

24. Data dictionary

Every table with its purpose, its row count and every column with its type. This section is generated from the live schema when the document is built, so it cannot be out of date; the notes beside non-obvious columns were written by hand.

`access_logs` — 42 rows, 5 columns

One row per sign-in, sign-up or Administrator hand-over: who and when. Shown to the Administrator as Access Logs.

Column	Type	Key	Notes
id	INTEGER	PK	
user_id	INTEGER		
event	TEXT		
date	TEXT		
company_id	INTEGER	NOT NULL	

`advances` — 4 rows, 11 columns

Money given to an employee ("given") or paid back by them ("repaid"). Keyed by employee name.

Column	Type	Key	Notes
id	INTEGER	PK	
employee	TEXT	NOT NULL	
date	TEXT		
amount	REAL		
type	TEXT		
notes	TEXT		
created_at	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		

`attendance` — 1 rows, 13 columns

One row per employee per day: Present, Absent, Leave or Half Day, with optional in/out time.

Column	Type	Key	Notes
id	INTEGER	PK	
employee_id	INTEGER	NOT NULL	
date	TEXT	NOT NULL	
status	TEXT	NOT NULL	
in_time	TEXT		
out_time	TEXT		
remarks	TEXT		
marked_by	TEXT		
created_at	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		

Column	Type	Key	Notes
updated_by	INTEGER		
updated_at	TEXT		

auth_attempts — 0 rows, 5 columns

Throttle counters: failed logins per account and address, SMS-code requests, wrong backup tokens. Old rows are swept daily.

Column	Type	Key	Notes
id	INTEGER	PK	
kind	TEXT	NOT NULL	
key	TEXT	NOT NULL	
at	REAL	NOT NULL	
company_id	INTEGER	NOT NULL	

batteries — 2 rows, 28 columns

Each battery bought: brand, model, capacity, serial, warranty months, vehicle it is fitted on and latest health.

Columns (name, type)		
id integer	install_date text	last_checked_date text
battery_no text	purchase_date text	status_override text
brand text	purchase_price real	notes text
model text	vendor_id integer	company_id integer
capacity_ah real	invoice_no text	created_by integer
battery_type text	warranty_months integer	created_at text
voltage_rating text	maintenance_id integer	updated_by integer
serial_no text	health_pct real	updated_at text
vehicle_id integer	voltage real	
installed_location text	temp_c real	

battery_checks — 7 rows, 13 columns

Dated health readings (voltage, temperature, health %) for a battery.

Column	Type	Key	Notes
id	INTEGER	PK	
battery_id	INTEGER		
date	TEXT		
event	TEXT		
health_pct	REAL		
voltage	REAL		
temp_c	REAL		
remarks	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

`companies` — 1 rows, 6 columns

The company record (one row today). Every other table points at it through company_id.

Column	Type	Key	Notes
id	INTEGER	PK	
name	TEXT	NOT NULL	
created_at	TEXT		
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		

`compliance_expenses` — 2 rows, 15 columns

Fees paid for fitness, permit and PUC renewals; each links to a maintenance bill so it can be paid like any bill.

Column	Type	Key	Notes
id	INTEGER	PK	
date	TEXT	NOT NULL	
vehicle_id	INTEGER		
compliance_type	TEXT	NOT NULL	
description	TEXT		
vendor_id	INTEGER		
amount	REAL	NOT NULL	
payment_mode	TEXT		
maintenance_id	INTEGER		
notes	TEXT		
created_at	TEXT		
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		
company_id	INTEGER	NOT NULL	

`employees` — 2 rows, 26 columns

Drivers and office staff: identity, pay, bank/UPI, documents, opening advance balance, status.

Column	Type	Key	Notes
id	INTEGER	PK	
name	TEXT	NOT NULL	
type	TEXT		
opening_balance	REAL		
opening_balance_date	TEXT		
employee_code	TEXT		
role	TEXT		
mobile	TEXT		
email	TEXT		
address	TEXT		
joining_date	TEXT		
date_of_birth	TEXT		

Column	Type	Key	Notes
bank_account	TEXT		
ifsc_code	TEXT		
upi_id	TEXT		
emergency_contact	TEXT		
aadhaar	TEXT		
pan	TEXT		
driving_license	TEXT		
status	TEXT		
basic_salary	REAL		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

insurance_policies — 18 rows, 26 columns

Vehicle insurance policies: insurer, policy number, dates, premium, IDV, NCB, agent, and uploaded documents.

Column	Type	Key	Notes
id	INTEGER	PK	
vehicle_id	INTEGER		
insurance_type	TEXT		
insurer_id	INTEGER		
policy_number	TEXT		
start_date	TEXT		
expiry_date	TEXT		
premium_amount	REAL		
idv	REAL		
ncb_pct	REAL		
gst_included	TEXT		
agent_name	TEXT		
agent_contact	TEXT		
agent_email	TEXT		
reminder_days	INTEGER		
notes	TEXT		
status_override	TEXT		
policy_doc_path	TEXT		
invoice_doc_path	TEXT		
rc_doc_path	TEXT		
maintenance_id	INTEGER		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		

Column	Type	Key	Notes
updated_by	INTEGER		
updated_at	TEXT		

invoice_batch_charge_exclusions — 0 rows, 7 columns

Trip charges deliberately left out of a generated invoice, so regenerating gives the same total.

Column	Type	Key	Notes
id	INTEGER	PK	
invoice_batch_id	INTEGER	NOT NULL	
trip_id	INTEGER	NOT NULL	
charge_key	TEXT	NOT NULL	
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		

invoice_batch_items — 5 rows, 11 columns

Extra charge or deduction lines on a generated invoice (optionally tied to a vendor).

Column	Type	Key	Notes
id	INTEGER	PK	
invoice_batch_id	INTEGER		
description	TEXT		
amount	REAL		
item_type	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		
vendor_id	INTEGER		

invoice_batch_trips — 3 rows, 8 columns

Which trips belong to which generated invoice.

Column	Type	Key	Notes
id	INTEGER	PK	
invoice_batch_id	INTEGER		
trip_id	INTEGER		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

invoice_batches — 3 rows, 21 columns

A generated invoice: number, type (party, vehicle_owner, tax, bill), party or vendor, dates, terms, GST/TDS rates, status and payment status.

Column	Type	Key	Notes
id	INTEGER	PK	
invoice_number	TEXT		
invoice_type	TEXT		
party_id	INTEGER		
vendor_id	INTEGER		
invoice_date	TEXT		
due_date	TEXT		
payment_terms	TEXT		
place_of_supply	TEXT		
remarks	TEXT		
gst_rate	REAL		
tds_rate	REAL		
loading_charges	REAL		
other_charges	REAL		
status	TEXT		
created_at	TEXT		
payment_status	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		

`invoice_items` — 5 rows, 11 columns

Extra lines on a single-trip (old style) invoice.

Column	Type	Key	Notes
id	INTEGER	PK	
trip_id	INTEGER		
description	TEXT		
amount	REAL		
item_type	TEXT		
vendor_id	INTEGER		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

`invoices` — 0 rows, 10 columns

Single-trip invoice header (older flow); also gives the ledger a real invoice number for a trip.

Column	Type	Key	Notes
id	INTEGER	PK	
trip_id	INTEGER		
invoice_number	TEXT		

Column	Type	Key	Notes
due_date	TEXT		
notes	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

login_otps — 0 rows, 9 columns

SMS login codes: user, phone, keyed hash of the code, expiry, tries used, used flag. The code itself is never stored.

Column	Type	Key	Notes
id	INTEGER	PK	
user_id	INTEGER	NOT NULL	
phone	TEXT	NOT NULL	
otp_hash	TEXT	NOT NULL	HMAC of id + code with the private secret key
expires_at	REAL	NOT NULL	
attempts	INTEGER	NOT NULL	
used	INTEGER	NOT NULL	
created_at	REAL	NOT NULL	
company_id	INTEGER	NOT NULL	

maintenance — 452 rows, 25 columns

The bill table for every repair or purchase: service, tyre, battery, urea, insurance and compliance fees. Amount, paid amount, vendor, vehicle, km, next-due.

Column	Type	Key	Notes
id	INTEGER	PK	
date	TEXT	NOT NULL	
vehicle_id	INTEGER		
category	TEXT		
amount	REAL		
paid_amount	REAL		how much of the bill is paid; never below what ledger allocations cover
vendor_id	INTEGER		
notes	TEXT		
km_reading	REAL		
service_type	TEXT		
next_due_km	REAL		
next_service_date	TEXT		
invoice_no	TEXT		
invoice_date	TEXT		
status	TEXT		'Open' or 'Completed'
checklist_done	TEXT		
tyre_action	TEXT		

Column	Type	Key	Notes
tyre_id	TEXT		
tyre_brand	TEXT		
tyre_position	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

maintenance_items — 0 rows, 13 columns

Checklist lines inside a service bill.

Column	Type	Key	Notes
id	INTEGER	PK	
maintenance_id	INTEGER		
item_name	TEXT		
category	TEXT		
qty	REAL		
unit	TEXT		
rate	REAL		
amount	REAL		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

overheads — 5 rows, 18 columns

Expense bills (office, fuel for non-trip use, rent, etc.) with Paid/Pending status, optional vendor, recurring flag and due date.

Column	Type	Key	Notes
id	INTEGER	PK	
date	TEXT	NOT NULL	
category	TEXT		
amount	REAL		
notes	TEXT		
payment_mode	TEXT		
receipt_number	TEXT		
vendor	TEXT		
description	TEXT		
status	TEXT		'Paid' or 'Pending'
is_recurring	INTEGER		
recurring_frequency	TEXT		
due_date	TEXT		
company_id	INTEGER	NOT NULL	

Column	Type	Key	Notes
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

parties — 70 rows, 18 columns

Customers: contact, GSTIN, credit limit, opening balance and date, category, status.

Column	Type	Key	Notes
id	INTEGER	PK	
name	TEXT	NOT NULL	
contact	TEXT		
notes	TEXT		
address	TEXT		
email	TEXT		
credit_limit	REAL		
since_date	TEXT		
opening_balance	REAL		signed: positive = they owed us before the app
opening_balance_date	TEXT		
gstin	TEXT		
category	TEXT		
status	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

payment_allocations — 22 rows, 11 columns

The link between a payment and what it settled: exactly one of a trip, a maintenance bill or an expense, with the amount.

Column	Type	Key	Notes
id	INTEGER	PK	
payment_id	INTEGER	NOT NULL	
trip_id	INTEGER		set when the payment settled a trip
maintenance_id	INTEGER		set when it settled a maintenance bill
overhead_id	INTEGER		set when it settled an expense bill
amount	REAL	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		
company_id	INTEGER	NOT NULL	

payments — 224 rows, 15 columns

Money received from a party or paid to a vendor: date, amount, mode, reference, how much has been allocated.

Column	Type	Key	Notes
id	INTEGER	PK	
date	TEXT	NOT NULL	
payment_type	TEXT		'received' (from a party) or 'paid' (to a vendor)
amount	REAL		
party_id	INTEGER		
vendor_id	INTEGER		
mode	TEXT		
reference_id	TEXT		
remarks	TEXT		
allocated_amount	REAL		how much of the amount was applied to trips / bills / expenses
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

salaries — 6 rows, 23 columns

One salary payment: employee, month, gross, deductions, advance recovery, net, mode, transaction id, payment status.

Column	Type	Key	Notes
id	INTEGER	PK	
employee	TEXT	NOT NULL	
month	TEXT	NOT NULL	
amount	REAL		
date	TEXT		
created_at	TEXT		
employee_id	INTEGER		
month_key	TEXT		
basic_salary	REAL		
gross_salary	REAL		
total_deductions	REAL		
advance_recovery	REAL		
net_salary	REAL		
payment_status	TEXT		
payment_date	TEXT		
payment_mode	TEXT		
transaction_id	TEXT		
paid_by	TEXT		
remarks	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
updated_by	INTEGER		

Column	Type	Key	Notes
updated_at	TEXT		

salary_items — 0 rows, 10 columns

Allowance or deduction lines of a salary.

Column	Type	Key	Notes
id	INTEGER	PK	
salary_id	INTEGER	NOT NULL	
item_type	TEXT	NOT NULL	
description	TEXT	NOT NULL	
amount	REAL	NOT NULL	
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

settings — 42 rows, 7 columns

Key/value store per company: company details, bank, invoice, tax, keys (masked), sync intervals, logo path, private secret key row.

Column	Type	Key	Notes
key	TEXT	PK	primary key (see _SECRET_SETTING_KEYS for masked ones)
value	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

sync_log — 1 rows, 9 columns

One row per background sync run: job, start/finish, synced / failed / skipped counts.

Column	Type	Key	Notes
id	INTEGER	PK	
job_name	TEXT	NOT NULL	
started_at	TEXT	NOT NULL	
finished_at	TEXT		
synced_count	INTEGER		
failed_count	INTEGER		
skipped_count	INTEGER		
note	TEXT		
company_id	INTEGER	NOT NULL	

toll_entries — 94 rows, 21 columns

Toll spend per vehicle (and optionally trip): plaza, highway, amount, source (fastag or manual), status (synced, approved, pending, rejected), receipt file.

Column	Type	Key	Notes
id	INTEGER	PK	
date	TEXT	NOT NULL	
time	TEXT		
vehicle_id	INTEGER		
trip_id	INTEGER		
toll_plaza	TEXT	NOT NULL	
highway	TEXT		
state	TEXT		
amount	REAL	NOT NULL	
source	TEXT	NOT NULL	
payment_mode	TEXT		
status	TEXT	NOT NULL	synced | approved | pending | rejected
reference_no	TEXT		
receipt_path	TEXT		
notes	TEXT		
maintenance_id	INTEGER		
created_at	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		

`trips` — 472 rows, 80 columns

The core table: one row per journey with route, rates, charges, deductions, fuel, advances, party money, owner terms and costs.

Columns (name, type)		
id integer	material_damage real	unloading_expense real
date text	shortage_amount real	wear_tear real
lr_number text	shortage_qty real	weighbridge_charges real
vehicle_id integer	tds real	other_expense real
type text	other_deductions real	misc_vendor_id integer
party_id integer	fuel_amount real	lr_received text
from_loc text	fuel_vendor_id integer	owner_vendor_id integer
to_loc text	driver_adv_amount real	permit_charges real
quantity real	driver_adv_vendor_id integer	end_date text
rate real	party_advance real	shortage_date text
driver_name text	payment_received real	end_time text
material text	owner_name text	actual_km real
rate_type text	fixed_rate_amount real	shortage_unit text
billed_amount real	owner_rate real	shortage_remarks text
driver_payment real	owner_amount real	remarks text
detention_charges real	paid_to_owner real	is_empty integer
gps_cost real	pending_to_owner real	owner_rate_type text

Columns (name, type)		
loading_charge real	agent_commission real	owner_fixed_amount real
unloading_charge real	builty_expense real	fuel_liters text
police_charges real	conductor_expense real	fuel_price real
sim_tracking real	fine real	company_id integer
union_charges real	labour_charges real	created_by integer
weight_charges real	parking real	created_at text
other_charges real	puncture real	updated_by integer
brokerage real	toll real	updated_at text
builty_commission real	urea real	guarantee_qty real
late_fees real	loading_expense real	

Column	Notes
billed_amount	freight + charges – deductions, calculated on save
payment_received	money received against this trip (typed on the trip and/or allocated from ledger payments)
party_advance	advance taken from the party at booking
guarantee_qty	minimum guaranteed weight; when non-zero PER_MT billing uses it
type	'own' or 'hired'; follows the vehicle
rate_type	'PER_MT' or 'FIXED'
is_empty	1 = an empty (no-load) run
lr_received	'Yes' when the signed lorry receipt has come back

tyre_stock — 2 rows, 19 columns

Tyres bought into stock and where each is fitted (vehicle, position) or scrapped.

Column	Type	Key	Notes
id	INTEGER	PK	
maintenance_id	INTEGER		
tyre_id	TEXT		
brand	TEXT		
tyre_type	TEXT		
purchase_date	TEXT		
purchase_cost	REAL		
vendor_id	INTEGER		
invoice_no	TEXT		
status	TEXT		
notes	TEXT		
installed_vehicle_id	INTEGER		
installed_position	TEXT		
installed_date	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

urea_transactions — 4 rows, 21 columns

AdBlue stock movements: stock_in (purchase) or stock_out (use, from stock or direct) with running balance.

Column	Type	Key	Notes
id	INTEGER	PK	
date	TEXT	NOT NULL	
txn_type	TEXT	NOT NULL	'stock_in' or 'stock_out'
source	TEXT	NOT NULL	
batch_no	TEXT		
supplier_id	INTEGER		
invoice_no	TEXT		
vehicle_id	INTEGER		
quantity_l	REAL	NOT NULL	
unit_price	REAL		
total_value	REAL		
balance_after_l	REAL		
location	TEXT		
odometer_km	REAL		
notes	TEXT		
maintenance_id	INTEGER		
created_at	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		

users — 4 rows, 17 columns

Sign-in accounts: username (mobile or email), password hash, role, access level, allowed screens, status, last login.

Column	Type	Key	Notes
id	INTEGER	PK	
username	TEXT	NOT NULL	
password_hash	TEXT	NOT NULL	
role	TEXT		free text; exactly 'Administrator' (any case) is the single admin seat
is_admin	INTEGER		legacy flag, set for Full Access users; real admin rights come from role
created_at	TEXT		
phone	TEXT		
full_name	TEXT		
email	TEXT		
access_level	TEXT		'Full Access', 'Read Only' or 'Limited Access'
module_access	TEXT		for Limited Access: e.g. 'Ledger:e,Reports:v' (e = edit, v = view)
status	TEXT		'Active' or 'Inactive' (inactive users are signed out at once)
last_login	TEXT		
company_id	INTEGER	NOT NULL	

Column	Type	Key	Notes
created_by	INTEGER		
updated_by	INTEGER		
updated_at	TEXT		

vehicle_challans — 9 rows, 33 columns

Traffic challans fetched from eChallan per vehicle, with amounts and status.

Columns (name, type)		
id integer	name_of_violator text	sent_to_reg_court text
vehicle_id integer	owner_name text	sent_to_virtual_court text
api_id text	dl_no text	offence_details text
challan_no text	document_impounded text	source_created_at text
challan_date_time text	remark text	source_updated_at text
challan_place text	rto_distric_name text	last_synced text
challan_status text	state_code text	company_id integer
fine_imposed real	court_name text	created_by integer
amount_of_fine_imposed real	court_address text	created_at text
department text	date_of_proceeding text	updated_by integer
driver_name text	sent_to_court_on text	updated_at text

vehicle_compliance — 54 rows, 19 columns

Fitness, PUC and permit records per vehicle: number, dates, source (manual, mock or live), sync status.

Column	Type	Key	Notes
id	INTEGER	PK	
vehicle_id	INTEGER	NOT NULL	
compliance_type	TEXT	NOT NULL	
document_number	TEXT		
issuing_authority	TEXT		
permit_subtype	TEXT		
valid_from	TEXT		
valid_upto	TEXT		
status_override	TEXT		
source	TEXT		
provider_name	TEXT		
last_sync_time	TEXT		
sync_status	TEXT		
remarks	TEXT		
created_at	TEXT		
updated_at	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
updated_by	INTEGER		

vehicles — 60 rows, 24 columns

Trucks (own or hired): registration, type, capacity, body, chassis/engine, document dates, cached RC data, challan counts.

Column	Type	Key	Notes
id	INTEGER	PK	
vehicle_no	TEXT	NOT NULL	
type	TEXT		
registration_date	TEXT		
capacity_mt	REAL		
insurance_expiry	TEXT		
fitness_expiry	TEXT		
notes	TEXT		
puc_valid_upto	TEXT		
permit_valid_upto	TEXT		
status	TEXT		
body_type	TEXT		
chassis_number	TEXT		
engine_number	TEXT		
rc_synced_data	TEXT		cached full eChallan RC answer (JSON text)
rc_last_synced	TEXT		
challan_count	INTEGER		
challan_amount	REAL		
challan_last_synced	TEXT		
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

vendors — 38 rows, 18 columns

People and firms the company pays: truck owners, fuel pumps, workshops; may be linked to a party.

Column	Type	Key	Notes
id	INTEGER	PK	
name	TEXT	NOT NULL	
category	TEXT		
contact	TEXT		
address	TEXT		
email	TEXT		
credit_limit	REAL		
since_date	TEXT		
linked_party_id	INTEGER		when the same organisation is also a party, the two ledgers are merged
opening_balance	REAL		
opening_balance_date	TEXT		
gstin	TEXT		
status	TEXT		

Column	Type	Key	Notes
company_id	INTEGER	NOT NULL	
created_by	INTEGER		
created_at	TEXT		
updated_by	INTEGER		
updated_at	TEXT		

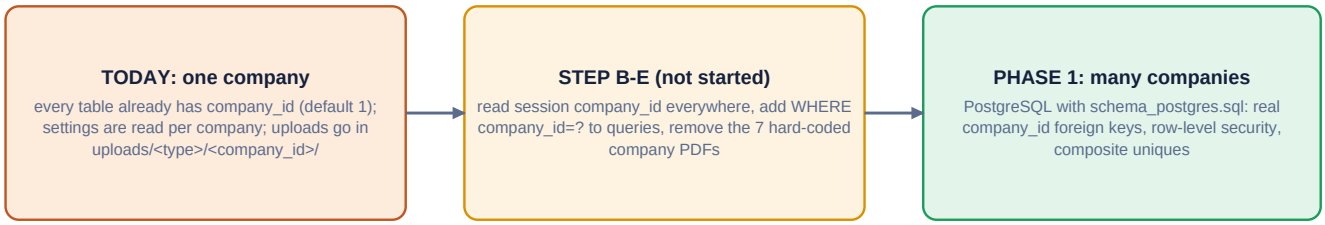
25. Conventions, rules and multi-company readiness

25.1 Rules enforced by the database itself

Rule	How
A payment allocation points at exactly one target	CHECK on payment_allocations: (trip_id is not null) + (maintenance_id is not null) + (overhead_id is not null) = 1
Allowed values	CHECK constraints: payments.payment_type (received, paid), advances.type (given, repaid), employees.type (Staff, Driver), attendance.status (Present, Absent, Leave, Half Day), invoice_batches.invoice_type (party, vehicle_owner, tax, bill), item types (charge, deduction, allowance), urea txn_type and source, toll source and status, compliance_type (fitness, puc, permit)
Unique mobile and email per user	unique index on users(phone) where not null; unique index on lower(email) where not null; usernames are unique
One active Administrator	unique index uq_one_active_administrator on users(company_id) where role is Administrator and status is not Inactive
Performance	62 indexes, including partial indexes on mostly-empty allocation columns (SQLite ignores an ordinary index on a column that is almost all NULL)

25.2 Sign and meaning conventions

- **Ledger balance** = debits – credits; positive means the party owes the company, negative means the company owes (payable).
- **Opening balances** are signed numbers with a date on parties, vendors and employees.
- **Status words** in use: vehicles Active/Inactive; users Active/Inactive; maintenance Open/Completed; expenses Paid/Pending; invoices payment status PENDING/PAID; toll synced/approved/pending/rejected; batteries Good/Weak/Replace Soon/Dead/In Stock; insurance Active/Expiring Soon/Expired/Cancelled.
- **Own versus hired** appears as type on both vehicles and trips and changes how costs and profit are read everywhere.



RULES FOR ANY NEW CODE (from CLAUDE.md)

- New tables get company_id NOT NULL DEFAULT 1 (and the Postgres twin with a foreign key + index)
- Natural-key UNIQUE constraints are (company_id, name), never bare
- Settings go through _get_all_settings(company_id) — never ad-hoc SELECTs
- Uploads: uploads/<type>/<company_id>/… ; never hard-code the company name in a page or PDF
- db.py is the only place a database swap happens

Figure 18 — Built to become a multi-company product later without a redesign.

25.3 The PostgreSQL twin

`schema_postgres.sql` is a complete, parse-verified PostgreSQL version of the schema (it has been checked with a SQL parser, not yet run against a live Postgres): every table carries company_id as a real foreign key with an index; natural-key uniques are composite (company_id, name); and row-level security policies make a connection that sets app.current_company_id physically unable to read another company's rows. Five bare UNIQUE constraints that would break the day a second company exists (vehicle number, party name, vendor name, employee name, username) were already corrected there. `db.py` holds the one-line placeholder translation for the day it is switched on.

26. Migrations: how the database layout has changed

Every change to the layout is a documented script, kept as history. The latest ones are also applied automatically at start-up (Chapter 8.2), so a deployment can never find the code ahead of the database.

File	What it changed
migrate.py	Original import from the Excel workbook into fleet.db.
migrate_audit_columns.sql	Adds created_by, created_at, updated_by, updated_at to the tables.
migrate_company_id.sql	Adds company_id (default 1) to every table: the multi-company preparation.
migrate_compliance_expenses.sql	Creates compliance_expenses.
migrate_drop_guarantee_applied.sql	Removes the old guarantee_applied flag.
migrate_extra.py	Extra one-time import steps.
migrate_guarantee_qty.sql	Adds trips.guarantee_qty (minimum guaranteed weight).
migrate_invoice_batch_items_vendor.sql	Lets invoice lines name a vendor.
migrate_invoice_charge_exclusions.sql	Creates invoice_batch_charge_exclusions.
migrate_payment_allocation_maintenance.sql	Lets a payment allocation target a maintenance bill.
migrate_payment_allocation_overhead.sql	Lets a payment allocation target an expense bill (also self-applied at start-up).
migrate_unique_phone_email.sql	Unique mobile and email per user.
migrate_username_nocase.sql	Case-insensitive usernames.
migrate_v2.py	Second-generation import / reshape.
migrate_vehicle_type_own_hired.sql	Normalises vehicle type to own / hired.

Changes made after these files are all self-applying and listed in Chapter 8.2: the payment_allocations.overhead_id column, the ~36 performance indexes, WAL mode, login_otps and auth_attempts, the secret key row, and the single-Administrator index.

E

Execution walkthroughs

Every action in the app, followed from the click to the database and back to the screen: what is sent, what is checked, which tables get which columns, and what is shown next.

IN THIS PART

- How to read a walkthrough
- Sign-in and sign-out
- Trips
- Payments, ledgers, parties and vendors
- Expenses
- Maintenance, vehicles, people, invoices, reports, administration
- The phone-app twins and background jobs

27. How to read a walkthrough

Each walkthrough follows one action from the user's finger to the database and back. They all share the same skeleton, so once you understand one you can read the others quickly.

Part	What it tells you
Where it starts	The screen, button or form the user uses.
What is sent	The URL, the HTTP method and the form fields or query parameters that travel to the server (names as they appear in the code).
Code entry	The route function that receives it (with its line number in app.py) and the helpers it calls.
Phone twin	The equivalent route used by the phone app, when there is one.
Step by step	Exactly what the code does, in order: validate, look up, calculate, write, answer.
What is saved	Every table written, whether it is an INSERT (new row), UPDATE (changed row) or DELETE, and which columns receive values.
What is read	The tables consulted to make the decision or to build the answer.
What the user sees next	The redirect or page returned, and how the saved data shows up on screen later.
Protections and edge cases	What is refused, what is repeated safely, and the rules that keep totals honest.

27.1 Things every write has in common

- **Post, then redirect.** Every form is sent as an HTTP POST. After the work is done the server answers with a redirect to a list or ledger page (so pressing Refresh never re-sends the form). Errors travel back as a short message in the address (for example `?pay_error=...`, `?exp_error=...`, `?end_error=...`) and the target page shows it as a banner.
- **Who and when.** Almost every write records `created_by` / `updated_by` (the signed-in user's id from the session) and `created_at` / `updated_at` (server time as YYYY-MM-DD HH:MM:SS). The tables below list only the business columns; read these four as always present on tables that have them.
- **Names become records.** Typing a new party, vendor or vehicle name on a trip or bill does not stop the user: `get_or_create_party / get_or_create_vendor / get_or_create_vehicle` look the name up and create a row if it is new, and the new id goes onto the transaction.
- **One commit.** A route opens one connection, performs all its reads and writes, then commits once. If anything fails before the commit nothing is saved.
- **Numbers are text until checked.** Browsers send everything as text. Amounts pass through `_parse_money` (or float with a guard) and are stored as numbers.
- **The same function serves both faces.** Wherever a "phone twin" is named, the twin calls the same `_insert_X_from_form` helper, so the saved columns are identical.

> **Where to find the complete lists**
>
> This Part walks through the important actions in prose. Part F then gives an automatically generated "execution card" for every one of the 199 URLs (inputs, helper chain, tables read and written with columns, page shown), and a "life of each table" listing which URLs create, change, delete and read it.

28. Sign-in and sign-out

28.1 Password sign-in

A user opens the website (or the phone app) and signs in with a mobile number or email and a password.

Where it starts	Website /login form (templates/login.html) or phone /app/login (templates/app/login.html).
What is sent	POST /login or /app/login: `username`, `password`, `remember_me`.
Code entry	`login` (10249) (website) and `app_login` (12533) (phone).
Phone twin	app_login

What happens, step by step

1. The username must look like a 10-digit Indian mobile or an email (`_username_login_format_ok` (92)), otherwise the page says so before touching the database.
2. Throttle check: `_login_throttle_message` (445) counts failures for this username and for this visitor address in the last 15 minutes (table `auth_attempts`). Five (account) or thirty (address) means "Too many failed sign-in attempts" with HTTP 429 and the password is not even checked.
3. The user row is fetched by username and the password is checked against the stored hash with werkzeug.
4. An Inactive account is refused with a clear message.
5. On success the failure counter for that username is cleared, the session is filled (user_id, username, is_admin, login_at, a one-time "zoom tip" flag) and made permanent (30 days) only if "remember me" was ticked.
6. last_login is updated and a row "Login" is added to access_logs; the browser is redirected to the Dashboard (or, for a Limited user without Dashboard, the first allowed screen).
7. On failure the attempt is recorded for both counters and the page shows "Invalid username or password."

What is saved, and where

Table	Operation	Columns / values written
`users`	UPDATE	last_login, updated_by, updated_at
`access_logs`	INSERT	user_id, event = "Login", date
`auth_attempts`	INSERT / DELETE	failure rows (kind login_fail_user and login_fail_ip, key, time); the user's failure rows are deleted after a successful login

What is read

- users (password_hash, status), auth_attempts (recent failures)

What the user sees next

- The signed cookie now identifies the user; every following request re-reads their level, role, status and screens.
- Access Logs (Administrator only) show the sign-in.

Protections and edge cases

- Failed attempts against unknown usernames are counted too, so the limit cannot be used to discover which usernames exist.
- Messages are identical for a wrong username and a wrong password.

28.2 Sign-in by SMS code

A user who has an account types only their mobile number and receives a six-digit code by SMS through MSG91.

Where it starts	Website: "Login with OTP" on /login. Phone: /app/login/otp then the verify screen.
What is sent	POST /login/otp/send (website) or /app/login/otp (phone): `phone`. Then POST /login/otp/verify or /app/login/otp/verify: `phone`, `otp`.

Code entry	`send_login_otp` (10334), `_start_otp_login` (10345), `verify_login_otp` (10410), `_finish_otp_login` (10382).
Phone twin	app_login_otp, app_verify_login_otp

What happens, step by step

1. The number must be a 10-digit Indian mobile. If MSG91 is not configured the page says SMS login is not set up.
2. Limits are checked (3 codes per number per 15 minutes, 10 per address per hour) and the request is counted.
3. If a matching active user exists, `_otp_issue` (461) stores a new row in login_otps (older unused codes for that user are marked used), generates six digits with secrets, stores only a keyed hash of id+code, and `_send_otp_msg91` (10312) asks MSG91 to send the SMS. A failed send marks the code used and shows "Could not send the code right now".
4. If no such user exists nothing is sent, but the same "code sent" screen is shown.
5. On verify, `_otp_check` (475) finds the code row from the session, checks it is unused, unexpired and for this number, counts the try, compares the keyed hash in constant time, and marks the code used on success.
6. The user is then signed in exactly like a password login (session filled, last_login updated, access_logs "Login (OTP)").

What is saved, and where

Table	Operation	Columns / values written
`login_otps`	INSERT / UPDATE	user_id, phone, otp_hash, expires_at, attempts, used, created_at
`auth_attempts`	INSERT	kind otp_send_phone / otp_send_ip, key, time
`users`	UPDATE	last_login
`access_logs`	INSERT	event = "Login (OTP)"

What is read

- settings (msg91_authkey, msg91_template_id), users (phone, status), login_otps

What the user sees next

- After a correct code: the Dashboard. After a wrong code: "Incorrect OTP"; after five wrong codes: "Too many incorrect codes. Please request a new OTP."

Protections and edge cases

- The browser cookie holds only the code row number, never anything that reveals the code.
- The SMS text is MSG91's DLT-approved template; the app only supplies the digits.

28.3 Sign-out and "sign-up"

Leaving the app, and the (closed) public registration page.

Where it starts	Sidebar "Logout" (website) or Profile "Log Out" (phone); /app/signup link (hidden).
What is sent	GET /logout (with ?ctx=app from the phone); GET or POST /app/signup.
Code entry	`logout` (10287), `app_signup` (12573).

What happens, step by step

1. Logout clears the whole session and redirects to the website login or, when it came from the phone app, to the app login (without replaying the splash animation).
2. Sign-up checks `_signup_open()`: unless the environment variable ALLOW_SELF_SIGNUP is 1 it renders the app login with "Sign-up is closed. Ask an Administrator to create your account." and HTTP 403; no row is written.

What the user sees next

- The login page.

Protections and edge cases

- If sign-up is ever reopened, new accounts are Staff / Read Only / Active, which can still read every screen — the reason it is closed.

29. Trips

Trips are the heart of the system. Five actions touch them: add, edit, end, delete (guarded) and view; plus the list that everyone looks at.

29.1 Add a trip

Dispatch enters a new journey: the party, the vehicle, the route, the rate, fuel and advances.

Where it starts	Website: Trips → Add Trip (trip_form.html, a multi-section form with live Trip Summary, estimated profit and payment tracking). Phone: Add Trip (+ menu) → app/trip_add.html wizard.
What is sent	POST /trips/add (phone: /app/trips/add). Fields: `is_empty`, `return_to`, `date`, `lr_number`, `vehicle_no`, `type`, `party_name`, `driver_name`, `material`, `rate_type`, `from_loc`, `to_loc`, `quantity`, `guarantee_qty`, `rate`, `fixed_rate_amount`, `detention_charges`, `gps_cost`, `loading_charge`, `unloading_charge`, `police_charges`, `sim_tracking`, `union_charges`, `weight_charges`, `other_charges`, `brokerage`, `builty_commission`, `late_fees`, `material_damage`, `shortage_amount`, `shortage_qty`, `tds`, `other_deductions`, `fuel_amount`, `fuel_vendor`, `fuel_liters`, `fuel_price`, `driver_adv_amount`, `driver_adv_vendor`, `agent_commission`, `builty_expense`, `fine`, `labour_charges`, `parking`, `puncture`, `toll`, `urea`, `loading_expense`, `unloading_expense`, `weighbridge_charges`, `other_expense`, `misc_vendor`, `owner_name`, `owner_rate_type`, `owner_rate`, `owner_fixed_amount`, `paid_to_owner`, `party_advance`, `payment_received`, `lr_received`, plus repeating rows `other_desc`, `other_type`, `other_rate`, `other_qty`, `other_vendor` for extra lines.
Code entry	`add_trip` (3227) → `_duplicate_lr_trip` (2990) → `_trip_form_error` (3041) → `_insert_trip_from_form` (3070).
Phone twin	app_add_trip

What happens, step by step

1. The LR number is checked for duplicates (case-insensitive, ignoring blanks). A clash re-shows the form with "LR Number ... is already used on trip #N (date, vehicle)".
2. _trip_form_error validates every one of the 40 money fields with _parse_money, refuses a negative computed bill and (for a NEW trip) a party advance larger than the bill.
3. _insert_trip_from_form resolves names: vehicle (get_or_create_vehicle, which also creates a vehicle row if the number is new), party, fuel vendor, driver-advance vendor, owner vendor (only if an owner name is typed), misc vendor. The trip type (own or hired) follows the vehicle.
4. The freight is computed by _trip_freight: FIXED → fixed_rate_amount; otherwise (guarantee_qty if non-zero, else quantity) × rate.
5. billed_amount = freight + (detention + GPS + loading + unloading + police + SIM tracking + union + weight + other charges) – (brokerage + builty commission + late fees + material damage + shortage amount + TDS + other deductions).
6. One INSERT into trips writes every business column. Then _save_trip_custom_items replaces the trip's "Others" lines in invoice_items (rate × quantity becomes the stored amount; an optional vendor name is resolved to vendor_id).
7. The route commits and redirects to the Trips list.

What is saved, and where

Table	Operation	Columns / values written
`trips`	INSERT	date, lr_number, vehicle_id, type, party_id, from_loc, to_loc, quantity, guarantee_qty, rate, driver_name, material, rate_type, billed_amount, the 9 charge columns, the 7 deduction columns (incl. shortage_qty), fuel_amount, fuel_vendor_id, fuel_liters, fuel_price, driver_adv_amount, driver_adv_vendor_id, party_advance, payment_received, owner_name, fixed_rate_amount, owner_rate, owner_rate_type, owner_fixed_amount, paid_to_owner, owner_vendor_id, the own-vehicle cost columns (agent_commission, builty_expense, fine, labour_charges, parking, puncture, toll, urea, loading_expense, unloading_expense, weighbridge_charges, other_expense), misc_vendor_id, lr_received, is_empty, created_by, created_at

Table	Operation	Columns / values written
`invoice_items`	DELETE then INSERT	for this trip: description, amount (rate × qty), item_type (charge or deduction), vendor_id, created_by, created_at
`parties / vendors / vehicles`	INSERT (only if the typed name is new)	name (+ type for a vehicle)

What is read

- trips (duplicate LR check), parties, vendors, vehicles (name lookups), employees (driver name suggestions on the form)

What the user sees next

- The new trip appears at the top of Trips (newest date first) with its LR, vehicle, party, route, Billed and Pending (= billed – payment received – party advance).
- It immediately feeds the party's ledger (a Trip Bill line), the Dashboard totals, Route Analytics, Performance and Fleet Utilization.
- Columns owner_amount, pending_to_owner, driver_payment, conductor_expense and wear_tear exist in the table but are not written by the current form: the owner's amount is computed when needed from owner_rate/owner_fixed_amount and quantity.

Protections and edge cases

- A trip on a hired vehicle needs an owner name to appear in the owner's (vendor) ledger.
- The vehicle cannot be changed later (locked at creation).

29.2 Edit a trip

Someone corrects a rate, adds an advance or fixes a route after the trip was entered.

Where it starts	Trips list → row → Edit (opens trip_form.html pre-filled). Phone: trip detail → Edit.
What is sent	POST /trips/edit/<id> (phone /app/trips/<id>/edit) with the same fields as Add (the vehicle field is ignored).
Code entry	`edit_trip` (11765) → `_update_trip_from_form` (3142).
Phone twin	app_edit_trip

What happens, step by step

1. Duplicate-LR check, excluding this trip itself. The "advance larger than bill" check is skipped on edits (a rate correction can legitimately leave more received than billed).
2. _update_trip_from_form reads the trip's existing vehicle_id (never changed), recomputes the type, resolves the same names, recomputes freight and billed_amount exactly like Add.
3. Two floors protect money already allocated: payment_received can never be saved below the sum of party payments already allocated to this trip, and paid_to_owner never below owner payments allocated (SELECT SUM(amount) from payment_allocations joined to payments).
4. One UPDATE rewrites all business columns; the custom invoice_items are replaced again; commit; redirect to the list (or to return_to).

What is saved, and where

Table	Operation	Columns / values written
`trips`	UPDATE	all columns listed under Add, except vehicle_id and the created_* fields; updated_by, updated_at
`invoice_items`	DELETE then INSERT	same as Add
`parties / vendors`	INSERT (only if a typed name is new)	name

What is read

- trips (existing vehicle_id), payment_allocations + payments (floors), vendors, parties

What the user sees next

- The list, the party ledger and every total show the new numbers on the next page load (nothing is cached).

Protections and edge cases

- A plain re-save can never reduce what a real payment already settled, so a ledger "Trip Bill" line cannot look like it double-subtracts a payment.

29.3 End a trip

When the truck is delivered the trip is closed with the actual end date, kilometres and any shortage.

Where it starts	Trips list → End Trip button (modal "End Trip"), or the phone trip detail End Trip sheet.
What is sent	POST /trips/<id>/end: `end_date`, `end_time`, `actual_km`, `lr_received`, `freight_amount`, `shortage_qty`, `shortage_unit`, `shortage_amount`, `shortage_remarks`, `remarks`.
Code entry	`end_trip` (7030) → `_end_trip_from_form` (6983).
Phone twin	app_end_trip

What happens, step by step

1. The trip must exist and must not already have an end_date (once ended, no further edits).
2. End date is required; freight amount, shortage quantity, shortage amount and kilometres are parsed with _parse_money.
3. The Freight Amount box is pre-filled with the trip's GROSS freight, so the new bill is simply freight_amount – shortage_amount. If that is negative the end is refused.
4. One UPDATE closes the trip.

What is saved, and where

Table	Operation	Columns / values written
`trips`	UPDATE	end_date, end_time, actual_km, lr_received, shortage_qty, shortage_unit, shortage_amount, shortage_date (= end date when there is a shortage), shortage_remarks, remarks, billed_amount (= freight – shortage), updated_by, updated_at

What is read

- trips

What the user sees next

- The row moves from "Active" to completed; the Pending balance reflects the new billed amount; Fleet Utilization counts the trip as active from its start date through its end date.

Protections and edge cases

- The shortage is taken off exactly once (earlier versions deducted it twice).
- Errors come back as ?end_error=... shown above the list.

29.4 Delete a trip

Removing a trip entered by mistake.

Where it starts	Trips list → Delete (confirmation modal).
What is sent	POST /trips/delete/<id>.
Code entry	`delete_trip` (6948).

What happens, step by step

1. Counts payment_allocations rows for the trip: if any exist the delete is refused (redirect with error=has_payment, the amount and the LR) because removing the trip would silently lose track of that money.
2. Counts toll_entries linked to the trip: if any exist the delete is refused (error=has_toll).
3. Otherwise one DELETE FROM trips; commit.

What is saved, and where

Table	Operation	Columns / values written
`trips`	DELETE	the row

What is read

- payment_allocations, toll_entries

What the user sees next

- Back on the list, a red message explains a refusal ("This trip has Rs X already received/paid against it ... reverse the payment first").

Protections and edge cases

- Observed gap: only the trips row is removed. The trip's invoice_items (its "Others" lines) and any invoice_batch_trips link are left behind and become orphans.

29.5 The Trips list (how trips are retrieved and shown)

The screen everyone opens first.

Where it starts	Sidebar → Trips (/trips).
What is sent	GET /trips with optional filters `vehicle`, `party`, `date_from`, `date_to`, `status`, `lr_received`, `lr_number`, `from_loc`, `to_loc`, `type`, `sort`, `dir`, `page`, `per_page`.
Code entry	`trips_list` (2879) → templates/trips_list.html.
Phone twin	app_trips (templates/app/trips.html)

What happens, step by step

1. Builds one SELECT over trips LEFT JOIN vehicles LEFT JOIN parties and appends an AND clause for each filter that was given (LIKE for vehicle, party, LR, from, to; comparisons for dates; IS NULL / IS NOT NULL on end_date for active/completed; the LR-received tests).
2. Runs five aggregate queries over the same WHERE clause: total billed, count, pending (SUM of billed – payment_received – party_advance), active count, LR-received count.
3. Pagination: default 50 per page (10/25/50/100 allowed). Sorting uses a whitelist (date or party) so the query string can never inject SQL; the page is cut with LIMIT/OFFSET.
4. Dropdown lists of active vehicles and parties are read for the filter boxes; the template is rendered.

What is read

- trips, vehicles, parties

What the user sees next

- Five cards: Total Trips, Total Billed, Pending Balance, Active Trips, LR Pending.
- A table with columns `Date` ■, `LR Number`, `Vehicle`, `Party` ■, `From`, `To`, `Billed`, `Pending`, `Action` and per-row End Trip / Delete / Edit actions.
- Sort arrows on Date and Party keep every filter in the address; Excel/PDF export buttons use the same filters.

Protections and edge cases

- All search text is passed as a bound parameter; only whitelisted column names are placed in the SQL text.

29.6 View one trip

A read-only look at everything about a trip, opened from a ledger line.

Where it starts	Ledger row "Trip Bill" → centred popup (website) or trip detail page (phone).
What is sent	GET /trips/<id>/view (phone /app/trips/<id>).
Code entry	`trip_view` (11894) → `_trip_detail_data` (11842).
Phone twin	app_trip_detail

What happens, step by step

1. One SELECT joins trips with vehicles (number, status, body, capacity, cached RC data), parties and the three vendor names; the trip's invoice_items are read.
2. Computed in Python: the owner's amount for hired trips (fixed amount or rate × quantity), additions total (charges + "charge" lines), deductions total (deductions + "deduction" lines) and balance_due = billed – payment received – party advance.

What is read

- trips, vehicles, parties, vendors, invoice_items

What the user sees next

- Sections for route and parties, rate and bill, charges, deductions, fuel and advance, payment tracking, owner terms for hired trips and the vehicle's RC summary.

30. Money: payments, ledgers, parties and vendors

This chapter holds the most important logic in the app. If you read only one chapter of Part E, read this one together with Figure 6.

30.1 Record money received from a party

A customer pays Rs 50,000. The clerk records it so that the right trips are marked as paid and the ledger balance falls.

Where it starts	Party ledger → "Add Payment" popup (it lists the party's pending trips with tick boxes). Phone: party ledger → payment sheet.
What is sent	POST /payment/party/<party_id>: `date`, `amount`, `mode`, `reference_id`, `remarks`, `trip_ids` (one per ticked trip).
Code entry	`add_party_payment` (6728) → `_insert_party_payment_from_form` (6682).
Phone twin	app_add_party_payment

What happens, step by step

1. The party must exist. `_validated_payment_basics` (6654) requires a valid ISO date, an amount greater than zero and a mode.
2. `_payment_is_duplicate` (6645): the same party, date, amount, mode and reference within the last 10 seconds is treated as a double-click and refused.
3. INSERT one row into payments (payment_type = "received", party_id, amount, mode, reference_id, remarks).
4. `_party_pending_trips` (6543) computes each of the party's trips still owing: pending = billed_amount – payment_received – party_advance, only where pending > 0.004, oldest first.
5. The order of settlement is: the ticked trips that really belong to this party (in ticked order), then every other pending trip oldest first.
6. For each trip in that order: alloc = min(pending, money left). UPDATE trips SET payment_received = payment_received + alloc; INSERT a payment_allocations row (payment_id, trip_id, alloc). Stop when the money is used up.
7. UPDATE payments SET allocated_amount = total allocated. Anything left (a real surplus, e.g. an advance or an opening-balance settlement) stays unallocated and appears as its own ledger line.
8. Commit; redirect to the party ledger.

What is saved, and where

Table	Operation	Columns / values written
`payments`	INSERT, then UPDATE	date, payment_type = received, amount, party_id, mode, reference_id, remarks; then allocated_amount
`payment_allocations`	INSERT (one per trip)	payment_id, trip_id, amount
`trips`	UPDATE (one per settled trip)	payment_received += allocation, updated_by, updated_at

What is read

- parties, trips (pending), payments (duplicate check)

What the user sees next

- Party ledger: new lines "Payment received — Applied to LR ..." (credit) and a lower balance; the Trips list shows a smaller Pending for each settled trip; Dashboard receivables fall.
- Errors come back as ?pay_error=... in a red banner above the ledger.

Protections and edge cases

- Because the same allocation rows feed the trip list and the ledger, the two cannot drift apart.
- A trip that was already paid in full is never over-allocated.

30.2 Record money paid to a vendor

The company pays a truck owner, fuel pump, workshop or any vendor, and the payment settles what is owed to them.

Where it starts	Vendor ledger (or a linked party's merged ledger) → "Add Payment".
What is sent	POST /payment/vendor/<vendor_id>: date, amount, mode, reference_id, remarks, trip_ids.
Code entry	`add_vendor_payment` (6806) → `_insert_vendor_payment_from_form` (6745).
Phone twin	app_add_vendor_payment

What happens, step by step

1. Same validation and duplicate guard as 30.1; INSERT payments with payment_type = "paid" and vendor_id.
2. Step A — owner-hire trips: `_vendor_pending_trips` (6554) lists trips where this vendor is the owner and something is still owed: owed = owner amount – paid_to_owner – fuel/advance the COMPANY paid on his behalf (see _company_paid_on_hired_trip). Ticked trips first, then oldest. UPDATE trips SET paid_to_owner += allocation; INSERT payment_allocations(trip).
3. Step B — unpaid bills: `_vendor_pending_maintenance` (6567) lists maintenance, tyre, battery, service and insurance bills of this vendor where amount – paid_amount > 0. UPDATE maintenance SET paid_amount += allocation; INSERT payment_allocations(maintenance_id).
4. Step C — pending expense bills: `_vendor_pending_overheads` (6577) lists Pending expenses whose vendor text equals this vendor's name, minus what is already allocated. INSERT payment_allocations(overhead_id); `_settle_overhead_status` (6597) flips the expense to Paid once fully covered (and clears its due date).
5. UPDATE payments SET allocated_amount; commit; redirect to the vendor ledger. A surplus stays unallocated.

What is saved, and where

Table	Operation	Columns / values written
`payments`	INSERT, then UPDATE	payment_type = paid, vendor_id, amount, mode, reference_id, remarks; allocated_amount
`payment_allocations`	INSERT (one per target)	payment_id and exactly one of trip_id, maintenance_id, overhead_id; amount
`trips`	UPDATE	paid_to_owner += allocation
`maintenance`	UPDATE	paid_amount += allocation
`overheads`	UPDATE	status → Paid, due_date → NULL when fully allocated

What is read

- vendors, trips, maintenance, overheads, payment_allocations

What the user sees next

- Vendor ledger shows the payment line(s) and a smaller payable; the Maintenance tab shows the bill as paid; Expenses → Pending Bills shrink; the vendor payment popup shows the new pending amounts.

Protections and edge cases

- The database CHECK guarantees an allocation points at exactly one of the three targets.

30.3 View a payment

Opening the detail of one recorded payment.

Where it starts	Ledger line "Payment In/Out" → side panel.

What is sent	GET /payments/<id>/view.
Code entry	`payment_view` (6824)

What happens, step by step

1. Reads the payment, the party or vendor name, and its allocations from three joins (to trips, to maintenance, to overheads).
2. The unallocated leftover is amount – allocated_amount (never negative).

What is read

- payments, parties or vendors, payment_allocations, trips, maintenance, overheads

What the user sees next

- A summary of date, amount, mode, reference, remarks, which LRs / bills / expenses it settled and any leftover, with a link back to the ledger.

30.4 The Ledger list (all parties and vendors)

Who owes what, at a glance.

Where it starts	Sidebar → Ledger (/accounts).
What is sent	GET /accounts with `search`, `role`, `group`, `status`, `page`, `per_page`.
Code entry	`accounts` (1145) → `_accounts_rows` (1109) → `_bulk_balances` (1010).
Phone twin	app_ledger

What happens, step by step

1. Reads all parties and all vendors (vendors that are linked to a party are excluded because they are merged into that party).
2. _bulk_balances computes every balance in a handful of queries: allocation totals per trip / bill / expense, per-payment totals, then trips, payments, maintenance, expenses, invoice items and opening balances grouped in Python — the same arithmetic as the detailed ledger, proven identical by a test.
3. Rows are sorted by name, filtered by search / role / group / status, then totals are computed: receivable (sum of positive balances) and how many parties owe, payable (sum of negative balances) and how many vendors, net.
4. Pagination 10 per page by default.

What is read

- parties, vendors, trips, payments, payment_allocations, maintenance, overheads, invoice_batches, invoice_batch_items, invoice_items

What the user sees next

- Cards for parties, vendors, receivable, payable and net; a table of accounts with a coloured balance (green receivable, red payable) and links to each ledger; tabs All / Parties / Vendors; search and group filters; add-account and Excel export.

Protections and edge cases

- Linked party/vendor pairs appear once, under the party, with the merged balance.

30.5 One party's ledger

The account statement of a customer: opening balance, every trip bill, every payment, running balance.

Where it starts	Ledger list → a party (/ledger/party/<id>).
What is sent	GET with `date_from`, `date_to`, `tab`, `txn_type`, `type`, `page`, `per_page`.

Code entry	`party_ledger` (5997) → `_get_party_ledger_entries` (6093) → `_ledger_page_context` (5963).
Phone twin	app_party_ledger

What happens, step by step

1. Reads the party row and its pending trips (for the payment popup).
2. _get_party_ledger_entries builds the lines: Opening Balance; one "Trip Bill" per trip (debit = billed, credit = the part of payment_received that did NOT come from a ledger payment, plus party_advance); one "Payment In" line per allocation (and one for any unallocated leftover); and, when a vendor is linked, all of that vendor's lines merged in.
3. Lines are sorted by date, a running balance (debit – credit) is added, and the list is reversed so the newest is on top.
4. _ledger_page_context applies date range, transaction type, vehicle type and tab (receivables / payables) filters in Python and paginates.

What is read

- parties, trips, payments, payment_allocations, invoices (for real invoice numbers), maintenance, overheads, invoice items, vendors (linked)

What the user sees next

- Header with contact, GSTIN, credit limit, opening balance and Edit/Opening-balance actions; a table of Date, Detail, Ref, Debit, Credit, Balance; clicking a Trip Bill opens the trip popup, a Payment line opens the payment panel; Add Payment and Excel/PDF export buttons.

Protections and edge cases

- Dates are normalised (a From after a To is swapped).

30.6 Add a party or vendor; edit contact; opening balance; activate/deactivate

Creating the people the business deals with and correcting their details.

Where it starts	Ledger → Add Account; ledger header → Edit contact / Opening balance; list → toggle.
What is sent	POST /accounts/add: `role`, `name`, `contact`, `email`, `address`, `category`, `gstin`, `credit_limit`, `opening_balance`. POST /ledger/<party|vendor>/<id>/contact. POST /ledger/<type>/<id>/opening-balance. POST /accounts/<role>/<id>/toggle-status.
Code entry	`add_account` (1259) → `_create_account_from_form` (1223); `update_contact` (9701); `update_opening_balance` (9739) → `_update_ledger_opening_balance_from_form` (9716); `toggle_account_status` (1266).
Phone twin	app_add_account, app_update_ledger_opening_balance

What happens, step by step

1. Create: a blank name does nothing; credit limit and opening balance go through _parse_money (opening balance may be negative); a nonsense amount creates nothing. role = Vendor inserts into vendors, anything else into parties. A duplicate name hits the UNIQUE constraint and is ignored.
2. Contact edit: UPDATE address, contact, email, credit_limit, gstin, category and status (only Active or Inactive accepted) on the chosen table.
3. Opening balance: requires a number (negative allowed) and an optional valid "as of" date; UPDATE opening_balance, opening_balance_date.
4. Toggle: flips Active ↔ Inactive.

What is saved, and where

Table	Operation	Columns / values written
`parties`	INSERT / UPDATE	name, contact, email, address, credit_limit, opening_balance, category, gstin; later status, opening_balance_date

Table	Operation	Columns / values written
`vendors`	INSERT / UPDATE	same columns for a vendor

What is read

- parties / vendors

What the user sees next

- The new account appears in the Ledger list; the opening balance becomes the first line of its ledger and moves the balance immediately; inactive accounts disappear from entry dropdowns (autocomplete lists only Active records).

Protections and edge cases

- The opening balance is a signed number: positive means they owed the company before the app was used.

31. Expenses

31.1 Add, edit and delete an expense

Recording money spent on things that are not a trip or a maintenance bill (office, rent, fuel for non-trip use, etc.), or a bill still to be paid to a vendor.

Where it starts	Expenses → Add Expense modal; row → Edit / Delete. Phone: + menu → Add Expense.
What is sent	POST /overheads/add or /overheads/edit/<id>: `tab`, `category`, `vendor`, `status`, `date_from`, `date_to`, `date`, `amount`, `payment_mode`, `receipt_number`, `description`, `due_date`, `is_recurring`, `recurring_frequency`, `notes`. POST /overheads/delete/<id>.
Code entry	`add_overhead` (6892) → `_insert_overhead_from_form` (6864); `edit_overhead` (6939) → `_update_overhead_from_form` (6903); `delete_overhead` (7253) → `_delete_overhead_row` (6604).
Phone twin	app_add_overhead, app_edit_overhead

What happens, step by step

1. Add: date, category and a positive amount are required. Status defaults to Paid. If the status is Pending and a vendor name is typed, the vendor is created if new so the bill has a ledger to land in. INSERT into overheads (due date only kept when Pending; recurring frequency only when the recurring box is ticked).
2. Edit: the same field set is rewritten. If ledger payments were already applied to the bill, the vendor cannot be changed and the amount cannot be lowered below what was paid. After the UPDATE, _settle_overhead_status marks it Paid when fully covered.
3. Delete: every payment allocation to this expense is removed and the payment's allocated_amount is reduced by that amount (the payment stays recorded and becomes partly unallocated again); then the expense row is deleted.

What is saved, and where

Table	Operation	Columns / values written
`overheads`	INSERT / UPDATE / DELETE	date, category, amount, notes, payment_mode, receipt_number, vendor, description, status, is_recurring, recurring_frequency, due_date
`vendors`	INSERT (new vendor, Pending only)	name
`payment_allocations`	DELETE (on delete)	rows for this expense
`payments`	UPDATE (on delete)	allocated_amount reduced

What is read

- overheads, payment_allocations

What the user sees next

- The list shows the new row; KPI cards (total, paid, pending, recurring), category and vendor summaries, donut and trend update; a Pending bill with a vendor appears in that vendor's ledger and in the payment popup.
- A refused edit shows ?exp_error=... above the list.

31.2 The Expenses list

Where all expense bills are reviewed.

Where it starts	Sidebar → Expenses (/overheads).
What is sent	GET with category, vendor, status, date range, tab and page filters.
Code entry	`overheads_list` (2705) → `_overhead_list_data` (2590).

Phone twin	app_expenses

What happens, step by step

1. One SELECT over overheads with the chosen filters; an additional per-row sum of allocations from payment_allocations (so a part-paid bill shows what is left).
2. Python builds KPI totals, per-category and per-vendor summaries, the monthly trend and the donut gradient (`_conic_gradient` (2572)).

What is read

- overheads, payment_allocations, vendors

What the user sees next

- Cards, summary tabs, a table of bills with status chips and actions, and charts drawn as CSS/SVG.

32. Maintenance: service, tyres, batteries, urea, tolls, compliance fees and insurance

Every purchase in this family follows one pattern: it creates (or updates) a row in `maintenance` — the bill — and, for kinds that have extra detail, a second row in a specialised table that points back with `maintenance_id`. The bill is what vendor payments settle; the specialised row is what the Tyres, Battery, Urea, Toll or Insurance tab shows.

Kind	Bill row (maintenance.category)	Detail row	Link
Service	category = service type, service_type set	maintenance_items (checklist and item lines)	maintenance_items.maintenance_id
Tyre bought and fitted	Tyres, tyre_action, tyre_id, tyre_brand, tyre_position	none (the bill row is the record)	—
Tyre bought for stock	Tyres, tyre_action = Stock Purchase, no vehicle	tyre_stock (In Stock)	tyre_stock.maintenance_id
Battery	Battery	batteries + battery_checks	batteries.maintenance_id
Urea purchase / direct fill	Urea	urea_transactions (stock_in or direct)	urea_transactions.maintenance_id
Toll	Toll (paid = amount)	toll_entries	toll_entries.maintenance_id
Compliance fee	the compliance type (fitness, permit, PUC, other)	compliance_expenses	compliance_expenses.maintenance_id
Insurance policy	Insurance (amount = premium)	insurance_policies	insurance_policies.maintenance_id

32.1 Add or edit a service bill

A truck goes to a workshop; the bill and what was done are recorded.

Where it starts	Maintenance → Service tab → Add Service (modal with a checklist and repeating item rows). Phone: + → Add Maintenance → Service.
What is sent	POST /maintenance/service/add or /maintenance/service/edit/<id>: `vehicle_no`, `service_type`, `date`, `vendor_name`, `amount`, `paid_amount`, `km_reading`, `next_due_km`, `next_service_date`, `invoice_no`, `invoice_date`, `status`, `notes`, `checklist` and repeating `item_name`, `item_category`, `item_qty`, `item_unit`, `item_rate`.
Code entry	`add_service` (4858) → `_insert_service_from_form` (4820) → `_save_service_items` (4800); `edit_service` (4889) → `_update_service_from_form` (4865).
Phone twin	app_add_service, app_edit_service

What happens, step by step

1. Vehicle, service type, date and total amount are required (the add route redirects to the Service tab either way).
2. The vehicle number is resolved (a new number creates a vehicle row without a type); the workshop is resolved with get_or_create_vendor and is optional.
3. INSERT into maintenance with category and service_type both set to the chosen service type, amount, paid_amount, km, next due km/date, status (default Completed) and the ticked checklist as comma-separated text.
4. _save_service_items deletes the bill's old maintenance_items and inserts one row per named item (qty × rate becomes amount).
5. Edit rewrites the same columns, then _enforce_maintenance_paid_floor lifts paid_amount if ledger payments already applied more.

What is saved, and where

Table	Operation	Columns / values written
`maintenance`	INSERT / UPDATE	date, vehicle_id, category, service_type, amount, paid_amount, vendor_id, notes, km_reading, next_due_km, next_service_date, invoice_no, invoice_date, status, checklist_done
`maintenance_items`	DELETE then INSERT	maintenance_id, item_name, category, qty, unit, rate, amount
`vendors / vehicles`	INSERT if new	name / number

What is read

- vehicles, vendors, payment_allocations (paid floor)

What the user sees next

- The Service tab lists the bill with workshop, amount, paid, balance and status; the Overview cards and charts include it; the workshop's vendor ledger shows a payable for the unpaid part.

32.2 Tyres: buy and fit, buy for stock, fit from stock, scrap, delete

Tyres can be bought and fitted at once, or bought ahead into stock and fitted later.

Where it starts	Maintenance → Tyres tab (vehicle wheel layout diagram, stock table, entries table).
What is sent	POST /maintenance/tyre/add (with optional stock_id), /maintenance/tyre/edit/<id>, /maintenance/tyre/stock/add, /maintenance/tyre/stock/<id>/install, /scrap, /delete.
Code entry	`add_tyre` (4936) → `_insert_tyre_from_form` (4896); `_insert_tyre_stock_from_form` (4998); `_install_tyre_stock_from_form` (5037); `edit_tyre` (4967); `scrap_tyre_stock` (5072); `delete_tyre_stock` (5081).
Phone twin	app_add_tyre, app_add_tyre_stock, app_install_tyre_stock

What happens, step by step

1. Buy and fit: vehicle, action, date, vendor (required) and amount are validated; one maintenance row (category Tyres) holds the cost, supplier, tyre id/brand, position and action.
2. Buy for stock: one maintenance row with no vehicle and tyre_action = Stock Purchase, then one tyre_stock row (status In Stock) linked by maintenance_id.
3. Fit from stock: the SAME maintenance row is updated (vehicle, position, action = New Tyre Fitted, km) and tyre_stock becomes Installed with vehicle, position and date — the supplier is not billed twice. add_tyre with a stock_id does the same.
4. Edit: updates the bill and also mirrors vehicle/position into tyre_stock so the wheel layout (reads maintenance.tyre_position) and the stock table (reads tyre_stock.installed_position) cannot disagree.
5. Scrap: sets a stock tyre to Scrapped. Delete: allowed only for In Stock tyres; removes the stock row then the bill (and its allocations).

What is saved, and where

Table	Operation	Columns / values written
`maintenance`	INSERT / UPDATE / DELETE	date, vehicle_id, category = Tyres, amount, paid_amount, vendor_id, notes, km_reading, invoice_no/date, tyre_action, tyre_id, tyre_brand, tyre_position, status
`tyre_stock`	INSERT / UPDATE / DELETE	maintenance_id, tyre_id, brand, tyre_type, purchase_date, purchase_cost, vendor_id, invoice_no, status (In Stock, Installed, Scrapped), installed_vehicle_id, installed_position, installed_date, notes

What is read

- tyre_stock, vehicles, vendors, payment_allocations

What the user sees next

- The Tyres tab shows KPI cards, the stock table with Fit / Scrap / Delete actions, the entries table and a wheel layout per vehicle (positions depend on the vehicle's wheel count).

32.3 Batteries

A battery is bought, fitted to a truck (or put in stock), checked from time to time and eventually marked dead.

Where it starts	Maintenance → Battery tab.
What is sent	POST /maintenance/battery/add (mode install or stock), /<id>/install, /<id>/check, /<id>/dead, /<id>/delete.
Code entry	`add_battery` (5163) → `_insert_battery_from_form` (5093); `install_battery` (5199); `check_battery` (5207); `mark_battery_dead` (5226); `delete_battery` (5239).
Phone twin	app_add_battery, app_install_battery

What happens, step by step

1. Add: brand, capacity (Ah), type, purchase date, warranty months, price and vendor are required (plus vehicle and installation date in install mode). A bill goes into maintenance (category Battery); the battery gets the next number from _next_battery_no; a batteries row stores the details.
2. Two history rows are written to battery_checks: "Purchased" and, when fitted, "Installed in <vehicle>" with the health, voltage and temperature typed.
3. Install later: updates batteries (vehicle, location, install date, last checked), the linked maintenance row's vehicle, and adds a history row.
4. Health check: updates the battery's health, voltage, temperature and last-checked date and adds a "Health Check" history row. Mark dead: status_override = Dead plus a "Marked Dead" row.
5. Delete: only for unfitted batteries; deletes checks, the battery and its bill.
6. Status shown to the user is derived: Dead/override wins, In Stock if no vehicle, else Good ≥70% health, Weak ≥40%, Replace Soon below.

What is saved, and where

Table	Operation	Columns / values written
`maintenance`	INSERT / UPDATE / DELETE	category = Battery, amount = price, paid_amount, vendor_id, vehicle_id, invoice_no
`batteries`	INSERT / UPDATE / DELETE	battery_no, brand, model, capacity_ah, battery_type, voltage_rating, serial_no, vehicle_id, installed_location, install_date, purchase_date, purchase_price, vendor_id, invoice_no, warranty_months, maintenance_id, health_pct, voltage, temp_c, last_checked_date, status_override, notes
`battery_checks`	INSERT / DELETE	battery_id, date, event, health_pct, voltage, temp_c, remarks

What is read

- batteries, battery_checks, vehicles, vendors

What the user sees next

- The Battery tab: cards (fitted, in stock, weak soon, dead, average health), a table with warranty end date and status, and a history drawer per battery.

32.4 Urea (AdBlue)

Urea is bought into a tank (stock in), drawn from stock into a truck (stock out) or bought and poured directly (direct).

Where it starts	Maintenance → Urea tab → Add Urea (one modal, three modes).

What is sent	POST /maintenance/urea/add with mode = stock_in, stock_out or direct; fields `date`, `quantity_l`, `unit_price`, `paid_amount`, `supplier_name`, `invoice_no`, `batch_no`, `vehicle_no`, `location`, `odometer_km`, `notes`. Edit/Delete: /maintenance/urea/<id>/edit, /delete.
Code entry	`add_urea` (5262) → `_insert_urea_from_form` (3711); `edit_urea` (5282); `delete_urea` (5310).
Phone twin	app_add_urea

What happens, step by step

1. Quantity must be positive. "Price" is the real total paid (it is not multiplied by quantity).
2. stock_in: a maintenance row (category Urea, no vehicle) holds the cost and a urea_transactions row (txn_type stock_in, source stock) records litres, batch, supplier, invoice and location.
3. stock_out (from stock): no cost row (already paid). First the current stock is computed from the whole ledger (purchases minus stock-sourced uses) rounded to 0.01 L; asking for more than is on hand is refused with "insufficient stock" and the available litres. Otherwise one urea_transactions row (stock_out, source stock) with the vehicle and odometer.
4. direct: a maintenance row for the vehicle plus a urea_transactions row (stock_out, source direct) with the supplier and cost.
5. Edit updates the transaction and keeps the linked bill's date, amount and notes equal; delete removes the transaction and its bill.

What is saved, and where

Table	Operation	Columns / values written
`urea_transactions`	INSERT / UPDATE / DELETE	date, txn_type, source, batch_no, supplier_id, vehicle_id, invoice_no, quantity_l, unit_price, total_value, location, odometer_km, notes, maintenance_id
`maintenance`	INSERT / UPDATE / DELETE	category = Urea, date, vehicle_id, amount, paid_amount, vendor_id, notes, invoice_no

What is read

- urea_transactions (stock check), vehicles, vendors

What the user sees next

- The Urea tab shows current stock, today/month use, average consumption, low-stock alerts, a running-balance ledger and a trend; a refused stock-out shows "insufficient stock".

Protections and edge cases

- Stock can never go negative.

32.5 Tolls: manual entry, Excel import, approve / reject, FASTag sync

Toll spend is added by hand, imported from a BlackBuck BOSS statement (or the template), and approved.

Where it starts	Maintenance → Toll tab; phone: + → Add Maintenance → Toll.
What is sent	POST /maintenance/toll/add (with a receipt file), /maintenance/toll/excel/preview (file), /maintenance/toll/excel/import (token + options), /<id>/approve, /<id>/reject, /<id>/delete, /maintenance/toll/sync-fastag.
Code entry	`add_toll` (5366) → `_insert_toll_from_form` (5322); `toll_excel_preview` (5726) → `_parse_toll_excel` (5546); `toll_excel_import` (5763); `approve_toll` (5447); `reject_toll` (5456); `sync_fastag_toll` (5465).
Phone twin	app_add_toll

What happens, step by step

1. Manual: the vehicle must be an OWN vehicle. An optional trip LR ("LR | vehicle | date") is looked up to set trip_id. A receipt file is saved by _save_toll_receipt under uploads/toll_receipts/<company_id>/. One maintenance row

(category Toll, paid = amount) and one toll_entries row (source manual → status pending, or fastag → synced).

2. Excel preview: the file is parsed (auto-detecting a BOSS statement, where only debit FasTag Recharge lines count, or the plain template). The parsed rows are stashed in memory under a random token (kept 30 minutes) and a summary is shown: new rows, duplicates (by transaction id), changed duplicates, invalid rows, total amount. Nothing is written yet.
3. Excel import: for each valid row a maintenance row and a toll_entries row are inserted (status synced for fastag, otherwise approved if "auto approve" was ticked, else pending); a duplicate is never inserted again — a changed duplicate may overwrite the existing record only if "override changed" was ticked; "auto-link trips" matches trip_no to a trip's LR.
4. Approve and Reject only change status from pending. FASTag sync is a MOCK generator (no live feed): it adds a small deterministic batch at most once per hour.

What is saved, and where

Table	Operation	Columns / values written
`toll_entries`	INSERT / UPDATE / DELETE	date, time, vehicle_id, trip_id, toll_plaza, highway, state, amount, source, payment_mode, status, reference_no, receipt_path, notes, maintenance_id
`maintenance`	INSERT / UPDATE / DELETE	category = Toll, date, vehicle_id, amount, paid_amount = amount, notes

What is read

- vehicles (own only), trips (LR lookup), toll_entries (duplicates)

What the user sees next

- The Toll tab lists entries with status chips, receipt links, totals per vehicle and an import wizard summary. Linked tolls reduce a trip's own toll estimate in reports (a real linked toll replaces the typed estimate, never added to it).

Protections and edge cases

- The in-memory import stash is per worker and expires; if a different worker answers the second step the user simply repeats the preview.

32.6 Compliance fees, insurance policies, generic bills

Fitness / permit / PUC fees, insurance premiums and any other maintenance bill.

Where it starts	Maintenance → Compliance tab; Vehicles → Insurance tab; Maintenance → Add (generic).
What is sent	POST /maintenance/compliance/add; /maintenance/insurance/add, /<id>/edit, /<id>/delete (with up to three files: policy_doc, invoice_doc, rc_doc); /maintenance/add; /maintenance/edit/<id>; /maintenance/delete/<id>.
Code entry	`add_compliance_expense` (5427) → `_insert_compliance_expense_from_form` (5385); `add_insurance` (5825); `edit_insurance` (5872); `delete_insurance` (5915); `add_maintenance` (4782); `edit_maintenance` (7052); `delete_maintenance` (7040).
Phone twin	app_add_compliance

What happens, step by step

1. Compliance fee: the vehicle must be an own vehicle. A maintenance row (category = the compliance type) and a compliance_expenses row are written; paid amount is separate from amount, so a blank paid amount leaves it payable to the vendor.
2. Insurance add: insurer becomes a vendor; a maintenance row (category Insurance, amount = premium, paid separate) is the bill; three uploaded documents are saved under uploads/insurance/<company_id>/; one insurance_policies row stores type, policy number, dates, premium, IDV, NCB, GST flag, agent details, reminder days, notes, optional Cancelled override and document paths; the vehicle's insurance_expiry is updated.
3. Insurance edit: rewrites the policy (keeping old document paths when no new file is sent), updates the linked bill and re-applies the paid floor; delete removes the policy and its bill.

4. Generic add/edit: a plain maintenance row with a free category; delete removes the row, its payment allocations and its checklist items.

What is saved, and where

Table	Operation	Columns / values written
`compliance_expenses`	INSERT / DELETE	date, vehicle_id, compliance_type, description, vendor_id, amount, payment_mode, maintenance_id, notes
`insurance_policies`	INSERT / UPDATE / DELETE	vehicle_id, insurance_type, insurer_id, policy_number, start_date, expiry_date, premium_amount, idv, ncb_pct, gst_included, agent_name, agent_contact, agent_email, reminder_days, notes, status_override, policy_doc_path, invoice_doc_path, rc_doc_path, maintenance_id
`maintenance`	INSERT / UPDATE / DELETE	the bill for each of the above
`vehicles`	UPDATE	insurance_expiry

What is read

- vehicles, vendors, insurance_policies

What the user sees next

- Insurance policies show Active / Expiring Soon (within 30 days) / Expired from their dates (Cancelled by assertion); the Vehicles page badges and the bell use the same dates; premiums appear in the insurer's ledger.

32.7 The Maintenance hub (how everything above is shown)

One page with tabs.

Where it starts	Sidebar → Maintenance (/maintenance?tab=overview|service|tyres|battery|urea|toll|compliance-expenses).
What is sent	GET /maintenance with tab and each tab's own filters (vehicle, vendor, dates, search, status, action, position …).
Code entry	`maintenance_list` (3409) → the tab function for the chosen tab (for example `_maintenance_service_tab` (4706), `_maintenance_tyres_tab` (4282), `_maintenance_battery_tab` (4506), `_maintenance_urea_tab` (3795), `_maintenance_toll_tab` (4032), `_maintenance_compliance_tab` (4157), `_maintenance_overview_tab` (3576)).
Phone twin	app_maintenance and one phone page per tab

What happens, step by step

1. Each tab calls its own _X_data function (shared with the phone) which runs filtered SELECTs over maintenance joined to vehicles and vendors plus its specialised table, then computes cards and chart series in Python.
2. The Overview covers OWN vehicles only (hired-vehicle bills are excluded by design) and uses _maintenance_classify so an entry lands in the same bucket everywhere.

What is read

- all of: `batteries`, `battery_checks`, `compliance_expenses`, `maintenance`, `maintenance_items`, `parties`, `settings`, `toll_entries`, `trips`, `tyre_stock`, `urea_transactions`, `vehicles`, `vendors`

What the user sees next

- KPI cards, charts drawn as SVG/CSS, tables with Add / Edit / Delete buttons and modals; the filter panel keeps its values in the address.

33. Vehicles and their documents

33.1 Add, edit, activate and delete a vehicle

Registering a truck (own or hired).

Where it starts	Vehicles → Add Vehicle (modal) or full Edit page; phone Add Vehicle.
What is sent	POST /vehicles/add (also used to update by number), /vehicles/edit/<id>, /vehicles/toggle_status/<id>, /vehicles/delete/<id>. Fields: `vehicle_no`, `type`, `body_type`, `status`, `registration_date`, `capacity_mt`, `chassis_number`, `engine_number`, `fitness_expiry`, `puc_valid_upto`, `permit_valid_upto`, `notes`.
Code entry	`add_vehicle` (7343) → `_insert_vehicle_from_form` (7295) → `_mirror_vehicle_compliance_from_form` (7260); `edit_vehicle` (7594); `toggle_vehicle_status` (7627); `delete_vehicle` (7642).
Phone twin	app_add_vehicle

What happens, step by step

1. Add requires a vehicle number. If that number already exists the existing row is UPDATED (so re-adding is an update), otherwise a row is inserted.
2. insurance_expiry is deliberately never written here: the Insurance tab (insurance_policies) is its only source.
3. For OWN vehicles the three dates (fitness, PUC, permit) are mirrored into vehicle_compliance with source manual and sync_status "Not Synced", so the compliance badges and the detail page agree with the form.
4. Delete runs a bare DELETE on the vehicle row.

What is saved, and where

Table	Operation	Columns / values written
`vehicles`	INSERT / UPDATE / DELETE	vehicle_no, type, registration_date, capacity_mt, fitness_expiry, puc_valid_upto, permit_valid_upto, status, body_type, chassis_number, engine_number, notes
`vehicle_compliance`	INSERT / UPDATE	vehicle_id, compliance_type (fitness, puc, permit), valid_upto, source = manual, sync_status

What is read

- vehicles, vehicle_compliance

What the user sees next

- The vehicle appears in lists, autocomplete (active ones), the trip form's vehicle-type map and Fleet Utilization (own only).

Protections and edge cases

- Observed gap: delete does not check for trips, bills or documents that still point at the vehicle, and foreign keys are not enforced, so such records would be left pointing at a missing vehicle.

33.2 Record a renewal; view RC details; refresh from the outside

Keeping the documents and challans current.

Where it starts	Vehicle detail / Vehicles list buttons.
What is sent	POST /vehicles/<id>/compliance/<fitness|puc|permit>/renew; GET /vehicles/<id>/rc-lookup; POST /vehicles/<id>/sync-now; POST /vehicles/<id>/compliance/sync; POST /vehicles/compliance/sync-all.
Code entry	`vehicle_compliance_renew` (7650); `vehicle_rc_lookup` (1683); `vehicle_sync_now` (1961); `vehicle_compliance_sync` (7698); `vehicles_compliance_sync_all` (1673).

What happens, step by step

1. Renewal: inserts or updates the vehicle_compliance row for that type with document number, authority, (permit subtype), valid-from/upto, source = manual; a manual entry does not overwrite whatever the last sync found.
2. RC view: reads vehicles.rc_synced_data (JSON cached from the last sync) and shows it; no live call is made.
3. Sync now (own vehicles only; skipped for hired): if an RC key is saved, calls eChallan for RC data → UPDATE vehicles.rc_synced_data / rc_last_synced and back-fill gaps (registration date, chassis, engine, expiry dates, body, capacity); then calls the challan lookup → DELETE the vehicle's vehicle_challans and INSERT the fresh set (33 columns), then UPDATE challan_count, challan_amount, challan_last_synced.
4. Compliance sync (single or all): runs the provider classes (mock until a key exists) and updates vehicle_compliance statuses.

What is saved, and where

Table	Operation	Columns / values written
`vehicle_compliance`	INSERT / UPDATE	document_number, issuing_authority, permit_subtype, valid_from, valid_upto, source, provider_name, last_sync_time, sync_status, remarks
`vehicles`	UPDATE	rc_synced_data, rc_last_synced, challan_count, challan_amount, challan_last_synced and back-filled spec/date columns
`vehicle_challans`	DELETE then INSERT	the full challan record per challan

What is read

- settings (rc_lookup_api_key), vehicles, vehicle_compliance

What the user sees next

- Vehicles page badges (Insurance, Fitness, PUC, Permit) turn red / amber / green; the bell and Vehicles menu badge count vehicles with pending challans; the eChallan tab lists each challan.

Protections and edge cases

- Every sync can spend paid credits, so none runs unless a person presses a button or the opt-in background job is enabled.

33.3 The Vehicles list and a vehicle's detail page

Fleet overview and one truck in depth.

Where it starts	Sidebar → Vehicles (/vehicles) and a vehicle (/vehicles/<id>).
What is sent	GET /vehicles?tab=all|insurance|permit …; GET /vehicles/<id>?tab=….
Code entry	`vehicles_list` (2049); `vehicle_detail` (1944) → `_vehicle_detail_data` (1853).
Phone twin	app_vehicles, app_vehicle_detail

What happens, step by step

1. The list reads vehicles with their trips counts and the date fields; Python assigns each date a bucket (expired / expiring within 30 days / ok) with _expiry_bucket and computes KPI cards.
2. The detail reads the vehicle, its insurance policies, maintenance history, challans and settings.

What is read

- vehicles list: `trips`, `vehicles`
- vehicle detail: `insurance_policies`, `maintenance`, `settings`, `vehicle_challans`, `vehicles`, `vendors`

What the user sees next

- The list: cards Total Vehicles, Active, In Maintenance, Insurance / PUC / Permit Expiring; tabs Overview, All vehicles, Insurance, eChallan; a table with document badges and quick actions. The detail page: vehicle facts (type, model, chassis, engine, registration date, age) and tabs Insurance, Permit, Fitness, PUC, eChallan and History.

34. People: employees, salaries, advances and attendance

34.1 Add or edit an employee

A driver or an office person joins.

Where it starts	Salaries → Add Employee modal; employee card → Edit.
What is sent	POST /employees/add and /employees/<id>/edit: `name`, `type`, `employee_code`, `role`, `mobile`, `email`, `address`, `joining_date`, `date_of_birth`, `bank_account`, `ifsc_code`, `upi_id`, `emergency_contact`, `aadhaar`, `pan`, `driving_license`, `basic_salary`.
Code entry	`add_employee` (8215) → `_insert_employee_from_form` (8177); `edit_employee` (7103); `deactivate_employee` (7120).
Phone twin	app_add_employee

What happens, step by step

1. Name and type (Driver or Staff) are required; basic salary is parsed. If an employee with that name already exists (case-insensitive) nothing new is created.
2. INSERT into employees with status Active; when no employee code was typed one is made (DR-001 for drivers, ST-001 for staff, from the new id).
3. Edit rewrites the personal and pay columns (it does not touch opening balances). Deactivate flips Active ↔ Inactive.

What is saved, and where

Table	Operation	Columns / values written
`employees`	INSERT / UPDATE	name, type, employee_code, role, mobile, email, address, joining_date, date_of_birth, bank_account, ifsc_code, upi_id, emergency_contact, aadhaar, pan, driving_license, status, basic_salary

What is read

- employees (duplicate name check)

What the user sees next

- The employee card and table appear on the Overview tab with attendance for today, advance outstanding and salary summary.

Protections and edge cases

- Observed gap: salaries and advances are matched to an employee by NAME. Renaming an employee through Edit does not rewrite the old name stored in earlier salary and advance rows, so those rows no longer match the renamed person.

34.2 Pay a salary, give an advance, record a repayment

Money moves between the company and an employee.

Where it starts	Employee ledger (/employee/<name>) → entry form, or a drawer on the Salaries page.
What is sent	POST /employee/<name> with entry_kind = salary, given or repaid, date, amount, payment_mode, notes. Also POST /employees/<id>/advance/add and POST /advances/add.
Code entry	`employee_ledger` (7812) → `_insert_employee_payment_from_form` (7770); `add_employee_advance` (7133); `add_advance` (7578).
Phone twin	app_employee_ledger, app_add_employee_payment

What happens, step by step

1. The entry kind, a valid date and a positive amount are required; the employee must exist.

2. salary: one row into salaries — month label (e.g. Mar 2026) and month_key (2026-03) from the date, gross = net = amount, no deductions, payment_status paid, payment date and mode, employee_id, basic salary from the employee record (or the amount).
3. given / repaid: one row into advances (employee name, date, amount, type, notes).

What is saved, and where

Table	Operation	Columns / values written
`salaries`	INSERT	employee, month, amount, date, employee_id, month_key, basic_salary, gross_salary, total_deductions = 0, advance_recovery = 0, net_salary, payment_status = paid, payment_date, payment_mode, remarks
`advances`	INSERT	employee, date, amount, type (given or repaid), notes

What is read

- employees

What the user sees next

- The employee ledger lists Opening Balance, Salary Paid, Advance Given and Advance Repaid with a running ADVANCE balance computed over the whole history (a date filter hides rows but never changes the balance). Salary payments do not change the advance balance.

34.3 Attendance and payslips

Marking who came to work and printing a payslip.

Where it starts	Salaries → Attendance tab (calendar and bulk toolbar); salary row → Payslip.
What is sent	POST /attendance/mark (employee_id, date, status, in_time, out_time, remarks); POST /attendance/bulk-mark (employee_ids, date, status); GET /salaries/<id>/payslip; POST /salaries/delete/<id>.
Code entry	`mark_attendance` (7157); `bulk_mark_attendance` (7176); `download_payslip` (7195); `delete_salary` (7094).
Phone twin	app_salary_attendance

What happens, step by step

1. Attendance is one row per employee per date: an INSERT ... ON CONFLICT(employee_id, date) DO UPDATE, so marking the same day again changes the status instead of adding a second row. Allowed statuses: Present, Absent, Leave, Half Day.
2. Payslip: reads the salary, the employee, its allowance/deduction items and the company settings and draws a PDF with ReportLab.
3. Delete salary removes its salary_items and the salary row.

What is saved, and where

Table	Operation	Columns / values written
`attendance`	INSERT or UPDATE	employee_id, date, status, in_time, out_time, remarks, marked_by
`salaries / salary_items`	DELETE (on delete)	the rows

What is read

- employees, attendance, salaries, salary_items, settings

What the user sees next

- The attendance calendar for the month, with per-employee present/absent counts; the payslip PDF downloads.

34.4 The Employees hub

Overview, Salary and Attendance in one page.

Where it starts	Sidebar → Salaries (/salaries?tab=overview|salary|attendance).
What is sent	GET with tab, dates, month, search.
Code entry	`salaries_list` (2283) → `_employees_overview_tab` (2292) / `_employees_salary_tab` (2440) / `_employees_attendance_tab` (2465).
Phone twin	app_salary

What happens, step by step

1. Overview: employees, today's attendance, per-person outstanding advance (`_employee_outstanding_advance` (2226)) and month calendar; Salary: paid-this-period totals and payments table; Attendance: month grid.

What is read

- `advances`, `attendance`, `employees`, `salaries`

What the user sees next

- KPI cards (employees, drivers, staff, present today, outstanding advances), employee cards, tables and drawers.

35. Invoices

35.1 Choose trips in the Invoice Center

Billing day: the clerk chooses a party (or an owner), a date range and the trips to put on one invoice.

Where it starts	Sidebar → Invoices (/invoice-center).
What is sent	GET /invoice-center with invoice_type, party_id, vendor_id, vehicle, lr_number, date_from, date_to, invoice_status.
Code entry	`invoice_center` (8248)

What happens, step by step

1. Reads active parties and vendors (plus parties not linked to a vendor, offered as possible owners for freight bills; picking one creates its vendor row on the fly — the only write on this page).
2. Reads matching trips with their already-invoiced status (from invoice_batch_trips), their extra lines (invoice_items) and tolls, and the invoice settings.

What is saved, and where

Table	Operation	Columns / values written
`vendors`	INSERT (rare)	name, when a party-only owner is picked

What is read

- `invoice_batch_trips`, `invoice_items`, `parties`, `settings`, `toll_entries`, `trips`, `vehicles`, `vendors`

What the user sees next

- A seven-step page: 1 Select invoice type, 2 Select party or owner, 3 Filter trips, 4 Select trips to include (tick boxes and an LR link per row), 5 Invoice summary, 6 Invoice preview, 7 Actions (Preview, Save as Draft, Generate); charge and deduction toggles per trip.

35.2 Review, then generate, draft or preview

Producing the invoice.

Where it starts	Invoice Center buttons.
What is sent	POST /invoice-center/review (to show the review screen) then POST /invoice-center/generate with mode = generate | draft | preview, trip_ids, reviewed_trip_ids, invoice_type, party_id / vendor_id, invoice_date, due_date, payment_terms, place_of_supply, remarks, payment_status, loading_charges, other_charges, tds_rate, gst_rate, excluded_charge_keys, included_charge_keys, extra_item_desc / extra_item_amount / extra_item_type.
Code entry	`invoice_center_review` (8412); `invoice_center_generate` (9172) → `_build_invoice_pdf` (8550).

What happens, step by step

1. For generate and draft the server checks that the LR link of EVERY selected trip was opened (reviewed_trip_ids); otherwise it redirects back with a message and a balloon at the LR column. Preview skips this check.
2. _invoice_numbers_error validates the numeric fields; money fields go through _parse_money.
3. Trips are loaded; their invoice_items are filtered by the unchecked charge keys; extra lines typed into the form are added.
4. The entity (party or owner vendor) is loaded; for a tax invoice the GST rate comes from the form or the CGST+SGST settings and is split in two.
5. The invoice number is prefix + current year + next number (e.g. ATS/INV/2026/0004).
6. Preview: builds the PDF and returns it without writing anything or consuming a number.
7. Draft / Generate: INSERT invoice_batches (status draft or generated), one invoice_batch_trips row per trip, one invoice_batch_items row per typed extra line, the charge choices into invoice_batch_charge_exclusions, then writes settings.next_invoice_number = next + 1. Commit.

8. Draft redirects to Generated Invoices; Generate returns the PDF as a download.

What is saved, and where

Table	Operation	Columns / values written
`invoice_batches`	INSERT	invoice_number, invoice_type, party_id, vendor_id, invoice_date, due_date, payment_terms, place_of_supply, remarks, gst_rate, tds_rate, loading_charges, other_charges, status, payment_status, created_by, created_at
`invoice_batch_trips`	INSERT (one per trip)	invoice_batch_id, trip_id
`invoice_batch_items`	INSERT	invoice_batch_id, description, amount, item_type
`invoice_batch_charge_exclusions`	INSERT	invoice_batch_id, trip_id, charge_key (a leading + means opted in)
`settings`	INSERT OR REPLACE	next_invoice_number

What is read

- trips, vehicles, parties or vendors, invoice_items, toll_entries, settings

What the user sees next

- A PDF with the company letterhead and logo, party details, one row per trip, charges, GST/TDS/round-off, amount in words, bank details and the RCM clause (tax invoices).

Protections and edge cases

- Saving requires the review gate; the number counter moves only on a saved invoice, so previews never burn numbers.

35.3 Generated invoices: list, edit, add/remove lines, PDF, delete; per-trip invoices

After saving, an invoice can be reopened.

Where it starts	Invoices → Generated Invoices (/invoices/generated).
What is sent	GET /invoices/generated; GET/POST /invoices/generated/<id>/edit; POST .../items/add; POST /invoices/generated/items/<id>/delete; GET .../pdf and .../pdf/preview; POST .../delete. Older per-trip invoices: /invoices, /invoices/<trip>, /invoices/<trip>/edit, .../items/add, /invoices/items/<id>/delete, /invoices/<trip>/pdf.
Code entry	`invoice_batches_list` (9306); `invoice_batch_edit` (9329); `invoice_batch_item_add` (9364); `invoice_batch_item_delete` (9381); `invoice_batch_pdf` (9400); `invoice_batch_delete` (9390).

What happens, step by step

1. The PDF is regenerated from the stored batch every time: trips from invoice_batch_trips, lines from invoice_batch_items and invoice_items, the saved charge choices from invoice_batch_charge_exclusions and the current company settings.
2. Delete removes the batch's trip links, lines and the batch itself.

What is saved, and where

Table	Operation	Columns / values written
`invoice_batches`	UPDATE / DELETE	dates, terms, remarks, status, payment_status …
`invoice_batch_items`	INSERT / DELETE	description, amount, item_type, vendor_id
`invoice_batch_trips`	DELETE (on delete)	the links

What is read

- `invoice_batch_charge_exclusions`, `invoice_batch_items`, `invoice_batch_trips`, `invoice_batches`, `invoice_items`, `parties`, `settings`, `toll_entries`, `trips`, `vehicles`, `vendors`

What the user sees next

- The list shows number, type, party/vendor, date, trip count, status and payment status with PDF, Edit and Delete.

Protections and edge cases

- Observed gap: deleting an invoice batch does not delete its invoice_batch_charge_exclusions rows, which are left behind unused.
- An invoice's total can differ from the ledger's billed amount when optional lines are included (a documented business decision).

36. Reports and exports: how the numbers are retrieved

All of these only read. They share the profit rule in Chapter 18. For each, the real tables read (found by analysing the code) are listed.

36.1 Dashboard

Where it starts	Sidebar → Dashboard.
What is sent	GET /dashboard with financial year, From, To, vehicle.
Code entry	`dashboard` (1371)
Phone twin	app_dashboard

What happens, step by step

1. _dashboard_financials_data loads the period's trips (optionally one vehicle) and totals revenue, own-vehicle revenue, hired-vehicle profit, fuel and maintenance cost, profit and the expense breakdown with the same own/hired rule as everywhere.
2. Receivables and payables come from _accounts_rows (the bulk ledger engine). A freight trend (this period versus last), recent activities (latest trips, payments, salaries, invoices, expenses), trip-status split and top vehicles are built in Python.

What is read

- `invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `trips`, `vehicle_compliance`, `vehicles`, `vendors`

What the user sees next

- Seven KPI cards, a freight revenue line chart, expense breakdown bars, recent activities, a trip-status donut, payables and receivables, top vehicles table.

36.2 Business Performance

Where it starts	Sidebar → Business Performance.
What is sent	GET with date_from, date_to (default: last full month).
Code entry	`business_performance` (11908)
Phone twin	app_business_performance

What happens, step by step

1. _period_financials is evaluated for the chosen period, the previous period, the quarter, the previous quarter and the previous year; growth percentages come from _pct_growth.

What is read

- `employees`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `toll_entries`, `trips`, `vehicles`, `vendors`

What the user sees next

- Revenue, costs, profit, receivable/payable and growth versus earlier periods, with an Excel export.

36.3 Performance (drivers and vehicles)

Where it starts	Sidebar → Performance (driver_performance redirects here).
What is sent	GET with period and dates.
Code entry	`performance` (8057)
Phone twin	app_performance

What happens, step by step

1. _performance_data groups trips by driver and by vehicle: trips, km, revenue, fuel and the full cost basis; profit = billed – cost (own) or billed – owner cost (hired); toll comes from linked toll entries else the trip estimate; trend lines per month.

What is read

- `maintenance`, `toll_entries`, `trips`, `vehicles`

What the user sees next

- Ranked tables of drivers and vehicles with revenue, cost, profit and trend charts.

36.4 Route Analytics

Where it starts	Sidebar → Route Analytics.
What is sent	GET with tab (best routes, route summary, route rates, finder), dates, from/to, vehicle type.
Code entry	`route_analytics` (11291)
Phone twin	app_analytics

What happens, step by step

1. _route_analytics_data groups trips by cleaned from/to place names (_clean_loc) and computes revenue, cost, margin and average rate; _route_rates_data finds lowest, average and highest rate per route and vehicle type.

What is read

- `toll_entries`, `trips`, `vehicles`

What the user sees next

- Top routes by margin and revenue, a route table, rate cards and the Route Finder tab.

36.5 Fleet Utilization

Where it starts	Sidebar → Fleet Utilization (idle-tracker and empty-runs redirect here).
What is sent	GET with dates, vehicle, period.
Code entry	`fleet_utilization` (11700)
Phone twin	app_fleet_utilization

What happens, step by step

1. _fleet_utilization_data counts, per OWN vehicle, the days covered by the union of each trip's start..end span (active) and the rest of the window (idle, starting no earlier than the registration date), empty runs, kilometres, fuel and efficiency.

What is read

- `toll_entries`, `trips`, `vehicles`

What the user sees next

- Utilization percentage per vehicle, idle trend chart, empty-run list and totals.

36.6 Exports

Taking data out as Excel or PDF.

Where it starts	Export buttons on list pages.
What is sent	GET /trips/export, /trips/export/pdf, /maintenance/export(/pdf), /overheads/export, /accounts/export, /ledger/party|vendor/<id>/export(/pdf), /performance/export, /route-analytics/export, /fleet-utilization/export, /business-performance/export, /settings/users/export, /export.
Code entry	`export_trips` (7353), `export_trips_pdf` (7401), `export_maintenance` (7468), `export_overheads` (2826), `export_accounts` (1195), `export_party_ledger` (6390), `export_excel` (12354) and the others.

What happens, step by step

1. Each export runs the same data function and filters as its screen and writes the result to an in-memory Excel workbook (openpyxl) or PDF (ReportLab), then returns it as a download.

What is read

- the same tables as the matching screen; the whole-database export reads `maintenance`, `parties`, `payments`, `trips`, `vehicles`, `vendors`

What the user sees next

- A file download (trips_export.pdf, ledger PDF, users.xlsx, and so on).

Protections and edge cases

- The whole-database Excel is refused for Limited Access users. Large PDFs of all trips take several seconds (about 5.8 s for 12,000 trips in the load test).

37. Administration: settings, users and your own profile

37.1 Save company settings (company, banking, invoice, tax, SMS, keys, sync)

The Administrator changes how the company appears on invoices, the GST rates, the bank account, the SMS and vehicle-data keys and the sync schedule.

Where it starts	Website: Profile and Settings (/settings) tabs Company Profile, Invoice Settings, Tax and Compliance, Users and Roles (the SMS, RC key and sync cards are inside it). Phone: Profile and Settings hub → five screens.
What is sent	Website: POST /settings?tab=… with form_type = company | banking | invoice_prefs | invoice_numbering | tax | rcm | sms | rc_lookup | sync_schedule and that group's fields. Phone: POST /app/settings/save with the same form_type values (JSON answer).
Code entry	`settings_page` (10636) and `app_settings_save` (14693) → `_settings_form_error` (10579) (phone only) → `_save_settings_group` (10552).
Phone twin	app_settings_company, app_settings_banking, app_settings_invoice, app_settings_tax, app_settings_users, app_settings_save

What happens, step by step

1. Only the Administrator may reach the routes (a Manager gets "Administrators only" / HTTP 403; a non-administrator who opens /settings is shown "My Profile" instead).
2. Phone only: validation first — company name required, valid email, GST/TDS between 0 and 100, round-off and due-day limits, next invoice number at least 1, sync days at least 1, and MSG91 key and template both present. Blank secret boxes are replaced with the saved value before validating.
3. _save_settings_group loops over the group's keys. Checkbox keys (show logo, show bank details, show signatory, print amount in words, reverse charge, RCM on transport, TDS applicable, e-way bill mandatory) are stored as 1 or 0 depending on whether the box was ticked. Every key is written with INSERT OR REPLACE into settings with the company id, so a missing row can never make a save silently do nothing.
4. Secret keys (msg91_authkey, rc_lookup_api_key) are never shown again; a blank box keeps the stored one.

What is saved, and where

Table	Operation	Columns / values written
`settings`	INSERT OR REPLACE (one row per key)	company: company_name, address, state, phone, email, gstin, pan, business_type, company_since, currency · banking: bank_name, account_holder, account_number, ifsc_code, branch, account_type · invoice_prefs: default_invoice_type, default_payment_terms, default_place_of_supply, default_due_days, show_company_logo, show_bank_details, show_signatory, print_amount_words · invoice_numbering: invoice_prefix, next_invoice_number · tax: cgst_rate, sgst_rate, igst_rate, reverse_charge_applicable, rcm_on_transport, tds_applicable, tds_rate_default, eway_bill_mandatory, round_off_limit · rcm: rcm_clause · sms: msg91_authkey, msg91_template_id · rc_lookup: rc_lookup_api_key · sync_schedule: rc_sync_interval_days, challan_sync_interval_days; each with company_id, created_by/at, updated_by/at

What is read

- settings, users, access_logs, sync_log (for the Users and Roles screen)

What the user sees next

- The next invoice, the company name in the sidebar and PDFs, the GST split on tax invoices and the sync cadence use the new values immediately; the phone reloads the screen with a "Saved" toast.

Protections and edge cases

- Invoice numbering: invoice_center_generate reads next_invoice_number, uses it and writes number + 1; editing it here changes where numbering continues.

37.2 Upload the company logo

The picture printed top-left on every invoice PDF and used as the browser tab icon.

Where it starts	Company Profile → Upload/Change Logo.
What is sent	POST /settings/logo/upload with a file named logo (PNG, JPG, GIF or WEBP).
Code entry	`upload_company_logo` (8511)

What happens, step by step

1. An allowed extension is required (otherwise a clear message in the app, nothing saved).
2. The file is saved as logo_<millisecond timestamp>.<ext> under static/uploads/logo/<company_id>/; the relative path is written to settings.logo_path.

What is saved, and where

Table	Operation	Columns / values written
`settings`	INSERT OR REPLACE	logo_path = uploads/logo/<company_id>/logo_<time>.<ext>

What the user sees next

- The new logo shows on the Company Profile, in the browser tab and on the next invoice PDF.

Protections and edge cases

- Old logo files are not deleted (three logos were already on disk when this document was prepared). Logos are public files; insurance and toll documents are protected.

37.3 Create, edit, hand over and remove users

The Administrator decides who can sign in and what they can do.

Where it starts	Settings → Users and Roles (website table and modals; phone: Users and Roles list and bottom sheet).
What is sent	POST /settings/users/add (full_name, username = mobile, email, password, role, access_level, modules, status); /settings/users/<id>/edit (the same plus phone); /settings/users/<id>/make-administrator; /settings/users/<id>/delete. The phone sheet sends the same forms with the header X-App-Request: 1 and gets JSON.
Code entry	`add_user` (10740); `edit_user` (10791); `make_administrator` (10840); `delete_user` (10880); `_modules_from_form` (9932); `_other_active_administrator` (10509).
Phone twin	the app sheet calls the same routes

What happens, step by step

1. Add: full name required; mobile must be a valid 10-digit number (it becomes both username and phone); duplicate mobile or email is refused with the account named; password optional (at least 8 if given — a blank one stores a random hash so only SMS login works); a role typed as Administrator is refused while another Administrator is active (unless the new user is Inactive); access level Limited needs at least one screen ("Ledger:e,Reports:v" format, unknown names dropped).
2. INSERT into users with is_admin = 1 only for Full Access, status (default Active), created_by.
3. Edit: same validations (excluding the user's own row); the last-Administrator rules stop demoting or deactivating the only one; the password changes only if a new one is typed; UPDATE.
4. Make Administrator: the target must exist, be Active and not be the caller; in one transaction the caller becomes Manager / Full Access (module_access cleared) and the target becomes Administrator / Full Access / Active; an access_logs line records the hand-over.
5. Delete: refused for yourself or the last Administrator; otherwise the user and their access_logs rows are deleted.

What is saved, and where

Table	Operation	Columns / values written
`users`	INSERT / UPDATE / DELETE	username, password_hash, role, is_admin, phone, full_name, email, access_level, module_access, status, created_at, created_by (insert); full_name, role, phone, email, access_level, module_access, status, is_admin, password_hash (edit)
`access_logs`	INSERT / DELETE	hand-over event; a deleted user's log rows are removed

What is read

- users, settings, access_logs

What the user sees next

- The users table updates; the affected user's very next request re-reads their row, so a change of level, screens or status applies at once (an Inactive user is signed out on the next click).

Protections and edge cases

- Every refusal is a readable message (a banner in the website modal, a red line in the phone sheet).

37.4 Your own profile and password

Anyone can change their own name, mobile, email and password.

Where it starts	Website: Profile and Settings (shows "My Profile" for non-administrators); phone: Profile → Edit Profile / Change Password.
What is sent	POST /settings/my-profile (full_name, phone, email); POST /settings/my-password (current_password, new_password, confirm_password); phone: POST /app/profile/update and /app/profile/change-password (JSON).
Code entry	`website_update_my_profile` (10481) → `_update_own_profile` (10439); `website_change_password` (10489) → `_change_own_password` (10458).
Phone twin	app_update_profile, app_change_password

What happens, step by step

1. Profile: name required; mobile must be 10 digits; email must look valid; another user's mobile or email is refused. Only name, email and phone are updated — a posted role or access level is ignored.
2. Password: all fields required, new equals confirmation, at least 8 characters, and the current password must verify before the hash is replaced.

What is saved, and where

Table	Operation	Columns / values written
`users`	UPDATE	full_name, email, phone | password_hash (with updated_by, updated_at)

What is read

- users

What the user sees next

- The header shows the new name/initials; Read Only users may still change their own password (the only write they are allowed).

38. The Route Finder

38.1 Type-ahead, search and stops

Finding a driving route between two places in India and the toll plazas, towns and fuel pumps along it.

Where it starts	Route Analytics → Route Finder tab; phone: Business Overview → Route analytics → Finder.
What is sent	GET /route-finder/suggest?q=…&lat=…&lon=… (after 3 letters); GET /route-finder/search?from=…&to=…&from_lat/from_lng/to_lat/to_lng (when a suggestion was picked); POST /route-finder/extras with JSON {geometry, distance_km, highways}.
Code entry	`route_finder_suggest` (11174), `route_finder_search` (11202), `route_finder_extras` (11254) and providers/osm_routes.py.

What happens, step by step

1. Suggest: fewer than 3 letters returns an empty list; results are cached for an hour per worker; a separate semaphore (_RF_SUGGEST_SLOTS) allows a limited number of simultaneous suggestion calls and answers "busy" otherwise; Photon is asked for places in India.
2. Search: at most 3 searches run at once (_RF_SLOTS), otherwise HTTP 429 "Several route searches are running"; each end is either the picked coordinate pair or a Nominatim lookup (geocodes are cached 24 hours; the To end is biased towards the From end); OSRM returns up to three routes (cached 1 hour by rounded coordinates); the result also includes what the company's OWN past trips on that route earned (_route_history reads trips and toll_entries).
3. Extras: the browser sends the chosen route's simplified path back (so any gunicorn worker can answer); the server checks it (2–4000 valid points, distance below 6000 km), caches by a hash of the path, and asks Overpass for toll booths and gantries, cities and towns, petrol pumps, airports, junctions and notable landmarks inside ~50 km boxes along the route; features are matched to the path by distance (tolls within 250 m, fuel 200 m, towns 4 km, landmarks 3 km, airports 8 km); places are ranked and the best spread along the route are chosen per screen size.
4. Failures: an Overpass failure is reported with its HTTP status and a "Try again" button; failures are never cached.

What is read

- trips and toll_entries only (for the company's own route history); everything else comes from the outside services

What the user sees next

- A map (Leaflet) with the route(s), cards with distance, time and highways, a "Route Stops" timeline with elapsed time, toll and petrol counts per stretch, and the honest notes about data limits.

Protections and edge cases

- Nothing from the outside services is stored in the database.

39. The phone app: how it relates to the website

The phone app has its own routes (60 of them under /app/) and its own templates (45 files), but wherever a business action exists on the website, the phone route calls the same helper, so the saved rows are identical. The table is generated from the code: for every phone route that shares a helper with the website, it lists the shared helper and the website routes that use it.

Phone URL	Phone function	Shared helper(s)	Website route(s) using the same helper
/app/dashboard	app_dashboard	_dashboard_financials_data, _route_analytics_data	dashboard, route_analytics
/app/trips/add	app_add_trip	_insert_trip_from_form, _save_trip_custom_items	add_trip, edit_trip
/app/trips/<int:trip_id>/edit	app_edit_trip	_update_trip_from_form, _save_trip_custom_items	add_trip, edit_trip
/app/maintenance	app_maintenance	_maintenance_overview_data	maintenance_list
/app/maintenance/service/add	app_add_service	_insert_service_from_form, _save_service_items	add_service, edit_service
/app/maintenance/service/<int:m_id>/edit	app_edit_service	_update_service_from_form, _save_service_items	add_service, edit_service
/app/tyres	app_tyres	_tyre_management_data	maintenance_list
/app/tyres/add	app_add_tyre	_insert_tyre_from_form	add_tyre
/app/tyres/stock/add	app_add_tyre_stock	_insert_tyre_stock_from_form	add_tyre_stock
/app/tyres/stock/<int:stock_id>/install	app_install_tyre_stock	_install_tyre_stock_from_form	install_tyre_stock
/app/battery	app_battery	_battery_management_data	maintenance_list
/app/battery/add	app_add_battery	_insert_battery_from_form	add_battery
/app/battery/<int:battery_id>/install	app_install_battery	_install_battery_from_form	install_battery
/app/urea	app_urea	_urea_management_data	maintenance_list
/app/urea/add	app_add_urea	_insert_urea_from_form	add_urea
/app/toll/add	app_add_toll	_insert_toll_from_form, _save_toll_receipt	add_toll
/app/compliance/add	app_add_compliance	_insert_compliance_expense_from_form	add_compliance_expense
/app/expenses	app_expenses	_overhead_list_data	overheads_list
/app/expenses/add	app_add_overhead	_insert_overhead_from_form	add_overhead
/app/expenses/<int:o_id>/edit	app_edit_overhead	_update_overhead_from_form	edit_overhead
/app/salary	app_salary	_employees_salary_data	salaries_list
/app/employee/<employee>/add	app_add_employee_payment	_insert_employee_payment_from_form	employee_ledger
/app/employee/<employee>/opening-balance	app_update_employee_opening_balance	_update_employee_opening_balance_from_form	update_employee_opening_balance
/app/salary/add	app_add_employee	_insert_employee_from_form	add_employee

Phone URL	Phone function	Shared helper(s)	Website route(s) using the same helper
/app/trips/<int:trip_id>	app_trip_detail	_trip_detail_data	trip_view
/app/trips/<int:trip_id>/end	app_end_trip	_end_trip_from_form	end_trip
/app/vehicles/<int:v_id>	app_vehicle_detail	_vehicle_detail_data	vehicle_detail
/app/vehicles/add	app_add_vehicle	_insert_vehicle_from_form	add_vehicle
/app/analytics	app_analytics	_route_analytics_data, _route_rates_data	route_analytics
/app/fleet-utilization	app_fleet_utilization	_fleet_utilization_data	fleet_utilization
/app/performance	app_performance	_performance_data	performance
/app/ledger/add	app_add_account	_create_account_from_form	add_account
/app/ledger/party/<int:party_id>/payment	app_add_party_payment	_insert_party_payment_from_form	add_party_payment
/app/ledger/vendor/<int:vendor_id>/payment	app_add_vendor_payment	_insert_vendor_payment_from_form	add_vendor_payment
/app/ledger/<entity_type>/<int:entity_id>/opening-balance	app_update_ledger_opening_balance	_update_ledger_opening_balance_from_form	update_opening_balance
/app/settings/save	app_settings_save	_save_settings_group	settings_page
/app/profile/update	app_update_profile	_update_own_profile	website_update_my_profile

- **Differences are only on the surface:** mobile templates (cards, bottom sheets, fans) instead of tables and modals, JSON answers to fetch() calls instead of redirects, and a few phone-only pages (the Administrator settings hub).
- **Access rules are the same hook.** require_login runs for /app/ exactly as for the website; a blocked phone request gets JSON or the friendly "No access" page.
- **Navigation shell.** The phone layout swaps page content with fetch() instead of reloading (Figure 15), but every page is still a normal server-rendered response.

40. Background jobs and operational endpoints

40.1 Scheduled vehicle-data refresh

Keeping RC data, compliance dates and challans fresh without anyone pressing a button.

Where it starts	Render cron (or the local APScheduler) every hour.
What is sent	POST /internal/sync-tick?job=sync_tick (default) or job=weekly_compliance, with header X-Sync-Token.
Code entry	`internal_sync_tick` (12475) → scheduler.run_sync_tick / run_weekly_sync.

What happens, step by step

1. The token is compared with INTERNAL_SYNC_TOKEN; if it is unset or wrong the answer is HTTP 401 and nothing runs.
2. run_sync_tick reads sync_log and the Settings intervals; if a job is due and enabled (ENABLE_RC_SYNC / ENABLE_CHALLAN_SYNC) it runs: one eChallan call per own vehicle, then the same writes as "Sync now".
3. weekly_compliance runs the provider sync for the whole own fleet unconditionally.
4. Every real run writes one sync_log row at start and updates it at the end with counts.

What is saved, and where

Table	Operation	Columns / values written
`sync_log`	INSERT / UPDATE	job_name, started_at, finished_at, synced_count, failed_count, skipped_count, note
`vehicles`	UPDATE	rc_synced_data, rc_last_synced, challan_count, challan_amount, challan_last_synced and back-filled columns
`vehicle_challans`	DELETE then INSERT	fresh challan rows
`vehicle_compliance`	INSERT / UPDATE	fitness, PUC, permit records

What is read

- settings (intervals, rc_lookup_api_key), vehicles, sync_log

What the user sees next

- Vehicles badges and the bell reflect the new data; Users and Roles → Integrations lists the latest runs.

Protections and edge cases

- Off by default; a failed call never overwrites good data; each call is a paid credit, so only deliberate opt-in turns it on. The token comparison is a plain inequality, not constant-time (low risk, noted).

40.2 Database backup download

Giving your Mac a verified copy of the live database.

Where it starts	pull_backup.py running daily on the Mac.
What is sent	POST /internal/backup with header X-Backup-Token.
Code entry	`internal_backup` (12427)

What happens, step by step

1. If more than 5 wrong tokens came from this address in 15 minutes the answer is HTTP 429. If BACKUP_TOKEN is unset or the header is wrong the answer is 403 and the attempt is recorded.
2. SQLite's online backup copies the database to a temp file, PRAGMA integrity_check runs, the file is gzipped and streamed as fleet_backup.db.gz; temp files are removed after the response.

What is saved, and where

Table	Operation	Columns / values written
`auth_attempts`	INSERT (wrong tries)	kind backup_fail_ip, key = address, time

What is read

- the whole database file, read through the backup API

What the user sees next

- The Mac script saves render_YYYYmmdd_HHMMSS.db after its own checks (integrity, key tables not empty, no sudden 20% drop in rows).

40.3 What happens at start-up

Every gunicorn worker, while importing app.py.

Where it starts	Render starting or restarting the service.
What is sent	none (not a request).
Code entry	Chapter 8.2 lists the seven steps.

What happens, step by step

1. Flask is created; cookie, proxy and upload settings are applied; the database path is resolved; then, in order: overhead allocation column, performance indexes, WAL mode, auth tables, secret key, single-Administrator index, Twilio cleanup.
2. Hooks and routes are registered; the worker begins to serve.

What is saved, and where

Table	Operation	Columns / values written
`payment_allocations / settings / users`	ALTER or INSERT only if missing	overhead_id column (after a backup copy), the private secret key row, indexes, the two auth tables

What the user sees next

- The server log prints one line per step ([schema] / [security]).

Protections and edge cases

- Safe to repeat and safe with two workers starting at once.

F

Complete reference, generated from the code

An execution card for every one of the URLs, the life story of every table, and a map from every screen to the data behind it.

IN THIS PART

- Execution card per URL: inputs, helpers, tables written (with columns), tables read, page shown
- Life of each table: who creates, changes, deletes and reads it
- Screen-to-data map for every page template

41. Execution cards: every URL

One card per URL (199 in all), grouped by the screen they belong to for Limited Access. Each card was produced by analysing the real source: which form fields and query parameters the code reads, which helper functions it runs (the generic ones such as get_db are omitted), which tables it writes and which columns it sets, which tables it reads, which template it renders and what that template shows (cards, table columns, sections), and where it goes next. Columns that are always set automatically (created_by, created_at, updated_by, updated_at, company_id) are left out to save space.

> **How to use the cards**
>
> To answer "what does pressing this do?": find the URL, read Writes (what changes), Reads (what it consults) and Page shown (what you will see). To answer "which URLs can change this table?": use the table list in Chapter 42. Cards for phone routes (names beginning app_) show the same helpers as their website twins.

Open pages and sign-in (9)

GET `/app-manifest.json` → `app_manifest` (line 9793)

PWA install manifest (name, icons, start page).

Writes	nothing (read-only)
Then	answers with JSON

GET / POST `/login` → `login` (line 10249)

Website sign-in form and password check (with lockout).

Form fields read	`password`, `remember_me`, `username`
Helpers run	`_login_failed`, `_login_throttle_message`, `_username_login_format_ok`
Writes	**INSERT** `access_logs` (date, event, user_id); **INSERT** `auth_attempts` (at, key, kind); **DELETE** `auth_attempts`; **UPDATE** `users` (last_login)
Reads	`auth_attempts`, `users`
Page shown	`login.html` — sections: Manage Your Transport Business. Drive Your Business Forward., Welcome Back!, Create Your Account, Verify Your Mobile Number, Account Created Successfully
Data given to the page	`error`
Then	redirects to `dashboard`

POST `/login/otp/send` → `send_login_otp` (line 10334)

Website: asks for a one-time SMS code for a mobile number.

Form fields read	`phone`
Helpers run	`_start_otp_login`, `_otp_issue`, `_otp_hash`, `_send_otp_sms`, `_send_otp_msg91`, `_sms_configured`
Writes	**INSERT** `auth_attempts` (at, key, kind); **DELETE** `auth_attempts`; **INSERT** `login_otps` (expires_at, otp_hash, phone, user_id); **UPDATE** `login_otps` (otp_hash, used)
Reads	`auth_attempts`, `settings`, `users`
Page shown	`login.html` — sections: Manage Your Transport Business. Drive Your Business Forward., Welcome Back!, Create Your Account, Verify Your Mobile Number, Account Created Successfully
Data given to the page	`otp_error`, `otp_mode`, `otp_phone`, `otp_sent`

POST `/login/otp/verify` → `verify_login_otp` (line 10410)

Website: checks the code the user typed and signs in.

Form fields read	`otp`, `phone`
Helpers run	`_finish_otp_login`, `_otp_check`, `_otp_hash`
Writes	**INSERT** `access_logs` (date, event, user_id); **UPDATE** `login_otps` (attempts, used); **UPDATE** `users` (last_login)
Reads	`login_otps`, `users`
Page shown	`login.html` — sections: Manage Your Transport Business. Drive Your Business Forward., Welcome Back!, Create Your Account, Verify Your Mobile Number, Account Created Successfully
Data given to the page	`error`, `otp_error`, `otp_mode`, `otp_phone`, `otp_sent`
Then	redirects to `dashboard`

GET `/logout` → `logout` (line 10287)

Ends the session and returns to the right login page (website or app).

Query parameters	`ctx`
Writes	nothing (read-only)
Then	redirects to `app_login`, `login`

GET / POST `/app/login` → `app_login` (line 12533)

Mobile app sign-in (password).

Form fields read	`password`, `remember_me`, `username`
Helpers run	`_login_failed`, `_login_throttle_message`, `_username_login_format_ok`
Writes	**INSERT** `access_logs` (date, event, user_id); **INSERT** `auth_attempts` (at, key, kind); **DELETE** `auth_attempts`; **UPDATE** `users` (last_login)
Reads	`auth_attempts`, `users`
Page shown	`app/login.html`
Data given to the page	`error`, `year`
Then	redirects to `app_dashboard`

GET / POST `/app/login/otp` → `app_login_otp` (line 14759)

Mobile app: ask for an SMS code.

Form fields read	`phone`
Helpers run	`_start_otp_login`, `_otp_issue`, `_otp_hash`, `_send_otp_sms`, `_send_otp_msg91`, `_sms_configured`
Writes	**INSERT** `auth_attempts` (at, key, kind); **DELETE** `auth_attempts`; **INSERT** `login_otps` (expires_at, otp_hash, phone, user_id); **UPDATE** `login_otps` (otp_hash, used)
Reads	`auth_attempts`, `settings`, `users`
Page shown	`app/login_otp.html` | `app/verify_otp.html`
Data given to the page	`error`, `phone`

POST `/app/login/otp/verify` → `app_verify_login_otp` (line 14775)	
Mobile app: verify the SMS code.	
Form fields read	`otp`, `phone`
Helpers run	`_finish_otp_login`, `_otp_check`, `_otp_hash`
Writes	**INSERT** `access_logs` (date, event, user_id); **UPDATE** `login_otps` (attempts, used); **UPDATE** `users` (last_login)
Reads	`login_otps`, `users`
Page shown	`app/login_otp.html` | `app/verify_otp.html`
Data given to the page	`error`, `phone`
Then	redirects to `app_dashboard`

GET / POST `/app/signup` → `app_signup` (line 12573)	
Self sign-up (closed by default: shows "Sign-up is closed").	
Form fields read	`confirm_password`, `full_name`, `password`, `username`
Helpers run	`_duplicate_user_contact`, `_signup_open`
Writes	**INSERT** `access_logs` (date, event, user_id); **INSERT** `users` (access_level, email, full_name, is_admin, password_hash, phone, role, status, username)
Reads	`users`
Page shown	`app/login.html` | `app/signup.html`
Data given to the page	`error`, `form_values`, `year`
Then	redirects to `app_dashboard`

Internal endpoints (token protected) (2)

POST `/internal/backup` → `internal_backup` (line 12427)	
Token-protected download of a verified, compressed copy of the whole database.	
Writes	**INSERT** `auth_attempts` (at, key, kind); **DELETE** `auth_attempts`
Reads	`auth_attempts`
Then	answers with JSON

POST `/internal/sync-tick` → `internal_sync_tick` (line 12475)	
Token-protected trigger for the background RC / challan / compliance sync jobs.	
Query parameters	`job`
Writes	nothing (read-only)

Profile, settings and users (19)

GET / POST `/settings` → `settings_page` (line 10636)

Administrator: company settings and users; everyone else: My Profile.

Form fields read	`form_type`
Query parameters	`tab`
Helpers run	`_render_my_profile`, `_role_suggestions`, `_is_admin_role`, `_other_active_administrator`, `_save_settings_group`, `_users_with_stats`
Writes	**INSERT** `settings` (key, value)
Reads	`access_logs`, `settings`, `sync_log`, `users`
Page shown	`profile_self.html` — cards: Role, Access level, Status, Login (username), Member since, Last login; sections: My Profile, My details, Change password | `settings.html` — cards: Company Name, GSTIN, Address, PAN, State, Business Type; columns: User, Role, Access Level, Mobile / Email, Status, Last Login, Users, Date Time, Event; sections: Profile Settings, Company Details, Edit Company Details, Banking Details, Edit Banking Details
Data given to the page	`access_logs`, `active`, `active_settings_tab`, `admin_users`, `company_name`, `error`, `inactive_users`, `invoice_example`, `limited_users`, `message`, `module_list`, `readonly_users`, `recent_sync_log`, `role_counts`, `role_suggestions`, `s`, `total_users`, `u`, `users`

POST `/settings/logo/upload` → `upload_company_logo` (line 8511)

Administrator: upload the logo shown on invoices.

Form fields read	`logo`
Writes	**INSERT** `settings` (key, value)
Then	redirects to `settings_page`; answers with JSON

POST `/settings/my-password` → `website_change_password` (line 10489)

Own password.

Form fields read	`confirm_password`, `current_password`, `new_password`
Helpers run	`_change_own_password`, `_render_my_profile`
Writes	**UPDATE** `users` (password_hash)
Reads	`users`
Page shown	`profile_self.html` — cards: Role, Access level, Status, Login (username), Member since, Last login; sections: My Profile, My details, Change password
Data given to the page	`active`, `error`, `message`, `u`

POST `/settings/my-profile` → `website_update_my_profile` (line 10481)

Own name, mobile, email.

Form fields read	`email`, `full_name`, `phone`
Helpers run	`_render_my_profile`, `_update_own_profile`, `_duplicate_user_contact`
Writes	**UPDATE** `users` (email, full_name, phone)
Reads	`users`
Page shown	`profile_self.html` — cards: Role, Access level, Status, Login (username), Member since, Last login; sections: My Profile, My details, Change password
Data given to the page	`active`, `error`, `message`, `u`

POST `/settings/users/<int:user_id>/delete` → `delete_user` (line 10880)

Administrator: remove a user (never self or last Administrator).

Helpers run	`_administrator_count`, `_render_settings_users_error`, `_role_suggestions`, `_is_admin_role`, `_other_active_administrator`, `_users_with_stats`
Writes	**DELETE** `access_logs`; **DELETE** `users`
Reads	`access_logs`, `settings`, `users`
Page shown	`settings.html` — cards: Company Name, GSTIN, Address, PAN, State, Business Type; columns: User, Role, Access Level, Mobile / Email, Status, Last Login, Users, Date Time, Event; sections: Profile Settings, Company Details, Edit Company Details, Banking Details, Edit Banking Details
Data given to the page	`access_logs`, `active`, `active_settings_tab`, `admin_users`, `company_name`, `inactive_users`, `invoice_example`, `limited_users`, `module_list`, `readonly_users`, `reopen_add_user`, `reopen_edit_user_id`, `role_counts`, `role_suggestions`, `s`, `total_users`, `user_error`, `users`
Then	redirects to `settings_page`; answers with JSON

POST `/settings/users/<int:user_id>/edit` → `edit_user` (line 10791)

Administrator: edit a user.

Form fields read	`access_level`, `email`, `full_name`, `modules`, `password`, `phone`, `role`, `status`
Helpers run	`_administrator_count`, `_duplicate_user_contact`, `_is_admin_role`, `_modules_from_form`, `_format_modules`, `_parse_modules`, `_other_active_administrator`, `_render_settings_users_error`, `_role_suggestions`, `_users_with_stats`
Writes	**UPDATE** `users` (access_level, email, full_name, is_admin, module_access, password_hash, phone, role, status)
Reads	`access_logs`, `settings`, `users`
Page shown	`settings.html` — cards: Company Name, GSTIN, Address, PAN, State, Business Type; columns: User, Role, Access Level, Mobile / Email, Status, Last Login, Users, Date Time, Event; sections: Profile Settings, Company Details, Edit Company Details, Banking Details, Edit Banking Details
Data given to the page	`access_logs`, `active`, `active_settings_tab`, `admin_users`, `company_name`, `inactive_users`, `invoice_example`, `limited_users`, `module_list`, `readonly_users`, `reopen_add_user`, `reopen_edit_user_id`, `role_counts`, `role_suggestions`, `s`, `total_users`, `user_error`, `users`
Then	redirects to `settings_page`; answers with JSON

POST `/settings/users/<int:user_id>/make-administrator` → `make_administrator` (line 10840)

Administrator: hand the single Administrator seat to another active user.

Helpers run	`_render_settings_users_error`, `_role_suggestions`, `_is_admin_role`, `_other_active_administrator`, `_users_with_stats`
Writes	**INSERT** `access_logs` (date, event, user_id); **UPDATE** `users` (access_level, is_admin, module_access, role, status)
Reads	`access_logs`, `settings`, `users`
Page shown	`settings.html` — cards: Company Name, GSTIN, Address, PAN, State, Business Type; columns: User, Role, Access Level, Mobile / Email, Status, Last Login, Users, Date Time, Event; sections: Profile Settings, Company Details, Edit Company Details, Banking Details, Edit Banking Details
Data given to the page	`access_logs`, `active`, `active_settings_tab`, `admin_users`, `company_name`, `inactive_users`, `invoice_example`, `limited_users`, `module_list`, `readonly_users`, `reopen_add_user`, `reopen_edit_user_id`, `role_counts`, `role_suggestions`, `s`, `total_users`, `user_error`, `users`
Then	redirects to `settings_page`; answers with JSON

POST `/settings/users/<int:user_id>/phone` → `update_user_phone` (line 10863)

Administrator: change a user's mobile.

Form fields read	`phone`
Helpers run	`_duplicate_user_contact`
Writes	**UPDATE** `users` (phone)
Reads	`users`
Then	redirects to `settings_page`

POST `/settings/users/add` → `add_user` (line 10740)

Administrator: create a user (role, access, screens).

Form fields read	`access_level`, `email`, `full_name`, `modules`, `password`, `role`, `status`, `username`
Helpers run	`_duplicate_user_contact`, `_is_admin_role`, `_modules_from_form`, `_format_modules`, `_parse_modules`, `_other_active_administrator`, `_render_settings_users_error`, `_role_suggestions`, `_users_with_stats`
Writes	**INSERT** `users` (access_level, email, full_name, is_admin, module_access, password_hash, phone, role, status, username)
Reads	`access_logs`, `settings`, `users`
Page shown	`settings.html` — cards: Company Name, GSTIN, Address, PAN, State, Business Type; columns: User, Role, Access Level, Mobile / Email, Status, Last Login, Users, Date Time, Event; sections: Profile Settings, Company Details, Edit Company Details, Banking Details, Edit Banking Details
Data given to the page	`access_logs`, `active`, `active_settings_tab`, `admin_users`, `company_name`, `inactive_users`, `invoice_example`, `limited_users`, `module_list`, `readonly_users`, `reopen_add_user`, `reopen_edit_user_id`, `role_counts`, `role_suggestions`, `s`, `total_users`, `user_error`, `users`
Then	redirects to `settings_page`; answers with JSON

GET `/settings/users/export` → `export_users` (line 10893)

Administrator: Excel list of users.

Helpers run	`_users_with_stats`
Writes	nothing (read-only)
Reads	`users`

GET `/app/profile` → `app_profile` (line 14613)

Phone profile (Administrator sees the settings hub).

Writes	nothing (read-only)
Reads	`settings`, `users`
Page shown	`app/profile.html` — sections: Personal Information, Account, Edit Profile, Change Password | `app/profile_admin.html` — sections: Profile Settings, Edit Profile, Change Password
Data given to the page	`access_level`, `active`, `company_name`, `email`, `full_name`, `is_admin`, `member_since`, `notif_count`, `phone`, `role_label`, `status`, `user_initials`, `username`
Then	redirects to `app_login`

POST `/app/profile/change-password` → `app_change_password` (line 14739)

Phone: change own password.

Form fields read	`confirm_password`, `current_password`, `new_password`
Helpers run	`_change_own_password`
Writes	**UPDATE** `users` (password_hash)
Reads	`users`
Then	answers with JSON

POST `/app/profile/update` → `app_update_profile` (line 14717)

Phone: edit own details.

Form fields read	`email`, `full_name`, `phone`
Helpers run	`_update_own_profile`, `_duplicate_user_contact`
Writes	**UPDATE** `users` (email, full_name, phone)
Reads	`users`
Then	redirects to `app_profile`; answers with JSON

GET `/app/settings/banking` → `app_settings_banking` (line 14662)

Administrator, phone: bank details.

Helpers run	`_app_settings_page`
Writes	nothing (read-only)
Reads	`settings`, `users`

GET `/app/settings/company` → `app_settings_company` (line 14658)

Administrator, phone: company profile.

Helpers run	`_app_settings_page`
Writes	nothing (read-only)
Reads	`settings`, `users`

GET `/app/settings/invoice` → `app_settings_invoice` (line 14666)

Administrator, phone: invoice numbering and preferences.

Helpers run	`_app_settings_page`
Writes	nothing (read-only)
Reads	`settings`, `users`

POST `/app/settings/save` → `app_settings_save` (line 14693)

Administrator, phone: saves any settings form (JSON).

Form fields read	`company_name`, `email`, `form_type`, `invoice_prefix`, `msg91_authkey`, `msg91_template_id`, `next_invoice_number`
Helpers run	`_save_settings_group`, `_settings_form_error`
Writes	**INSERT** `settings` (key, value)
Reads	`settings`
Then	answers with JSON

GET `/app/settings/tax` → `app_settings_tax` (line 14670)

Administrator, phone: GST, TDS, RCM.

Helpers run	`_app_settings_page`
Writes	nothing (read-only)
Reads	`settings`, `users`

GET `/app/settings/users` → `app_settings_users` (line 14674)

Administrator, phone: users, roles, logs, integrations.

Helpers run	`_app_settings_page`, `_role_suggestions`, `_is_admin_role`, `_other_active_administrator`, `_users_with_stats`
Writes	nothing (read-only)
Reads	`access_logs`, `settings`, `sync_log`, `users`

Screen: Dashboard (3)

GET `/` → `home` (line 1278)

Redirects / to the Dashboard.

Writes	nothing (read-only)
Then	redirects to `dashboard`

GET `/dashboard` → `dashboard` (line 1371)

Website Dashboard: revenue, profit, receivables, expense mix, trends, top vehicles.

Query parameters	`date_from`, `date_to`, `vehicle`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_dashboard_financials_data`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `trips`, `vehicle_compliance`, `vehicles`, `vendors`
Page shown	`dashboard.html` — cards: Total Trips, Total Revenue, Total Profit, Own Vehicle Revenue, Hired Vehicle Profit, Fuel Expense; columns: Vehicle, Trips, Revenue, Avg/Trip, Util.; sections: Dashboard, Freight Revenue Trend View Details, Expense Breakdown, Recent Activities, Trip Status View All
Data given to the page	`active`, `activities`, `adj_revenue`, `alerts`, `driveradv_total`, `exp_breakdown`, `f_date_from`, `f_date_to`, `f_fy`, `f_vehicle`, `fleet_running_pct`, `freight_chart_bottom`, `freight_trend`, `freight_y_ticks`, `fuel_total`, `hired_trip_count`, `hired_vehicle_chart`, `hired_vehicle_profit`, `idle_count`, `maint_by_category`, `maint_total`, `other_exp_total`, `overdue_payables`, `overdue_receivables`, `own_idle_count`, `own_vehicle_count` (+19 more)

GET `/app/dashboard` → `app_dashboard` (line 12740)

Phone Dashboard: today, fleet status, alerts, expenses, business overview, routes, recent activity.

Query parameters	`date_from`, `date_to`, `period`, `vehicle`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_dashboard_financials_data`, `_month_bounds`, `_pct_growth`, `_route_analytics_data`, `_toll_by_trip`, `_trip_toll`, `_shift_period_back`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `toll_entries`, `trips`, `users`, `vehicles`, `vendors`
Page shown	`app/dashboard.html` — sections: Hi, Fleet Status, Needs Your Attention!, Top Expenses, Business Overview
Data given to the page	`active`, `activities`, `alert_count`, `alert_summary`, `bo_expense_growth`, `bo_expenses`, `bo_revenue`, `bo_revenue_growth`, `bo_trip_growth`, `bo_trips`, `challans_pending`, `dash_vehicles`, `documents_expired`, `documents_expiring`, `documents_expiring_soon`, `f_date_from`, `f_date_to`, `f_vehicle`, `fs_idle_pct`, `fs_on_trip_pct`, `fs_workshop_pct`, `greeting_name`, `more_routes_count`, `notif_count`, `on_trip_count`, `payable_count` (+17 more)

Screen: Trips (13)

GET `/trips` → `trips_list` (line 2879)

Trips table with filters, search, paging, totals and balance due.

Query parameters	`amount`, `date_from`, `date_to`, `dir`, `end_error`, `error`, `from_loc`, `lr`, `lr_number`, `lr_received`, `page`, `party`, `per_page`, `sort`, `status`, `to_loc`, `toll_count`, `type`, `vehicle`
Writes	nothing (read-only)
Reads	`parties`, `trips`, `vehicles`
Page shown	`trips_list.html` — cards: Total Trips, Total Billed, Pending Balance, Active Trips, LR Pending; columns: Date ■, LR Number, Vehicle, Party ■, From, To, Billed, Pending, Action; sections: Trips, End Trip, Export — Trips
Data given to the page	`active`, `active_count`, `base_qs`, `delete_error`, `delete_error_amount`, `delete_error_lr`, `delete_error_toll_count`, `end_error`, `f_date_from`, `f_date_to`, `f_dir`, `f_from`, `f_lr`, `f_lr_number`, `f_party`, `f_sort`, `f_status`, `f_to`, `f_type`, `f_vehicle`, `filter_qs`, `lr_pending_count`, `lr_received_count`, `page`, `page_tokens`, `parties` (+7 more)

POST `/trips/<int:trip_id>/end` → `end_trip` (line 7030)

End a running trip: end date, shortage, extra expenses.

Form fields read	`actual_km`, `end_date`, `end_time`, `freight_amount`, `lr_received`, `remarks`, `shortage_amount`, `shortage_qty`, `shortage_remarks`, `shortage_unit`
Helpers run	`_end_trip_from_form`, `_parse_money`
Writes	**UPDATE** `trips` (actual_km, billed_amount, end_date, end_time, lr_received, remarks, shortage_amount, shortage_date, shortage_qty, shortage_remarks, shortage_unit)
Reads	`trips`
Then	redirects to `trips_list`

GET `/trips/<int:trip_id>/view` → `trip_view` (line 11894)

One trip in full: charges, deductions, payments, owner terms.

Helpers run	`_trip_detail_data`
Writes	nothing (read-only)
Reads	`invoice_items`, `parties`, `trips`, `vehicles`, `vendors`
Page shown	`trip_view.html` — cards: Trip Type, Party, Material, Driver, LR Received, Billed Amount; columns: Description, Type, Amount, Vendor; sections: Trip Details
Data given to the page	`active`
Then	redirects to `trips_list`

GET / POST `/trips/add` → `add_trip` (line 3227)

Trip form and save (checks duplicate LR, validates every amount).

Form fields read	`agent_commission`, `brokerage`, `builty_commission`, `builty_expense`, `date`, `detention_charges`, `driver_adv_amount`, `driver_adv_vendor`, `driver_name`, `fine`, `fixed_rate_amount`, `from_loc`, `fuel_amount`, `fuel_liters`, `fuel_price`, `fuel_vendor`, `gps_cost`, `guarantee_qty`, `is_empty`, `labour_charges`, `late_fees`, `loading_charge`, `loading_expense`, `lr_number`, `lr_received`, `material`, `material_damage`, `misc_vendor`, `other_charges`, `other_deductions`, `other_desc`, `other_expense`, `other_qty`, `other_rate`, `other_type`, `other_vendor`, `owner_fixed_amount`, `owner_name`, `owner_rate`, `owner_rate_type` (+25 more)
Helpers run	`_duplicate_lr_trip`, `_get_autocomplete_lists`, `_insert_trip_from_form`, `_resolve_trip_type`, `_save_trip_custom_items`, `get_or_create_vendor`, `_trip_freight`, `get_or_create_party`, `get_or_create_vehicle`, `_trip_form_error`, `_parse_money`, `_vehicle_type_map`
Writes	**INSERT** `invoice_items` (amount, description, item_type, trip_id, vendor_id); **DELETE** `invoice_items`; **INSERT** `parties` (name); **INSERT** `trips` (agent_commission, billed_amount, brokerage, builty_commission, builty_expense, date, detention_charges, driver_adv_amount, driver_adv_vendor_id, driver_name, fine, fixed_rate_amount, from_loc, fuel_amount, fuel_liters, fuel_price, +45 more); **INSERT** `vehicles` (type, vehicle_no); **INSERT** `vendors` (linked_party_id, name); **UPDATE** `vendors` (linked_party_id)
Reads	`employees`, `parties`, `trips`, `vehicles`, `vendors`
Page shown	`trip_form.html` — cards: Net Amount (Billed), Estimated Profit; columns: #, Description, Type, Amount / Rate ( ), Quantity (MT/Nos), Vendor (Optional), Total ( ); sections: Trip Summary, Hired Vehicle Profit, Payment Tracking, Quick Tips
Data given to the page	`active`, `combined_names`, `custom_items`, `employees`, `error`, `mode`, `parties`, `return_to`, `t`, `vehicle_type_map`, `vehicles`, `vendors`
Then	redirects to `trips_list`

POST `/trips/delete/<int:trip_id>` → `delete_trip` (line 6948)

Deletes a trip only when no payment is allocated to it and no toll entry is linked (otherwise redirects with an explanation); only the trips row is removed.

Writes	**DELETE** `trips`
Reads	`payment_allocations`, `toll_entries`, `trips`
Then	redirects to `trips_list`

GET / POST `/trips/edit/<int:trip_id>` → `edit_trip` (line 11765)

Edit a trip (recomputes the bill; re-checks the numbers).

Form fields read	`agent_commission`, `brokerage`, `builty_commission`, `builty_expense`, `date`, `detention_charges`, `driver_adv_amount`, `driver_adv_vendor`, `driver_name`, `fine`, `fixed_rate_amount`, `from_loc`, `fuel_amount`, `fuel_liters`, `fuel_price`, `fuel_vendor`, `gps_cost`, `guarantee_qty`, `is_empty`, `labour_charges`, `late_fees`, `loading_charge`, `loading_expense`, `lr_number`, `lr_received`, `material`, `material_damage`, `misc_vendor`, `other_charges`, `other_deductions`, `other_desc`, `other_expense`, `other_qty`, `other_rate`, `other_type`, `other_vendor`, `owner_fixed_amount`, `owner_name`, `owner_rate`, `owner_rate_type` (+24 more)
Query parameters	`return_to`
Helpers run	`_get_autocomplete_lists`, `_trip_form_error`, `_parse_money`, `_trip_freight`, `_update_trip_from_form`, `_resolve_trip_type`, `_save_trip_custom_items`, `get_or_create_vendor`, `get_or_create_party`, `_vehicle_type_map`
Writes	**INSERT** `invoice_items` (amount, description, item_type, trip_id, vendor_id); **DELETE** `invoice_items`; **INSERT** `parties` (name); **UPDATE** `trips` (agent_commission, billed_amount, brokerage, builty_commission, builty_expense, date, detention_charges, driver_adv_amount, driver_adv_vendor_id, driver_name, fine, fixed_rate_amount, from_loc, fuel_amount, fuel_liters, fuel_price, +45 more); **INSERT** `vendors` (linked_party_id, name); **UPDATE** `vendors` (linked_party_id)
Reads	`employees`, `invoice_items`, `parties`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Page shown	`trip_form.html` — cards: Net Amount (Billed), Estimated Profit; columns: #, Description, Type, Amount / Rate ( ), Quantity (MT/Nos), Vendor (Optional), Total ( ); sections: Trip Summary, Hired Vehicle Profit, Payment Tracking, Quick Tips
Data given to the page	`active`, `combined_names`, `custom_items`, `employees`, `error`, `mode`, `parties`, `return_to`, `t`, `vehicle_type_map`, `vehicles`, `vendors`
Then	redirects to `trips_list`

GET `/trips/export` → `export_trips` (line 7353)

Excel export of the trips list.

Query parameters	`date_from`, `date_to`, `lr_number`, `lr_received`, `party`, `status`, `vehicle`
Writes	nothing (read-only)
Reads	`parties`, `trips`, `vehicles`

GET `/trips/export/pdf` → `export_trips_pdf` (line 7401)

PDF export of the trips list.

Query parameters	`date_from`, `date_to`, `lr_number`, `lr_received`, `party`, `status`, `vehicle`
Writes	nothing (read-only)
Reads	`parties`, `trips`, `vehicles`

GET `/app/trips` → `app_trips` (line 13740)

Phone trips list.

Query parameters	`date_from`, `date_to`, `lr_received`, `search`, `show`, `sort_dir`, `status`, `type`
Helpers run	`_month_bounds`
Writes	nothing (read-only)
Reads	`parties`, `trips`, `users`, `vehicles`
Page shown	`app/trips.html` — sections: Trips, Filters, Trips List
Data given to the page	`active`, `completed_month`, `f_date_from`, `f_date_to`, `f_lr_received`, `f_search`, `f_sort_dir`, `f_status`, `f_type`, `has_more`, `notif_count`, `ongoing_count`, `show`, `total_count`, `total_freight_month`, `total_trips_month`, `trips`, `user_initials`

GET `/app/trips/<int:trip_id>` → `app_trip_detail` (line 13844)

Phone trip detail.

Query parameters	`tab`
Helpers run	`_trip_detail_data`
Writes	nothing (read-only)
Reads	`invoice_items`, `parties`, `settings`, `trips`, `users`, `vehicles`, `vendors`
Page shown	`app/trip_detail.html` — cards: Trip Type, Party, Material, Driver, LR Received, Billed Amount; sections: Trip Information, Fuel, Hired Vehicle Owner, Billing Payments, Additional Charges ■
Data given to the page	`active`, `company_name`, `notif_count`, `tab`, `user_initials`
Then	redirects to `app_trips`

GET / POST `/app/trips/<int:trip_id>/edit` → `app_edit_trip` (line 13005)

Phone edit-trip.

Form fields read	`agent_commission`, `brokerage`, `builty_commission`, `builty_expense`, `date`, `detention_charges`, `driver_adv_amount`, `driver_adv_vendor`, `driver_name`, `fine`, `fixed_rate_amount`, `from_loc`, `fuel_amount`, `fuel_liters`, `fuel_price`, `fuel_vendor`, `gps_cost`, `guarantee_qty`, `is_empty`, `labour_charges`, `late_fees`, `loading_charge`, `loading_expense`, `lr_number`, `lr_received`, `material`, `material_damage`, `misc_vendor`, `other_charges`, `other_deductions`, `other_desc`, `other_expense`, `other_qty`, `other_rate`, `other_type`, `other_vendor`, `owner_fixed_amount`, `owner_name`, `owner_rate`, `owner_rate_type` (+23 more)
Helpers run	`_duplicate_lr_trip`, `_get_autocomplete_lists`, `_get_route_locations`, `_trip_form_error`, `_parse_money`, `_trip_freight`, `_update_trip_from_form`, `_resolve_trip_type`, `_save_trip_custom_items`, `get_or_create_vendor`, `get_or_create_party`, `_vehicle_type_map`
Writes	**INSERT** `invoice_items` (amount, description, item_type, trip_id, vendor_id); **DELETE** `invoice_items`; **INSERT** `parties` (name); **UPDATE** `trips` (agent_commission, billed_amount, brokerage, builty_commission, builty_expense, date, detention_charges, driver_adv_amount, driver_adv_vendor_id, driver_name, fine, fixed_rate_amount, from_loc, fuel_amount, fuel_liters, fuel_price, +45 more); **INSERT** `vendors` (linked_party_id, name); **UPDATE** `vendors` (linked_party_id)
Reads	`employees`, `invoice_items`, `parties`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Page shown	`app/trip_add.html` — sections: Trip Information, Additional Charges, Deductions, Fuel Driver Advance, Other Expenses
Data given to the page	`combined_names`, `custom_items`, `employees`, `error`, `location_options`, `mode`, `t`, `trip_id`, `vehicle_type_map`, `vehicles`
Then	redirects to `app_trip_detail`, `app_trips`

POST `/app/trips/<int:trip_id>/end` → `app_end_trip` (line 13870)

Phone end-trip.

Form fields read	`actual_km`, `end_date`, `end_time`, `freight_amount`, `lr_received`, `remarks`, `shortage_amount`, `shortage_qty`, `shortage_remarks`, `shortage_unit`
Helpers run	`_end_trip_from_form`, `_parse_money`
Writes	**UPDATE** `trips` (actual_km, billed_amount, end_date, end_time, lr_received, remarks, shortage_amount, shortage_date, shortage_qty, shortage_remarks, shortage_unit)
Reads	`trips`
Then	redirects to `app_trip_detail`; answers with JSON

GET / POST `/app/trips/add` → `app_add_trip` (line 12954)

Phone add-trip form and save.

Form fields read	`agent_commission`, `brokerage`, `builty_commission`, `builty_expense`, `date`, `detention_charges`, `driver_adv_amount`, `driver_adv_vendor`, `driver_name`, `fine`, `fixed_rate_amount`, `from_loc`, `fuel_amount`, `fuel_liters`, `fuel_price`, `fuel_vendor`, `gps_cost`, `guarantee_qty`, `is_empty`, `labour_charges`, `late_fees`, `loading_charge`, `loading_expense`, `lr_number`, `lr_received`, `material`, `material_damage`, `misc_vendor`, `other_charges`, `other_deductions`, `other_desc`, `other_expense`, `other_qty`, `other_rate`, `other_type`, `other_vendor`, `owner_fixed_amount`, `owner_name`, `owner_rate`, `owner_rate_type` (+24 more)
Helpers run	`_duplicate_lr_trip`, `_get_autocomplete_lists`, `_get_route_locations`, `_insert_trip_from_form`, `_resolve_trip_type`, `_save_trip_custom_items`, `get_or_create_vendor`, `_trip_freight`, `get_or_create_party`, `get_or_create_vehicle`, `_trip_form_error`, `_parse_money`, `_vehicle_type_map`
Writes	**INSERT** `invoice_items` (amount, description, item_type, trip_id, vendor_id); **DELETE** `invoice_items`; **INSERT** `parties` (name); **INSERT** `trips` (agent_commission, billed_amount, brokerage, builty_commission, builty_expense, date, detention_charges, driver_adv_amount, driver_adv_vendor_id, driver_name, fine, fixed_rate_amount, from_loc, fuel_amount, fuel_liters, fuel_price, +45 more); **INSERT** `vehicles` (type, vehicle_no); **INSERT** `vendors` (linked_party_id, name); **UPDATE** `vendors` (linked_party_id)
Reads	`employees`, `parties`, `trips`, `vehicles`, `vendors`
Page shown	`app/trip_add.html` — sections: Trip Information, Additional Charges, Deductions, Fuel Driver Advance, Other Expenses
Data given to the page	`combined_names`, `custom_items`, `employees`, `error`, `location_options`, `mode`, `t`, `vehicle_type_map`, `vehicles`
Then	redirects to `app_dashboard`

Screen: Vehicles (18)

GET `/vehicles` → `vehicles_list` (line 2049)

Vehicles with compliance badges, challans and filters.

Query parameters	`age`, `dir`, `page`, `per_page`, `sort`, `status`, `type`, `vehicle`
Helpers run	`_body_type_color`, `_body_type_icon`, `_expiry_bucket`, `_short_body_type`, `_vehicle_age_years`
Writes	nothing (read-only)
Reads	`trips`, `vehicles`
Page shown	`vehicles_list.html` — cards: Total Vehicles, Active, In Maintenance, Insurance Expiring, PUC Expiring, Permit Expiring; columns: Vehicle No, Age ■, Insurance, Fitness, PUC, Permit, Road Tax, Status, Challan ■; sections: Vehicles, Edit Vehicle —, Add Vehicle
Data given to the page	`active`, `active_count`, `all_vehicle_nos`, `base_qs`, `body_type_dist`, `challan_total_amount`, `challan_vehicles_count`, `compliance_by_vehicle`, `compliance_overview`, `f_age`, `f_dir`, `f_sort`, `f_status`, `f_type`, `f_vehicle`, `fit_expiring`, `inactive_count`, `ins_expiring`, `maint_count`, `page`, `page_tokens`, `per_page`, `permit_expiring`, `puc_expiring`, `rows`, `tab` (+2 more)

GET `/vehicles/<int:v_id>` → `vehicle_detail` (line 1944)

One vehicle: documents, compliance, challans, trips, maintenance.

Query parameters	`tab`
Helpers run	`_vehicle_detail_data`, `_body_type_icon`, `_rc_month_year`, `_short_body_type`, `_vehicle_age_years`
Writes	nothing (read-only)
Reads	`insurance_policies`, `maintenance`, `settings`, `vehicle_challans`, `vehicles`, `vendors`
Page shown	`vehicle_detail.html` — cards: Vehicle Type, Manufacturer, Model, Fuel Type, Year, Vehicle Age; columns: Policy Number, Insurer, Type, Start Date, End Date, IDV, Status; sections: Insurance —, Edit Permit —, Edit Fitness —, Edit PUC —
Data given to the page	`active`, `tab`
Then	redirects to `vehicles_list`

GET `/vehicles/<int:v_id>/challans` → `vehicle_challan_detail` (line 1840)

Redirect to the vehicle's eChallan tab.

Writes	nothing (read-only)
Then	redirects to `vehicle_detail`

GET `/vehicles/<int:v_id>/rc-lookup` → `vehicle_rc_lookup` (line 1683)

Shows the RC details last cached from eChallan (no live call is made).

Writes	nothing (read-only)
Reads	`vehicles`
Page shown	`vehicle_rc_view.html` — cards: Registration No., Status, Status As On, Registration Date, Registration Valid Upto, RTO Code; sections: RC Details
Data given to the page	`active`, `rc_last_synced`, `result`, `v`
Then	redirects to `vehicles_list`

GET `/vehicles/<int:v_id>/rc-lookup/download` → `vehicle_rc_download` (line 1705)

RC details as a PDF.

Helpers run	`_build_rc_pdf`, `_get_invoice_settings`
Writes	nothing (read-only)
Reads	`settings`, `vehicles`
Then	redirects to `vehicle_rc_lookup`

POST `/vehicles/<int:v_id>/sync-now` → `vehicle_sync_now` (line 1961)

Own vehicles only: with an RC key saved, fetches RC and challans from eChallan now (paid) and stores them.

Form fields read	`return_to`
Writes	**INSERT** `vehicle_challans` (amount_of_fine_imposed, api_id, challan_date_time, challan_no, challan_place, challan_status, court_address, court_name, date_of_proceeding, department, dl_no, document_impounded, driver_name, fine_imposed, last_synced, name_of_violator, +11 more); **DELETE** `vehicle_challans`; **UPDATE** `vehicles` (challan_amount, challan_count, challan_last_synced, rc_last_synced, rc_synced_data)
Reads	`settings`, `vehicles`
Then	redirects to `vehicle_detail`, `vehicles_list`

POST `/vehicles/<int:vehicle_id>/compliance/<ctype>/renew` → `vehicle_compliance_renew` (line 7650)

Record a renewal of insurance / fitness / PUC / permit.

Form fields read	`document_number`, `permit_subtype`, `return_to`, `valid_upto`
Writes	**INSERT** `vehicle_compliance` (compliance_type, document_number, permit_subtype, source, sync_status, valid_upto, vehicle_id); **UPDATE** `vehicle_compliance` (document_number, permit_subtype, source, sync_status, valid_upto); **UPDATE** `vehicles`
Reads	`vehicle_compliance`
Then	redirects to `vehicles_list`

POST `/vehicles/<int:vehicle_id>/compliance/sync` → `vehicle_compliance_sync` (line 7698)

Refreshes one vehicle's fitness/PUC/permit through the compliance providers (mock data until real keys exist).

Writes	nothing (read-only)
Then	redirects to `vehicles_list`

GET / POST `/vehicles/add` → `add_vehicle` (line 7343)

Adds a vehicle, or updates it when that vehicle number already exists (own vehicles also mirror fitness/PUC/permit dates into vehicle_compliance).

Form fields read	`body_type`, `capacity_mt`, `chassis_number`, `engine_number`, `fitness_expiry`, `notes`, `permit_valid_upto`, `puc_valid_upto`, `registration_date`, `status`, `type`, `vehicle_no`
Helpers run	`_insert_vehicle_from_form`, `_mirror_vehicle_compliance_from_form`
Writes	**INSERT** `vehicle_compliance` (compliance_type, source, sync_status, valid_upto, vehicle_id); **UPDATE** `vehicle_compliance` (source, sync_status, valid_upto); **INSERT** `vehicles` (body_type, capacity_mt, chassis_number, engine_number, fitness_expiry, notes, permit_valid_upto, puc_valid_upto, registration_date, status, type, vehicle_no); **UPDATE** `vehicles` (body_type, capacity_mt, chassis_number, engine_number, fitness_expiry, notes, permit_valid_upto, puc_valid_upto, registration_date, status, type)
Reads	`vehicle_compliance`, `vehicles`
Page shown	`add_vehicle.html` — sections: Add Vehicle
Data given to the page	`active`
Then	redirects to `vehicles_list`

POST `/vehicles/compliance/sync-all` → `vehicles_compliance_sync_all` (line 1673)

Same refresh for the whole own fleet.

Writes	nothing (read-only)
Then	redirects to `vehicles_list`

POST `/vehicles/delete/<int:vehicle_id>` → `delete_vehicle` (line 7642)

Deletes the vehicle row; there is no check for trips or bills that still use it.

Writes	**DELETE** `vehicles`
Then	redirects to `vehicles_list`

GET / POST `/vehicles/edit/<int:vehicle_id>` → `edit_vehicle` (line 7594)

Edit a vehicle.

Form fields read	`body_type`, `capacity_mt`, `chassis_number`, `engine_number`, `fitness_expiry`, `notes`, `permit_valid_upto`, `puc_valid_upto`, `registration_date`, `status`, `type`, `vehicle_no`
Helpers run	`_mirror_vehicle_compliance_from_form`
Writes	**INSERT** `vehicle_compliance` (compliance_type, source, sync_status, valid_upto, vehicle_id); **UPDATE** `vehicle_compliance` (source, sync_status, valid_upto); **UPDATE** `vehicles` (body_type, capacity_mt, chassis_number, engine_number, fitness_expiry, notes, permit_valid_upto, puc_valid_upto, registration_date, status, type, vehicle_no)
Reads	`vehicle_compliance`, `vehicles`
Page shown	`edit_vehicle.html` — sections: Edit Vehicle
Data given to the page	`active`, `v`
Then	redirects to `vehicles_list`

POST `/vehicles/toggle_status/<int:vehicle_id>` → `toggle_vehicle_status` (line 7627)

Activate / deactivate a vehicle.

Query parameters	`tab`
Writes	**UPDATE** `vehicles` (status)
Reads	`vehicles`
Then	redirects to `vehicles_list`

GET `/app/compliance` → `app_compliance` (line 13497)

Phone compliance list.

Query parameters	`compliance_type`, `date_from`, `date_to`, `page`, `per_page`, `search`, `show`, `vehicle`
Helpers run	`_compliance_expense_tab_context`, `_month_bounds`
Writes	nothing (read-only)
Reads	`compliance_expenses`, `settings`, `users`, `vehicles`, `vendors`
Page shown	`app/compliance.html` — sections: Compliance, Filters, Compliance Expense List (), Compliance Expenses Trend (6 months), Expenses by Compliance Type
Data given to the page	`active`, `company_name`, `compliance_expense_colors`, `has_more`, `notif_count`, `rows`, `show`, `total_count`, `user_initials`

POST `/app/compliance/add` → `app_add_compliance` (line 13533)

Phone: record a compliance fee.

Form fields read	`amount`, `compliance_type`, `date`, `description`, `notes`, `paid_amount`, `payment_mode`, `vehicle_no`, `vendor_name`
Helpers run	`_insert_compliance_expense_from_form`, `get_or_create_vendor`
Writes	**INSERT** `compliance_expenses` (amount, compliance_type, date, description, maintenance_id, notes, payment_mode, vehicle_id, vendor_id); **INSERT** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Then	redirects to `app_compliance`; answers with JSON

GET `/app/vehicles` → `app_vehicles` (line 13885)

Phone vehicles list.

Query parameters	`age`, `search`, `show`, `status`, `type`
Helpers run	`_expiry_bucket`, `_short_body_type`, `_vehicle_age_years`
Writes	nothing (read-only)
Reads	`users`, `vehicles`
Page shown	`app/vehicles.html` — sections: Vehicles, Filters, Add Vehicle
Data given to the page	`active`, `active_count`, `active_pct`, `challan_total_amount`, `challan_vehicle_count`, `f_age`, `f_search`, `f_status`, `f_type`, `has_more`, `ins_expiring`, `maint_count`, `notif_count`, `show`, `total_count`, `total_vehicles`, `user_initials`, `vehicles`

GET `/app/vehicles/<int:v_id>` → `app_vehicle_detail` (line 14035)

Phone vehicle detail.

Query parameters	`tab`
Helpers run	`_vehicle_detail_data`, `_body_type_icon`, `_rc_month_year`, `_short_body_type`, `_vehicle_age_years`
Writes	nothing (read-only)
Reads	`insurance_policies`, `maintenance`, `settings`, `users`, `vehicle_challans`, `vehicles`, `vendors`
Page shown	`app/vehicle_detail.html` — cards: Vehicle Type, Manufacturer, Model, Fuel Type, Year, Vehicle Age
Data given to the page	`active`, `company_name`, `notif_count`, `tab`, `user_initials`
Then	redirects to `app_vehicles`

POST `/app/vehicles/add` → `app_add_vehicle` (line 14064)

Phone add-vehicle.

Form fields read	`body_type`, `capacity_mt`, `chassis_number`, `engine_number`, `fitness_expiry`, `notes`, `permit_valid_upto`, `puc_valid_upto`, `registration_date`, `status`, `type`, `vehicle_no`
Helpers run	`_insert_vehicle_from_form`, `_mirror_vehicle_compliance_from_form`
Writes	**INSERT** `vehicle_compliance` (compliance_type, source, sync_status, valid_upto, vehicle_id); **UPDATE** `vehicle_compliance` (source, sync_status, valid_upto); **INSERT** `vehicles` (body_type, capacity_mt, chassis_number, engine_number, fitness_expiry, notes, permit_valid_upto, puc_valid_upto, registration_date, status, type, vehicle_no); **UPDATE** `vehicles` (body_type, capacity_mt, chassis_number, engine_number, fitness_expiry, notes, permit_valid_upto, puc_valid_upto, registration_date, status, type)
Reads	`vehicle_compliance`, `vehicles`
Then	redirects to `app_vehicles`; answers with JSON

Screen: Maintenance (50)

GET `/maintenance` → `maintenance_list` (line 3409)

Maintenance hub; tab parameter picks Overview, Service, Tyres, Battery, Urea, Toll or Compliance.

Query parameters	`action`, `available`, `battery_type`, `brand`, `compliance_type`, `date_from`, `date_to`, `error`, `location`, `lv`, `page`, `per_page`, `position`, `search`, `service_type`, `source`, `status`, `supplier`, `tab`, `type`, `vehicle`, `vendor`, `workshop`
Helpers run	`_maintenance_battery_tab`, `_battery_management_data`, `_battery_enrich`, `_battery_status`, `_battery_warranty_upto`, `_get_autocomplete_lists`, `_maintenance_category_tab`, `_maintenance_classify`, `_maintenance_compliance_tab`, `_compliance_expense_tab_context`, `_month_bounds`, `_maintenance_overview_tab`, `_maintenance_overview_data`, `_vehicle_active_idle_days` (+11 more)
Writes	nothing (read-only)
Reads	`batteries`, `battery_checks`, `compliance_expenses`, `maintenance`, `maintenance_items`, `parties`, `settings`, `toll_entries`, `trips`, `tyre_stock`, `urea_transactions`, `vehicles`, `vendors`
Page shown	`maintenance.html` — cards: Total Fleet, Vehicles Serviced, Pending Payments, Total Maintenance Cost, Avg Cost / KM, Fleet Availability; columns: Date, Vehicle, Amount, Cost, Service Type, KM Reading, Workshop, Status, Next Due; sections: Maintenance, Urea Entry —, Toll Entry —, Compliance Expense —, Add New Service
Data given to the page	`action_breakdown`, `actions`, `active`, `avg_consumption`, `avg_cost`, `avg_cost_km`, `avg_health_pct`, `avg_life_months`, `avg_unit_cost`, `base_qs`, `battery_brands`, `battery_types`, `battery_vendors`, `breakdown_count`, `category_cards`, `chart_h`, `chart_w`, `checklist_items`, `checks_by_battery`, `combined_names`, `completed_jobs`, `cost_per_battery`, `current_stock`, `dead_count`, `due_soon`, `f_action` (+78 more)
Then	redirects to `vehicles_list`

GET `/maintenance/<int:m_id>/view` → `maintenance_view` (line 7080)

One maintenance bill in full.

Writes	nothing (read-only)
Reads	`maintenance`, `vehicles`, `vendors`
Page shown	`maintenance_view.html` — cards: Amount, Paid Amount, Balance, Date, Category, Status; sections: Maintenance Details
Data given to the page	`active`, `m`
Then	redirects to `maintenance_list`

GET / POST `/maintenance/add` → `add_maintenance` (line 4782)

Generic maintenance entry form.

Form fields read	`amount`, `category`, `date`, `notes`, `paid_amount`, `vehicle_no`, `vendor_name`
Helpers run	`_get_autocomplete_lists`, `_get_or_create_vehicle_no_type`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Page shown	`add_maintenance.html` — sections: Add Maintenance
Data given to the page	`active`, `combined_names`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/battery/<int:battery_id>/check` → `check_battery` (line 5207)

Log a battery health check.

Form fields read	`date`, `health_pct`, `remarks`, `temp_c`, `voltage`
Writes	**UPDATE** `batteries` (health_pct, last_checked_date, temp_c, voltage); **INSERT** `battery_checks` (battery_id, date, event, health_pct, remarks, temp_c, voltage)
Then	redirects to `maintenance_list`

POST `/maintenance/battery/<int:battery_id>/dead` → `mark_battery_dead` (line 5226)

Mark a battery dead.

Writes	**UPDATE** `batteries` (status_override); **INSERT** `battery_checks` (battery_id, date, event)
Then	redirects to `maintenance_list`

POST `/maintenance/battery/<int:battery_id>/delete` → `delete_battery` (line 5239)

Deletes a battery that is not fitted, with its checks and purchase bill.

Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `batteries`; **DELETE** `battery_checks`; **DELETE** `maintenance`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount)
Reads	`batteries`, `payment_allocations`
Then	redirects to `maintenance_list`

POST `/maintenance/battery/<int:battery_id>/install` → `install_battery` (line 5199)

Fit a battery on a vehicle.

Form fields read	`install_date`, `installed_location`, `vehicle_no`
Helpers run	`_install_battery_from_form`, `_get_or_create_vehicle_no_type`
Writes	**UPDATE** `batteries` (install_date, installed_location, last_checked_date, vehicle_id); **INSERT** `battery_checks` (battery_id, date, event); **UPDATE** `maintenance` (vehicle_id); **INSERT** `vehicles` (vehicle_no)
Reads	`batteries`, `vehicles`
Then	redirects to `maintenance_list`

POST `/maintenance/battery/add` → `add_battery` (line 5163)

Add a battery (serial, warranty).

Form fields read	`battery_type`, `brand`, `capacity_ah`, `health_pct`, `install_date`, `installed_location`, `invoice_no`, `mode`, `model`, `notes`, `paid_amount`, `purchase_date`, `purchase_price`, `serial_no`, `temp_c`, `vehicle_no`, `vendor_name`, `voltage`, `voltage_rating`, `warranty_months`
Helpers run	`_insert_battery_from_form`, `_get_or_create_vehicle_no_type`, `_next_battery_no`, `get_or_create_vendor`
Writes	**INSERT** `batteries` (battery_no, battery_type, brand, capacity_ah, health_pct, install_date, installed_location, invoice_no, last_checked_date, maintenance_id, model, notes, purchase_date, purchase_price, serial_no, temp_c, +5 more); **INSERT** `battery_checks` (battery_id, date, event, health_pct, temp_c, voltage); **INSERT** `maintenance` (amount, category, date, invoice_no, notes, paid_amount, status, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`batteries`, `parties`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/compliance/<int:entry_id>/delete` → `delete_compliance_expense` (line 5434)

Delete a compliance fee.

Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `compliance_expenses`; **DELETE** `maintenance`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount)
Reads	`compliance_expenses`, `payment_allocations`
Then	redirects to `maintenance_list`

POST `/maintenance/compliance/add` → `add_compliance_expense` (line 5427)

Records a fitness/permit/PUC fee: a compliance_expenses row plus its mirrored maintenance bill.

Form fields read	`amount`, `compliance_type`, `date`, `description`, `notes`, `paid_amount`, `payment_mode`, `vehicle_no`, `vendor_name`
Helpers run	`_insert_compliance_expense_from_form`, `get_or_create_vendor`
Writes	**INSERT** `compliance_expenses` (amount, compliance_type, date, description, maintenance_id, notes, payment_mode, vehicle_id, vendor_id); **INSERT** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/delete/<int:m_id>` → `delete_maintenance` (line 7040)

Delete a bill (and its allocations).

Form fields read	`tab`
Query parameters	`tab`
Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `maintenance`; **DELETE** `maintenance_items`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount)
Reads	`payment_allocations`
Then	redirects to `maintenance_list`, `vehicles_list`

GET / POST `/maintenance/edit/<int:m_id>` → `edit_maintenance` (line 7052)

Edit a maintenance bill.

Form fields read	`amount`, `category`, `date`, `notes`, `paid_amount`, `return_to`, `vehicle_no`, `vendor_name`
Query parameters	`return_to`
Helpers run	`_enforce_maintenance_paid_floor`, `_get_autocomplete_lists`, `_get_or_create_vehicle_no_type`, `get_or_create_vendor`
Writes	**UPDATE** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`maintenance`, `parties`, `payment_allocations`, `vehicles`, `vendors`
Page shown	`edit_maintenance.html` — sections: Edit Maintenance
Data given to the page	`active`, `combined_names`, `m`, `return_to`, `vehicles`
Then	redirects to `maintenance_list`

GET `/maintenance/export` → `export_maintenance` (line 7468)

Excel export.

Query parameters	`category`, `date_from`, `date_to`, `vehicle`, `vendor`
Writes	nothing (read-only)
Reads	`maintenance`, `vehicles`, `vendors`

GET `/maintenance/export/pdf` → `export_maintenance_pdf` (line 7508)

PDF export.

Query parameters	`category`, `date_from`, `date_to`, `vehicle`, `vendor`
Writes	nothing (read-only)
Reads	`maintenance`, `vehicles`, `vendors`

POST `/maintenance/insurance/<int:policy_id>/delete` → `delete_insurance` (line 5915)

Delete a policy.

Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `insurance_policies`; **DELETE** `maintenance`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount)
Reads	`insurance_policies`, `payment_allocations`
Then	redirects to `vehicles_list`

POST `/maintenance/insurance/<int:policy_id>/edit` → `edit_insurance` (line 5872)

Edit a policy.

Form fields read	`agent_contact`, `agent_email`, `agent_name`, `expiry_date`, `gst_included`, `idv`, `insurance_type`, `insurer_name`, `invoice_doc`, `ncb_pct`, `notes`, `paid_amount`, `policy_doc`, `policy_number`, `policy_status`, `premium_amount`, `rc_doc`, `reminder_days`, `return_to`, `start_date`, `vehicle_no`
Helpers run	`_enforce_maintenance_paid_floor`, `_get_or_create_vehicle_no_type`, `_save_insurance_doc`, `get_or_create_vendor`
Writes	**UPDATE** `insurance_policies` (agent_contact, agent_email, agent_name, expiry_date, gst_included, idv, insurance_type, insurer_id, invoice_doc_path, ncb_pct, notes, policy_doc_path, policy_number, premium_amount, rc_doc_path, reminder_days, +3 more); **UPDATE** `maintenance` (amount, date, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **UPDATE** `vehicles` (insurance_expiry); **INSERT** `vendors` (linked_party_id, name)
Reads	`insurance_policies`, `parties`, `payment_allocations`, `vehicles`, `vendors`
Then	redirects to `vehicles_list`

POST `/maintenance/insurance/add` → `add_insurance` (line 5825)

Adds a policy (+ up to three documents) with a premium bill, and updates the vehicle's insurance expiry.

Form fields read	`agent_contact`, `agent_email`, `agent_name`, `expiry_date`, `gst_included`, `idv`, `insurance_type`, `insurer_name`, `invoice_doc`, `ncb_pct`, `notes`, `paid_amount`, `policy_doc`, `policy_number`, `policy_status`, `premium_amount`, `rc_doc`, `reminder_days`, `return_to`, `start_date`, `vehicle_no`
Helpers run	`_get_or_create_vehicle_no_type`, `_save_insurance_doc`, `get_or_create_vendor`
Writes	**INSERT** `insurance_policies` (agent_contact, agent_email, agent_name, expiry_date, gst_included, idv, insurance_type, insurer_id, invoice_doc_path, maintenance_id, ncb_pct, notes, policy_doc_path, policy_number, premium_amount, rc_doc_path, +4 more); **INSERT** `maintenance` (amount, category, date, notes, paid_amount, status, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **UPDATE** `vehicles` (insurance_expiry); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Then	redirects to `vehicles_list`

POST `/maintenance/service/add` → `add_service` (line 4858)

Adds a service bill (vehicle, type, amount, paid, vendor, km, next due, checklist, item lines).

Form fields read	`amount`, `checklist`, `date`, `invoice_date`, `invoice_no`, `item_category`, `item_name`, `item_qty`, `item_rate`, `item_unit`, `km_reading`, `next_due_km`, `next_service_date`, `notes`, `paid_amount`, `service_type`, `status`, `vehicle_no`, `vendor_name`
Helpers run	`_insert_service_from_form`, `_get_or_create_vehicle_no_type`, `_save_service_items`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, checklist_done, date, invoice_date, invoice_no, km_reading, next_due_km, next_service_date, notes, paid_amount, service_type, status, vehicle_id, vendor_id); **INSERT** `maintenance_items` (amount, category, item_name, maintenance_id, qty, rate, unit); **DELETE** `maintenance_items`; **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/service/edit/<int:m_id>` → `edit_service` (line 4889)

Edit a service record.

Form fields read	`amount`, `checklist`, `date`, `invoice_date`, `invoice_no`, `item_category`, `item_name`, `item_qty`, `item_rate`, `item_unit`, `km_reading`, `next_due_km`, `next_service_date`, `notes`, `paid_amount`, `service_type`, `status`, `vehicle_no`, `vendor_name`
Helpers run	`_update_service_from_form`, `_enforce_maintenance_paid_floor`, `_get_or_create_vehicle_no_type`, `_save_service_items`, `get_or_create_vendor`
Writes	**UPDATE** `maintenance` (amount, category, checklist_done, date, invoice_date, invoice_no, km_reading, next_due_km, next_service_date, notes, paid_amount, service_type, status, vehicle_id, vendor_id); **INSERT** `maintenance_items` (amount, category, item_name, maintenance_id, qty, rate, unit); **DELETE** `maintenance_items`; **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `payment_allocations`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/toll/<int:entry_id>/approve` → `approve_toll` (line 5447)

Approve a pending toll entry.

Writes	**UPDATE** `toll_entries` (status)
Then	redirects to `maintenance_list`

POST `/maintenance/toll/<int:entry_id>/delete` → `delete_toll` (line 5373)

Deletes one toll entry and its linked maintenance bill.

Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `maintenance`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount); **DELETE** `toll_entries`
Reads	`payment_allocations`, `toll_entries`
Then	redirects to `maintenance_list`

POST `/maintenance/toll/<int:entry_id>/reject` → `reject_toll` (line 5456)

Reject a pending toll entry.

Writes	**UPDATE** `toll_entries` (status)
Then	redirects to `maintenance_list`

POST `/maintenance/toll/add` → `add_toll` (line 5366)

Adds one toll entry plus its mirrored maintenance bill (receipt file optional).

Form fields read	`amount`, `date`, `highway`, `notes`, `payment_mode`, `receipt`, `source`, `state`, `time`, `toll_plaza`, `trip_lr`, `vehicle_no`
Helpers run	`_insert_toll_from_form`, `_save_toll_receipt`
Writes	**INSERT** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id); **INSERT** `toll_entries` (amount, date, highway, maintenance_id, notes, payment_mode, receipt_path, source, state, status, time, toll_plaza, trip_id, vehicle_id)
Reads	`trips`, `vehicles`
Then	redirects to `maintenance_list`

POST `/maintenance/toll/excel/import` → `toll_excel_import` (line 5763)

Import the previewed toll rows.

Form fields read	`auto_approve`, `auto_link_trips`, `override_changed`, `token`
Writes	**INSERT** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id); **UPDATE** `maintenance` (amount, date, notes, paid_amount, vehicle_id); **INSERT** `toll_entries` (amount, date, highway, maintenance_id, notes, payment_mode, reference_no, source, state, status, time, toll_plaza, trip_id, vehicle_id); **UPDATE** `toll_entries` (amount, date, highway, notes, payment_mode, state, time, toll_plaza, trip_id, vehicle_id)
Reads	`toll_entries`, `trips`
Then	redirects to `maintenance_list`

POST `/maintenance/toll/excel/preview` → `toll_excel_preview` (line 5726)

Upload an Excel file and preview what would be imported (writes nothing).

Form fields read	`excel_file`
Query parameters	`date_from`, `date_to`, `page`, `per_page`, `search`, `source`, `status`, `vehicle`
Helpers run	`_parse_toll_excel`, `_toll_tab_base_context`, `_month_bounds`
Writes	nothing (read-only)
Reads	`settings`, `toll_entries`, `trips`, `vehicles`
Page shown	`maintenance.html` — cards: Total Fleet, Vehicles Serviced, Pending Payments, Total Maintenance Cost, Avg Cost / KM, Fleet Availability; columns: Date, Vehicle, Amount, Cost, Service Type, KM Reading, Workshop, Status, Next Due; sections: Maintenance, Urea Entry —, Toll Entry —, Compliance Expense —, Add New Service
Data given to the page	`active`, `excel_changed_count`, `excel_credit_excl_amount`, `excel_credit_excl_count`, `excel_dup_count`, `excel_error`, `excel_format`, `excel_invalid_count`, `excel_row_count`, `excel_rows`, `excel_token`, `excel_valid_amount`, `excel_valid_count`, `tab`
Then	redirects to `maintenance_list`

POST `/maintenance/toll/sync-fastag` → `sync_fastag_toll` (line 5465)

Pull FASTag toll lines into the toll table.

Writes	**INSERT** `maintenance` (amount, category, date, paid_amount, vehicle_id); **INSERT** `settings` (key, value); **INSERT** `toll_entries` (amount, date, highway, maintenance_id, payment_mode, reference_no, source, state, status, time, toll_plaza, vehicle_id)
Reads	`toll_entries`, `vehicles`
Then	redirects to `maintenance_list`

GET `/maintenance/toll/template` → `toll_excel_template` (line 5515)

Download the toll Excel import template.

Writes	nothing (read-only)

POST `/maintenance/tyre/add` → `add_tyre` (line 4936)

Record a tyre purchase / change.

Form fields read	`amount`, `date`, `invoice_date`, `invoice_no`, `km_reading`, `notes`, `paid_amount`, `stock_id`, `tyre_action`, `tyre_brand`, `tyre_id`, `tyre_position`, `vehicle_no`, `vendor_name`
Helpers run	`_get_or_create_vehicle_no_type`, `_insert_tyre_from_form`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, invoice_date, invoice_no, km_reading, notes, paid_amount, status, tyre_action, tyre_brand, tyre_id, tyre_position, vehicle_id, vendor_id); **UPDATE** `maintenance` (km_reading, tyre_action, tyre_position, vehicle_id); **UPDATE** `tyre_stock` (installed_date, installed_position, installed_vehicle_id, status); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `tyre_stock`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/tyre/edit/<int:m_id>` → `edit_tyre` (line 4967)

Edit a tyre record.

Form fields read	`amount`, `date`, `invoice_date`, `invoice_no`, `km_reading`, `notes`, `paid_amount`, `tyre_action`, `tyre_brand`, `tyre_id`, `tyre_position`, `vehicle_no`, `vendor_name`
Helpers run	`_enforce_maintenance_paid_floor`, `_get_or_create_vehicle_no_type`, `get_or_create_vendor`
Writes	**UPDATE** `maintenance` (amount, date, invoice_date, invoice_no, km_reading, notes, paid_amount, tyre_action, tyre_brand, tyre_id, tyre_position, vehicle_id, vendor_id); **UPDATE** `tyre_stock` (installed_position, installed_vehicle_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `payment_allocations`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/tyre/stock/<int:stock_id>/delete` → `delete_tyre_stock` (line 5081)

Deletes a tyre that is still In Stock together with its purchase bill.

Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `maintenance`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount); **DELETE** `tyre_stock`
Reads	`payment_allocations`, `tyre_stock`
Then	redirects to `maintenance_list`

POST `/maintenance/tyre/stock/<int:stock_id>/install` → `install_tyre_stock` (line 5064)

Fit a stock tyre on a vehicle position.

Form fields read	`install_date`, `km_reading`, `position`, `vehicle_no`
Helpers run	`_install_tyre_stock_from_form`, `_get_or_create_vehicle_no_type`
Writes	**UPDATE** `maintenance` (km_reading, tyre_action, tyre_position, vehicle_id); **UPDATE** `tyre_stock` (installed_date, installed_position, installed_vehicle_id, status); **INSERT** `vehicles` (vehicle_no)
Reads	`tyre_stock`, `vehicles`
Then	redirects to `maintenance_list`

POST `/maintenance/tyre/stock/<int:stock_id>/scrap` → `scrap_tyre_stock` (line 5072)

Mark a stock tyre scrapped.

Writes	**UPDATE** `tyre_stock` (status)
Then	redirects to `maintenance_list`

POST `/maintenance/tyre/stock/add` → `add_tyre_stock` (line 5030)

Add a tyre to stock.

Form fields read	`brand`, `invoice_no`, `notes`, `paid_amount`, `purchase_cost`, `purchase_date`, `tyre_id`, `tyre_type`, `vendor_name`
Helpers run	`_insert_tyre_stock_from_form`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, invoice_no, notes, paid_amount, status, tyre_action, tyre_brand, tyre_id, vehicle_id, vendor_id); **INSERT** `tyre_stock` (brand, invoice_no, maintenance_id, notes, purchase_cost, purchase_date, status, tyre_id, tyre_type, vendor_id); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vendors`
Then	redirects to `maintenance_list`

POST `/maintenance/urea/<int:txn_id>/delete` → `delete_urea` (line 5310)

Deletes one urea movement and its linked bill.

Helpers run	`_delete_maintenance_row`
Writes	**DELETE** `maintenance`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount); **DELETE** `urea_transactions`
Reads	`payment_allocations`, `urea_transactions`
Then	redirects to `maintenance_list`

GET / POST `/maintenance/urea/<int:txn_id>/edit` → `edit_urea` (line 5282)

Edit a urea movement.

Form fields read	`batch_no`, `date`, `location`, `notes`, `odometer_km`, `quantity_l`, `unit_price`
Writes	**UPDATE** `maintenance` (amount, date, notes); **UPDATE** `urea_transactions` (batch_no, date, location, notes, odometer_km, quantity_l, total_value, unit_price)
Reads	`urea_transactions`
Then	redirects to `maintenance_list`

POST `/maintenance/urea/add` → `add_urea` (line 5262)

Urea purchase or usage.

Form fields read	`batch_no`, `date`, `invoice_no`, `location`, `mode`, `notes`, `odometer_km`, `paid_amount`, `quantity_l`, `supplier_name`, `unit_price`, `vehicle_no`
Helpers run	`_insert_urea_from_form`, `_get_or_create_vehicle_no_type`, `_parse_money`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, invoice_no, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `urea_transactions` (batch_no, date, invoice_no, location, maintenance_id, notes, odometer_km, quantity_l, source, supplier_id, total_value, txn_type, unit_price, vehicle_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `urea_transactions`, `vehicles`, `vendors`
Then	redirects to `maintenance_list`

GET `/app/battery` → `app_battery` (line 13272)

Phone batteries.

Query parameters	`battery_type`, `brand`, `date_from`, `date_to`, `search`, `show`, `status`, `vehicle`
Helpers run	`_battery_management_data`, `_battery_enrich`, `_battery_status`, `_battery_warranty_upto`, `_get_autocomplete_lists`
Writes	nothing (read-only)
Reads	`batteries`, `parties`, `settings`, `users`, `vehicles`, `vendors`
Page shown	`app/battery.html` — sections: Battery Inventory, Filters, Add Battery
Data given to the page	`active`, `avg_health_pct`, `battery_brands`, `battery_types`, `battery_vendors`, `combined_names`, `company_name`, `dead_count`, `f_battery_type`, `f_brand`, `f_date_from`, `f_date_to`, `f_search`, `f_status`, `f_vehicle`, `has_more`, `in_stock`, `in_use`, `notif_count`, `rows`, `show`, `stock_batteries`, `total_batteries`, `total_count`, `user_initials`, `vehicles` (+1 more)

POST `/app/battery/<int:battery_id>/install` → `app_install_battery` (line 13322)

Phone fit battery.

Form fields read	`install_date`, `installed_location`, `vehicle_no`
Helpers run	`_install_battery_from_form`, `_get_or_create_vehicle_no_type`
Writes	**UPDATE** `batteries` (install_date, installed_location, last_checked_date, vehicle_id); **INSERT** `battery_checks` (battery_id, date, event); **UPDATE** `maintenance` (vehicle_id); **INSERT** `vehicles` (vehicle_no)
Reads	`batteries`, `vehicles`
Then	redirects to `app_battery`; answers with JSON

POST `/app/battery/add` → `app_add_battery` (line 13305)

Phone add-battery.

Form fields read	`battery_type`, `brand`, `capacity_ah`, `health_pct`, `install_date`, `installed_location`, `invoice_no`, `mode`, `model`, `notes`, `paid_amount`, `purchase_date`, `purchase_price`, `serial_no`, `temp_c`, `vehicle_no`, `vendor_name`, `voltage`, `voltage_rating`, `warranty_months`
Helpers run	`_insert_battery_from_form`, `_get_or_create_vehicle_no_type`, `_next_battery_no`, `get_or_create_vendor`
Writes	**INSERT** `batteries` (battery_no, battery_type, brand, capacity_ah, health_pct, install_date, installed_location, invoice_no, last_checked_date, maintenance_id, model, notes, purchase_date, purchase_price, serial_no, temp_c, +5 more); **INSERT** `battery_checks` (battery_id, date, event, health_pct, temp_c, voltage); **INSERT** `maintenance` (amount, category, date, invoice_no, notes, paid_amount, status, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`batteries`, `parties`, `vehicles`, `vendors`
Then	redirects to `app_battery`; answers with JSON

GET `/app/maintenance` → `app_maintenance` (line 13078)

Phone maintenance hub.

Query parameters	`date_from`, `date_to`, `month`, `search`, `service_type`, `show`, `status`, `tab`, `vehicle`, `workshop`
Helpers run	`_get_autocomplete_lists`, `_maintenance_classify`, `_maintenance_overview_data`, `_month_bounds`, `_vehicle_active_idle_days`, `_maintenance_service_rows`
Writes	nothing (read-only)
Reads	`maintenance`, `parties`, `trips`, `users`, `vehicles`, `vendors`
Page shown	`app/maintenance.html` — sections: Maintenance, Fleet Health Overview, Action Required, Maintenance Cost Trend, Service Records
Data given to the page	`actions`, `active`, `avg_cost`, `breakdown_count`, `checklist_items`, `chosen_month`, `combined_names`, `completed_jobs`, `due_soon`, `f_date_from`, `f_date_to`, `f_search`, `f_service_type`, `f_status`, `f_vehicle`, `f_workshop`, `has_more`, `health_bands`, `month_options`, `notif_count`, `open_jobs`, `pending_count`, `period_label`, `rows`, `service_types`, `show` (+11 more)

POST `/app/maintenance/service/<int:m_id>/edit` → `app_edit_service` (line 13173)

Phone edit-service.

Form fields read	`amount`, `checklist`, `date`, `invoice_date`, `invoice_no`, `item_category`, `item_name`, `item_qty`, `item_rate`, `item_unit`, `km_reading`, `next_due_km`, `next_service_date`, `notes`, `paid_amount`, `service_type`, `status`, `vehicle_no`, `vendor_name`
Helpers run	`_update_service_from_form`, `_enforce_maintenance_paid_floor`, `_get_or_create_vehicle_no_type`, `_save_service_items`, `get_or_create_vendor`
Writes	**UPDATE** `maintenance` (amount, category, checklist_done, date, invoice_date, invoice_no, km_reading, next_due_km, next_service_date, notes, paid_amount, service_type, status, vehicle_id, vendor_id); **INSERT** `maintenance_items` (amount, category, item_name, maintenance_id, qty, rate, unit); **DELETE** `maintenance_items`; **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `payment_allocations`, `vehicles`, `vendors`
Then	redirects to `app_maintenance`; answers with JSON

POST `/app/maintenance/service/add` → `app_add_service` (line 13157)

Phone add-service.

Form fields read	`amount`, `checklist`, `date`, `invoice_date`, `invoice_no`, `item_category`, `item_name`, `item_qty`, `item_rate`, `item_unit`, `km_reading`, `next_due_km`, `next_service_date`, `notes`, `paid_amount`, `service_type`, `status`, `vehicle_no`, `vendor_name`
Helpers run	`_insert_service_from_form`, `_get_or_create_vehicle_no_type`, `_save_service_items`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, checklist_done, date, invoice_date, invoice_no, km_reading, next_due_km, next_service_date, notes, paid_amount, service_type, status, vehicle_id, vendor_id); **INSERT** `maintenance_items` (amount, category, item_name, maintenance_id, qty, rate, unit); **DELETE** `maintenance_items`; **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Then	redirects to `app_maintenance`; answers with JSON

GET `/app/toll` → `app_toll` (line 13436)

Phone tolls.

Query parameters	`date_from`, `date_to`, `page`, `per_page`, `period`, `search`, `show`, `source`, `status`, `vehicle`
Helpers run	`_resolve_toll_period`, `_month_bounds`, `_toll_period_presets`, `_toll_tab_base_context`
Writes	nothing (read-only)
Reads	`settings`, `toll_entries`, `trips`, `users`, `vehicles`
Page shown	`app/toll.html` — sections: Toll Management, Toll Entries, Toll Summary (Last 6 Months), Top Toll Plazas, Select Period
Data given to the page	`active`, `company_name`, `has_more`, `notif_count`, `period`, `period_label`, `period_presets`, `rows`, `show`, `total_count`, `user_initials`

POST `/app/toll/add` → `app_add_toll` (line 13481)

Phone add-toll.

Form fields read	`amount`, `date`, `highway`, `notes`, `payment_mode`, `receipt`, `source`, `state`, `time`, `toll_plaza`, `trip_lr`, `vehicle_no`
Helpers run	`_insert_toll_from_form`, `_save_toll_receipt`
Writes	**INSERT** `maintenance` (amount, category, date, notes, paid_amount, vehicle_id); **INSERT** `toll_entries` (amount, date, highway, maintenance_id, notes, payment_mode, receipt_path, source, state, status, time, toll_plaza, trip_id, vehicle_id)
Reads	`trips`, `vehicles`
Then	redirects to `app_toll`; answers with JSON

GET `/app/tyres` → `app_tyres` (line 13188)

Phone tyres.

Query parameters	`action`, `date_from`, `date_to`, `position`, `search`, `show`, `vehicle`, `vendor`
Helpers run	`_tyre_management_data`, `_get_autocomplete_lists`, `_maintenance_classify`
Writes	nothing (read-only)
Reads	`maintenance`, `parties`, `settings`, `tyre_stock`, `users`, `vehicles`, `vendors`
Page shown	`app/tyres.html` — sections: Tyre Management, Tyre Stock, Tyre Entries, Filters, Add Tyre
Data given to the page	`active`, `avg_cost`, `combined_names`, `company_name`, `f_action`, `f_date_from`, `f_date_to`, `f_position`, `f_search`, `f_vehicle`, `f_vendor`, `has_more`, `notif_count`, `puncture_count`, `rows`, `show`, `stock_in_count`, `stock_new_count`, `stock_resole_count`, `stock_rows`, `stock_value`, `total_cost`, `total_count`, `tyre_actions`, `tyre_brands`, `tyre_positions` (+5 more)

POST `/app/tyres/add` → `app_add_tyre` (line 13224)

Phone add-tyre.

Form fields read	`amount`, `date`, `invoice_date`, `invoice_no`, `km_reading`, `notes`, `paid_amount`, `tyre_action`, `tyre_brand`, `tyre_id`, `tyre_position`, `vehicle_no`, `vendor_name`
Helpers run	`_insert_tyre_from_form`, `_get_or_create_vehicle_no_type`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, invoice_date, invoice_no, km_reading, notes, paid_amount, status, tyre_action, tyre_brand, tyre_id, tyre_position, vehicle_id, vendor_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vehicles`, `vendors`
Then	redirects to `app_tyres`; answers with JSON

POST `/app/tyres/stock/<int:stock_id>/install` → `app_install_tyre_stock` (line 13256)

Phone fit tyre.

Form fields read	`install_date`, `km_reading`, `position`, `vehicle_no`
Helpers run	`_install_tyre_stock_from_form`, `_get_or_create_vehicle_no_type`
Writes	**UPDATE** `maintenance` (km_reading, tyre_action, tyre_position, vehicle_id); **UPDATE** `tyre_stock` (installed_date, installed_position, installed_vehicle_id, status); **INSERT** `vehicles` (vehicle_no)
Reads	`tyre_stock`, `vehicles`
Then	redirects to `app_tyres`; answers with JSON

POST `/app/tyres/stock/add` → `app_add_tyre_stock` (line 13240)

Phone add tyre stock.

Form fields read	`brand`, `invoice_no`, `notes`, `paid_amount`, `purchase_cost`, `purchase_date`, `tyre_id`, `tyre_type`, `vendor_name`
Helpers run	`_insert_tyre_stock_from_form`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, invoice_no, notes, paid_amount, status, tyre_action, tyre_brand, tyre_id, vehicle_id, vendor_id); **INSERT** `tyre_stock` (brand, invoice_no, maintenance_id, notes, purchase_cost, purchase_date, status, tyre_id, tyre_type, vendor_id); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vendors`
Then	redirects to `app_tyres`; answers with JSON

GET `/app/urea` → `app_urea` (line 13339)

Phone urea.

Query parameters	`date_from`, `date_to`, `location`, `search`, `show`, `supplier`, `type`
Helpers run	`_urea_management_data`, `_get_autocomplete_lists`
Writes	nothing (read-only)
Reads	`maintenance`, `parties`, `settings`, `trips`, `urea_transactions`, `users`, `vehicles`, `vendors`
Page shown	`app/urea.html` — sections: Urea Management, Urea Entries, Stock Trend (Last 30 Days), Top Vehicles by Usage (30d), Filters
Data given to the page	`active`, `avg_consumption`, `avg_unit_cost`, `combined_names`, `company_name`, `current_stock`, `f_date_from`, `f_date_to`, `f_location`, `f_search`, `f_supplier`, `f_type`, `has_more`, `locations`, `low_stock_alerts`, `month_usage`, `notif_count`, `rows`, `show`, `suppliers`, `today_usage`, `top_vehicles`, `total_count`, `total_value`, `trend_max`, `trend_rows` (+2 more)

POST `/app/urea/add` → `app_add_urea` (line 13381)	
Phone add-urea.	
Form fields read	`batch_no`, `date`, `invoice_no`, `location`, `mode`, `notes`, `odometer_km`, `paid_amount`, `quantity_l`, `supplier_name`, `unit_price`, `vehicle_no`
Helpers run	`_insert_urea_from_form`, `_get_or_create_vehicle_no_type`, `_parse_money`, `get_or_create_vendor`
Writes	**INSERT** `maintenance` (amount, category, date, invoice_no, notes, paid_amount, vehicle_id, vendor_id); **INSERT** `urea_transactions` (batch_no, date, invoice_no, location, maintenance_id, notes, odometer_km, quantity_l, source, supplier_id, total_value, txn_type, unit_price, vehicle_id); **INSERT** `vehicles` (vehicle_no); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `urea_transactions`, `vehicles`, `vendors`
Then	redirects to `app_urea`; answers with JSON

Screen: Ledger (including payments) (22)

GET `/accounts` → `accounts` (line 1145)	
Ledger list: every party and vendor with its balance.	
Query parameters	`group`, `page`, `per_page`, `role`, `search`, `status`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_filter_accounts_rows`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `trips`, `vendors`
Page shown	`home.html` — cards: Total Parties, Total Vendors, Total Receivable, Total Payable, Net Balance; columns: Name, Type, Group / Category, Phone, Email / GSTIN, Receivable, Payable, Balance, Status; sections: Ledger, Add Party / Vendor
Data given to the page	`active`, `active_parties`, `active_vendors`, `base_qs`, `f_group`, `f_role`, `f_search`, `f_status`, `groups`, `net_balance`, `page`, `page_tokens`, `payable_to`, `per_page`, `receivable_from`, `rows`, `tab_qs`, `total_count`, `total_pages`, `total_parties`, `total_payable`, `total_receivable`, `total_vendors`

POST `/accounts/<role>/<int:account_id>/toggle-status` → `toggle_account_status` (line 1266)	
Activate / deactivate a party or vendor.	
Writes	nothing (read-only)
Then	redirects to `accounts`

POST `/accounts/add` → `add_account` (line 1259)	
Add a party or vendor.	
Form fields read	`address`, `category`, `contact`, `credit_limit`, `email`, `gstin`, `name`, `opening_balance`, `role`
Helpers run	`_create_account_from_form`, `_parse_money`
Writes	**INSERT** `parties` (address, category, contact, credit_limit, email, gstin, name, opening_balance); **INSERT** `vendors` (address, category, contact, credit_limit, email, gstin, name, opening_balance)
Then	redirects to `accounts`

GET `/accounts/export` → `export_accounts` (line 1195)	
Excel export of the ledger list.	
Query parameters	`group`, `role`, `search`, `status`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_filter_accounts_rows`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `trips`, `vendors`

POST `/ledger/<entity_type>/<int:entity_id>/contact` → `update_contact` (line 9701)

Edit contact, address, GSTIN, credit limit.

Form fields read	`address`, `category`, `contact`, `credit_limit`, `email`, `gstin`, `status`
Writes	nothing (read-only)
Then	redirects to `party_ledger`, `vendor_ledger`

POST `/ledger/<entity_type>/<int:entity_id>/opening-balance` → `update_opening_balance` (line 9739)

Set a party/vendor opening balance and date.

Form fields read	`opening_balance`, `opening_balance_date`
Helpers run	`_update_ledger_opening_balance_from_form`, `_parse_money`
Writes	nothing (read-only)
Then	redirects to `accounts`, `party_ledger`, `vendor_ledger`

GET `/ledger/party/<int:party_id>` → `party_ledger` (line 5997)

One party's ledger with running balance and a payment popup.

Query parameters	`date_from`, `date_to`, `page`, `per_page`, `tab`, `txn_type`, `type`
Helpers run	`_get_party_ledger_entries`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_ledger_page_context`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_tab_filter_entries`, `_party_pending_trips`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Page shown	`ledger.html` — cards: Total Receivable, Total Payable, Net Balance, Credit Limit, Total Receivables, Total Payables; columns: Date, Type, Details, Ref / Document, Debit (■), Credit (■), Balance (■), Vehicle, LR Number; sections: Add Payment —, Edit Contact Details —
Data given to the page	`active`, `address`, `category`, `contact`, `credit_limit`, `email`, `entity_id`, `entity_type`, `export_url`, `final_balance`, `gstin`, `name`, `opening_balance`, `opening_balance_date`, `payment_url`, `pending_trips`, `return_to`, `role`, `since_date`, `status`, `total_pending_trips`
Then	redirects to `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `payment_view`, `trip_view`

GET `/ledger/party/<int:party_id>/export` → `export_party_ledger` (line 6390)

Party ledger to Excel.

Query parameters	`date_from`, `date_to`, `tab`, `txn_type`, `type`
Helpers run	`_export_ledger_entries`, `_get_party_ledger_entries`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_ledger_export_entries`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_tab_filter_entries`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Then	redirects to `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `payment_view`, `trip_view`

GET `/ledger/party/<int:party_id>/export/pdf` → `export_party_ledger_pdf` (line 6482)

Party ledger to PDF.

Query parameters	`date_from`, `date_to`, `tab`, `txn_type`, `type`
Helpers run	`_export_ledger_pdf`, `_get_invoice_settings`, `_get_party_ledger_entries`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_ledger_export_entries`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_tab_filter_entries`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `settings`, `trips`, `vehicles`, `vendors`
Then	redirects to `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `payment_view`, `trip_view`

GET `/ledger/vendor/<int:vendor_id>` → `vendor_ledger` (line 6017)

One vendor's ledger.

Query parameters	`date_from`, `date_to`, `page`, `per_page`, `tab`, `txn_type`, `type`
Helpers run	`_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_ledger_page_context`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_tab_filter_entries`, `_vendor_pending_maintenance`, `_vendor_pending_overheads`, `_vendor_pending_trips`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Page shown	`ledger.html` — cards: Total Receivable, Total Payable, Net Balance, Credit Limit, Total Receivables, Total Payables; columns: Date, Type, Details, Ref / Document, Debit (■), Credit (■), Balance (■), Vehicle, LR Number; sections: Add Payment —, Edit Contact Details —
Data given to the page	`active`, `address`, `category`, `contact`, `credit_limit`, `email`, `entity_id`, `entity_type`, `export_url`, `final_balance`, `gstin`, `name`, `opening_balance`, `opening_balance_date`, `payment_url`, `pending_maint_total`, `pending_trips`, `return_to`, `role`, `since_date`, `status`, `total_pending_trips`
Then	redirects to `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `party_ledger`, `payment_view`

GET `/ledger/vendor/<int:vendor_id>/export` → `export_vendor_ledger` (line 6399)

Vendor ledger to Excel.

Query parameters	`date_from`, `date_to`, `tab`, `txn_type`, `type`
Helpers run	`_export_ledger_entries`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_ledger_export_entries`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_tab_filter_entries`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Then	redirects to `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `payment_view`, `trip_view`

GET `/ledger/vendor/<int:vendor_id>/export/pdf` → `export_vendor_ledger_pdf` (line 6491)

Vendor ledger to PDF.

Query parameters	`date_from`, `date_to`, `tab`, `txn_type`, `type`
Helpers run	`_export_ledger_pdf`, `_get_invoice_settings`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_ledger_export_entries`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_tab_filter_entries`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `payment_allocations`, `payments`, `settings`, `trips`, `vehicles`, `vendors`
Then	redirects to `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `payment_view`, `trip_view`

GET / POST `/payment/party/<int:party_id>` → `add_party_payment` (line 6728)

Record money received from a party and allocate it to trips.

Form fields read	`amount`, `date`, `mode`, `reference_id`, `remarks`, `trip_ids`
Helpers run	`_insert_party_payment_from_form`, `_party_pending_trips`, `_payment_is_duplicate`, `_ticked_ids`, `_validated_payment_basics`, `_parse_money`, `_redirect_payment_error`
Writes	**INSERT** `payment_allocations` (amount, payment_id, trip_id); **INSERT** `payments` (amount, date, mode, party_id, payment_type, reference_id, remarks); **UPDATE** `payments` (allocated_amount); **UPDATE** `trips` (payment_received)
Reads	`parties`, `payments`, `trips`
Page shown	`add_payment.html` — sections: Add Payment —
Data given to the page	`action_url`, `active`, `name`, `role`
Then	redirects to `accounts`, `party_ledger`

GET / POST `/payment/vendor/<int:vendor_id>` → `add_vendor_payment` (line 6806)

Record money paid to a vendor and allocate it to trips, bills and expenses.

Form fields read	`amount`, `date`, `mode`, `reference_id`, `remarks`, `trip_ids`
Helpers run	`_insert_vendor_payment_from_form`, `_payment_is_duplicate`, `_settle_overhead_status`, `_overhead_alloc_sum`, `_ticked_ids`, `_validated_payment_basics`, `_parse_money`, `_vendor_pending_maintenance`, `_vendor_pending_overheads`, `_vendor_pending_trips`, `_redirect_payment_error`
Writes	**UPDATE** `maintenance` (paid_amount); **UPDATE** `overheads` (due_date, status); **INSERT** `payment_allocations` (amount, maintenance_id, overhead_id, payment_id, trip_id); **INSERT** `payments` (amount, date, mode, payment_type, reference_id, remarks, vendor_id); **UPDATE** `payments` (allocated_amount); **UPDATE** `trips` (paid_to_owner)
Reads	`maintenance`, `overheads`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Page shown	`add_payment.html` — sections: Add Payment —
Data given to the page	`action_url`, `active`, `name`, `role`
Then	redirects to `accounts`, `vendor_ledger`

GET `/payments/<int:payment_id>/view` → `payment_view` (line 6824)

One payment: what it settled.

Helpers run	`_maintenance_label`
Writes	nothing (read-only)
Reads	`maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `trips`, `vendors`
Page shown	`payment_view.html` — cards: Total Amount, Allocated, Unallocated, Date, Type, Mode; columns: LR Number, Route, Amount; sections: Payment Details
Data given to the page	`active`, `allocations`, `entity_id`, `entity_name`, `entity_type`, `leftover`, `p`
Then	redirects to `accounts`

GET `/app/ledger` → `app_ledger` (line 14284)

Phone ledger list.

Query parameters	`group`, `role`, `search`, `show`, `status`
Helpers run	`_accounts_rows`, `_bulk_balances`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `trips`, `users`, `vendors`
Page shown	`app/ledger.html` — sections: Ledger, Add Party / Vendor, Filters
Data given to the page	`active`, `active_parties`, `active_vendors`, `f_group`, `f_role`, `f_search`, `f_status`, `groups`, `has_more`, `net_balance`, `notif_count`, `payable_to`, `receivable_from`, `rows`, `show`, `total_count`, `total_parties`, `total_payable`, `total_receivable`, `total_vendors`, `user_initials`

POST `/app/ledger/<entity_type>/<int:entity_id>/opening-balance` → `app_update_ledger_opening_balance` (line 14545)

Phone: opening balance.

Form fields read	`opening_balance`, `opening_balance_date`
Helpers run	`_update_ledger_opening_balance_from_form`, `_parse_money`
Writes	nothing (read-only)
Then	answers with JSON

POST `/app/ledger/add` → `app_add_account` (line 14368)

Phone: add party/vendor.

Form fields read	`address`, `category`, `contact`, `credit_limit`, `email`, `gstin`, `name`, `opening_balance`, `return_role`, `return_search`, `role`
Helpers run	`_create_account_from_form`, `_parse_money`
Writes	**INSERT** `parties` (address, category, contact, credit_limit, email, gstin, name, opening_balance); **INSERT** `vendors` (address, category, contact, credit_limit, email, gstin, name, opening_balance)
Then	redirects to `app_ledger`

GET `/app/ledger/party/<int:party_id>` → `app_party_ledger` (line 14490)

Phone party ledger.

Query parameters	`date_from`, `date_to`, `show`, `tab`, `txn_type`, `type`
Helpers run	`_app_entity_ledger`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_get_party_ledger_entries`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_party_pending_trips`, `_tab_filter_entries`, `_vendor_pending_maintenance`, `_vendor_pending_overheads`, `_vendor_pending_trips`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `settings`, `trips`, `users`, `vehicles`, `vendors`
Page shown	`app/ledger_detail.html` — sections: Contact Information, Transactions, Add Payment —
Data given to the page	`active`, `company_name`, `notif_count`, `user_initials`
Then	redirects to `app_ledger`, `invoice_batch_edit`, `maintenance_view`, `overheads_list`, `payment_view`

POST `/app/ledger/party/<int:party_id>/payment` → `app_add_party_payment` (line 14512)	
Phone: payment from a party.	
Form fields read	`amount`, `date`, `mode`, `reference_id`, `remarks`, `trip_ids`
Helpers run	`_insert_party_payment_from_form`, `_party_pending_trips`, `_payment_is_duplicate`, `_ticked_ids`, `_validated_payment_basics`, `_parse_money`
Writes	**INSERT** `payment_allocations` (amount, payment_id, trip_id); **INSERT** `payments` (amount, date, mode, party_id, payment_type, reference_id, remarks); **UPDATE** `payments` (allocated_amount); **UPDATE** `trips` (payment_received)
Reads	`parties`, `payments`, `trips`
Then	redirects to `app_party_ledger`; answers with JSON

GET `/app/ledger/vendor/<int:vendor_id>` → `app_vendor_ledger` (line 14500)	
Phone vendor ledger.	
Query parameters	`date_from`, `date_to`, `show`, `tab`, `txn_type`, `type`
Helpers run	`_app_entity_ledger`, `_filter_ledger_entries`, `_filter_entries_by_date`, `_get_party_ledger_entries`, `_get_vendor_ledger_entries`, `_maintenance_label`, `_payment_base_detail`, `_party_pending_trips`, `_tab_filter_entries`, `_vendor_pending_maintenance`, `_vendor_pending_overheads`, `_vendor_pending_trips`
Writes	nothing (read-only)
Reads	`batteries`, `insurance_policies`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `invoices`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `settings`, `trips`, `users`, `vehicles`, `vendors`
Page shown	`app/ledger_detail.html` — sections: Contact Information, Transactions, Add Payment —
Data given to the page	`active`, `company_name`, `notif_count`, `user_initials`
Then	redirects to `app_ledger`, `app_party_ledger`, `invoice_batch_edit`, `maintenance_view`, `overheads_list`

POST `/app/ledger/vendor/<int:vendor_id>/payment` → `app_add_vendor_payment` (line 14529)	
Phone: payment to a vendor.	
Form fields read	`amount`, `date`, `mode`, `reference_id`, `remarks`, `trip_ids`
Helpers run	`_insert_vendor_payment_from_form`, `_payment_is_duplicate`, `_settle_overhead_status`, `_overhead_alloc_sum`, `_ticked_ids`, `_validated_payment_basics`, `_parse_money`, `_vendor_pending_maintenance`, `_vendor_pending_overheads`, `_vendor_pending_trips`
Writes	**UPDATE** `maintenance` (paid_amount); **UPDATE** `overheads` (due_date, status); **INSERT** `payment_allocations` (amount, maintenance_id, overhead_id, payment_id, trip_id); **INSERT** `payments` (amount, date, mode, payment_type, reference_id, remarks, vendor_id); **UPDATE** `payments` (allocated_amount); **UPDATE** `trips` (paid_to_owner)
Reads	`maintenance`, `overheads`, `payment_allocations`, `payments`, `trips`, `vehicles`, `vendors`
Then	redirects to `app_vendor_ledger`; answers with JSON

Screen: Invoices (16)

GET `/invoice-center` → `invoice_center` (line 8248)

Choose invoice type and filters; pick trips to invoice.

Query parameters	`date_from`, `date_to`, `invoice_status`, `invoice_type`, `lr_number`, `party_id`, `vehicle`, `vendor_id`
Helpers run	`_company_paid_on_hired_trip`, `_get_invoice_settings`, `_gstin_state`, `_toll_by_trip`, `_trip_toll`, `get_or_create_vendor`
Writes	**INSERT** `vendors` (linked_party_id, name)
Reads	`invoice_batch_trips`, `invoice_items`, `parties`, `settings`, `toll_entries`, `trips`, `vehicles`, `vendors`
Page shown	`invoice_center.html` — cards: GSTIN, State; columns: LR Number, Vehicle, From To, Date, Freight (■), Status; sections: Invoice Center, 1. Select Invoice Type, 2. Select, 3. Filter Trips, 4. Select Trips to Include in Invoice
Data given to the page	`active`, `entity_gstin`, `entity_state`, `f_date_from`, `f_date_to`, `f_lr`, `f_party_id`, `f_status`, `f_vehicle`, `f_vendor_id`, `invoice_settings`, `invoice_type`, `owner_options`, `parties`, `selected_party`, `selected_vendor`, `today`, `trip_charges_map`, `trip_rows`, `vehicles`, `vendors`
Then	redirects to `invoice_center`

POST `/invoice-center/generate` → `invoice_center_generate` (line 9172)

Generate, save as draft or preview an invoice (PDF).

Form fields read	`due_date`, `excluded_charge_keys`, `extra_item_amount`, `extra_item_desc`, `extra_item_type`, `gst_rate`, `included_charge_keys`, `invoice_date`, `invoice_type`, `loading_charges`, `mode`, `other_charges`, `party_id`, `payment_status`, `payment_terms`, `place_of_supply`, `remarks`, `reviewed_trip_ids`, `tds_rate`, `trip_ids`, `vendor_id`
Helpers run	`_build_invoice_pdf`, `_company_paid_on_hired_trip`, `_trip_freight`, `_trip_toll`, `number_to_words_inr`, `_filter_trip_items`, `_get_invoice_settings`, `_invoice_numbers_error`, `_parse_money`, `_save_invoice_charge_exclusions`, `_toll_by_trip`
Writes	**INSERT** `invoice_batch_charge_exclusions` (charge_key, invoice_batch_id, trip_id); **INSERT** `invoice_batch_items` (amount, description, invoice_batch_id, item_type); **INSERT** `invoice_batch_trips` (invoice_batch_id, trip_id); **INSERT** `invoice_batches` (due_date, gst_rate, invoice_date, invoice_number, invoice_type, loading_charges, other_charges, party_id, payment_status, payment_terms, place_of_supply, remarks, status, tds_rate, vendor_id); **INSERT** `settings` (key, value)
Reads	`invoice_items`, `parties`, `settings`, `toll_entries`, `trips`, `vehicles`, `vendors`
Then	redirects to `invoice_batches_list`, `invoice_center`

POST `/invoice-center/review` → `invoice_center_review` (line 8412)

Review screen for the picked trips (every LR link must be opened).

Form fields read	`invoice_type`, `party_id`, `trip_ids`, `vendor_id`
Helpers run	`_company_paid_on_hired_trip`, `_trip_freight`
Writes	nothing (read-only)
Reads	`parties`, `settings`, `trips`, `vehicles`, `vendors`
Page shown	`invoice_review.html` — columns: LR Number, Vehicle, Date, Freight, Net Payable; sections: Invoice Summary Generate, Trips Included (), Invoice Details, Invoice Summary
Data given to the page	`account_holder`, `account_number`, `active`, `bank_name`, `entity`, `ifsc_code`, `invoice_type`, `line_items`, `party_id`, `rcm_clause`, `today`, `total_freight`, `total_trips`, `vendor_id`

GET `/invoices` → `invoices_search` (line 9485)

Search single-trip invoices.

Query parameters	`lr_number`
Writes	nothing (read-only)
Reads	`parties`, `trips`, `vehicles`
Page shown	`invoices_search.html` — columns: Date, LR Number, Vehicle, Party, From, To, Billed, Action; sections: Invoices
Data given to the page	`active`, `f_lr`, `results`

GET `/invoices/<int:trip_id>` → `invoice_preview` (line 9499)

Per-trip invoice preview.

Helpers run	`_get_invoice_settings`, `_trip_freight`
Writes	nothing (read-only)
Reads	`invoice_items`, `invoices`, `parties`, `settings`, `trips`, `vehicles`
Page shown	`invoice_preview.html` — cards: Invoice No., LR Number, Date, Due Date, Bill To, Vehicle No; columns: Description, Amount (■)
Data given to the page	`active`, `amount_paid`, `balance_due`, `charges`, `deductions`, `extra_items`, `freight`, `inv`, `invoice_total`, `s`, `t`

GET / POST `/invoices/<int:trip_id>/edit` → `invoice_edit` (line 9535)

Edit a per-trip invoice's details.

Form fields read	`due_date`, `invoice_number`, `notes`
Writes	**INSERT** `invoices` (due_date, invoice_number, notes, trip_id); **UPDATE** `invoices` (due_date, invoice_number, notes)
Reads	`invoice_items`, `invoices`, `trips`
Page shown	`invoice_edit.html` — columns: Description, Type, Amount; sections: Edit Invoice —
Data given to the page	`active`, `default_invoice_no`, `extra_items`, `inv`, `t`, `trip_id`
Then	redirects to `invoice_preview`

POST `/invoices/<int:trip_id>/items/add` → `invoice_item_add` (line 9560)

Add an extra line to a trip invoice.

Form fields read	`amount`, `description`, `item_type`
Helpers run	`_parse_money`
Writes	**INSERT** `invoice_items` (amount, description, item_type, trip_id)
Then	redirects to `invoice_edit`

GET `/invoices/<int:trip_id>/pdf` → `invoice_pdf` (line 9589)

Per-trip invoice PDF.

Helpers run	`_get_invoice_settings`, `_trip_freight`
Writes	nothing (read-only)
Reads	`invoice_items`, `invoices`, `parties`, `settings`, `trips`, `vehicles`

GET `/invoices/generated` → `invoice_batches_list` (line 9306)

All generated invoices.

Query parameters	`invoice_type`, `search`
Writes	nothing (read-only)
Reads	`invoice_batch_trips`, `invoice_batches`, `parties`, `vendors`
Page shown	`invoice_batches_list.html` — columns: Invoice No., Type, Bill To, Invoice Date, Due Date, Trips, Payment Status; sections: Generated Invoices + New Invoice
Data given to the page	`active`, `f_search`, `f_type`, `rows`

POST `/invoices/generated/<int:batch_id>/delete` → `invoice_batch_delete` (line 9390)

Delete a generated invoice.

Writes	**DELETE** `invoice_batch_items`; **DELETE** `invoice_batch_trips`; **DELETE** `invoice_batches`
Then	redirects to `invoice_batches_list`

GET / POST `/invoices/generated/<int:batch_id>/edit` → `invoice_batch_edit` (line 9329)

Edit a generated invoice.

Form fields read	`due_date`, `gst_rate`, `invoice_date`, `item_amount`, `item_desc`, `item_type`, `item_vendor`, `loading_charges`, `other_charges`, `payment_status`, `payment_terms`, `place_of_supply`, `remarks`, `tds_rate`
Helpers run	`_invoice_numbers_error`, `_parse_money`, `_save_invoice_batch_items`, `get_or_create_vendor`
Writes	**INSERT** `invoice_batch_items` (amount, description, invoice_batch_id, item_type, vendor_id); **DELETE** `invoice_batch_items`; **UPDATE** `invoice_batches` (due_date, gst_rate, invoice_date, loading_charges, other_charges, payment_status, payment_terms, place_of_supply, remarks, tds_rate); **INSERT** `vendors` (linked_party_id, name)
Reads	`invoice_batch_items`, `invoice_batches`, `parties`, `vendors`
Page shown	`invoice_batch_edit.html` — columns: Description, Vendor, Type, Amount (■); sections: Edit Invoice —, Additional Line Items, Invoice Preview
Data given to the page	`active`, `b`, `batch_items`, `vendors`
Then	redirects to `invoice_batch_edit`, `invoice_batches_list`

POST `/invoices/generated/<int:batch_id>/items/add` → `invoice_batch_item_add` (line 9364)

Add a line to a generated invoice.

Form fields read	`amount`, `description`, `item_type`, `vendor_name`
Helpers run	`_parse_money`, `get_or_create_vendor`
Writes	**INSERT** `invoice_batch_items` (amount, description, invoice_batch_id, item_type, vendor_id); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vendors`
Then	redirects to `invoice_batch_edit`

GET `/invoices/generated/<int:batch_id>/pdf` → `invoice_batch_pdf` (line 9400)

Download the invoice PDF.

Helpers run	`_build_invoice_pdf`, `_company_paid_on_hired_trip`, `_trip_freight`, `_trip_toll`, `number_to_words_inr`, `_filter_trip_items`, `_get_invoice_settings`, `_load_invoice_charge_exclusions`, `_toll_by_trip`
Writes	nothing (read-only)
Reads	`invoice_batch_charge_exclusions`, `invoice_batch_items`, `invoice_batch_trips`, `invoice_batches`, `invoice_items`, `parties`, `settings`, `toll_entries`, `trips`, `vehicles`, `vendors`

GET `/invoices/generated/<int:batch_id>/pdf/preview` → `invoice_batch_pdf_preview` (line 9442)

Inline PDF preview.

Helpers run	`_build_invoice_pdf`, `_company_paid_on_hired_trip`, `_trip_freight`, `_trip_toll`, `number_to_words_inr`, `_filter_trip_items`, `_get_invoice_settings`, `_load_invoice_charge_exclusions`, `_toll_by_trip`
Writes	nothing (read-only)
Reads	`invoice_batch_charge_exclusions`, `invoice_batch_items`, `invoice_batch_trips`, `invoice_batches`, `invoice_items`, `parties`, `settings`, `toll_entries`, `trips`, `vehicles`, `vendors`

POST `/invoices/generated/items/<int:item_id>/delete` → `invoice_batch_item_delete` (line 9381)

Remove a line.

Writes	**DELETE** `invoice_batch_items`
Reads	`invoice_batch_items`
Then	redirects to `invoice_batch_edit`, `invoice_batches_list`

POST `/invoices/items/<int:item_id>/delete` → `invoice_item_delete` (line 9576)

Remove an extra line.

Writes	**DELETE** `invoice_items`
Reads	`invoice_items`
Then	redirects to `invoice_center`, `invoice_edit`

Screen: Expenses (8)

GET `/overheads` → `overheads_list` (line 2705)

Expenses: list, categories, pending bills, recurring items.

Query parameters	`category`, `date_from`, `date_to`, `page`, `per_page`, `search`, `status`, `tab`, `vendor`
Helpers run	`_conic_gradient`, `_expense_category_visual`, `_overhead_list_data`
Writes	nothing (read-only)
Reads	`overheads`, `payment_allocations`, `vendors`
Page shown	`overheads_list.html` — cards: Total Expenses, This Month, Pending Bills, Recurring (Monthly), Avg Monthly Expense, Categories; columns: Date, Category, Vendor, Description, Payment Mode, Status, Amount, Actions, Expenses; sections: Expenses, —, Edit Expense, Add Expense
Data given to the page	`active`, `active_categories`, `avg_monthly_expense`, `base_qs`, `categories`, `category_summary_rows`, `current_year`, `f_category`, `f_date_from`, `f_date_to`, `f_status`, `f_vendor`, `month_labels_full`, `page`, `page_tokens`, `pending_amount`, `pending_count`, `per_page`, `recent_bills`, `recurring_count`, `recurring_monthly_amount`, `rows`, `status_gradient`, `status_legend`, `tab`, `this_month_amount` (+12 more)

GET / POST `/overheads/add` → `add_overhead` (line 6892)

Add an expense (optionally naming a vendor).

Form fields read	`amount`, `category`, `date`, `description`, `due_date`, `is_recurring`, `notes`, `payment_mode`, `receipt_number`, `recurring_frequency`, `status`, `vendor`
Helpers run	`_insert_overhead_from_form`, `_parse_money`, `get_or_create_vendor`
Writes	**INSERT** `overheads` (amount, category, date, description, due_date, is_recurring, notes, payment_mode, receipt_number, recurring_frequency, status, vendor); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vendors`
Page shown	`add_overhead.html` — sections: Add Expense Entry
Data given to the page	`active`
Then	redirects to `overheads_list`

POST `/overheads/delete/<int:o_id>` → `delete_overhead` (line 7253)

Delete an expense.

Helpers run	`_delete_overhead_row`
Writes	**DELETE** `overheads`; **DELETE** `payment_allocations`; **UPDATE** `payments` (allocated_amount)
Reads	`payment_allocations`
Then	redirects to `overheads_list`

POST `/overheads/edit/<int:o_id>` → `edit_overhead` (line 6939)

Edit an expense.

Form fields read	`amount`, `category`, `date`, `description`, `due_date`, `is_recurring`, `notes`, `payment_mode`, `receipt_number`, `recurring_frequency`, `status`, `vendor`
Helpers run	`_update_overhead_from_form`, `_overhead_alloc_sum`, `_parse_money`, `_settle_overhead_status`, `get_or_create_vendor`
Writes	**UPDATE** `overheads` (amount, category, date, description, due_date, is_recurring, notes, payment_mode, receipt_number, recurring_frequency, status, vendor); **INSERT** `vendors` (linked_party_id, name)
Reads	`overheads`, `parties`, `payment_allocations`, `vendors`
Then	redirects to `overheads_list`

GET `/overheads/export` → `export_overheads` (line 2826)

Excel export.

Query parameters	`category`, `date_from`, `date_to`, `payment_mode`, `status`, `tab`, `vendor`
Writes	nothing (read-only)
Reads	`overheads`

GET `/app/expenses` → `app_expenses` (line 13548)

Phone expenses.

Query parameters	`category`, `date_from`, `date_to`, `search`, `show`, `status`, `tab`, `vendor`
Helpers run	`_overhead_list_data`, `_expense_category_visual`
Writes	nothing (read-only)
Reads	`overheads`, `payment_allocations`, `settings`, `users`, `vendors`
Page shown	`app/expenses.html` — sections: Expenses, Expenses List, Filters, Add Expense, Expense
Data given to the page	`active`, `active_categories`, `avg_monthly_expense`, `categories`, `company_name`, `f_category`, `f_date_from`, `f_date_to`, `f_search`, `f_status`, `f_vendor`, `has_more`, `month_labels_full`, `notif_count`, `pending_amount`, `pending_count`, `recurring_count`, `recurring_monthly_amount`, `rows`, `show`, `this_month_amount`, `today_str`, `total_amount`, `total_count`, `total_count_filtered`, `user_initials` (+1 more)

POST `/app/expenses/<int:o_id>/edit` → `app_edit_overhead` (line 13597)

Phone edit expense.

Form fields read	`amount`, `category`, `date`, `description`, `due_date`, `is_recurring`, `notes`, `payment_mode`, `receipt_number`, `recurring_frequency`, `status`, `vendor`
Helpers run	`_update_overhead_from_form`, `_overhead_alloc_sum`, `_parse_money`, `_settle_overhead_status`, `get_or_create_vendor`
Writes	**UPDATE** `overheads` (amount, category, date, description, due_date, is_recurring, notes, payment_mode, receipt_number, recurring_frequency, status, vendor); **INSERT** `vendors` (linked_party_id, name)
Reads	`overheads`, `parties`, `payment_allocations`, `vendors`
Then	redirects to `app_expenses`; answers with JSON

POST `/app/expenses/add` → `app_add_overhead` (line 13581)

Phone add expense.

Form fields read	`amount`, `category`, `date`, `description`, `due_date`, `is_recurring`, `notes`, `payment_mode`, `receipt_number`, `recurring_frequency`, `status`, `vendor`
Helpers run	`_insert_overhead_from_form`, `_parse_money`, `get_or_create_vendor`
Writes	**INSERT** `overheads` (amount, category, date, description, due_date, is_recurring, notes, payment_mode, receipt_number, recurring_frequency, status, vendor); **INSERT** `vendors` (linked_party_id, name)
Reads	`parties`, `vendors`
Then	redirects to `app_expenses`; answers with JSON

Screen: Salaries (19)

GET `/advances` → `advances_list` (line 7566)

Advances outstanding by employee.

Writes	nothing (read-only)
Reads	`advances`
Page shown	`advances_list.html` — columns: Employee, Given, Repaid, Outstanding, Date, Type, Amount, Notes; sections: Employee Advances + Add Advance
Data given to the page	`active`, `entries`, `rows`

GET / POST `/advances/add` → `add_advance` (line 7578)

Give an advance / record repayment.

Form fields read	`amount`, `date`, `employee`, `notes`, `type`
Writes	**INSERT** `advances` (amount, date, employee, notes, type)
Page shown	`add_advance.html` — sections: Add Advance Entry
Data given to the page	`active`
Then	redirects to `advances_list`

POST `/attendance/bulk-mark` → `bulk_mark_attendance` (line 7176)

Mark attendance for many days / people.

Form fields read	`date`, `employee_ids`, `status`
Writes	**INSERT** `attendance` (date, employee_id, marked_by, status)
Then	redirects to `salaries_list`

POST `/attendance/mark` → `mark_attendance` (line 7157)

Mark one day.

Form fields read	`date`, `employee_id`, `in_time`, `out_time`, `remarks`, `status`
Writes	**INSERT** `attendance` (date, employee_id, in_time, marked_by, out_time, remarks, status)
Then	redirects to `salaries_list`

GET / POST `/employee/<employee>` → `employee_ledger` (line 7812)

One employee's ledger (opening, salary, advances).

Form fields read	`amount`, `date`, `entry_kind`, `from_tab`, `notes`, `payment_mode`
Query parameters	`date_from`, `date_to`, `from_tab`
Helpers run	`_employee_ledger_entries`, `_insert_employee_payment_from_form`, `_parse_money`
Writes	**INSERT** `advances` (amount, date, employee, notes, type); **INSERT** `salaries` (advance_recovery, amount, basic_salary, date, employee, employee_id, gross_salary, month, month_key, net_salary, payment_date, payment_mode, payment_status, remarks, total_deductions)
Reads	`advances`, `employees`, `salaries`
Page shown	`employee_ledger.html` — cards: Debit, Credit, Notes, Entry Recorded; columns: Date, Type, Debit, Credit, Advance Balance; sections: + Add Payment, —, Add Payment —
Data given to the page	`active`, `advance_balance`, `brought_forward`, `employee`, `entries`, `f_date_from`, `f_date_to`, `from_tab`, `opening_balance`, `opening_balance_date`, `total_salary_paid`
Then	redirects to `employee_ledger`

POST `/employee/<employee>/opening-balance` → `update_employee_opening_balance` (line 7854)

Employee opening balance.

Form fields read	`from_tab`, `opening_balance`, `opening_balance_date`
Helpers run	`_update_employee_opening_balance_from_form`, `_parse_money`
Writes	**UPDATE** `employees` (opening_balance, opening_balance_date)
Reads	`employees`
Then	redirects to `employee_ledger`

POST `/employees/<int:employee_id>/advance/add` → `add_employee_advance` (line 7133)

Advance from an employee card.

Form fields read	`amount`, `date`, `notes`, `return_tab`, `type`
Helpers run	`_parse_money`
Writes	**INSERT** `advances` (amount, date, employee, notes, type)
Reads	`employees`
Then	redirects to `salaries_list`

POST `/employees/<int:employee_id>/deactivate` → `deactivate_employee` (line 7120)

Mark employee inactive.

Writes	**UPDATE** `employees` (status)
Reads	`employees`
Then	redirects to `salaries_list`

POST `/employees/<int:employee_id>/edit` → `edit_employee` (line 7103)

Edit employee details.

Form fields read	`aadhaar`, `address`, `bank_account`, `basic_salary`, `date_of_birth`, `driving_license`, `email`, `emergency_contact`, `ifsc_code`, `joining_date`, `mobile`, `name`, `pan`, `return_tab`, `role`, `type`, `upi_id`
Writes	**UPDATE** `employees` (aadhaar, address, bank_account, basic_salary, date_of_birth, driving_license, email, emergency_contact, ifsc_code, joining_date, mobile, name, pan, role, type, upi_id)
Then	redirects to `salaries_list`

GET / POST `/employees/add` → `add_employee` (line 8215)

Add an employee.

Form fields read	`aadhaar`, `address`, `bank_account`, `basic_salary`, `date_of_birth`, `driving_license`, `email`, `emergency_contact`, `employee_code`, `ifsc_code`, `joining_date`, `mobile`, `name`, `next`, `pan`, `role`, `type`, `upi_id`
Helpers run	`_insert_employee_from_form`, `_parse_money`
Writes	**INSERT** `employees` (aadhaar, address, bank_account, basic_salary, date_of_birth, driving_license, email, emergency_contact, employee_code, ifsc_code, joining_date, mobile, name, pan, role, status, +2 more); **UPDATE** `employees` (employee_code)
Reads	`employees`
Page shown	`add_employee.html` — sections: Add Employee
Data given to the page	`active`
Then	redirects to `employee_ledger`, `salaries_list`

GET `/salaries` → `salaries_list` (line 2283)

Employees hub; tab = Overview, Salary or Attendance.

Query parameters	`date_from`, `date_to`, `page`, `per_page`, `role`, `search`, `status`, `tab`
Helpers run	`_employees_attendance_tab`, `_employee_attendance_stats`, `_employee_avatar_colors`, `_employee_initials`, `_employee_month_calendar`, `_employees_overview_tab`, `_employee_outstanding_advance`, `_month_bounds`, `_employees_salary_tab`, `_employees_salary_data`
Writes	nothing (read-only)
Reads	`advances`, `attendance`, `employees`, `salaries`
Page shown	`employees.html` — cards: Total Employees, Drivers, Office Staff, Present Today, On Leave, Attendance %; columns: Employee, Role, Joining Date, Attendance, Salary Status, Advance, Total Paid, Payments, Last Paid; sections: Employees, Edit Employee —, Add Employee, Mark Attendance
Data given to the page	`absent_kpi`, `active`, `all_employees_list`, `avg_salary`, `base_qs`, `calendar_month_key`, `dept_dist`, `dept_max`, `drivers`, `drivers_pct`, `employees_paid`, `f_date_from`, `f_date_to`, `f_role`, `f_search`, `f_status`, `half_kpi`, `late_kpi`, `leave_kpi`, `office_staff`, `on_leave_today`, `overall_attendance_pct`, `overall_pct`, `page`, `page_tokens`, `paid_payroll` (+18 more)

GET `/salaries/<int:salary_id>/payslip` → `download_payslip` (line 7195)

Payslip PDF.

Writes	nothing (read-only)
Reads	`employees`, `salaries`, `salary_items`, `settings`
Then	redirects to `salaries_list`

POST `/salaries/delete/<int:s_id>` → `delete_salary` (line 7094)

Delete a salary payment.

Writes	**DELETE** `salaries`; **DELETE** `salary_items`
Then	redirects to `salaries_list`

GET `/app/employee/<employee>` → `app_employee_ledger` (line 13654)

Phone employee ledger.

Query parameters	`date_from`, `date_to`, `search`, `show`
Helpers run	`_employee_ledger_entries`
Writes	nothing (read-only)
Reads	`advances`, `employees`, `salaries`, `settings`, `users`
Page shown	`app/employee_ledger.html` — sections: Ledger Entries, Filters, Add Payment —
Data given to the page	`active`, `advance_balance`, `brought_forward`, `company_name`, `employee`, `entries`, `f_date_from`, `f_date_to`, `f_search`, `has_more`, `notif_count`, `opening_balance`, `opening_balance_date`, `show`, `total_count`, `total_salary_paid`, `user_initials`

POST `/app/employee/<employee>/add` → `app_add_employee_payment` (line 13688)

Phone: salary / advance entry.

Form fields read	`amount`, `date`, `entry_kind`, `notes`, `payment_mode`
Helpers run	`_insert_employee_payment_from_form`, `_parse_money`
Writes	**INSERT** `advances` (amount, date, employee, notes, type); **INSERT** `salaries` (advance_recovery, amount, basic_salary, date, employee, employee_id, gross_salary, month, month_key, net_salary, payment_date, payment_mode, payment_status, remarks, total_deductions)
Reads	`employees`
Then	redirects to `app_employee_ledger`; answers with JSON

POST `/app/employee/<employee>/opening-balance` → `app_update_employee_opening_balance` (line 13705)

Phone: opening balance.

Form fields read	`opening_balance`, `opening_balance_date`
Helpers run	`_update_employee_opening_balance_from_form`, `_parse_money`
Writes	**UPDATE** `employees` (opening_balance, opening_balance_date)
Reads	`employees`
Then	redirects to `app_employee_ledger`; answers with JSON

GET `/app/salary` → `app_salary` (line 13614)

Phone salaries.

Query parameters	`date_from`, `date_to`, `role`, `search`, `show`
Helpers run	`_employees_salary_data`, `_employee_avatar_colors`, `_employee_initials`, `_employee_outstanding_advance`
Writes	nothing (read-only)
Reads	`advances`, `employees`, `salaries`, `settings`, `users`
Page shown	`app/salary.html` — sections: Salary, Salary Summary, Add Employee, Filters
Data given to the page	`active`, `avg_salary`, `company_name`, `employees_paid`, `f_date_from`, `f_date_to`, `f_role`, `f_search`, `has_more`, `notif_count`, `roles`, `rows`, `show`, `tab`, `total_count`, `total_employees_all`, `total_outstanding_advances`, `total_payroll`, `user_initials`

POST `/app/salary/add` → `app_add_employee` (line 13722)

Phone: add employee.

Form fields read	`aadhaar`, `address`, `bank_account`, `basic_salary`, `date_of_birth`, `driving_license`, `email`, `emergency_contact`, `employee_code`, `ifsc_code`, `joining_date`, `mobile`, `name`, `pan`, `role`, `type`, `upi_id`
Helpers run	`_insert_employee_from_form`, `_parse_money`
Writes	**INSERT** `employees` (aadhaar, address, bank_account, basic_salary, date_of_birth, driving_license, email, emergency_contact, employee_code, ifsc_code, joining_date, mobile, name, pan, role, status, +2 more); **UPDATE** `employees` (employee_code)
Reads	`employees`
Then	redirects to `app_salary`; answers with JSON

GET `/app/salary/attendance` → `app_salary_attendance` (line 13644)

Phone attendance.

Writes	nothing (read-only)
Reads	`settings`, `users`
Page shown	`app/salary.html` — sections: Salary, Salary Summary, Add Employee, Filters
Data given to the page	`active`, `company_name`, `notif_count`, `tab`, `user_initials`

Screen: Reports (19)

GET `/business-performance` → `business_performance` (line 11908)

Month/quarter/year profit comparison (revenue, costs, growth).

Query parameters	`date_from`, `date_to`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_month_bounds`, `_pct_growth`, `_period_financials`, `_quarter_bounds`, `_shift_period_back`, `_shift_period_year`, `_toll_by_trip`, `_trip_toll`
Writes	nothing (read-only)
Reads	`employees`, `invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `toll_entries`, `trips`, `vehicles`, `vendors`
Page shown	`business_performance.html` — cards: Total Revenue, Net Profit, Profit Margin, Outstanding Receivables, Outstanding Payables, Cash Collected; columns: Customer, Revenue, %, Profit; sections: Business Performance, Financial Overview, Monthly Profit Trend, Revenue Comparison, Top Customers by Revenue
Data given to the page	`active`, `avg_collection_days`, `avg_daily_profit`, `avg_daily_revenue`, `avg_invoice_value`, `avg_profit_per_trip`, `avg_revenue_per_employee`, `avg_revenue_per_trip`, `avg_revenue_per_vehicle`, `cash_collected`, `cash_paid`, `collection_efficiency`, `collection_growth`, `curr_q_revenue`, `curr_revenue`, `curr_y_revenue`, `customer_growth`, `customers_other_revenue`, `ebitda`, `exp_total`, `expense_breakdown`, `expense_growth`, `f_date_from`, `f_date_to`, `f_fy`, `health_kind` (+34 more)

GET `/business-performance/export` → `export_business_performance` (line 12255)

Excel export.

Query parameters	`date_from`, `date_to`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_month_bounds`, `_period_financials`, `_toll_by_trip`, `_trip_toll`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `toll_entries`, `trips`, `vehicles`, `vendors`

GET `/driver-performance` → `driver_performance` (line 7865)

Redirects to Performance (drivers and vehicles).

Writes	nothing (read-only)
Then	redirects to `performance`

GET `/empty-runs` → `empty_runs` (line 11519)

Redirects to Fleet Utilization.

Writes	nothing (read-only)
Then	redirects to `fleet_utilization`

GET `/fleet-utilization` → `fleet_utilization` (line 11700)

Idle vs active days, empty runs, km and fuel.

Query parameters	`date_from`, `date_to`, `vehicle`
Helpers run	`_fleet_utilization_data`, `_short_body_type`, `_toll_by_trip`, `_trip_toll`, `_vehicle_active_idle_days`
Writes	nothing (read-only)
Reads	`toll_entries`, `trips`, `vehicles`
Page shown	`fleet_utilization.html` — cards: Total Vehicles, Active Vehicles, Idle Vehicles (Any Day), Empty Run Trips, Avg Idle Days / Vehicle, Avg Days / Vehicle; columns: Vehicle No, Type, Group, Active Days (This Period), Idle Days (This Period), Total Days, Empty Run Trips, Idle %, Last Trip Date; sections: Fleet Utilization Efficiency, Idle vs Active Days (This Period), Idle Days Trend (Last 12 Weeks), Vehicle Utilization Summary, Most Idle Vehicles
Data given to the page	`active`, `active_days_pct`, `active_pct`, `active_vehicles`, `avg_active_days`, `avg_idle_days`, `chart_bottom`, `empty_rows`, `empty_run_pct_overall`, `empty_run_trips`, `f_date_from`, `f_date_to`, `f_vehicle`, `idle_days_pct`, `idle_vehicles`, `idle_vehicles_pct`, `most_idle`, `rows`, `total_active_days`, `total_empty_cost`, `total_fuel`, `total_idle_days`, `total_toll`, `total_vehicles`, `trend`, `trend_max` (+2 more)

GET `/fleet-utilization/export` → `export_fleet_utilization` (line 11720)

Excel export.

Query parameters	`date_from`, `date_to`, `vehicle`
Helpers run	`_vehicle_active_idle_days`
Writes	nothing (read-only)
Reads	`trips`, `vehicles`

GET `/idle-tracker` → `idle_tracker` (line 11514)

Redirects to Fleet Utilization.

Writes	nothing (read-only)
Then	redirects to `fleet_utilization`

GET `/performance` → `performance` (line 8057)

Driver and vehicle performance.

Query parameters	`d_page`, `date_from`, `date_to`, `per_page`, `v_page`
Helpers run	`_performance_data`, `_toll_by_trip`, `_trend_chart_coords`, `_trip_toll`
Writes	nothing (read-only)
Reads	`maintenance`, `toll_entries`, `trips`, `vehicles`
Page shown	`performance.html` — cards: Total Drivers, Completed Trips, Total Billed, Total Fuel Cost, Total Profit, Avg Trips / Driver; columns: Driver Name, Completed Trips, Total Billed, Fuel Cost, Profit, Last Trip Date, Vehicle No, Type, Maintenance; sections: Performance, Top 5 Drivers by Completed Trips, Trips Trend (All Drivers), Driver Performance Detail, Top 5 Vehicles by Completed Trips
Data given to the page	`active`, `base_qs`, `d_page`, `d_page_tokens`, `d_total_pages`, `driver_avg_trips`, `driver_chart_bottom`, `driver_monthly`, `driver_rows`, `driver_total_billed`, `driver_total_fuel`, `driver_total_profit`, `driver_total_trips`, `driver_trend_max`, `driver_y_ticks`, `f_date_from`, `f_date_to`, `per_page`, `top5_drivers`, `top5_vehicles`, `total_drivers`, `total_vehicles`, `v_page`, `v_page_tokens`, `v_total_pages`, `vehicle_avg_trips` (+10 more)

GET `/performance/export` → `export_performance` (line 8095)

Excel export.

Query parameters	`date_from`, `date_to`
Writes	nothing (read-only)
Reads	`maintenance`, `trips`, `vehicles`

GET `/route-analytics` → `route_analytics` (line 11291)

Best routes, route rates, route finder tab.

Query parameters	`date_from`, `date_to`, `from`, `page`, `per_page`, `rf_from`, `rf_to`, `rr_page`, `tab`, `to`, `vehicle_type`
Helpers run	`_route_analytics_data`, `_toll_by_trip`, `_trip_toll`, `_route_finder_ctx`, `_rf_text`, `_route_finder_texts`, `_route_rates_data`
Writes	nothing (read-only)
Reads	`toll_entries`, `trips`, `vehicles`
Page shown	`route_analytics.html` — cards: Total Routes, Total Trips, Total Revenue, Avg Rate, Avg Margin, Highest Rate; columns: Rank, Route, Trips, Revenue, Profit, Margin %, Total Revenue, Total Cost, Action; sections: Route Analytics, Top 5 Best Performing Routes (By Profit Margin %), Top Routes by Revenue, Route Summary, Route Rates ()
Data given to the page	`active`, `all_from_list`, `all_to_list`, `avg_margin`, `avg_rate`, `base_qs`, `da_average`, `da_highest`, `da_lowest`, `f_date_from`, `f_date_to`, `f_from`, `f_to`, `f_vehicle_type`, `fuel_average`, `fuel_highest`, `fuel_lowest`, `highest_entry`, `hired_avg`, `hired_pct`, `lowest_entry`, `own_avg`, `own_pct`, `page`, `page_tokens`, `per_page` (+20 more)

GET `/route-analytics/export` → `export_route_analytics` (line 11410)

Excel export.

Query parameters	`date_from`, `date_to`, `from`, `to`, `vehicle_type`
Helpers run	`_toll_by_trip`, `_trip_toll`
Writes	nothing (read-only)
Reads	`toll_entries`, `trips`

POST `/route-finder/extras` → `route_finder_extras` (line 11254)

Toll plazas, towns, petrol pumps along a chosen route.

Helpers run	`_rf_cache_get`, `_rf_cache_put`, `_rf_coord`
Writes	nothing (read-only)
Then	answers with JSON

GET `/route-finder/search` → `route_finder_search` (line 11202)

Geocode + OSRM routes for a From/To pair.

Query parameters	`from`, `from_lat`, `from_lng`, `to`, `to_lat`, `to_lng`
Helpers run	`_rf_cache_get`, `_rf_cache_put`, `_rf_coord`, `_rf_geocode`, `_rf_text`, `_route_history`, `_toll_by_trip`, `_trip_toll`
Writes	nothing (read-only)
Reads	`toll_entries`, `trips`
Then	answers with JSON

GET `/route-finder/suggest` → `route_finder_suggest` (line 11174)

Place suggestions as you type (Photon).

Query parameters	`lat`, `lon`, `q`
Helpers run	`_rf_cache_get`, `_rf_cache_put`, `_rf_coord`, `_rf_text`
Writes	nothing (read-only)
Then	answers with JSON

GET `/app/analysis` → `app_analysis` (line 14082)

Phone Business Overview hub.

Writes	nothing (read-only)
Then	redirects to `app_business_performance`

GET `/app/analytics` → `app_analytics` (line 14094)

Phone route analytics + finder.

Query parameters	`date_from`, `date_to`, `from`, `period`, `rf_from`, `rf_to`, `show`, `tab`, `to`, `vehicle_type`
Helpers run	`_analysis_period_presets`, `_resolve_analysis_period`, `_month_bounds`, `_route_analytics_data`, `_toll_by_trip`, `_trip_toll`, `_route_finder_ctx`, `_rf_text`, `_route_finder_texts`, `_route_rates_data`
Writes	nothing (read-only)
Reads	`toll_entries`, `trips`, `users`, `vehicles`
Page shown	`app/analytics.html` — sections: Route Finder, Top 5 Best Performing Routes, Top Routes by Revenue, Route Rates, Filters

GET `/app/business-performance` → `app_business_performance` (line 14219)

Phone business performance.

Query parameters	`date_from`, `date_to`, `period`
Helpers run	`_accounts_rows`, `_bulk_balances`, `_analysis_period_presets`, `_month_bounds`, `_pct_growth`, `_period_financials`, `_resolve_analysis_period`, `_shift_period_back`
Writes	nothing (read-only)
Reads	`invoice_batch_items`, `invoice_batches`, `invoice_items`, `maintenance`, `overheads`, `parties`, `payment_allocations`, `payments`, `salaries`, `settings`, `trips`, `users`, `vehicles`, `vendors`
Page shown	`app/business_performance.html` — sections: Revenue vs Expenses, Business Overview, Select Period
Data given to the page	`active`, `active_ov`, `cash_in_hand`, `company_name`, `expense_growth`, `f_date_from`, `f_date_to`, `margin`, `monthly_rows`, `net_profit`, `notif_count`, `period`, `period_label`, `period_presets`, `profit_growth`, `revenue_growth`, `total_expenses`, `total_receivables`, `total_revenue`, `trend_max`, `trip_count`, `user_initials`

GET `/app/fleet-utilization` → `app_fleet_utilization` (line 14166)

Phone utilization.

Query parameters	`date_from`, `date_to`, `period`, `tab`
Helpers run	`_analysis_period_presets`, `_fleet_utilization_data`, `_short_body_type`, `_toll_by_trip`, `_trip_toll`, `_vehicle_active_idle_days`, `_resolve_analysis_period`, `_month_bounds`
Writes	nothing (read-only)
Reads	`settings`, `toll_entries`, `trips`, `users`, `vehicles`
Page shown	`app/fleet_utilization.html` — sections: Idle Days Trend, Utilization by Vehicle Type, Most Idle Vehicles, Vehicles Running Empty Most, Routes Running Empty Most
Data given to the page	`active`, `active_ov`, `company_name`, `d`, `f_date_from`, `f_date_to`, `notif_count`, `period`, `period_label`, `period_presets`, `tab`, `user_initials`

GET `/app/performance` → `app_performance` (line 14192)

Phone performance.

Query parameters	`date_from`, `date_to`, `period`, `tab`
Helpers run	`_analysis_period_presets`, `_performance_data`, `_toll_by_trip`, `_trend_chart_coords`, `_trip_toll`, `_resolve_analysis_period`, `_month_bounds`
Writes	nothing (read-only)
Reads	`maintenance`, `settings`, `toll_entries`, `trips`, `users`, `vehicles`
Page shown	`app/performance.html` — sections: Top 5 Drivers (By Trips), Monthly Trips Trend, Top 5 Vehicles (By Trips), Business Overview, Select Period
Data given to the page	`active`, `active_ov`, `company_name`, `d`, `f_date_from`, `f_date_to`, `notif_count`, `period`, `period_label`, `period_presets`, `tab`, `user_initials`

Whole-database export (1)

GET `/export` → `export_excel` (line 12354)

Excel of the whole database (refused for Limited Access users).

Writes	nothing (read-only)
Reads	`maintenance`, `parties`, `payments`, `trips`, `vehicles`, `vendors`

42. The life of each table

For every table: which URLs create rows, which change them, which delete them, and which only read them. This is the answer to "when is this table written, and who looks at it?". It is computed from the same analysis as the cards (including everything each URL's helper functions do).

`access_logs` — One row per sign-in, sign-up or Administrator hand-over: who and when. Shown to the Administrator as Access Logs.

Created by	`app_login`, `app_signup`, `app_verify_login_otp`, `login`, `make_administrator`, `verify_login_otp`
Changed by	—
Deleted by	`delete_user`
Read by (screens and reports)	`add_user`, `app_settings_users`, `edit_user`, `settings_page`

`advances` — Money given to an employee ("given") or paid back by them ("repaid"). Keyed by employee name.

Created by	`add_advance`, `add_employee_advance`, `app_add_employee_payment`, `employee_ledger`
Changed by	—
Deleted by	—
Read by (screens and reports)	`advances_list`, `app_employee_ledger`, `app_salary`, `salaries_list`

`attendance` — One row per employee per day: Present, Absent, Leave or Half Day, with optional in/out time.

Created by	`bulk_mark_attendance`, `mark_attendance`
Changed by	—
Deleted by	—
Read by (screens and reports)	`salaries_list`

`auth_attempts` — Throttle counters: failed logins per account and address, SMS-code requests, wrong backup tokens. Old rows are swept daily.

Created by	`app_login`, `app_login_otp`, `internal_backup`, `login`, `send_login_otp`
Changed by	—
Deleted by	`app_login`, `app_login_otp`, `internal_backup`, `login`, `send_login_otp`
Read by (screens and reports)	—

`batteries` — Each battery bought: brand, model, capacity, serial, warranty months, vehicle it is fitted on and latest health.

Created by	`add_battery`, `app_add_battery`
Changed by	`app_install_battery`, `check_battery`, `install_battery`, `mark_battery_dead`
Deleted by	`delete_battery`
Read by (screens and reports)	`app_battery`, `app_party_ledger`, `app_vendor_ledger`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `maintenance_list`, `party_ledger`, `vendor_ledger`

battery_checks — Dated health readings (voltage, temperature, health %) for a battery.

Created by	add_battery, app_add_battery, app_install_battery, check_battery, install_battery, mark_battery_dead
Changed by	—
Deleted by	delete_battery
Read by (screens and reports)	maintenance_list

companies — The company record (one row today). Every other table points at it through company_id.

Created by	—
Changed by	—
Deleted by	—
Read by (screens and reports)	—

compliance_expenses — Fees paid for fitness, permit and PUC renewals; each links to a maintenance bill so it can be paid like any bill.

Created by	add_compliance_expense, app_add_compliance
Changed by	—
Deleted by	delete_compliance_expense
Read by (screens and reports)	app_compliance, maintenance_list

employees — Drivers and office staff: identity, pay, bank/UPI, documents, opening advance balance, status.

Created by	add_employee, app_add_employee
Changed by	add_employee, app_add_employee, app_update_employee_opening_balance, deactivate_employee, edit_employee, update_employee_opening_balance
Deleted by	—
Read by (screens and reports)	add_employee_advance, add_trip, app_add_employee_payment, app_add_trip, app_edit_trip, app_employee_ledger, app_salary, business_performance, download_payslip, edit_trip, employee_ledger, salaries_list

insurance_policies — Vehicle insurance policies: insurer, policy number, dates, premium, IDV, NCB, agent, and uploaded documents.

Created by	add_insurance
Changed by	edit_insurance
Deleted by	delete_insurance
Read by (screens and reports)	app_party_ledger, app_vehicle_detail, app_vendor_ledger, export_party_ledger, export_party_ledger_pdf, export_vendor_ledger, export_vendor_ledger_pdf, party_ledger, vehicle_detail, vendor_ledger

`invoice_batch_charge_exclusions` — Trip charges deliberately left out of a generated invoice, so regenerating gives the same total.

Created by	`invoice_center_generate`
Changed by	—
Deleted by	—
Read by (screens and reports)	`invoice_batch_pdf`, `invoice_batch_pdf_preview`

`invoice_batch_items` — Extra charge or deduction lines on a generated invoice (optionally tied to a vendor).

Created by	`invoice_batch_edit`, `invoice_batch_item_add`, `invoice_center_generate`
Changed by	—
Deleted by	`invoice_batch_delete`, `invoice_batch_edit`, `invoice_batch_item_delete`
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_ledger`, `app_party_ledger`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `invoice_batch_pdf`, `invoice_batch_pdf_preview`, `party_ledger`, `vendor_ledger`

`invoice_batch_trips` — Which trips belong to which generated invoice.

Created by	`invoice_center_generate`
Changed by	—
Deleted by	`invoice_batch_delete`
Read by (screens and reports)	`invoice_batch_pdf`, `invoice_batch_pdf_preview`, `invoice_batches_list`, `invoice_center`

`invoice_batches` — A generated invoice: number, type (party, vehicle_owner, tax, bill), party or vendor, dates, terms, GST/TDS rates, status and payment status

Created by	`invoice_center_generate`
Changed by	`invoice_batch_edit`
Deleted by	`invoice_batch_delete`
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_ledger`, `app_party_ledger`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `invoice_batch_pdf`, `invoice_batch_pdf_preview`, `invoice_batches_list`, `party_ledger`, `vendor_ledger`

`invoice_items` — Extra lines on a single-trip (old style) invoice.

Created by	`add_trip`, `app_add_trip`, `app_edit_trip`, `edit_trip`, `invoice_item_add`
Changed by	—
Deleted by	`add_trip`, `app_add_trip`, `app_edit_trip`, `edit_trip`, `invoice_item_delete`
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_ledger`, `app_party_ledger`, `app_trip_detail`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `invoice_batch_pdf`, `invoice_batch_pdf_preview`, `invoice_center`, `invoice_center_generate`, `invoice_edit`, `invoice_pdf`, `invoice_preview`, `party_ledger`, `trip_view`, `vendor_ledger`

`invoices` — Single-trip invoice header (older flow); also gives the ledger a real invoice number for a trip.

Created by	`invoice_edit`
Changed by	`invoice_edit`
Deleted by	—
Read by (screens and reports)	`app_party_ledger`, `app_vendor_ledger`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `invoice_pdf`, `invoice_preview`, `party_ledger`, `vendor_ledger`

`login_otps` — SMS login codes: user, phone, keyed hash of the code, expiry, tries used, used flag. The code itself is never stored.

Created by	`app_login_otp`, `send_login_otp`
Changed by	`app_login_otp`, `app_verify_login_otp`, `send_login_otp`, `verify_login_otp`
Deleted by	—
Read by (screens and reports)	—

`maintenance` — The bill table for every repair or purchase: service, tyre, battery, urea, insurance and compliance fees. Amount, paid amount, vendor, vehic

Created by	`add_battery`, `add_compliance_expense`, `add_insurance`, `add_maintenance`, `add_service`, `add_toll`, `add_tyre`, `add_tyre_stock`, `add_urea`, `app_add_battery`, `app_add_compliance`, `app_add_service`, `app_add_toll`, `app_add_tyre`, `app_add_tyre_stock`, `app_add_urea`, `sync_fastag_toll`, `toll_excel_import`
Changed by	`add_tyre`, `add_vendor_payment`, `app_add_vendor_payment`, `app_edit_service`, `app_install_battery`, `app_install_tyre_stock`, `edit_insurance`, `edit_maintenance`, `edit_service`, `edit_tyre`, `edit_urea`, `install_battery`, `install_tyre_stock`, `toll_excel_import`
Deleted by	`delete_battery`, `delete_compliance_expense`, `delete_insurance`, `delete_maintenance`, `delete_toll`, `delete_tyre_stock`, `delete_urea`
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_ledger`, `app_maintenance`, `app_party_ledger`, `app_performance`, `app_tyres`, `app_urea`, `app_vehicle_detail`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_excel`, `export_maintenance`, `export_maintenance_pdf`, `export_party_ledger`, `export_party_ledger_pdf`, `export_performance`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `maintenance_list`, `maintenance_view`, `party_ledger`, `payment_view`, `performance`, `vehicle_detail`, `vendor_ledger`

`maintenance_items` — Checklist lines inside a service bill.

Created by	`add_service`, `app_add_service`, `app_edit_service`, `edit_service`
Changed by	—
Deleted by	`add_service`, `app_add_service`, `app_edit_service`, `delete_maintenance`, `edit_service`
Read by (screens and reports)	`maintenance_list`

`overheads` — Expense bills (office, fuel for non-trip use, rent, etc.) with Paid/Pending status, optional vendor, recurring flag and due date.

Created by	`add_overhead`, `app_add_overhead`
Changed by	`add_vendor_payment`, `app_add_vendor_payment`, `app_edit_overhead`, `edit_overhead`
Deleted by	`delete_overhead`
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_expenses`, `app_ledger`, `app_party_ledger`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_overheads`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `overheads_list`, `party_ledger`, `payment_view`, `vendor_ledger`

`parties` — Customers: contact, GSTIN, credit limit, opening balance and date, category, status.

Created by	`add_account`, `add_trip`, `app_add_account`, `app_add_trip`, `app_edit_trip`, `edit_trip`
Changed by	—
Deleted by	—
Read by (screens and reports)	`accounts`, `add_battery`, `add_compliance_expense`, `add_insurance`, `add_maintenance`, `add_overhead`, `add_party_payment`, `add_service`, `add_tyre`, `add_tyre_stock`, `add_urea`, `app_add_battery`, `app_add_compliance`, `app_add_overhead`, `app_add_party_payment`, `app_add_service`, `app_add_tyre`, `app_add_tyre_stock`, `app_add_urea`, `app_battery`, `app_business_performance`, `app_dashboard`, `app_edit_overhead`, `app_edit_service`, `app_ledger`, `app_maintenance`, `app_party_ledger`, `app_trip_detail`, `app_trips`, `app_tyres` (+32 more)

`payment_allocations` — The link between a payment and what it settled: exactly one of a trip, a maintenance bill or an expense, with the amount.

Created by	`add_party_payment`, `add_vendor_payment`, `app_add_party_payment`, `app_add_vendor_payment`
Changed by	—
Deleted by	`delete_battery`, `delete_compliance_expense`, `delete_insurance`, `delete_maintenance`, `delete_overhead`, `delete_toll`, `delete_tyre_stock`, `delete_urea`
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_edit_overhead`, `app_edit_service`, `app_edit_trip`, `app_expenses`, `app_ledger`, `app_party_ledger`, `app_vendor_ledger`, `business_performance`, `dashboard`, `delete_trip`, `edit_insurance`, `edit_maintenance`, `edit_overhead`, `edit_service`, `edit_trip`, `edit_tyre`, `export_accounts`, `export_business_performance`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `overheads_list`, `party_ledger`, `payment_view`, `vendor_ledger`

`payments` — Money received from a party or paid to a vendor: date, amount, mode, reference, how much has been allocated.

Created by	`add_party_payment`, `add_vendor_payment`, `app_add_party_payment`, `app_add_vendor_payment`
Changed by	`add_party_payment`, `add_vendor_payment`, `app_add_party_payment`, `app_add_vendor_payment`, `delete_battery`, `delete_compliance_expense`, `delete_insurance`, `delete_maintenance`, `delete_overhead`, `delete_toll`, `delete_tyre_stock`, `delete_urea`
Deleted by	—
Read by (screens and reports)	`accounts`, `app_business_performance`, `app_dashboard`, `app_edit_trip`, `app_ledger`, `app_party_ledger`, `app_vendor_ledger`, `business_performance`, `dashboard`, `edit_trip`, `export_accounts`, `export_business_performance`, `export_excel`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `party_ledger`, `payment_view`, `vendor_ledger`

`salaries` — One salary payment: employee, month, gross, deductions, advance recovery, net, mode, transaction id, payment status.

Created by	`app_add_employee_payment`, `employee_ledger`
Changed by	—
Deleted by	`delete_salary`
Read by (screens and reports)	`app_business_performance`, `app_dashboard`, `app_employee_ledger`, `app_salary`, `business_performance`, `dashboard`, `download_payslip`, `export_business_performance`, `salaries_list`

`salary_items` — Allowance or deduction lines of a salary.

Created by	—
Changed by	—
Deleted by	`delete_salary`
Read by (screens and reports)	`download_payslip`

`settings` — Key/value store per company: company details, bank, invoice, tax, keys (masked), sync intervals, logo path, private secret key row.

Created by	`app_settings_save`, `invoice_center_generate`, `settings_page`, `sync_fastag_toll`, `upload_company_logo`
Changed by	—
Deleted by	—
Read by (screens and reports)	`add_user`, `app_battery`, `app_business_performance`, `app_compliance`, `app_employee_ledger`, `app_expenses`, `app_fleet_utilization`, `app_login_otp`, `app_party_ledger`, `app_performance`, `app_profile`, `app_salary`, `app_salary_attendance`, `app_settings_banking`, `app_settings_company`, `app_settings_invoice`, `app_settings_tax`, `app_settings_users`, `app_toll`, `app_trip_detail`, `app_tyres`, `app_urea`, `app_vehicle_detail`, `app_vendor_ledger`, `delete_user`, `download_payslip`, `edit_user`, `export_party_ledger_pdf`, `export_vendor_ledger_pdf`, `invoice_batch_pdf` (+12 more)

`sync_log` — One row per background sync run: job, start/finish, synced / failed / skipped counts.

Created by	—
Changed by	—
Deleted by	—
Read by (screens and reports)	`app_settings_users`, `settings_page`

`toll_entries` — Toll spend per vehicle (and optionally trip): plaza, highway, amount, source (fastag or manual), status (synced, approved, pending, rejected

Created by	`add_toll`, `app_add_toll`, `sync_fastag_toll`, `toll_excel_import`
Changed by	`approve_toll`, `reject_toll`, `toll_excel_import`
Deleted by	`delete_toll`
Read by (screens and reports)	`app_analytics`, `app_dashboard`, `app_fleet_utilization`, `app_performance`, `app_toll`, `business_performance`, `delete_trip`, `export_business_performance`, `export_route_analytics`, `fleet_utilization`, `invoice_batch_pdf`, `invoice_batch_pdf_preview`, `invoice_center`, `invoice_center_generate`, `maintenance_list`, `performance`, `route_analytics`, `route_finder_search`, `toll_excel_preview`

`trips` — The core table: one row per journey with route, rates, charges, deductions, fuel, advances, party money, owner terms and costs.

Created by	`add_trip`, `app_add_trip`
Changed by	`add_party_payment`, `add_vendor_payment`, `app_add_party_payment`, `app_add_vendor_payment`, `app_edit_trip`, `app_end_trip`, `edit_trip`, `end_trip`
Deleted by	`delete_trip`
Read by (screens and reports)	`accounts`, `add_toll`, `app_add_toll`, `app_analytics`, `app_business_performance`, `app_dashboard`, `app_fleet_utilization`, `app_ledger`, `app_maintenance`, `app_party_ledger`, `app_performance`, `app_toll`, `app_trip_detail`, `app_trips`, `app_urea`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_excel`, `export_fleet_utilization`, `export_party_ledger`, `export_party_ledger_pdf`, `export_performance`, `export_route_analytics`, `export_trips`, `export_trips_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf` (+22 more)

`tyre_stock` — Tyres bought into stock and where each is fitted (vehicle, position) or scrapped.

Created by	`add_tyre_stock`, `app_add_tyre_stock`
Changed by	`add_tyre`, `app_install_tyre_stock`, `edit_tyre`, `install_tyre_stock`, `scrap_tyre_stock`
Deleted by	`delete_tyre_stock`
Read by (screens and reports)	`app_tyres`, `maintenance_list`

`urea_transactions` — AdBlue stock movements: stock_in (purchase) or stock_out (use, from stock or direct) with running balance.

Created by	`add_urea`, `app_add_urea`
Changed by	`edit_urea`
Deleted by	`delete_urea`
Read by (screens and reports)	`app_urea`, `maintenance_list`

`users` — Sign-in accounts: username (mobile or email), password hash, role, access level, allowed screens, status, last login.

Created by	`add_user`, `app_signup`
Changed by	`app_change_password`, `app_login`, `app_update_profile`, `app_verify_login_otp`, `edit_user`, `login`, `make_administrator`, `update_user_phone`, `verify_login_otp`, `website_change_password`, `website_update_my_profile`
Deleted by	`delete_user`
Read by (screens and reports)	`app_analytics`, `app_battery`, `app_business_performance`, `app_compliance`, `app_dashboard`, `app_employee_ledger`, `app_expenses`, `app_fleet_utilization`, `app_ledger`, `app_login_otp`, `app_maintenance`, `app_party_ledger`, `app_performance`, `app_profile`, `app_salary`, `app_salary_attendance`, `app_settings_banking`, `app_settings_company`, `app_settings_invoice`, `app_settings_tax`, `app_settings_users`, `app_toll`, `app_trip_detail`, `app_trips`, `app_tyres`, `app_urea`, `app_vehicle_detail`, `app_vehicles`, `app_vendor_ledger`, `export_users` (+2 more)

`vehicle_challans` — Traffic challans fetched from eChallan per vehicle, with amounts and status.

Created by	`vehicle_sync_now`
Changed by	—
Deleted by	`vehicle_sync_now`
Read by (screens and reports)	`app_vehicle_detail`, `vehicle_detail`

`vehicle_compliance` — Fitness, PUC and permit records per vehicle: number, dates, source (manual, mock or live), sync status.

Created by	`add_vehicle`, `app_add_vehicle`, `edit_vehicle`, `vehicle_compliance_renew`
Changed by	`add_vehicle`, `app_add_vehicle`, `edit_vehicle`, `vehicle_compliance_renew`
Deleted by	—
Read by (screens and reports)	`dashboard`

`vehicles` — Trucks (own or hired): registration, type, capacity, body, chassis/engine, document dates, cached RC data, challan counts.

Created by	`add_battery`, `add_insurance`, `add_maintenance`, `add_service`, `add_trip`, `add_tyre`, `add_urea`, `add_vehicle`, `app_add_battery`, `app_add_service`, `app_add_trip`, `app_add_tyre`, `app_add_urea`, `app_add_vehicle`, `app_edit_service`, `app_install_battery`, `app_install_tyre_stock`, `edit_insurance`, `edit_maintenance`, `edit_service`, `edit_tyre`, `install_battery`, `install_tyre_stock`
Changed by	`add_insurance`, `add_vehicle`, `app_add_vehicle`, `edit_insurance`, `edit_vehicle`, `toggle_vehicle_status`, `vehicle_compliance_renew`, `vehicle_sync_now`
Deleted by	`delete_vehicle`
Read by (screens and reports)	`add_compliance_expense`, `add_toll`, `add_vendor_payment`, `app_add_compliance`, `app_add_toll`, `app_add_vendor_payment`, `app_analytics`, `app_battery`, `app_business_performance`, `app_compliance`, `app_dashboard`, `app_edit_trip`, `app_fleet_utilization`, `app_maintenance`, `app_party_ledger`, `app_performance`, `app_toll`, `app_trip_detail`, `app_trips`, `app_tyres`, `app_urea`, `app_vehicle_detail`, `app_vehicles`, `app_vendor_ledger`, `business_performance`, `dashboard`, `edit_trip`, `export_business_performance`, `export_excel`, `export_fleet_utilization` (+32 more)

`vendors` — People and firms the company pays: truck owners, fuel pumps, workshops; may be linked to a party.

Created by	`add_account`, `add_battery`, `add_compliance_expense`, `add_insurance`, `add_maintenance`, `add_overhead`, `add_service`, `add_trip`, `add_tyre`, `add_tyre_stock`, `add_urea`, `app_add_account`, `app_add_battery`, `app_add_compliance`, `app_add_overhead`, `app_add_service`, `app_add_trip`, `app_add_tyre`, `app_add_tyre_stock`, `app_add_urea`, `app_edit_overhead`, `app_edit_service`, `app_edit_trip`, `edit_insurance`, `edit_maintenance`, `edit_overhead` (+6 more)
Changed by	`add_trip`, `app_add_trip`, `app_edit_trip`, `edit_trip`
Deleted by	—
Read by (screens and reports)	`accounts`, `add_vendor_payment`, `app_add_vendor_payment`, `app_battery`, `app_business_performance`, `app_compliance`, `app_dashboard`, `app_expenses`, `app_ledger`, `app_maintenance`, `app_party_ledger`, `app_trip_detail`, `app_tyres`, `app_urea`, `app_vehicle_detail`, `app_vendor_ledger`, `business_performance`, `dashboard`, `export_accounts`, `export_business_performance`, `export_excel`, `export_maintenance`, `export_maintenance_pdf`, `export_party_ledger`, `export_party_ledger_pdf`, `export_vendor_ledger`, `export_vendor_ledger_pdf`, `invoice_batch_pdf`, `invoice_batch_pdf_preview`, `invoice_batches_list` (+10 more)

43. Screen-to-data map

Every page template with the route function that renders it, the tables behind it, and what it shows. Phone templates are listed with the website pages they mirror.

Template	Rendered by	Tables read	Shows (cards / columns / sections)
add_advance.html	add_advance		sections: Add Advance Entry
add_employee.html	add_employee	employees	sections: Add Employee
add_maintenance.html	add_maintenance	parties, vehicles, vendors	sections: Add Maintenance
add_overhead.html	add_overhead	parties, vendors	sections: Add Expense Entry
add_payment.html	add_party_payment, add_vendor_payment	parties, payments, trips	sections: Add Payment —
add_vehicle.html	add_vehicle	vehicle_compliance, vehicles	sections: Add Vehicle
advances_list.html	advances_list	advances	columns: Employee, Given, Repaid, Outstanding, Date, Type, Amount, Notes; sections: Employee Advances + Add Advance
app/analytics.html	app_analytics	toll_entries, trips, users, vehicles	sections: Route Finder, Top 5 Best Performing Routes, Top Routes by Revenue, Route Rates, Filters
app/battery.html	app_battery	batteries, parties, settings, users, vehicles, vendors	sections: Battery Inventory, Filters, Add Battery
app/business_performance.html	app_business_performance	invoice_batch_items, invoice_batches, invoice_items, maintenance, overheads, parties, payment_allocations, payments, salaries, settings, trips, users, vehicles, vendors	sections: Revenue vs Expenses, Business Overview, Select Period
app/compliance.html	app_compliance	compliance_expenses, settings, users, vehicles, vendors	sections: Compliance, Filters, Compliance Expense List (), Compliance Expenses Trend (6 months), Expenses by Compliance Type
app/dashboard.html	app_dashboard	invoice_batch_items, invoice_batches, invoice_items, maintenance, overheads, parties, payment_allocations, payments, salaries, toll_entries, trips, users, vehicles, vendors	sections: Hi, Fleet Status, Needs Your Attention!, Top Expenses, Business Overview
app/employee_ledger.html	app_employee_ledger	advances, employees, salaries, settings, users	sections: Ledger Entries, Filters, Add Payment —
app/expenses.html	app_expenses	overheads, payment_allocations, settings, users, vendors	sections: Expenses, Expenses List, Filters, Add Expense, Expense
app/fleet_utilization.html	app_fleet_utilization	settings, toll_entries, trips, users, vehicles	sections: Idle Days Trend, Utilization by Vehicle Type, Most Idle Vehicles, Vehicles Running Empty Most, Routes Running Empty Most
app/ledger.html	app_ledger	invoice_batch_items, invoice_batches, invoice_items, maintenance, overheads, parties, payment_allocations, payments, trips, users, vendors	sections: Ledger, Add Party / Vendor, Filters

Template	Rendered by	Tables read	Shows (cards / columns / sections)
app/ledger_detail.html	app_party_ledger, app_vendor_ledger	batteries, insurance_policies, invoice_batch_items, invoice_batches, invoice_items, invoices, maintenance, overheads, parties, payment_allocations, payments, settings, trips, users, vehicles, vendors	sections: Contact Information, Transactions, Add Payment —
app/login.html	app_login, app_signup	auth_attempts, users	
app/login_otp.html	app_login_otp, app_verify_login_otp	auth_attempts, settings, users	
app/maintenance.html	app_maintenance	maintenance, parties, trips, users, vehicles, vendors	sections: Maintenance, Fleet Health Overview, Action Required, Maintenance Cost Trend, Service Records
app/performance.html	app_performance	maintenance, settings, toll_entries, trips, users, vehicles	sections: Top 5 Drivers (By Trips), Monthly Trips Trend, Top 5 Vehicles (By Trips), Business Overview, Select Period
app/profile.html	app_profile	settings, users	sections: Personal Information, Account, Edit Profile, Change Password
app/profile_admin.html	app_profile	settings, users	sections: Profile Settings, Edit Profile, Change Password
app/salary.html	app_salary, app_salary_attendance	advances, employees, salaries, settings, users	sections: Salary, Salary Summary, Add Employee, Filters
app/signup.html	app_signup	users	
app/toll.html	app_toll	settings, toll_entries, trips, users, vehicles	sections: Toll Management, Toll Entries, Toll Summary (Last 6 Months), Top Toll Plazas, Select Period
app/trip_add.html	app_add_trip, app_edit_trip	employees, parties, trips, vehicles, vendors	sections: Trip Information, Additional Charges, Deductions, Fuel Driver Advance, Other Expenses
app/trip_detail.html	app_trip_detail	invoice_items, parties, settings, trips, users, vehicles, vendors	cards: Trip Type, Party, Material, Driver, LR Received, Billed Amount; sections: Trip Information, Fuel, Hired Vehicle Owner, Billing Payments, Additional Charges ■
app/trips.html	app_trips	parties, trips, users, vehicles	sections: Trips, Filters, Trips List
app/tyres.html	app_tyres	maintenance, parties, settings, tyre_stock, users, vehicles, vendors	sections: Tyre Management, Tyre Stock, Tyre Entries, Filters, Add Tyre
app/urea.html	app_urea	maintenance, parties, settings, trips, urea_transactions, users, vehicles, vendors	sections: Urea Management, Urea Entries, Stock Trend (Last 30 Days), Top Vehicles by Usage (30d), Filters
app/vehicle_detail.html	app_vehicle_detail	insurance_policies, maintenance, settings, users, vehicle_challans, vehicles, vendors	cards: Vehicle Type, Manufacturer, Model, Fuel Type, Year, Vehicle Age
app/vehicles.html	app_vehicles	users, vehicles	sections: Vehicles, Filters, Add Vehicle
app/verify_otp.html	app_login_otp, app_verify_login_otp	auth_attempts, settings, users	
business_performance.html	business_performance	employees, invoice_batch_items, invoice_batches, invoice_items, maintenance, overheads, parties, payment_allocations, payments, salaries, toll_entries, trips, vehicles, vendors	cards: Total Revenue, Net Profit, Profit Margin, Outstanding Receivables, Outstanding Payables, Cash Collected; columns: Customer, Revenue, %, Profit; sections: Business Performance, Financial Overview, Monthly Profit Trend, Revenue Comparison, Top Customers by Revenue

Template	Rendered by	Tables read	Shows (cards / columns / sections)
dashboard.html	dashboard	invoice_batch_items, invoice_batches, invoice_items, maintenance, overheads, parties, payment_allocations, payments, salaries, trips, vehicle_compliance, vehicles, vendors	cards: Total Trips, Total Revenue, Total Profit, Own Vehicle Revenue, Hired Vehicle Profit, Fuel Expense; columns: Vehicle, Trips, Revenue, Avg/Trip, Util.; sections: Dashboard, Freight Revenue Trend View Details, Expense Breakdown, Recent Activities, Trip Status View All
edit_maintenance.html	edit_maintenance	maintenance, parties, payment_allocations, vehicles, vendors	sections: Edit Maintenance
edit_vehicle.html	edit_vehicle	vehicle_compliance, vehicles	sections: Edit Vehicle
employee_ledger.html	employee_ledger	advances, employees, salaries	cards: Debit, Credit, Notes, Entry Recorded; columns: Date, Type, Debit, Credit, Advance Balance; sections: + Add Payment, —, Add Payment —
employees.html	_employees_attendance_tab, _employees_overview_tab, _employees_salary_tab		cards: Total Employees, Drivers, Office Staff, Present Today, On Leave, Attendance %; columns: Employee, Role, Joining Date, Attendance, Salary Status, Advance, Total Paid, Payments; sections: Employees, Edit Employee —, Add Employee, Mark Attendance
fleet_utilization.html	fleet_utilization	toll_entries, trips, vehicles	cards: Total Vehicles, Active Vehicles, Idle Vehicles (Any Day), Empty Run Trips, Avg Idle Days / Vehicle, Avg Days / Vehicle; columns: Vehicle No, Type, Group, Active Days (This Period), Idle Days (This Period), Total Days, Empty Run Trips, Idle %; sections: Fleet Utilization Efficiency, Idle vs Active Days (This Period), Idle Days Trend (Last 12 Weeks), Vehicle Utilization Summary, Most Idle Vehicles
home.html	accounts	invoice_batch_items, invoice_batches, invoice_items, maintenance, overheads, parties, payment_allocations, payments, trips, vendors	cards: Total Parties, Total Vendors, Total Receivable, Total Payable, Net Balance; columns: Name, Type, Group / Category, Phone, Email / GSTIN, Receivable, Payable, Balance; sections: Ledger, Add Party / Vendor
invoice_batch_edit.html	invoice_batch_edit	invoice_batch_items, invoice_batches, parties, vendors	columns: Description, Vendor, Type, Amount (■); sections: Edit Invoice —, Additional Line Items, Invoice Preview
invoice_batches_list.html	invoice_batches_list	invoice_batch_trips, invoice_batches, parties, vendors	columns: Invoice No., Type, Bill To, Invoice Date, Due Date, Trips, Payment Status; sections: Generated Invoices + New Invoice
invoice_center.html	invoice_center	invoice_batch_trips, invoice_items, parties, settings, toll_entries, trips, vehicles, vendors	cards: GSTIN, State; columns: LR Number, Vehicle, From To, Date, Freight (■), Status; sections: Invoice Center, 1. Select Invoice Type, 2. Select, 3. Filter Trips, 4. Select Trips to Include in Invoice
invoice_edit.html	invoice_edit	invoice_items, invoices, trips	columns: Description, Type, Amount; sections: Edit Invoice —
invoice_preview.html	invoice_preview	invoice_items, invoices, parties, settings, trips, vehicles	cards: Invoice No., LR Number, Date, Due Date, Bill To, Vehicle No; columns: Description, Amount (■)
invoice_review.html	invoice_center_review	parties, settings, trips, vehicles, vendors	columns: LR Number, Vehicle, Date, Freight, Net Payable; sections: Invoice Summary Generate, Trips Included (), Invoice Details, Invoice Summary
invoices_search.html	invoices_search	parties, trips, vehicles	columns: Date, LR Number, Vehicle, Party, From, To, Billed, Action; sections: Invoices
ledger.html	party_ledger, vendor_ledger	batteries, insurance_policies, invoice_batch_items, invoice_batches, invoice_items, invoices, maintenance, overheads, parties, payment_allocations, payments, trips, vehicles, vendors	cards: Total Receivable, Total Payable, Net Balance, Credit Limit, Total Receivables, Total Payables; columns: Date, Type, Details, Ref / Document, Debit (■), Credit (■), Balance (■), Vehicle; sections: Add Payment —, Edit Contact Details —

Template	Rendered by	Tables read	Shows (cards / columns / sections)
login.html	login, send_login_otp, verify_login_otp	auth_attempts, users	sections: Manage Your Transport Business. Drive Your Business Forward., Welcome Back!, Create Your Account, Verify Your Mobile Number, Account Created Successfully
maintenance.html	_maintenance_battery_tab, _maintenance_compliance_tab, _maintenance_overview_tab		cards: Total Fleet, Vehicles Serviced, Pending Payments, Total Maintenance Cost, Avg Cost / KM, Fleet Availability; columns: Date, Vehicle, Amount, Cost, Service Type, KM Reading, Workshop, Status; sections: Maintenance, Urea Entry —, Toll Entry —, Compliance Expense —, Add New Service
maintenance_view.html	maintenance_view	maintenance, vehicles, vendors	cards: Amount, Paid Amount, Balance, Date, Category, Status; sections: Maintenance Details
overheads_list.html	overheads_list	overheads, payment_allocations, vendors	cards: Total Expenses, This Month, Pending Bills, Recurring (Monthly), Avg Monthly Expense, Categories; columns: Date, Category, Vendor, Description, Payment Mode, Status, Amount, Actions; sections: Expenses, —, Edit Expense, Add Expense
payment_view.html	payment_view	maintenance, overheads, parties, payment_allocations, payments, trips, vendors	cards: Total Amount, Allocated, Unallocated, Date, Type, Mode; columns: LR Number, Route, Amount; sections: Payment Details
performance.html	performance	maintenance, toll_entries, trips, vehicles	cards: Total Drivers, Completed Trips, Total Billed, Total Fuel Cost, Total Profit, Avg Trips / Driver; columns: Driver Name, Completed Trips, Total Billed, Fuel Cost, Profit, Last Trip Date, Vehicle No, Type; sections: Performance, Top 5 Drivers by Completed Trips, Trips Trend (All Drivers), Driver Performance Detail, Top 5 Vehicles by Completed Trips
profile_self.html	_render_my_profile		cards: Role, Access level, Status, Login (username), Member since, Last login; sections: My Profile, My details, Change password
route_analytics.html	route_analytics	toll_entries, trips, vehicles	cards: Total Routes, Total Trips, Total Revenue, Avg Rate, Avg Margin, Highest Rate; columns: Rank, Route, Trips, Revenue, Profit, Margin %, Total Revenue, Total Cost; sections: Route Analytics, Top 5 Best Performing Routes (By Profit Margin %), Top Routes by Revenue, Route Summary, Route Rates ()
settings.html	_render_settings_users_error, settings_page		cards: Company Name, GSTIN, Address, PAN, State, Business Type; columns: User, Role, Access Level, Mobile / Email, Status, Last Login, Users, Date Time; sections: Profile Settings, Company Details, Edit Company Details, Banking Details, Edit Banking Details
trip_form.html	add_trip, edit_trip	employees, parties, trips, vehicles, vendors	cards: Net Amount (Billed), Estimated Profit; columns: #, Description, Type, Amount / Rate ( ), Quantity (MT/Nos), Vendor (Optional), Total ( ); sections: Trip Summary, Hired Vehicle Profit, Payment Tracking, Quick Tips
trip_view.html	trip_view	invoice_items, parties, trips, vehicles, vendors	cards: Trip Type, Party, Material, Driver, LR Received, Billed Amount; columns: Description, Type, Amount, Vendor; sections: Trip Details
trips_list.html	trips_list	parties, trips, vehicles	cards: Total Trips, Total Billed, Pending Balance, Active Trips, LR Pending; columns: Date ■, LR Number, Vehicle, Party ■, From, To, Billed, Pending; sections: Trips, End Trip, Export — Trips
vehicle_detail.html	vehicle_detail	insurance_policies, maintenance, settings, vehicle_challans, vehicles, vendors	cards: Vehicle Type, Manufacturer, Model, Fuel Type, Year, Vehicle Age; columns: Policy Number, Insurer, Type, Start Date, End Date, IDV, Status; sections: Insurance —, Edit Permit —, Edit Fitness —, Edit PUC —

Template	Rendered by	Tables read	Shows (cards / columns / sections)
`vehicle_rc_view.html`	`vehicle_rc_lookup`	vehicles	cards: Registration No., Status, Status As On, Registration Date, Registration Valid Upto, RTO Code; sections: RC Details
`vehicles_list.html`	`vehicles_list`	trips, vehicles	cards: Total Vehicles, Active, In Maintenance, Insurance Expiring, PUC Expiring, Permit Expiring; columns: Vehicle No, Age ■, Insurance, Fitness, PUC, Permit, Road Tax, Status; sections: Vehicles, Edit Vehicle —, Add Vehicle

G

The front-end

What the user sees and touches: the two layouts, every template, the scripts, the installable app and the design rules.

IN THIS PART

- The two layouts (website and phone)
- Template inventory
- Static files: CSS, JavaScript, images
- Behaviours that run in the browser
- The installable app (PWA) and in-app navigation
- Design system

44. The two layouts

There is no JavaScript framework. Each screen is an HTML page produced on the server by Jinja2 from a template, styled with plain CSS, and made interactive by small amounts of vanilla JavaScript. Two "base" templates provide the frame; every other template fills in the middle.

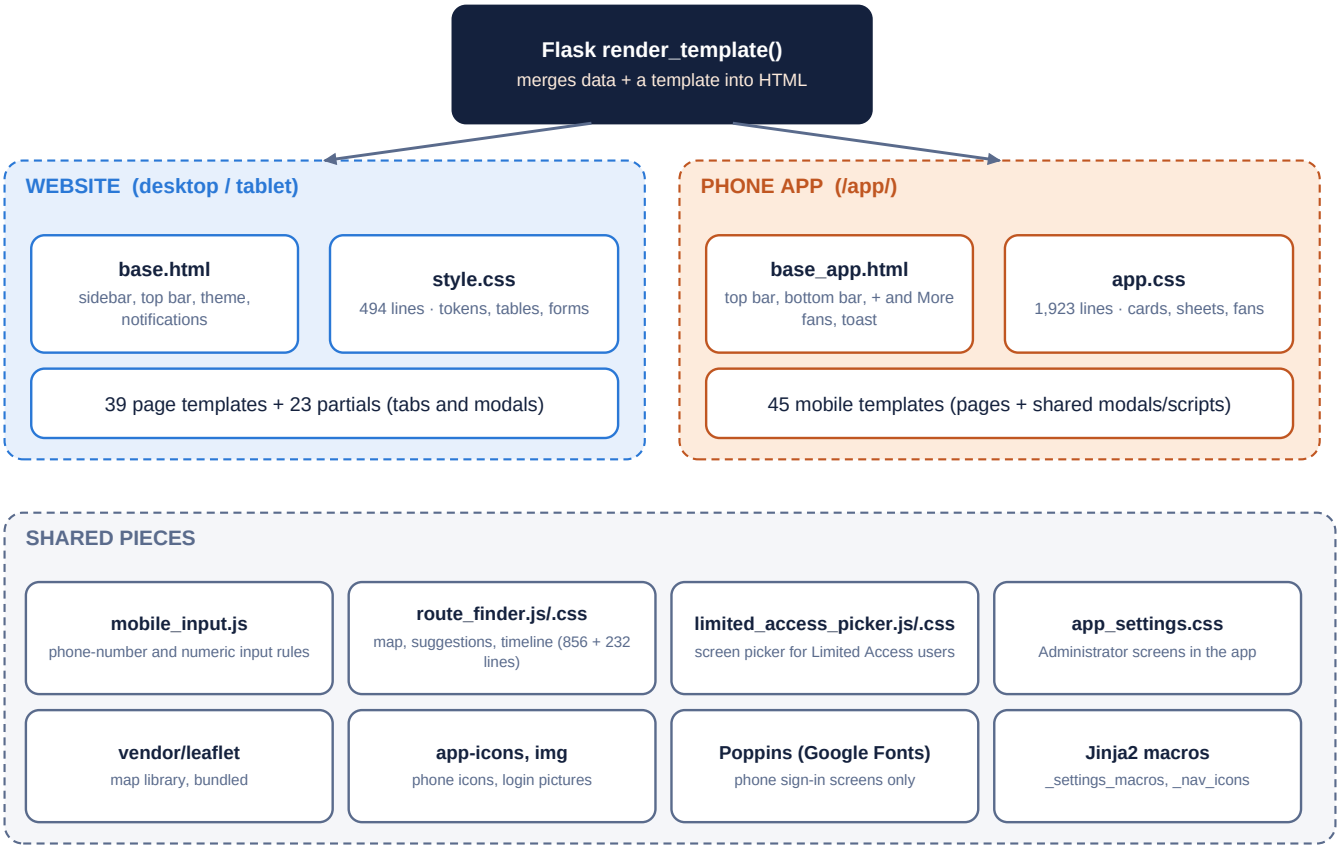


Figure 14 — Two front-ends, one back-end: both are built from Jinja2 templates; they differ in their base layout and stylesheet.

44.1 Website layout: templates/base.html

- **Left sidebar** with the brand and four groups — Overview (Dashboard, Ledger), Operations (Trips, Maintenance, Vehicles), Analysis (Route Analytics, Fleet Utilization, Performance, Business Performance), Finance (Salaries, Invoices, Expenses) — then Profile and Settings and Logout. Each item is wrapped in `can('Screen')` so a Limited Access user only sees theirs; a group disappears when none of its items is allowed. The sidebar collapses to the hamburger (state kept in the browser's localStorage).
- **Top bar** with the hamburger, a light/dark switch (stored in localStorage as `theme`, applied by setting data-theme on the page root), the notification bell (a dropdown of vehicles with pending challans, supplied by the context processor) and the user's name, role and initials.
- **Blocks** the child templates fill: `title`, `extra_head` (page-specific CSS), `content` (inside the main container) and scripts.
- **Soft navigation.** Clicking a sidebar link fetches the next page, parses it, and swaps only the main container and its page styles (function swapContent), so the sidebar and top bar do not flicker; any failure falls back to a normal page load.
- **Shared popups.** Ledger rows open a centred record popup (trip) or a slide-in detail panel (payment) loaded from the read-only view routes; an RC modal shows cached vehicle data.
- **Autocomplete.** Inputs with a datalist are enhanced (function enhanceAutocompleteInputs) into a styled suggestion list.

JavaScript functions defined in base.html: `applyTheme`, `attachHandlers`, `closeDetailPanel`, `closeRcModal`, `closeRecordModal`, `enhanceAutocompleteInputs`, `goTo`, `hideVehiclesNavBadgeIfDismissed`, `initTheme`, `navRecordModal`, `openDetailPanel`, `openRcModal`, `openRecordModal`, `positionDropdown`, `restoreSidebarState`, `showMatches`, `showRecordTab`, `swapContent`, `toggleDetailPanel`, `toggleNotifDropdown`, `toggleSidebar`, `toggleTheme`, `updateActiveNav`, `updateRecordNavButtons`.

44.2 Phone layout: templates/app/base_app.html

- **Top bar**: brand badge and name, notification bell and a round avatar with the user's initials linking to the profile.
- **Main area** (`#appMain`) holding each page; **extra scripts** (`#appExtraScripts`) holding that page's own JavaScript.
- **Bottom bar**: Home, Trips, a round + button, Ledger and More. The + button fans out four quick-add actions in an arc (Add Trip, Add Maintenance → a second arc of six kinds, Add Expense, Add Vehicle); More fans a vertical list (Maintenance, Vehicles, Expenses, Salary, Business Overview). For Limited Access users this whole block is generated from their screens by `_limited_nav()`.
- **Shared components**: a toast ("Saved"), a custom calendar and time picker that replace the phone's native pickers, a translucent loading overlay with an animated truck shown only when a real navigation is slow, scroll locking for sheets, and a viewport-height variable that keeps the layout stable when mobile address bars show and hide.
- **Sheets, not pages** for most add/edit forms: a bottom sheet (class vam-modal) slides up over the list.

JavaScript functions defined in base_app.html: `closeCalPicker`, `closeFabMenu`, `closeMoreSheet`, `closeTimePicker`, `confirmCalPicker`, `confirmTimePicker`, `currentOptions`, `enhanceAllDatalistInputs`, `enhanceAllDateInputs`, `enhanceAllTimeInputs`, `enhanceDatalistInput`, `enhanceDateInput`, `enhanceTimeInput`, `findSizeReference`, `hideOverlay`, `iso`, `lockPageScroll`, `navigate`, `onFromChange`, `onToChange`, `openCalPicker`, `openFabMenu`, `openMoreSheet`, `openTimePicker`, `pad2`, `pad2t`, `pretty`, `prettyTime`, `refreshTrigger`, `reloadWithFlag`, `render`, `renderCal`, `renderMonthGrid`, `renderTimeCol`, `renderTimePicker`, `renderYearList`, `runPageCleanup`, `runScripts`, `setAppVh`, `showMaintFan`, `showOverlay`, `showToast`, `syncActiveNav`, `to24`, `toggleFabMenu`, `toggleMoreSheet`, `unlockPageScroll`, `wireAllDateRangePairs`, `wireDateRangePair`.

45. Template inventory

All 107 templates. "Extends" names the layout they fill; "Rendered by" lists the route functions that return them (partials are included into other templates instead).

Website pages (39)

File	Lines	Extends / includes	Title or role	Rendered by
add_advance.html	28	base.html	Add Advance	add_advance
add_employee.html	19	base.html	Add Employee	add_employee
add_maintenance.html	71	base.html	Add Maintenance	add_maintenance
add_overhead.html	22	base.html	Add Expense	add_overhead
add_payment.html	21	base.html	Add Payment	add_party_payment, add_vendor_payment
add_vehicle.html	55	base.html	Add Vehicle	add_vehicle
advances_list.html	42	base.html	Employee Advances	advances_list
base.html	781		Anil Transport Service	
business_performance.html	424	base.html	Business Performance	business_performance
dashboard.html	247	base.html	Dashboard	dashboard
edit_maintenance.html	93	base.html	Edit Maintenance	edit_maintenance
edit_vehicle.html	62	base.html	Edit Vehicle	edit_vehicle
employee_ledger.html	128	base.html	Employee Ledger — …	employee_ledger
employees.html	40	base.html · includes partials/_employees_attendance.html, partials/_employees_modals.html, partials/_employees_overview.html	Employees	_employees_attendance_tab, _employees_overview_tab, _employees_salary_tab
fleet_utilization.html	277	base.html · includes partials/_idle_trend_chart.html	Fleet Utilization & Efficiency	fleet_utilization
home.html	170	base.html	Ledger	accounts
invoice_batch_edit.html	209	base.html	Edit Invoice — …	invoice_batch_edit
invoice_batches_list.html	55	base.html	Generated Invoices	invoice_batches_list
invoice_center.html	610	base.html	Invoice Center	invoice_center
invoice_edit.html	52	base.html	Edit Invoice	invoice_edit
invoice_preview.html	82	base.html	Invoice — …	invoice_preview
invoice_review.html	91	base.html	Invoice Summary	invoice_center_review
invoices_search.html	35	base.html	Invoices	invoices_search
ledger.html	473	base.html	… — Ledger	party_ledger, vendor_ledger
login.html	421			login, send_login_otp, verify_login_otp
maintenance.html	73	base.html · includes partials/_maintenance_battery.html, partials/_maintenance_battery_modals.html, partials/_maintenance_category.html	Maintenance	_maintenance_battery_tab, _maintenance_compliance_tab, _maintenance_overview_tab

File	Lines	Extends / includes	Title or role	Rendered by
maintenance_view.html	91	base.html	Maintenance Details	maintenance_view
overheads_list.html	418	base.html	Expenses	overheads_list
payment_view.html	84	base.html	Payment Details	payment_view
performance.html	212	base.html · includes partials/_perf_trend_chart.html	Performance	performance
profile_self.html	84	base.html	My Profile	_render_my_profile
route_analytics.html	329	base.html · includes partials/_route_finder.html, partials/_route_finder_scripts.html	Route Analytics	route_analytics
settings.html	700	base.html	Profile & Settings	_render_settings_users_error, settings_page
trip_form.html	546	base.html	…	add_trip, edit_trip
trip_view.html	232	base.html	Trip Details	trip_view
trips_list.html	256	base.html	Trips	trips_list
vehicle_detail.html	568	base.html	…	vehicle_detail
vehicle_rc_view.html	107	base.html		vehicle_rc_lookup
vehicles_list.html	504	base.html	Vehicles	vehicles_list

Partials (tabs, modals, shared blocks) (23)

File	Lines	Extends / includes	Title or role	Rendered by
partials/_employees_attendance.html	8			
partials/_employees_modals.html	103			
partials/_employees_overview.html	259			
partials/_employees_salary.html	57			
partials/_idle_trend_chart.html	17			
partials/_maintenance_battery.html	147			
partials/_maintenance_battery_modals.html	345			
partials/_maintenance_category.html	46			
partials/_maintenance_compliance.html	153			
partials/_maintenance_compliance_modals.html	88			
partials/_maintenance_overview.html	121			
partials/_maintenance_service.html	132			
partials/_maintenance_service_modals.html	262			

File	Lines	Extends / includes	Title or role	Rendered by
partials/_maintenance_toll.html	176			
partials/_maintenance_toll_modals.html	197			
partials/_maintenance_tyre_modals.html	307			
partials/_maintenance_tyres.html	269			
partials/_maintenance_urea.html	136			
partials/_maintenance_urea_modals.html	157			
partials/_pager.html	20			
partials/_perf_trend_chart.html	21			
partials/_route_finder.html	86			
partials/_route_finder_scripts.html	18			

Phone-app templates (45)

File	Lines	Extends / includes	Title or role	Rendered by
app/_auth_hero.html	212			
app/_business_overview_header.html	47	· includes app/_overview_tabstrip.html		
app/_maintenance_tabstrip.html	63			
app/_nav_icons.html	4			
app/_overview_tabstrip.html	22			
app/_page_loader.html	106			
app/_period_macros.html	16			
app/_profile_modals.html	46			
app/_profile_scripts.html	120			
app/_settings_macros.html	17			
app/_settings_scripts.html	35			
app/_splash.html	169			
app/analytics.html	334	app/base_app.html · includes app/_business_overview_header.html, partials/_route_finder.html, partials/_route_finder_scripts.html	Route Analytics — …	app_analytics

File	Lines	Extends / includes	Title or role	Rendered by
app/base_app.html	1243	· includes app/_page_loader.html, app/_splash.html	Anil Transport	
app/battery.html	406	app/base_app.html · includes app/_maintenance_tabstrip.html	Battery — …	app_battery
app/business_performance.html	84	app/base_app.html · includes app/_business_overview_header.html	Business Performance — …	app_business_performance
app/compliance.html	380	app/base_app.html · includes app/_maintenance_tabstrip.html	Compliance — …	app_compliance
app/dashboard.html	382	app/base_app.html	Dashboard — Anil Transport	app_dashboard
app/employee_ledger.html	275	app/base_app.html	… — …	app_employee_ledger
app/expenses.html	490	app/base_app.html	Expenses — …	app_expenses
app/fleet_utilization.html	203	app/base_app.html · includes app/_business_overview_header.html	Fleet Utilization — …	app_fleet_utilization
app/ledger.html	256	app/base_app.html	Ledger — …	app_ledger
app/ledger_detail.html	339	app/base_app.html	… — …	app_party_ledger, app_vendor_ledger
app/login.html	619	· includes app/_splash.html		app_login, app_signup
app/login_otp.html	149	· includes app/_auth_hero.html, app/_splash.html		app_login_otp, app_verify_login_otp
app/maintenance.html	658	app/base_app.html · includes app/_maintenance_tabstrip.html	Maintenance — …	app_maintenance
app/performance.html	143	app/base_app.html · includes app/_business_overview_header.html	Performance — …	app_performance
app/profile.html	105	app/base_app.html · includes app/_profile_modals.html, app/_profile_scripts.html	Profile — …	app_profile
app/profile_admin.html	74	app/base_app.html · includes app/_profile_modals.html, app/_profile_scripts.html	Profile & Settings — …	app_profile
app/salary.html	316	app/base_app.html	Salary — …	app_salary, app_salary_attendance
app/settings_banking.html	43	app/base_app.html · includes app/_settings_scripts.html	Banking Details — …	
app/settings_company.html	57	app/base_app.html · includes app/_settings_scripts.html	Company Profile — …	
app/settings_invoice.html	44	app/base_app.html · includes app/_settings_scripts.html	Invoice Settings — …	

File	Lines	Extends / includes	Title or role	Rendered by
app/settings_tax.html	41	app/base_app.html · includes app/_settings_scripts.html	Tax & Compliance — …	
app/settings_users.html	232	app/base_app.html · includes app/_settings_scripts.html	Users & Roles — …	
app/signup.html	314	· includes app/_auth_hero.html, app/_splash.html		app_signup
app/toll.html	395	app/base_app.html · includes app/_maintenance_tabstrip.html	Toll — …	app_toll
app/trip_add.html	884			app_add_trip, app_edit_trip
app/trip_detail.html	324	app/base_app.html	… — …	app_trip_detail
app/trips.html	207	app/base_app.html	Trips — …	app_trips
app/tyres.html	443	app/base_app.html · includes app/_maintenance_tabstrip.html	Tyres — …	app_tyres
app/urea.html	375	app/base_app.html · includes app/_maintenance_tabstrip.html	Urea — …	app_urea
app/vehicle_detail.html	149	app/base_app.html	… — …	app_vehicle_detail
app/vehicles.html	367	app/base_app.html	Vehicles — …	app_vehicles
app/verify_otp.html	167			app_login_otp, app_verify_login_otp

46. Static files: styles, scripts, images

File	Lines	What it is
`static/style.css`	494	Website stylesheet: design tokens (colours), light and dark themes, tables, forms, KPI cards, modals, drawers, maintenance and employee layouts.
`static/app.css`	1923	Phone-app stylesheet: tokens (--a-*), top and bottom bars, + and More fans, cards, bottom sheets, calendar/time pickers, loading overlay.
`static/app-auth.css`	113	Extra rules for the phone sign-in screens.
`static/app_settings.css`	140	Administrator Profile and Settings screens in the app (hub, forms, switches, user list).
`static/route_finder.css`	232	Route Finder form, map box, route cards and stops timeline (used on both layouts).
`static/limited_access_picker.css`	37	Screen picker for Limited Access users.
`static/route_finder.js`	856	Type-ahead, search, map drawing with Leaflet, route selection, stops timeline, retry logic and responsive layout switching (by the finder's own width).
`static/limited_access_picker.js`	76	Presets, per-screen switches and View/Edit toggles, live phone preview; writes the hidden "modules" fields.
`static/mobile_input.js`	24	Keeps mobile-number boxes to 10 digits (strips +91, leading 0, non-digits) while typing or pasting; the server stays the real check.
`static/app-sw.js`	35	Service worker for the phone app: caches two icons only; never pages or data.
`static/vendor/leaflet/`	—	The Leaflet map library (bundled so no outside CDN is needed) and its images.
`static/app-icons/`	—	Four PNG icons (180, 192, 512, 512 maskable) for the installed app.
`static/img/`	—	Login hero picture and splash highway image.
`static/uploads/`	—	Uploaded files: logo/<company>/, insurance/<company>/, toll_receipts/<company>/ (documents are served only to signed-in users).

Cache busting. Templates link CSS and JavaScript through `asset_v('file')`, which appends the file's modification time as ?v=…, so a changed stylesheet is fetched at once even by a phone that has had the app open for days.

47. Behaviours that run in the browser

Behaviour	Where	What it does
Light / dark theme	base.html + style.css	A switch in the top bar flips a data-theme attribute and remembers the choice in localStorage; because every colour is a CSS variable, the whole site re-themes at once.
Sidebar collapse and soft navigation	base.html	Remembers whether the sidebar is closed; swaps the main area on link clicks instead of reloading.
Notification bell	base.html	Shows how many vehicles have pending challans or documents expiring; a dismissal is remembered for the current sign-in.
Record popups and panels	base.html + ledger.html	Clicking a ledger line fetches the trip or payment detail page and shows it in a popup or side panel, with previous/next buttons.
Trip form intelligence	trip_form.html, app/trip_add.html	Live "Trip Summary" (net amount, estimated profit, payment tracking); the Type box follows the chosen vehicle (a normalised map of vehicle number to own/hired); extra "Others" lines can be added; hired-vehicle fields appear only for hired trucks.
Invoice review gate	invoice_center.html	Tracks which LR links were opened. Generate and Draft stay locked until every selected trip is reviewed; a single speech balloon above the LR column explains what is missing and disappears after a few seconds. The server checks the same list.
Payment popup	ledger.html	Lists the pending trips with tick boxes and an amount box; shows how the entered amount will be applied.
Read-only layer	injected by the server	For Read Only users and View-only screens: a "View-only access" badge, add/edit/delete buttons hidden, forms and fetch() writes intercepted with a friendly message.
Limited Access picker	static/limited_access_picker.js	Preset chips, per-screen switch and View/Edit toggle, phone preview; used in the website user modal and the phone user sheet.
Custom date and time pickers	base_app.html	Replace the phone's native pickers with a themed calendar (day, month and year views) and a time wheel; date-range pairs stop a To before a From.
Datalist enhancement	base.html and base_app.html	Turns native suggestion lists (vehicles, parties, vendors) into a styled list that works on phones.
Mobile number guard	mobile_input.js	Keeps mobile boxes to ten digits.
Route Finder	route_finder.js	Debounced type-ahead (350 ms), map, route cards, timeline with skeleton placeholders while stops load, automatic retry after 4 seconds when the free server is busy.
Settings forms in the app	_settings_scripts.html	Every Administrator form posts with fetch() and the header X-App-Request; success reloads the screen with a "Saved" toast, an error appears inside the form.

48. The installable app (PWA) and in-app navigation

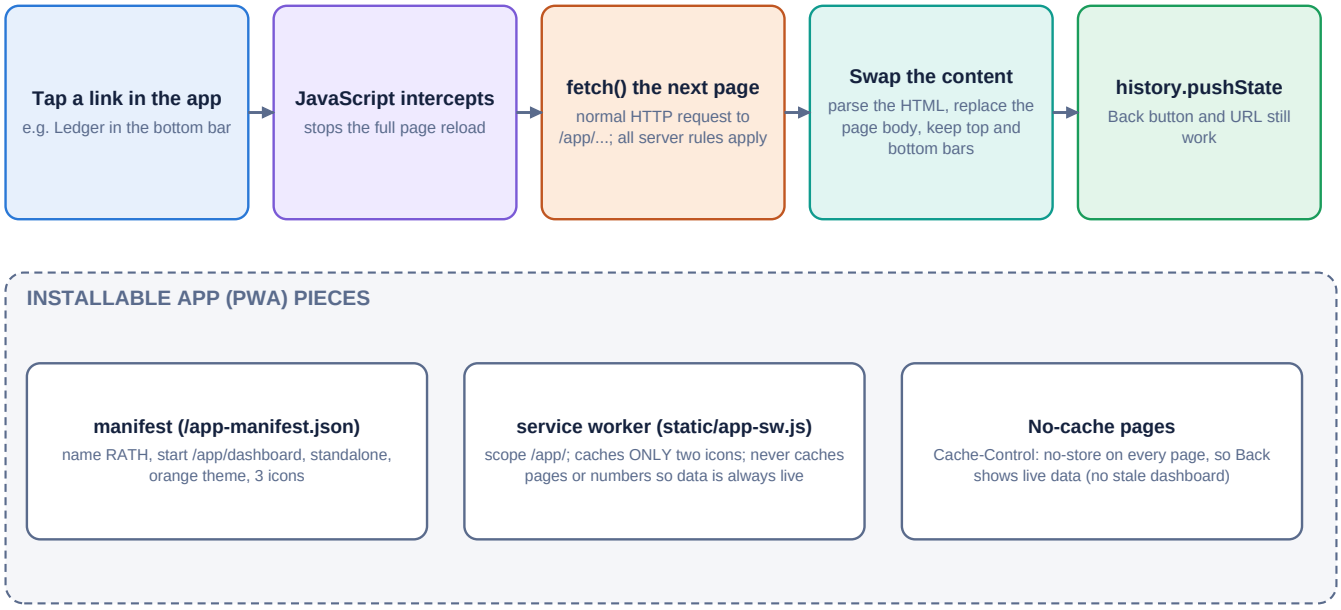


Figure 15 — How the phone app feels instant: in-app navigation swaps page content; the installed shell stays put.

- **Installability.** /app-manifest.json declares the app name (RATH), start page /app/dashboard, scope /app/, standalone display, orange theme and three icons. A service worker registered with scope /app/ satisfies the browser's install requirement.
- **No offline data, on purpose.** The service worker caches only two icon files. Every page and every number comes from the server, so what a user sees is never stale.
- **Navigation.** Tapping a link in the bottom bar calls navigate(url): it fetches the next page, parses it, replaces only #appMain and the page scripts, updates the address with history.pushState, re-wires the date and time pickers on the new content and highlights the right tab. If anything unexpected happens (not signed in, network error, odd response) it falls back to a normal page load.
- **Page transitions.** If a real navigation takes longer than about 0.4 seconds a translucent overlay with a moving truck appears; it never shows for quick switches or for fetch() calls that stay on the page.

49. Design system

Item	Website	Phone app
Colour tokens	--ink #14213d, --steel #5a6b8c, --line #e2e6ed, --paper #f5f6f9, --accent #c05621 (brand orange), --green #1a7a4c, --red #b3392f; dark theme: --ink #eef1f8, --paper #0d1220, --white #161c2e and brighter accent/green/red	--a-ink, --a-steel, --a-line, --a-paper, --a-accent #c05621 and --a-accent-dark #9a4419, --a-green #1a9c5b, --a-red #e34948, --a-blue #2a78d6, --a-purple #7a5ad6, --a-amber #eda100, --a-teal #0f9b8e (light only)
Typeface	Segoe UI, then system fonts	System font stack for pages; Poppins on the sign-in, SMS-code and sign-up screens
Base size and spacing	15 px body, cards with 12–16 px radius, tables with zebra rows and fixed dark header bars	Cards with 16 px radius, 0.7–0.9 rem text, bottom sheets with 20 px top radius, 44 px touch targets
Charts	SVG paths, CSS bars and conic-gradient donuts written by the templates	Same approach, simplified
Responsive	Breakpoints around 480, 640, 760, 900, 1000, 1100, 1200 and 1300 px; sidebar can collapse	Single column; layout locks to portrait; env(safe-area) insets respected
Motion	Short transitions on buttons and switches	Fans, sheets and the loading overlay; all disabled under prefers-reduced-motion
Dark mode	Yes (switch in the top bar)	No (single light theme)

A known inconsistency

Font sizes, spacing and a few colours are repeated as literal values in many page-level style blocks instead of coming from shared tokens, and the website and phone token sets are separate. It is cosmetic, tracked in code_health_report.md, and the clean-up path is to define one set of typography and spacing tokens and replace the literals.

H

Quality: tests, speed and honest findings

How correctness is proven, how fast it runs under load, and the gaps found while documenting the code.

IN THIS PART

- Test architecture and results
- Performance engineering and the load test
- Findings recorded while preparing this document
- Business decisions still open

50. How the app is tested

The biggest risk in a money application is a screen showing a number that is quietly wrong. The test strategy attacks that directly: drive the real screens with thousands of invented situations and compare every number shown with an independent calculation.

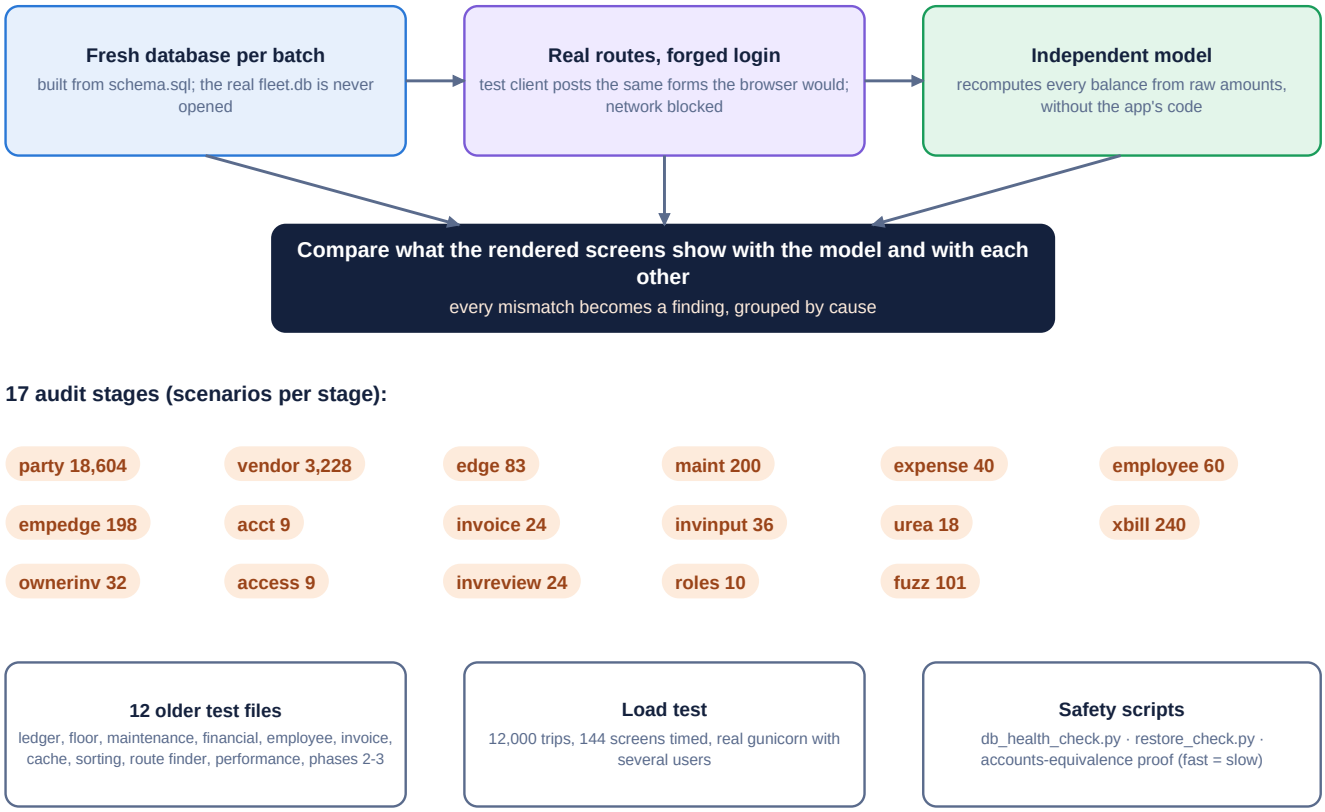


Figure 16 — How the app is tested: real screens versus an independent calculation, on throw-away data.

50.1 The audit harness (test_balance_audit.py, 3,000+ lines)

- **Isolation.** Each batch of scenarios builds a fresh SQLite database from schema.sql. The real fleet.db is never opened. All outside network calls are blocked, so no paid API can be called and no SMS can be sent.
- **Realism.** Scenarios are executed through the real routes with a forged session for a real test user (the app re-reads the user on every request, so the test user exists in the scratch database). Website forms and phone routes are both used.
- **Independent model.** After every step the harness reads the numbers off the rendered pages (using an HTML parser) and compares them with its own first-principles arithmetic and with each other (for example Trips "Balance due" versus Ledger balance versus Dashboard receivable).
- **Findings, not failures.** Every disagreement is recorded with the exact scenario that caused it and grouped by cause so a few hundred identical symptoms become one finding. Remaining findings are classified DESIGN (a business definition difference) or BUG; the run for this document shows only DESIGN items.

Stage	Scenarios	Checks	What it proves
`party`	18,604	918,301	every sequence of trip, advance, payment (ticked and unticked), edit, end and delete for a party; Trips vs Ledger vs Dashboard; remaining differences are the opening-balance and unallocated-payment DESIGN items
`vendor`	3,228	92,992	owner-hire trips, fuel and advance paid by the company, maintenance bills, expense bills and vendor payments; vendor ledger vs popup
`edge`	83	145	awkward inputs and orderings

Stage	Scenarios	Checks	What it proves
maint	200	2,040	maintenance bills and allocations
expense	40	396	expenses, pending bills, vendor linking
employee	60	3,026	salary, advances, repayments, date filters
empedge	198	350	employee edge cases
acct	9	504	Ledger list, bulk balances
invoice	24	162	invoice totals, charges, deductions, large invoices
invinput	36	50	invoice number and amount validation
urea	18	388	stock in/out/direct and stock floor
xbill	240	5,172	expense bills naming vendors and payment settlement
ownerinv	32	128	owner (freight bill) invoice rules
access	9	94	Read Only: every write refused, nothing changes
invreview	24	36	invoice review gate
roles	10	218	Administrator-only screens, own profile, last-Administrator rules, security controls, the one-Administrator rule, all of Limited Access (screen map, view/edit, navigation, dashboard cards), MSG91
fuzz	101	1,212	garbage into every POST route: no 500s, no data change

Latest full run (all 17 stages): no BUG findings; the only differences are the DESIGN items listed in Chapter 52.

50.2 The other test files

File	Purpose (first line of its own description)
test_accounts_equivalence.py	Proves the fast bulk _accounts_rows() returns exactly the same balances as the original per-ledger
test_business_performance.py	Focused test pass on Business Performance (/business-performance) -- every computed metric on
test_employee_ledger.py	Phase 1d: Employee Ledger integrity suite -- opening balance, Salary Paid, Advance Given/
test_financial_integrity.py	Phase 1 financial-integrity test suite — the actual money-flow correctness checks across
test_floor_protection.py	Phase 1e: edit_trip floor-protection regression check -- the exact "past bug class" the
test_invoice_charge_exclusions.py	Regression test for the new Invoice Center "Trip Charges & Costs" popup feature --
test_ledger_integrity.py	Phase 1b: exhaustive Ledger integrity suite -- every source that can write a party/vendor
test_maintenance_ledger.py	Phase 1c: Maintenance sub-tabs (Tyre/Battery/Service/Insurance) vs Ledger integrity suite.
test_multitenancy_isolation.py	Two-company isolation test — the actual test that matters for multi-tenancy Step A.
test_no_bfcache.py	Regression test: dynamic pages (Dashboard, Trips, etc.) must carry Cache-Control: no-store so
test_phase2_ledger_trips_maintenance.py	Phase 2: combination testing across Ledger x Trips x Maintenance -- the three genuine
test_phase3_analysis_combinations.py	Phase 3: cross-domain testing of the Analysis section (Route Analytics, Fleet Utilization,
test_route_finder.py	Regression test for the live Route Finder (providers/osm_routes.py + /route-finder/* in app.py).
test_support.py	Shared helper for the test scripts: the app now checks, on every request, that the logged-in user
test_vehicles_sort_persist.py	Regression test: the Vehicles page sidebar link (bare /vehicles, no query string) should
load_test.py	Load test for the fleet app (website + mobile /app/ screens).
db_health_check.py	Read-only database health check for the fleet app.

The Postgres isolation test (test_multitenancy_isolation.py) needs a real PostgreSQL connection string and is skipped until Phase 1 starts. The accounts-equivalence test proves the fast bulk balances equal the original per-account ones (on a copy of the real data and on the 12,000-trip database). Run any test with `python3 test_x.py`; the harness stage with `python3 test_balance_audit.py --stage roles`.

51. Performance

A load test seeds 12,000 trips, 4,215 payments (17,086 allocations), 4,500 maintenance bills, 2,280 salaries, 3,500 expenses, 24,000 attendance rows and 6,000 toll entries (a 10.9 MB database) and times every screen. Rating bands: under 100 ms excellent, under 300 ms good, under 1 s OK, 1–3 s slow, over 3 s poor.

Group	Screens	Excellent	Good	OK	Slow	Poor
Website screens	87	76	8	2	1	0
Phone screens	36	31	2	3	0	0
Excel / PDF downloads	21	10	4	5	1	1
Buttons that save data	13	12	1	0	0	0

Slowest items: Business Performance over a long date range about 1.0 s (189 queries); Dashboard with a financial-year filter about 0.5 s (plain 0.27 s); the full Trips PDF about 5.8 s and the whole-database Excel about 1.3 s. Everything else is under half a second.

51.1 Many people at once

Real gunicorn with the Render settings (2 workers × 4 threads). Each virtual user signs in and clicks around (about 1–2 clicks a second, one in ten is a save):

Users at once	Requests/s	Typical (p50) ms	p95 ms	Slowest ms	Errors
1	1.5	76	313	562	0
5	7.0	60	320	1,732	0
10	10.1	73	1,726	5,733	0
25	7.5	1,783	6,062	11,471	0
50	5.1	5,637	15,091	18,806	0

Reading it: up to 10 simultaneous users the app stays snappy and error-free; at 25 or more heavy simultaneous users pages slow to several seconds because only 8 requests can run at once and SQLite allows one writer. For about five staff this is far inside the safe zone. The next steps if growth demands it are more workers, trimming the heaviest reports and PostgreSQL.

51.2 What made it fast

- **Indexes.** About 36 were added (including partial indexes for mostly-empty allocation columns). The foreign-key columns of every table with 200+ rows are indexed (checked by db_health_check.py).
- **Bulk balances.** The Ledger list went from about 9,000 queries to a handful (3.5 s → 0.1 s; Dashboard 3.9 s → 0.27 s).
- **WAL mode and a 30-second lock wait.** Saves no longer fail with "database is locked" when 25 or more users work at once.
- **Single connection per request,** closed at teardown even after a crash.
- **No caching of data.** Speed comes from fewer queries, never from remembering old answers.

52. Findings recorded while preparing this document

Writing down every path exposed several small gaps. None changes today's numbers, but each is worth knowing and fixing. They are listed so nothing is hidden; the first group are data-integrity gaps, the second are design and operations notes.

52.1 Data-integrity gaps (left behind rows or silent no-ops)

#	Where	What happens	Suggested fix
1	delete_vehicle	A bare DELETE with no check; trips, bills, documents and tolls that point at the vehicle are left pointing at a missing id (foreign keys are not enforced).	Refuse when the vehicle is used, like delete_trip does; offer Deactivate instead.
2	delete_trip	Removes only the trips row. The trip's invoice_items ("Others" lines) and any invoice_batch_trips link remain as orphans.	Delete or refuse on those links too.
3	invoice_batch_delete	Leaves the batch's invoice_batch_charge_exclusions rows behind.	Delete them in the same transaction.
4	edit_employee	Salaries and advances are matched by employee NAME; renaming an employee leaves the old name on earlier rows, which then no longer match.	Match by employee_id, or rewrite the name on related rows when renaming.
5	add_service, add_tyre_stock, add_battery, install_tyre_stock, add_compliance_expense	The website routes ignore the validation message returned by their helper and just redirect, so a request missing a required field (possible only if the browser form checks are bypassed) silently saves nothing.	Pass the error back like add_overhead does.
6	invoice numbers	invoice_batches.invoice_number has no UNIQUE constraint and the next number is read, then written at the end of the request. Two generations at the same moment could use the same number.	Add a unique index on (company, invoice_number) and take the number inside the same write transaction.
7	payments	The duplicate guard covers only 10 seconds, and payments have no edit or reverse screen, only the allocation of what was recorded.	A "reverse payment" action that restores allocations.
8	upload files	Toll and insurance uploads accept any file type and size up to the 16 MB request cap, and live in the app folder, not on the persistent disk.	Allow PDF/JPG/PNG/WEBP, limit size, resize photos, store on the disk.

52.2 Design and operations notes

- **app.py is one 14,800-line file.** It works and is tested, but splitting it into modules (trips, ledger, maintenance, invoices, security, phone) would make change safer.
- **Money is stored as floating-point numbers** rounded to paise on writes. Tests compare totals with independent arithmetic, but a fixed-point (integer paise) design would remove any rounding doubt.
- **Foreign keys are declared but not enforced** by SQLite at run time; consistency is kept by code and watched by db_health_check.py (currently 4 old orphan rows from mock compliance and insurance data, one trip over-received by Rs 434, one duplicate LR).
- **FASTag sync is a mock.** The toll Excel import (BOSS statements) is the real route for toll data.
- **The toll-import stash is per worker and in memory.** A restart between "preview" and "import" means repeating the preview.
- **Sync endpoint token compare** (sync-tick) is a plain inequality; the backup endpoint uses a constant-time compare.
- **Security still open:** full CSRF tokens (today SameSite + Origin check), a Content-Security-Policy, encryption of stored keys, pinned dependency versions, error alerting, closing the old Twilio account, removing old database backups from the GitHub history.

- **Uploads and the persistent disk** must be confirmed for production (Chapters 6 and 9).

52.3 Business decisions still open (screens may differ by definition)

- **Ledger versus Trips:** a party's opening balance and any payment not allocated to a trip count in the Ledger but not in the Trips balance.
- **Invoice versus ledger:** an invoice's grand total (optional lines, discounts, detention, RCM) can differ from the ledger's billed amount.
- **Linked party + vendor ledger** has no "pay the owner" popup; vendor popup gaps for opening balances and maintenance.
- **Maintenance Overview** counts own vehicles only (hired-vehicle bills are excluded).
- **Employees:** a repayment can exceed what was given (no cap).

Running, changing and extending

The practical handbook: day-to-day commands, a file-by-file reference and a checklist for adding something new without breaking the rules.

IN THIS PART

- Runbook: the commands and tasks
- File-by-file reference
- How to add a screen, a table or a rule safely
- Roadmap

53. Runbook

53.1 Run it on your computer

```
cd /Users/beenash/Downloads/fleet_local_app
python3 app.py                                  # website + phone app at http://127.0.0.1:5050  (debug on)
FLASK_HOST=0.0.0.0 FLASK_DEBUG=0 python3 app.py # also reachable from phones on the same Wi-Fi (use debug OFF)
python3 test_balance_audit.py --stage roles     # run one test stage (omit --stage for everything)
python3 db_health_check.py                      # read-only audit of fleet.db
```

53.2 Everyday tasks

Task	How
Create a user	Sign in as the Administrator → Profile and Settings → Users and Roles → Add User (mobile number, name, role, access level; for Limited Access tick screens and choose View or Edit). Password is optional (SMS login only if blank).
Give a user fewer screens	Edit the user → Access level Limited Access → presets or switches. Takes effect on their next click.
Hand the Administrator seat to someone	Edit that (active) user → Make Administrator. You become a Manager with Full Access.
Turn on SMS login	Create a DLT-approved MSG91 OTP template containing ##OTP##; copy the Auth Key and Template ID; switch off the key's IP restriction or add Render's address; paste both in Users and Roles → Integrations (or the website SMS card).
Change invoice numbering or GST	Settings → Invoice Settings / Tax and Compliance.
Take a backup now	python3 backup_db.py (local) or BACKUP_URL=… BACKUP_TOKEN=… python3 pull_backup.py (live).
Check a backup restores	python3 restore_check.py path/to/backup.db
Deploy	Run the tests, commit and push the branch Render follows; Render builds and restarts; the app applies its own safe schema steps. Check the server log for the [schema] / [security] lines.
Rotate a key	MSG91 or eChallan: create a new key in the provider's console, paste it into Settings (blank box keeps the old one), then disable the old key. SECRET_KEY: change it in Render (everyone is signed out once).
Reopen public sign-up	Set ALLOW_SELF_SIGNUP=1 (not recommended).

53.3 When something is wrong

Symptom	First place to look
A page is blank or error 500	The server log; then the layer that owns it (Figure 2). Bad form input is answered with a friendly 400, so a 500 is a real bug worth a test.
"Database is locked"	Should not appear (30 s wait, WAL). If it does: a long report plus many saves; check the load test numbers and worker count.
A user says "no access"	Their access level and screens in Users and Roles; Read Only and View screens refuse writes by design.
Balances disagree between screens	Run python3 db_health_check.py; then the audit stages party / vendor; compare against Chapter 52.3 (known definition differences).
SMS code not arriving	MSG91 template approved? Auth Key IP restriction? Balance? The app says "Could not send the code right now" and logs the provider's reason.
Everything vanished after a deploy	The persistent disk is missing: restore from Render's snapshot or the latest pull_backup copy (BACKUP_AND_RESTORE.md).
A vehicle shows no RC data	No RC key saved, or the vehicle is hired (not synced), or the paid sync has never run: use "Sync Latest Data" on the vehicle.

54. File-by-file reference

File / folder	Lines	Purpose
app.py	14,834	The whole back-end: routes, business rules, security hooks, PDF/Excel builders, phone-app routes, self-applying schema steps.
db.py	113	Connection wrapper (pass-through today; the one place a PostgreSQL switch would be wired).
scheduler.py	341	APScheduler jobs and the sync-tick logic: weekly compliance, RC sync, challan sync, back-fill rules, sync_log.
compliance_service.py	279	Fitness/PUC/permit status calculation and the sync of a vehicle or the whole own fleet through the providers.
providers/base_provider.py	38	The shared provider interface and the "mock or live" rule.
providers/echallan_client.py	146	HTTP client for the eChallan RC and challan APIs (X-API-Key header, one call serves several providers).
providers/vahan_provider.py	51	Fitness slice of the RC answer (mock until a key exists).
providers/puc_provider.py	47	PUC slice of the RC answer.
providers/permit_provider.py	49	Permit slice of the RC answer.
providers/insurance_provider.py	35	Placeholder for a future insurance cross-check API.
providers/osm_routes.py	726	Route Finder engine: Photon suggestions, Nominatim geocode, OSRM routes, Overpass features, ranking, rate limiting and budgets.
config/apis.py	58	Compliance provider defaults and environment-variable names.
schema.sql	880	Authoritative SQLite schema (also used to build the test databases).
schema_postgres.sql	1,059	PostgreSQL twin with real company_id enforcement and row-level security (parse-verified, not yet run).
migrate.py, migrate_extra.py, migrate_v2.py	114	One-time imports from Fleet_Master_Tracker.xlsx (historical).
migrate_*.sql (12 files)		Documented schema changes; see Chapter 26.
backfill_rc_fields.py	69	One-off script that back-filled vehicle spec/date fields from cached RC data.
Procfile		The gunicorn command Render runs.
requirements.txt		Python packages: flask, openpyxl, reportlab, gunicorn, apscheduler, requests.
backup_db.py	57	Local verified backup of fleet.db.
pull_backup.py	94	Daily download of a verified copy of the live database from Render.
restore_check.py	56	Proves a backup restores and the app runs on it.
db_health_check.py	136	Read-only database audit (integrity, drift, anomalies).
load_test.py	706	Seeds 12,000 trips and times every screen and concurrent users.
test_balance_audit.py	3,143	The 17-stage audit harness.
test_*.py (15 more)		Earlier regression suites; see Chapter 50.2.
test_support.py	20	Helper that creates an active Administrator test user in disposable databases.
CLAUDE.md	71	Project rules (multi-company readiness, test on copies, deploy database and code separately).
README.md, DEPLOY_TO_CLOUD.md, BACKUP_AND_RESTORE.md	59	Original local guide, an older DigitalOcean deployment guide (superseded by Render), and the current backup/restore guide.
production_readiness_report.md, code_health_report.md, balance_audit_findings.md, load_test_report.md	85	Review reports that feed Part H.

File / folder	Lines	Purpose
`docs/blueprint/`		This document's generator: lib.py (layout), diagrams.py, facts.py and analysis.py (read the real code), content_*.py (text), build.py.
`archive/unused_ui/`		Retired templates kept for reference (old Analysis page, old salary list, mock-ups).
`audit_reports/`		Machine-written reports of earlier audit runs.
`static/, templates/`		Front-end files; see Chapters 45 and 46.

In numbers: app.py 14,834 lines; 434 functions; 199 URLs; 107 templates (23,192 lines); 2,939 lines of CSS and 991 lines of JavaScript of our own; 36 tables and 62 indexes; 16 test files.

55. Adding something new without breaking the rules

A short checklist, distilled from the project rules (CLAUDE.md) and from every defect found during this project.

If you add…	Do this
A table	company_id INTEGER NOT NULL DEFAULT 1 (and the Postgres twin with a foreign key and an index; natural-key uniques as (company_id, name)); add created_by/at, updated_by/at; add it to schema.sql and schema_postgres.sql; write a guarded self-applying step if the live database must change.
A form that saves	One _insert_X_from_form / _update_X_from_form helper used by both website and phone routes; validate every number with _parse_money; return (ok, message) and SHOW the message; resolve names with get_or_create_*; redirect after POST.
A screen	Put its numbers in an _X_data function shared by website and phone; add its routes to the right screen in _MODULE_ENDPOINTS (a test fails otherwise); wrap its menu entries in can('Screen'); read company settings through _get_all_settings(company_id).
Money logic	Change the shared helper, never a screen; add a scenario to the audit harness (party / vendor / the relevant stage) and run the full run; compare every screen that shows the number.
A setting or secret	Add the key to ALL_SETTING_KEYS and to a group in _SETTINGS_FIELD_GROUPS; if secret, add to _SECRET_SETTING_KEYS (never printed back; blank keeps the saved value).
An upload	Save under uploads/<type>/<company_id>/; restrict type and size; serve private documents only to signed-in users.
A call to a paid or outside service	Opt-in switch, hard timeout, failure message that never leaks details, tests with the network blocked.
A new company-specific text	Never hard-code the company name, address or GSTIN in a page or PDF; use get_company_name(company_id) and settings.

55.1 Roadmap

- **Before real users:** confirm the persistent disk and daily backups on Render, set SECRET_KEY, rotate old keys, make sure exactly one Administrator exists there, test on real phones.
- **Soon:** fix the integrity gaps in Chapter 52.1, put uploads on the disk with type/size limits and image shrinking, add a disk-usage indicator, pin package versions, add uptime and error alerts, full CSRF tokens and a Content-Security-Policy.
- **Quality of the code base:** split app.py into modules, one set of design tokens for fonts and spacing, integer-paise money.
- **Scale:** multi-company (read company_id from the session everywhere, remove the seven hard-coded company PDFs), PostgreSQL with row-level security, more workers.

End of the blueprint

This document was generated from the project files by docs/blueprint/build.py: re-run it after any change and the counts, the tables, the data dictionary, the execution cards and the table lifecycle regenerate from the real code.